

UZA – Universelle Zusammenhangsarchitektur

Gesamtspezifikation 0.9.1 Master · Formalrevision T6 kernelgeprüft

Konsolidierte Referenz für Architektur, Semantik, Grammatik, Engine,
Dynamik, Skalierung, Interaktion und formale Theorie

Arbeitsstand: 9. August 2026 · Dokumentstatus: konsolidierte Schließungsfassung 0.9.1 ·
Formalrevision T6 kernelgeprüft

Fortschreibungsbasis: vollständige T5-Masterfassung · Isabelle2025-2-Fresh-Profile-Build mit elf Theorien · T1–T6-Erhaltung kernelgeprüft · nicht-vakuoser gerichteter T6-Zeuge bestanden
Statusprovenienz: Alle bis T5 ausgewiesenen Isabelle-Stände bleiben historisch dokumentiert und inhaltlich erhalten. Maßgeblich für T6 ist der am 9. August 2026 aus einem leeren Isabelle-Benutzerprofil ausgeführte Build des gepinnten Gesamtstands mit elf Theorien. Er nutzte die offizielle Isabelle2025-2-Pure/HOL-Basis, kompilierte die UZA-Session vollständig aus den Quellen und endete mit Exit-Code 0. Rohlog, Exit-Code, ROOT, Theoriequellen, Prüfsummen, Source Audit und Verifikationsbericht werden als getrennte Einzeldateien geführt.

Inhaltsverzeichnis

Dokumentidentität und normativer Status.....	4
Leseschlüssel.....	5
Teil I – Ursprung, Entwicklung und Gesamtarchitektur.....	6
1. Ursprung und Leitfrage.....	6
2. Entwicklungslinie und Konsolidierungsentscheidungen.....	6
3. Viergliedriger Kern mit orthogonalen Erweiterungsbereichen.....	8
4. Zwölf architektonische Grundaxiome RA1–RA12.....	9
5. Verbindliche Invariantenfamilien.....	10
6. Systemgrenze und Anwendungsprinzip.....	11
6.1 Meta-ontologische Geltungsgrenze.....	11
Teil II – Ontologie und semantischer Kern.....	13
7. Zusammenhang als formale Grundeinheit.....	13
8. Drei Zeitebenen.....	13
9. Primäres Typensystem.....	14
10. Objektidentität, Versionierung und Provenienz.....	15
11. Lifecycle und Unsicherheit.....	15
12. Perspektivsemantik.....	16
13. Beziehungen und Wirkungen.....	16
14. Konsolidierter Wirkungsvektor.....	17
15. Resonanzsemantik.....	20
16. Relevanzsemantik.....	20
17. Orientierung, Gestaltung und Rückwirkung.....	21
18. Beschreibbarkeit und Emergenz.....	22
Teil III – Zustandsmaschine und formale Grammatik.....	23
19. Konsolidierte Zustandsmaschine.....	23

20. Drei Wandeltypen.....	24
21. Transformationsgrammatik T1–T13.....	24
22. Regelkalkül und Ableitungsprinzipien.....	26
23. Grammatikklassen G0–G4.....	26
24. Konsistenzbedingungen C1–C10.....	27
25. Meta-Evolution.....	27
Teil IV – Grammar Engine und operationale Ausführung.....	30
26. Engine-Zielbild.....	30
27. Komponenten der Grammar Engine.....	30
28. Event-Sourcing und Copy-on-Write.....	32
29. Commit Ledger.....	32
30. Drei Validierungsebenen.....	32
31. Regel-Executor.....	33
32. Fehler- und Konfliktmodell.....	33
33. Fünfzehn Mindestanforderungen einer konformen Engine.....	33
34. Bereinigter Engine-Hauptzyklus.....	34
Teil V – Zusammenhangsdynamik.....	35
35. Dynamikmodell.....	35
36. Zehn dynamische Grundmuster.....	35
37. Flow Analyzer und Dynamics Detector.....	36
38. Relevance Feedback Loop.....	36
39. Perspective Tracker.....	37
40. Complexity Monitor und Zusammenhangskompetenz.....	37
Teil VI – Skalierung und Interaktion.....	38
41. ScaleProfile.....	38
42. ContainmentGraph.....	38
43. SUBZ, Projektion und LossProfile.....	38
44. InteractionNetwork und Kopplungsobjekt.....	39
45. Kontrollierte Wirkungsübertragung.....	39
46. Transmission Lifecycle.....	40
47. Grenztransformationen.....	40
48. Interaktionskomplex.....	41
49. Interaktionsoperationen I1–I11.....	41
50. Konsistenzbedingungen C11–C18.....	42
51. Interaktions-Emergenz.....	42
Teil VII – Durchgängige Demonstratoren.....	43
52. Methodischer Status der Demonstratoren.....	43
53. Demonstrator A – KI-Einsatz in einer Organisation.....	43
54. Demonstrator B – Unternehmen, Betriebsrat und Regulierung.....	47
Teil VIII – Referenzimplementierungsdesign und Conformance 0.6/0.9.1.....	49
55. Technische Architektur.....	49
56. Logisches Datenmodell.....	49
57. API-Entwurf.....	50
58. Conformance- und Teststrategie.....	51
59. Implementierungsstatus.....	52
Teil IX – Formale Theorie und mathematische Schließung.....	53
60. Logische Grundlage.....	53
61. Konsolidierte Signaturentscheidungen.....	53
62. Natürlichsprachliches Axiomregister A1–A22.....	54
63. Konditionale Existenzaxiome der Interaktion.....	55
64. Relative Konsistenz und Modell-Expansion.....	55
65. Beweisziele.....	56
66. Roadmap M0–M6.....	57
67. Mathematischer Netto-Status.....	58
Teil X – Gesamteinordnung.....	60
68. Abschließende Definition.....	60
Anhang A – Verbindliche Artefakt- und Statusmatrix.....	61
Anhang B – Version Bundle und Reproduzierbarkeitsmanifest.....	63

Anhang C – Vollständiger Referenz-Pseudocode.....	64
Anhang D – UZA HOL Theory 0.9.1: T1–T6 Kernel-Checked.....	66
D.0 Leistungsstand auf einen Blick.....	66
D.1 Haupttheorie.....	68
D.2 Expliziter minimaler Basismodellzeuge.....	76
D.3 Konkrete T1-Semantik und Erhaltung für RegisterAppearance.....	80
D.4 Reichhaltiger nicht-vakuoser Modellzeuge.....	87
D.5 Konkrete T2-Semantik und Erhaltung für DifferentiatePotential.....	95
D.6 Konkrete T3-Semantik und Erhaltung für CreateRelation	102
D.7 Konkrete T4-Semantik und Erhaltung für HypothesizeEffect	111
D.8 Konkrete T5-Semantik und Erhaltung für IntegrateEvidence	122
D.9 Nicht-vakuoser Ausführungszeuge für T5	143
D.10 Gerichtete T6-Semantik und Erhaltung für ClassifyResonance	149
D.11 Nicht-vakuoser Richtungszeuge für T6	166
Anhang E – Offene mathematische Beweisaufgaben.....	172
Anhang F – Glossar.....	173
Anhang G – Konsolidierungsprotokoll 0.9.1.....	175
Anhang H – Validierungsprotokoll 0.9.1.....	178
H.1 Formale Quell-, Modell- und Operationsprüfung.....	178
H.2 Ausführbare Conformance.....	184

Dokumentidentität und normativer Status

Diese Masterfassung führt die belastbaren Ergebnisse der bisherigen UZA-Entwicklung in einem Dokument zusammen. Sie ersetzt die verstreute Lektüre der Versionen 0.1 bis 0.9.1, bewahrt deren Entwicklungslinie und trennt drei Reifearten konsequent:

- **Architektonisch konsolidiert:** Begriffe, Abhängigkeiten und Operationsklassen sind stabil genug, um weitere Arbeit darauf aufzubauen.
- **Spezifiziert:** Daten, Regeln, Zustandsübergänge und Validierungsbedingungen sind so beschrieben, dass eine Referenzimplementierung erstellt werden kann.
- **Formal bewiesen oder mechanisiert:** Aussagen sind in einer Beweisumgebung parsergeprüft und durch Beweise oder Modellkonstruktionen abgesichert.

Für die UZA gilt aktuell: Kern, Grammar Engine sowie Scale & Interaction sind spezifikatorisch weitgehend konsolidiert. Das Conformance-Harness reproduziert drei Golden Runs, ist jedoch kein Produktivsystem. Die HOL-Theorie 0.9.1 enthält ausführbare Referenzdefinitionen, A1–A22 als typisierte Locale-Annahmen, einen minimalen und einen reichhaltigen Modellzeugen sowie konstruktive Semantiken für T1 bis T6. T6 ClassifyResonance konstruiert aus zwei verschiedenen aktiven Effekten ein frisches K_RES-Objekt und einen typisierten Katalogeintrag. Eine versionierte gerichtete Interaktionsmatrix bindet Quelle an Ziel; Perspektive, Kontext, Modell, Zeithorizont, Unsicherheit, Begründung, Score, Form und Stärke werden explizit persistiert. Die endliche V10-Domäne ist in HOL vollständig charakterisiert. Ein nicht-vakuoser Zeuge beweist $A \rightarrow B = 1/\text{Amplification}$ und $B \rightarrow A = 0/\text{Neutral}$. Der Isabelle2025-2-Fresh-Profile-Build aller elf Theorien endete am 9. August 2026 mit Exit-Code 0. Nicht daraus folgen empirische Angemessenheit eines Resonanzmodells, Vollständigkeit der fünf Formen, allgemeine Modell-Expansion, Konservativität oder Produktivengine-Äquivalenz.

Verbindliche Kurzform: Architecture & Engine Specification 0.9.1-RC; Formal Theory – Isabelle2025-2 Kernel-Checked 0.9.1 für Haupttheorie, A9, minimale und reichhaltige Modellzeugen sowie T1–T6 Preservation; Fresh-Profile-Build mit elf Theorien und Exit-Code 0; T5 und T6 zusätzlich durch nicht-vakuoese Ausführungszeugen belegt; T6 mit mechanisch bewiesener gerichteter Asymmetrie; Conformance Harness 0.1, Golden Runs bestanden. T7–T13, allgemeine Expansion, Konservativität und Engine-Äquivalenz bleiben offen. „1.0“ bleibt der weitergehend beweistheoretisch, modelltheoretisch und technisch geschlossenen Fassung vorbehalten.

Nachweisprovenienz der Isabelle-Statusangaben

Die früher als kernelgeprüft ausgewiesenen Sessionstände bis einschließlich T5 bleiben als historische Nachweisprovenienz erhalten. Maßgeblich ist nun der gepinnte T6-Gesamtstand: eine aus einem leeren Isabelle-Benutzerprofil gebaute Isabelle2025-2-Session mit elf Theorien. Der unveränderte Konsolenoutput steht in Isabelle2025-2_Fresh_Profile_Build_T6.log; Quellen, ROOT, Exit-Code, Quellhashes, Source Audit und Verifikationsbericht werden als einzeln auffindbare Artefakte geführt.

Die frühere Aussage, in einer zwischenzeitlichen Bearbeitungsumgebung sei Isabelle nicht verfügbar gewesen, beschreibt nur einen damaligen Kandidatenstand. Sie ist kein aktueller

Prüfstatus. Maßgeblich ist der Fresh-Profile-Build vom 9. August 2026 mit Exit-Code 0. Statusbehauptungen bleiben strikt quellen- und laufgebunden: Der Build belegt die Kernelakzeptanz der gepinnten Definitionen und Theoreme, aber weder empirische Universalität noch allgemeine Konservativität oder Gleichheit mit einer künftigen Produktivengine.

Leseschlüssel

Die Wörter **muss**, **darf nur** und **unzulässig** kennzeichnen normative Anforderungen. **Soll** bezeichnet eine starke Empfehlung mit begründungspflichtiger Abweichung. **Kann** bezeichnet eine zulässige Option. Mathematische Formeln präzisieren die Semantik; sie ersetzen erst dann einen Beweis, wenn dies ausdrücklich als Theorem mit Beweisstatus ausgewiesen ist.

Die Dokumentteile haben folgende Funktion:

- Teile I–III definieren den stabilen Kern K .
- Teil IV beschreibt die operationale Grammar Engine.
- Teil V integriert die Zusammenhangsdynamik.
- Teil VI spezifiziert Skalierung und Interaktion mehrerer Zusammenhänge.
- Teil VII dokumentiert die Demonstratorläufe.
- Teil VIII legt das Referenzimplementierungsdesign fest.
- Teil IX ordnet die mathematische Theorie und ihren Beweisstatus ein.
- Die Anhänge enthalten vollständige Register, Pseudocode, die HOL-Signatur und offene Nachweise.

Teil I – Ursprung, Entwicklung und Gesamtarchitektur

1. Ursprung und Leitfrage

Die UZA entstand aus einer wiederkehrenden Struktur in Arbeiten zu Ontologie, Perspektivenanalyse, lernenden Systemen, Organisationsgestaltung und regelbasierten Analysemaschinen. Trotz verschiedener Gegenstände erschien dieselbe Aufgabe:

Wie entsteht Orientierung in einem komplexen Zusammenhang, ohne dass Wesentliches verloren geht?

Die Antwort wurde zunächst als „Zusammenhangs-Engine“ formuliert. Gemeint war eine gemeinsame Architektur, deren Grundoperationen in verschiedenen Domänen gleich bleiben, während sich Beobachtungen, Perspektiven, Evidenzen, Relevanzmodelle und Gestaltungsoptionen ändern.

Die UZA ist deshalb zugleich:

- eine Ontologie der für Zusammenhangsanalyse benötigten Objekttypen,
- eine Semantik ihrer Bedeutung und Gültigkeit,
- eine Grammatik zulässiger Transformationen,
- eine ausführbare Engine-Spezifikation,
- eine Architektur für dynamische, verschachtelte und interagierende Zusammenhänge,
- und ein kontrolliert erweiterbares formales Theorieprojekt.

Ihre Grundeinheit ist der **Zusammenhang**. Beobachtungen werden als Erscheinungen registriert; ihre Bedeutung bildet sich über Beziehungen, Wirkungen, Resonanzen, Perspektiven, Kontexte und Relevanzordnungen. Orientierung entsteht als begründeter Möglichkeitsraum. Gestaltung wird ausgewählt, ausgeführt, beobachtet und über Rückwirkung erneut in den Zusammenhang integriert.

2. Entwicklungslinie und Konsolidierungsentscheidungen

Version	Hauptgegenstand	Konsolidierter Status
Konzept	Ereignis, Beziehung, Resonanz und Gestaltung; Periodensystem der Erkenntnis	Ursprungsidee
UZA 0.1	Operative Zustandsmaschine, Elementtypen, Wirkungsvektor, Relevanzfunktion, erster KI- Demonstrator	in 0.9 integriert
UZA 0.2	Referenzarchitektur, Potentialraum, Emergenz, Wandeltypen, zwölf Grundaxiome, Invarianten, Meta- Evolution	in 0.9 integriert
UZA 0.3	Semantischer Kern, Signaturen, C1–C10, Unsicherheit, Perspektivsemantik, Describe, G0–G4	Spezifikation 0.9-RC

Version	Hauptgegenstand	Konsolidierter Status
UZA 0.4	Transformationsgrammatik T1–T13 und Regelkalkül	in 0.3/0.5 integriert
UZA 0.5	Grammar Engine mit Event-Sourcing, Copy-on-Write, Ledger, Validierung und Meta-Evolution	Spezifikation 0.9-RC
UZA 0.6	Datenmodell, API, Regel-Executor und vollständiger demonstrativer Commit-Durchlauf	Referenzimplementierungsdesign ; kein Produktivcode
UZA 0.7	Vierdimensionale Dynamik, drei Zeitebenen, Flow-Analyse und zehn Muster	spezifikatorisch integriert; formal offen
UZA 0.8	Flow Analyzer, Dynamics Detector, Complexity Monitor, Perspective Tracker, Feedback Loop, EvolutionMemory	in 0.9 integriert
UZA 0.9	ScaleProfile, ContainmentGraph, SUBZ/PROJ, LossProfile, I1–I11, Transmission und Grenztransformationen	Scale & Interaction 0.9-RC
UZA 0.9.1	EMG/FDB-Bereinigung, Minimality-Korrektur, V10-Begründung und Migration, Machtverschiebung, Scheiter- und Patch-Abbruchregeln	Architecture & Engine 0.9.1-RC
Formal Theory 0.9.1	Referenzsemantik für V10, gerichtete Resonanz, Relevanz, Describe, Emergenz und Step; A1–A22 mit geschlossener A9-Evidenzintegrität; minimale und reichhaltige Modellzeugen; konkrete T1–T6-Semantik; nicht-vakuose T5-/T6-Ausführungszeugen	Isabelle2025-2-Fresh-Profile-Build mit elf Theorien bestanden; T1–T6 kernelgeprüft
Conformance Harness 0.1	deterministisches Replay, Golden Run A, Golden Run B und negative Gate-Tests	lokal ausgeführt und bestanden

2.1 Aufgelöste Statuswidersprüche

Referenzimplementierung. „Vollständiger Demonstrator-Durchlauf“ bedeutet, dass Objekte, Operationen, Commits und Ergebnisse vollständig spezifiziert und nachvollziehbar durchgespielt wurden. Seit 0.9.1 existiert zusätzlich ein kleines ausführbares Conformance-Harness mit reproduzierbaren Golden Runs. Es ersetzt weder eine produktive Engine noch deren Performance-, Sicherheits- und Freigabeproofung.

Dynamik. Die dynamische Schicht ist architektonisch und operational spezifiziert. Domänenspezifische Detektoren und eine HOL-Axiomatik der Dynamik sind weiterhin offen. Deshalb lautet ihr Status: in die Engine-Spezifikation integriert, mathematisch noch nicht geschlossen.

HOL-Theorie. Die Typ- und Funktionsarchitektur ist bereinigt; die zentralen Referenzprädikate und A1–A22 liegen als konkrete Isabelle/HOL-Theorie vor. Ein integrierter Fresh-Profile-Build mit Isabelle2025-2 ist für Haupttheorie, Basismodell, Rich Model sowie T1, T2, T3, T4, T5, T6 und die nicht-vakuen T5-/T6-Zeugen bestanden. Parser, Typsystem und Kernel haben die

ausgewiesenen Sentinels, A9-Lemmata, Locale-Interpretationen und T1–T6-Erhaltungssätze akzeptiert. T6 besitzt eine konstruktive gerichtete Semantik mit getrennten Source-/Target-Referenzen, wohldefiniertem versioniertem Kernprofil, begrenztem Score und Stärke, totaler Formklassifikation, exaktem Zustandsdelta, Katalogfortschreibung, Erhaltung der Ausgangskataloge, *state_well_formed*, Identität, Referenzen, Zeit, Perspektive und Kontext sowie Refinement zu step. Der Zeuge aktiviert zwei validierte Effekte und beweist eine echte Richtungsdivergenz. Nicht belegt sind T7–T13, allgemeine Modell-Expansion, Konservativität, empirische Güte der Schwellen oder Produktionsäquivalenz.

Wirkungsvektor. Die UZA 0.1 verwendete ein polares Zehnerprofil aus Verbindung/Trennung, Stabilisierung/Destabilisierung, Öffnung/Begrenzung, Beschleunigung/Verlangsamung und Kooperation/Konflikt. Die konsolidierte Fassung verwendet zehn eigenständige Gestaltungsdimensionen. 0.9.1 begründet ihre funktionale Auswahl, definiert EffectProfile als normative Schnittstelle, spezifiziert die verlustbehaftete Migration und trennt Machtverschiebung als relationales Zusatzprofil. Das frühere Profil bleibt Legacy-Profil und ist kein zweiter normativer Kern.

3. Viergliedriger Kern mit orthogonalen Erweiterungsbereichen

Der stabile Kern lautet:

$$K = \{ O, S, G, E \}$$

mit:

- **O – Ontologie:** Welche Objekte und Strukturen können vorkommen?
- **S – Semantik:** Was bedeuten sie und unter welchen Bedingungen sind sie gültig?
- **G – Grammatik:** Welche Transformationen sind zulässig?
- **E – Engine:** Wie werden Transformationen reproduzierbar ausgeführt, validiert und protokolliert?

Der Ausdruck „viergliedriger Kern“ bezeichnet exakt diese vier voneinander abhängigen Architekturbereiche. Auf der äußeren Ebene bildet K zusammen mit vier orthogonalen Erweiterungsbereichen eine fünfteilige Gesamtarchitektur. Kernglieder und Erweiterungsbereiche werden nicht als eine einzige lineare Schichtenfolge gezählt.

Die Abhängigkeitsordnung lautet:

$$O \rightarrow S \rightarrow G \rightarrow E$$

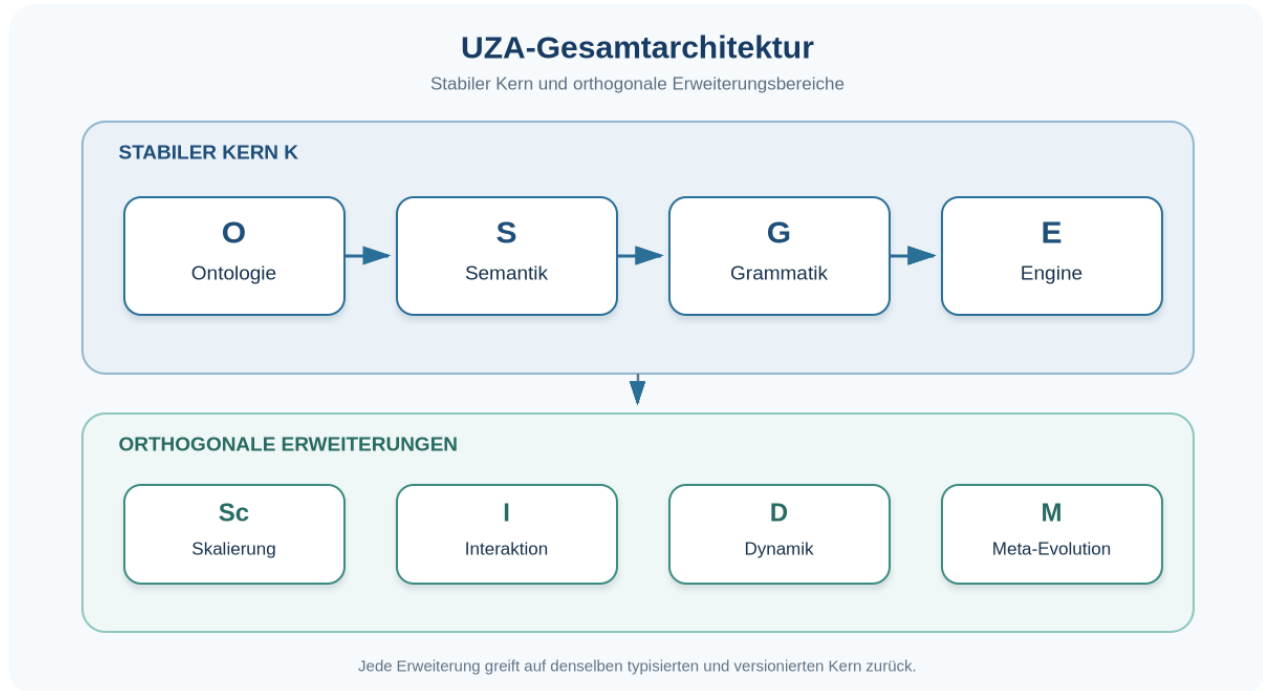
Eine Engineoperation darf keinen Objekttyp erzeugen, den Ontologie und Semantik nicht tragen. Eine Grammatikregel darf keine semantisch unbestimmte Operation legitimieren. Umgekehrt können mehrere Engines dieselbe Grammatik implementieren, sofern sie dieselben Gültigkeits- und Reproduzierbarkeitsbedingungen erfüllen.

Orthogonale Erweiterungsbereiche sind:

$$UZA = \{ K, Sc, I, D, M \}$$

- **Sc** – Skalierung, Einbettung und Projektion,

- *I* – Interaktion mehrerer Zusammenhänge,
- *D* – Verlauf, Rückkopplung und dynamische Muster,
- *M* – kontrollierte Meta-Evolution der Grammatik.



4. Zwölf architektonische Grundaxiome RA1–RA12

Diese zwölf Aussagen sind konsolidierte Architekturaxiome. Sie bilden den semantischen Entwurf aus UZA 0.2 ab. Sie sind von den als Locale-Annahmen formalisierten HOL-Axiomen A1–A22 zu unterscheiden.

RA1 – Zusammenhangsgebundenheit

Jedes UZA-Objekt erhält Bedeutung durch seine typisierten Beziehungen zu Perspektive, Kontext, Zeit, Herkunft und weiteren Objekten. Isolierte Speicherung genügt nicht für semantische Gültigkeit.

RA2 – Relationale Herkunft von Wirkung

Jede validierte Wirkung besitzt eine ausgewiesene relationale oder interaktionale Herkunft. Eine Wirkungsbehauptung ohne Herkunft bleibt Hypothese oder wird verworfen.

RA3 – Perspektivische Beobachtung

Beobachtungen, Relevanzen und Orientierungen werden einer versionierten Perspektive zugeordnet. Perspektivgebundenheit ist eine Transparenzbedingung und keine Beliebigkeitserlaubnis.

RA4 – Kontextbindung

Semantische Gültigkeit gilt relativ zu einem angegebenen Kontext. Kontextwechsel erzeugen eine neue Prüfung und können neue Objektversionen erforderlich machen.

RA5 – Dreifache Zeitbindung

Ereigniszeit, Zusammenhangszeit und Versionszeit werden getrennt geführt. Dadurch bleiben Auftreten, Wirksamkeitsverlauf und Dokumentänderung unterscheidbar.

RA6 – Explizite Unsicherheit

Unsicherheit ist Bestandteil eines Objekts und seiner Begründung. Änderungen von Klasse oder Konfidenz erfolgen über dokumentierte Evidenz oder Revision.

RA7 – Versionierte Wahrheit

Veröffentlichte Zustände und Objektversionen bleiben unverändert. Korrektur erzeugt eine neue Version mit Rückverweis auf ihre Vorgänger.

RA8 – Begründete Relevanz

Relevanz entsteht durch ein ausgewiesenes Relevanzmodell in Bezug auf Objekt, Perspektive, Kontext und Zeit. Sie wird als mehrdimensionales Ergebnis mit Begründung ausgegeben.

RA9 – Hergeleitete Orientierung

Orientierungen werden aus Relevanzen, Resonanzen, Zielen, Einschränkungen, Unsicherheiten und erwarteten Wirkungen hergeleitet. Die Engine darf „unzureichende Grundlage“ ausgeben.

RA10 – Rückwirkungsgebundene Gestaltung

Gestaltung verweist auf eine Orientierung; Ausführung verweist auf die Gestaltung; Beobachtung und Rückwirkung verweisen auf die Ausführung. Damit bleibt der Gestaltungszyklus geschlossen und prüfbar.

RA11 – Grammatikgebundene Emergenz

Emergenz ist keine bloße Überraschungsbezeichnung. Sie wird über die Beschreibbarkeit des Zustands durch Typen, Beziehungen und Übergänge der aktiven Grammatik klassifiziert.

RA12 – Kontrollierte Meta-Evolution

Grammatikänderungen erfolgen über Proposal, Validierung, Freigabe, VersionRelease und Migration. Laufende Fälle bleiben bis zu einer expliziten Migration an ihre aktive Grammatik gebunden.

5. Verbindliche Invariantenfamilien

Die UZA unterscheidet drei Ebenen von Verbindlichkeit:

1. **Theorieinvarianten** gelten für jedes Modell der UZA-Theorie.

2. **Zustandsgültigkeitsbedingungen** bestimmen, ob ein veröffentlichter Zustand akzeptiert wird.
3. **Implementierungskonformität** bestimmt, ob eine Engine die Spezifikation korrekt ausführt.

Zu den übergreifenden Invarianten gehören:

- eindeutige Objekt- und Versionsidentität,
- referenzielle Integrität,
- vollständige Herkunft validierter Wirkungen,
- explizite Perspektiv-, Kontext- und Zeitbindung,
- zulässige Lifecycle-Übergänge,
- Trennung von Simulation und Beobachtung,
- dokumentierte Orientierung vor Auswahl einer Gestaltung,
- Rückführung jeder Zustandsänderung auf einen atomaren Commit,
- vollständige Lineage bei Fission und Disposition bei Fusion,
- grammar-versionstreue Reproduzierbarkeit,
- sichtbare Konflikte statt stiller Auflösung,
- kontrollierte Unsicherheits- und Evidenzänderung.

6. Systemgrenze und Anwendungsprinzip

Die UZA legt fest, wie ein Zusammenhang formal beschrieben, verändert und geprüft wird. Domänenwissen, Messverfahren, Relevanzmodelle und Entscheidungsbefugnisse werden als versionierte Eingaben oder Module angebunden. Dadurch bleibt der Kern domänenübergreifend, während die jeweilige Anwendung ihre eigenen Begriffe, Evidenzstandards und Ziele einbringt.

Eine UZA-Analyse produziert deshalb keine kontextfreie „Gesamtwahrheit“. Sie produziert eine nachvollziehbare, reproduzierbare und revisionsfähige Darstellung dessen, was unter angegebenen Perspektiven, Daten, Regeln und Modellen beschrieben und begründet werden kann.

6.1 Meta-ontologische Geltungsgrenze: POT und reine Potenzialität

Die UZA setzt erst dort ein, wo Unterscheidungen typisierbar, beschreibbar oder als Übergang prüfbar werden. Sie formalisiert bestimmte Möglichkeiten innerhalb einer Grammatik; sie erhebt keinen Anspruch, die Möglichkeit von Bestimmtheit und Grammatik überhaupt als internes Objekt abzubilden.

Dimension	UZA-POT	Reine/schöpferische Potenzialität
Status	formaler, grammatik- und zustandsabhängiger Möglichkeitsraum	meta-ontologischer Grenzbegriff

Dimension	UZA-POT	Reine/schöpferische Potenzialität
Form	typisierte Kandidaten, Bedingungen und erreichbare Zustände	keine Objektart, kein Payload und keine Relation
Zeit	an Zusammenhangs-, Ereignis- und Versionszeit anschließbar	weder Zeitpunkt noch Dauer innerhalb des Modells
Grenze	durch Kontext, Auflösung und Grammatik begrenzt	nicht als Skala oder Modellgrenze repräsentiert
Funktion	liefert prüfbare Alternativen für Operationen	bezeichnet allenfalls die Ermöglichung von Grammatik und Bestimmtheit

MG1 – Nicht-Objektivierbarkeit. Reine oder schöpferische Potenzialität darf nicht als UZAObject, Zustand, Ressource, Kontext, Perspektive oder verborgene Ursache persistiert werden.

MG2 – Keine Gleichsetzung. POT bezeichnet nur den innerhalb einer versionierten Grammatik modellierten Möglichkeitsraum; POT ist nicht mit reiner Potenzialität identisch.

MG3 – Keine Vollständigkeitsüberschreitung. Aus vollständiger Describe-Abdeckung innerhalb einer Grammatik folgt keine ontologische Vollständigkeit des Modells.

MG4 – Meta-Evolution bleibt intern. Ein GrammarPatch erweitert die Beschreibbarkeit bestimmter Phänomene, erfasst aber nicht den Ermöglichungsgrund von Grammatik selbst.

MG5 – Formale Anschlussstelle. Erst bestimmte Erscheinungen, Unterscheidungen, Relationen oder Übergänge dürfen in UZA eingehen; ihre Herkunft muss dann als Evidenz-, Unsicherheits- oder Geltungsfrage ausgewiesen werden.

Diese fünf Regeln sind meta-theoretische Scope-Klauseln. Sie werden nicht als A1–A22-Erweiterung in Isabelle kodiert: Eine Formalisierung des ausdrücklich Nicht-Objektivierbaren würde die gesetzte Grenze performativ verletzen.

Teil II – Ontologie und semantischer Kern

7. Zusammenhang als formale Grundeinheit

Ein Zusammenhang ist eine zeitlich und perspektivisch zugängliche Konfiguration von Objekten, Beziehungen, Wirkungen, Relevanzordnungen, Regeln und Geschichte:

$$Z_t = \{ E_t, R_t, W_t, S_t, Q_t, O_t, A_t, P_t, C_t, G_t, H_t \}$$

Dabei bezeichnen *E* Erscheinungen, *R* Beziehungen, *W* Wirkungen, *S* Resonanzen, *Q* Relevanzen, *O* Orientierungen, *A* Gestaltungen, *P* Perspektiven, *C* Kontexte, *G* die aktive Grammatik und *H* die versionierte Geschichte.

Für die dynamische Betrachtung wird derselbe Zusammenhang in vier komplementären Aspekten gelesen:

$$Z = \{ Structure, Process, Potential, History \}$$

- **Structure** enthält die gegenwärtig publizierten Objekte und ihre Relationen.
- **Process** enthält aktive Übergänge, Rückkopplungen und Übertragungen.
- UZA-Potential bezeichnet die unter einer aktiven, versionierten Grammatik sowie unter Ressourcen- und Kontextbedingungen erreichbaren Folgezustände.
- **History** enthält die geordnete Trajektorie der Zustände, Commits, Modellwechsel und Revisionen.

UZA-Potential wird nicht als gewöhnliches persistiertes Objekt behandelt. Es ist eine zustands- und grammatikabhängige Funktion. Diese formale Größe darf nicht mit reiner oder schöpferischer Potenzialität als meta-ontologischem Grenzbegriff gleichgesetzt werden:

$$POT(Z_t, G_t, C_t, P_t) \subseteq P(State)$$

8. Drei Zeitebenen

Die UZA führt drei getrennte Zeitachsen:

Zeitebene	Frage	Typische Felder
Ereigniszeit	Wann trat ein Ereignis auf oder wurde eine Wirkung beobachtet?	occurred_at, observed_at
Zusammenhangszeit	Über welchen Zeitraum gilt oder wirkt ein Objekt im modellierten Zusammenhang?	valid_from, valid_to, horizon, delay
Versionszeit	Wann wurde eine Objekt- oder Grammatikversion erzeugt, freigegeben oder ersetzt?	created_at, committed_at, released_at

Die Trennung verhindert, dass eine spät dokumentierte Beobachtung fälschlich als spät eingetretenes Ereignis behandelt wird. Ebenso bleibt erkennbar, ob eine Wirkung historisch wirksam war, obwohl ihre aktuelle Objektversion inzwischen revidiert wurde.

9. Primäres Typensystem

9.1 Persistierte Kernobjekte

Code	Typ	Semantische Funktion
ENT	Erscheinung/Entität	unterscheidbare Gestalt oder registrierte Beobachtungseinheit
REL	Beziehung	typisierte Verbindung zwischen Erscheinungen
EFF	Wirkung	gerichtete Veränderung mit Herkunft, Ziel und Wirkungsprofil
RES	Resonanz	qualifizierte Wechselwirkung zwischen Wirkungen
REV	Relevanz	perspektivisch-kontextuelle Bewertung eines Objekts
ORI	Orientierung	begründete Entwicklungs- oder Handlungsoption
ACT	Gestaltung	ausgewählte, freigegebene oder ausgeführte Option
FDB	Rückwirkung	beobachtete Folge und Abweichung nach Gestaltung
EMG	Emergenzfall	klassifizierte Beschreibungs- oder Strukturgrenze
PER	Perspektive	versionierter Beobachtungs-, Wissens- und Wertungsstandpunkt
CTX	Kontext	Bedingungsrahmen eines Falls oder Teilzusammenhangs
META	Metaobjekt	Grammatikpatch, Evolutionsvorschlag oder Reflexionsobjekt
CONFLICT	Konfliktobjekt	explizit erhaltener, noch nicht aufgelöster Widerspruch

9.2 Erweiterungsobjekte

Code	Typ	Semantische Funktion
SYSCTX	Systemzusammenhang	adressierbarer Einzel- oder Teilzusammenhang im Interaktionsnetz

Code	Typ	Semantische Funktion
COUP	Kopplung	versionierte Verbindung zwischen Systemzusammenhängen
CHAN	Kanal	Übertragungsweg innerhalb einer Kopplung
TRANSF	Kanaltransformation	typisierte Abbildung einer Quellwirkung in einen Zielkontext
TEV	TransmissionEvent	lifecycle-gebundene Übertragung
SUBS	Einbettungsrelation	direkte Container-Teil-Beziehung
PROJ	Projektion	ausgewählte Sicht eines Teilzusammenhangs mit LossProfile
BEV	BoundaryEvent	Fusion, Fission, Aggregation, Embedding oder Detachment

Alle persistierten Typen werden in einem gemeinsamen Summentyp `UZAObject` gehalten. Spezialisierte Stores sind zulässige Indizes oder Projektionen, jedoch keine unabhängigen Wahrheitsbestände.

10. Objektidentität, Versionierung und Provenienz

Jedes persistierte Objekt trägt mindestens:

<code>object_id</code>	stabile Identität des Gegenstands
<code>version_id</code>	Identität dieser konkreten Version
<code>revision</code>	monotone Revisionsnummer
<code>payload</code>	typisierter <code>UZAObject</code> -Inhalt
<code>uncertainty</code>	Klasse und Konfidenz
<code>lifecycle_status</code>	gültiger Lifecycle-Zustand
<code>valid_from/to</code>	Zusammenhangszeit
<code>created_at/by</code>	Versionszeit und Urheber/Akteur
<code>previous_version</code>	unmittelbarer Vorgänger
<code>perspective_refs</code>	beteiligte Perspektiven
<code>context_refs</code>	gültige Kontexte
<code>source_refs</code>	Quellen und Eingaben
<code>evidence_refs</code>	stützende oder widersprechende Evidenz
<code>annotations</code>	typisierte Zusatzangaben

Es gilt:

$$objectId(x_i) = objectId(x_j) \Rightarrow sameEntity(x_i, x_j)$$

und:

$$versionId(x_i) \neq versionId(x_j) \Rightarrow x_i \neq x_j$$

Eine neue Version überschreibt keine frühere Bedeutung. Sie setzt die nachvollziehbare Entwicklung desselben Gegenstands fort. Quellen-, Evidenz- und Commitreferenzen bilden gemeinsam die Provenienz.

11. Lifecycle und Unsicherheit

Der allgemeine Objektlebenszyklus lautet:

DRAFT → *HYPOTHESIS* → *SUPPORTED* → *VALIDATED* → *REVISED*

Zusätzliche End- oder Seitenzustände sind *REJECTED* und *ARCHIVED*.

- **DRAFT:** formal angelegt, semantisch noch unvollständig.
- **HYPOTHESIS:** ausdrücklich als Annahme ausgewiesen.
- **SUPPORTED:** durch mindestens eine zulässige Evidenz gestützt.
- **VALIDATED:** erfüllt alle für den Typ geltenden Integritäts- und Evidenzbedingungen.
- **REVISED:** durch eine neuere Version fortgeführt.
- **REJECTED:** nach Prüfung verworfen, historisch erhalten.
- **ARCHIVED:** für aktive Verarbeitung inaktiv, weiterhin referenzierbar.

Unsicherheit besteht aus Klasse und Konfidenz:

$$U(x) = \langle class(x), conf(x) \rangle, conf(x) \in [0, 1]$$

mit *KNOWN*, *PLAUSIBLE*, *HYPOTHETICAL* und *OPEN*. Die Klassen kennzeichnen den epistemischen Status, nicht den persönlichen Grad des Gefallens. Neue Evidenz kann Konfidenz erhöhen, vermindern oder den Lifecycle verändern. Jede Änderung erzeugt eine protokollierte Objektversion oder einen Commit-Eintrag.

12. Perspektivsemantik

Eine Perspektive ist eine dynamische Entität:

$$p_t = \langle O_p, K_p, V_p, I_p, H_p, A_p, M_p, R_p, F_p \rangle_t$$

mit Beobachtungsraum O_p , Wissensstand K_p , Wertungsstruktur V_p , Interessenhypothesen I_p , historischer Erfahrung H_p , Handlungsspielraum A_p , Machtprofil M_p , Verantwortungsprofil R_p und Freiheitsprofil F_p .

Die Beobachtungsfunktion lautet:

$$observe(p_t, Z_t) = Z_t^{(p)}$$

Der Vergleich zweier Perspektiven erzeugt ein Differenzprofil und keine automatische Rangordnung:

$$Dist(p_i, p_j) = \langle d_O, d_K, d_V, d_I, d_H, d_A, d_M, d_R, d_F \rangle$$

Perspektiven werden versioniert, wenn sich Wissen, Wertungen, Interessen, Zugänge, Macht, Verantwortung oder Handlungsspielräume ändern. Aussagen über „die Perspektive einer Gruppe“ benötigen entweder eine ausgewiesene Aggregationsregel oder mehrere Einzelperspektiven; die Gruppenzuschreibung darf nicht stillschweigend homogenisiert werden.

13. Beziehungen und Wirkungen

13.1 Beziehung

Eine Beziehung ist eine typisierte, zeit- und kontextgebundene Verbindung:

$$REL: E \times E \times RelationType \times C \times T \rightarrow R$$

Pflichtfelder sind source, target, relation_type, directed, context, validity, uncertainty und provenance. Beziehungen können unter anderem verbinden, bedingen, verstärken, hemmen, widersprechen, erzeugen oder transformieren.

13.2 Wirkungshypothese

Eine Wirkungshypothese wird aus einer Beziehung, einer Interaktion oder einer bereits dokumentierten Wirkung abgeleitet:

$$EFF_H = \{cause, target?, vector?, p, c, t, u, evidence^*\}$$

Noch unbekannte Ziele oder Wirkungsdimensionen sind zulässig, solange die Offenheit explizit typisiert ist.

13.3 Validierte Wirkung

Eine validierte Wirkung besitzt vollständige Herkunft, Ziel, Profil, Perspektive, Kontext, Zeit und zulässige Evidenz:

$$EFF_V = \{cause, target, vector, p, c, t, u, evidence^+\}$$

Für Einzelzusammenhänge gilt als Grundform:

$$EFF: REL \times P \times C \times T \rightarrow W$$

Bei Interaktionen kann die Ursache zusätzlich ein TransmissionEvent oder BoundaryEvent sein. Diese Erweiterung wird über die Lineage referenziert und hebt die Herkunftspflicht nicht auf.

14. Konsolidierter Wirkungsvektor

14.1 Normative EffectProfile-Schnittstelle und V10-Referenzprofil

Die universelle Semantik bindet Wirkungsbewertung nicht an eine stillschweigend veränderbare Dimensionsliste. Normativ ist zunächst die Schnittstelle EffectProfile. Jedes Profil muss mindestens ausweisen:

- stabile Profil- und Versionsidentität,
- benannte Dimensionen mit eindeutiger Semantik und Wertebereich,
- Bewertungs-, Nullwert- und Unsicherheitsregeln,
- Perspektiv-, Kontext- und Zeitbindung,
- Regeln für Aggregation, Abhängigkeiten und fehlende Werte,
- Migrations- und LossProfile für Vorgänger- oder Fremdprofile.

V10 ist das verbindliche Referenzprofil der UZA 0.9.1. Eine Engine mit dem Konformitätsanspruch UZA-0.9.1-V10 muss es unverändert unterstützen. Domänenspezifische Zusatzprofile sind zulässig, sofern sie separat versioniert werden und V10 weder überschreiben noch stillschweigend erweitern.

Der konsolidierte Wirkungsvektor bewertet zehn eigenständige, aber nicht als mathematisch orthogonal behauptete Gestaltungsdimensionen:

$$V_{10}(w) = (v_{con}, v_{sta}, v_{ope}, v_{lea}, v_{coo}, v_{act}, v_{tra}, v_{fre}, v_{res}, v_{reg})$$

mit $v_i \in \{-3, -2, -1, 0, 1, 2, 3\}$.

Dimension	Leitfrage für die Bewertung
Verbindung	Wie verändert die Wirkung tragfähige Beziehungen und Anschlussfähigkeit?
Stabilität	Wie verändert sie Verlässlichkeit, Beständigkeit und Belastbarkeit?
Offenheit	Wie verändert sie erreichbare Optionen und Zugänglichkeit?
Lernfähigkeit	Wie verändert sie Wahrnehmung, Rückmeldung, Revision und Erkenntnisgewinn?
Kooperation	Wie verändert sie die Fähigkeit zu wechselseitiger Abstimmung und gemeinsamem Handeln?
Handlungsfähigkeit	Wie verändert sie die realen Möglichkeiten wirksamer Gestaltung?
Transparenz	Wie verändert sie Nachvollziehbarkeit, Sichtbarkeit und Erklärbarkeit?
Freiheit	Wie verändert sie echte Wahl- und Gestaltungsspielräume?
Verantwortung	Wie verändert sie Zurechenbarkeit, Folgenbezug und verlässliche Übernahme?
Regeneration	Wie verändert sie Erneuerungs-, Erholungs- und Wiederherstellungsfähigkeit?

Die Werte sind perspektivisch und kontextuell. +3 bedeutet eine stark fördernde Wirkung auf die betreffende Dimension, -3 eine stark mindernde Wirkung, 0 eine neutrale, ausgeglichene oder nicht feststellbare Wirkung. Der Grund für 0 muss über zero_reason zwischen neutral, kompensiert und unbekannt unterschieden werden.

14.2 Begründung, Abdeckung und Abhängigkeiten

Die Auswahl folgt vier funktionalen Abdeckungsgruppen:

1. **relationale Tragfähigkeit:** Verbindung und Kooperation,
2. **strukturelle Fortdauer:** Stabilität und Regeneration,
3. **Möglichkeits- und Entwicklungsraum:** Offenheit, Lernfähigkeit, Handlungsfähigkeit und Freiheit,
4. **epistemische und verantwortliche Nachvollziehbarkeit:** Transparenz und Verantwortung.

Damit werden Beziehung, Beständigkeit, Entwicklung, wirksame Gestaltung, Nachvollziehbarkeit und Folgenübernahme jeweils explizit sichtbar. Diese Begründung ist eine Architekturentscheidung und kein Vollständigkeits- oder Unabhängigkeitsbeweis. Überschneidungen sind erwartbar: Offenheit kann Freiheit fördern, Transparenz kann Verantwortung stärken, Lernfähigkeit kann Handlungsfähigkeit verändern. Deshalb darf eine Engine die zehn Werte nicht zu unabhängigen Messgrößen umdeuten. Sie muss relevante Abhängigkeiten in einem

DimensionDependencyReport ausweisen und darf einen Gesamtscore nur erzeugen, wenn ein versioniertes Aggregationsmodell ihn ausdrücklich definiert.

14.3 Legacy-Profil V10-POLAR und Migration

Das frühere Profil mit Verbindung, Trennung, Stabilisierung, Destabilisierung, Öffnung, Begrenzung, Beschleunigung, Verlangsamung, Kooperation und Konflikt bleibt für historische Fälle verfügbar. Es wird als eigenes EffectProfile versioniert. Eine automatische Eins-zu-eins-Übersetzung in V10 ist unzulässig, weil polare Prozessbeschreibungen und eigenständige Gestaltungsqualitäten verschiedene Bedeutungen tragen. Ein Adapter muss seine Abbildungsregeln und Verluste ausweisen.

14.3.1 Migrationsdispositionen

Jede Altdimension erhält je Zielfall genau eine Disposition: Mapped, Derived, Unmapped oder Conflicted. Die Referenzmigration lautet:

V10-POLAR	mögliche V10-Zuordnung	Referenzdisposition
Verbindung	Verbindung	Mapped; Vorzeichen und Begründung bleiben erhalten
Trennung	Verbindung mit umgekehrter Richtung	Derived; semantischer Verlust ist auszuweisen
Stabilisierung	Stabilität	Mapped
Destabilisierung	Stabilität mit umgekehrter Richtung	Derived
Öffnung	Offenheit	Mapped
Begrenzung	Offenheit mit umgekehrter Richtung	Derived
Beschleunigung	keine feste V10-Dimension	Unmapped; kontextuell unter anderem Handlungsfähigkeit oder Regeneration prüfbar
Verlangsamung	keine feste V10-Dimension	Unmapped; kontextuelle Neubewertung erforderlich
Kooperation	Kooperation	Mapped
Konflikt	Kooperation, Verbindung oder Resonanzspannung	Conflicted; menschlich oder modellgebunden aufzulösen

14.3.2 Normative Migrationsregeln

1. Das ursprüngliche V10-POLAR-Objekt und sein Version Bundle bleiben unverändert reproduzierbar.
2. Eine Migration erzeugt neue V10-Objekte mit Verweis auf Quelle, Adapterversion und Bearbeitungsentscheidung.
3. Für jede Zieldimension werden Disposition, Konfidenz und semantischer Verlust dokumentiert.
4. Unmapped und Conflicted dürfen nicht als Nullwert ausgegeben werden; sie bleiben offene Migrationslücken.

5. Ein migrierter Fall darf erst als V10-konform gelten, wenn alle für seine Orientierungs- oder Gestaltungsentscheidung relevanten Dimensionen bewertet oder ausdrücklich als nicht entscheidungsrelevant begründet sind.
6. Golden-Run- und Replay-Tests müssen sowohl den historischen Lauf als auch die neue Migration reproduzieren.

14.4 Macht als relationales Zusatzprofil

Macht ist keine elfte V10-Gestaltungsqualität, sondern eine relationale Verteilung von Möglichkeiten und Folgen. Sie wird deshalb in einem getrennten PowerProfile erfasst:

$$Power(p, t) = \{resource, access, decision, risk, correction\}$$

mit Ressourcenmacht, Zugangskontrolle, Entscheidungsbefugnis, Risikotragung und Korrekturfähigkeit. Eine Machtverschiebung lautet:

$$\Delta Power_p(t_0, t_1) = Power(p, t_1) - Power(p, t_0)$$

Ein PowerShiftReport bindet diese Differenz an Perspektive, Kontext, Zeit, verursachende Beziehungen und Wirkungen. Er zeigt zusätzlich, welche V10-Dimensionen betroffen sind, etwa Freiheit, Handlungsfähigkeit, Transparenz oder Verantwortung. PowerProfile und V10 werden gemeinsam gelesen, aber weder miteinander verrechnet noch durch Mittelung homogenisiert.

15. Resonanzsemantik

Resonanz qualifiziert das geordnete Verhältnis zweier Wirkungen relativ zu Perspektive, Kontext, Modellversion und Zeithorizont. Die normative T6-Signatur lautet $EFF \times EFF \times Perspective \times Context \times ResonanceModel \times TimeHorizon \rightarrow RES$. Ein ResonanceModel enthält mindestens eine gerichtete V10-Interaktionsmatrix und geordnete Klassifikationsschwellen.

$$RES: W \times W \times P \times C \rightarrow R$$

mit:

- R^+ – wechselseitige Verstärkung,
- $R^\sim$ – funktionale Ergänzung,
- $R^\sim$ – produktive Spannung,
- R^- – hemmende Spannung,
- R^0 – keine belastbare Resonanzbestimmung.

Resonanz ist gerichtet: $RES(A, B, p, c, m, h)$ darf von $RES(B, A, p, c, m, h)$ abweichen. T6 berechnet den normalisierten Score aus $K(source_dimension, target_dimension)$, klassifiziert ihn total in Amplification, Complement, ProductiveTension, Inhibition oder Neutral und dämpft die Stärke nachvollziehbar durch die deklarierte Unsicherheit. Jede Klassifikation persistiert Quell- und Zieleffekt, Profilmodell, Form, Score, Stärke, Zeithorizont, Perspektive, Kontext, Begründungsreferenzen, Begründungstext und Unsicherheit.

Abgrenzung der Referenzsemantiken: Die ältere symmetrische Funktion `classify_resonance(a, b)` auf Basis des V10-Skalarprodukts bleibt als einfache Alignment-Baseline und Legacy-Referenz

erhalten. Die normative operationale T6-Semantik verwendet $\text{directional_resonance_score}(\text{profile}, \text{source}, \text{target})$. Zwischen beiden wird keine stille Gleichheit behauptet. Ein symmetrischer T6-Fall muss durch ein entsprechend symmetrisches Kernelfprofil explizit modelliert werden. Die Kernelprüfung belegt Rechen- und Erhaltungseigenschaften der Spezifikation, nicht die empirische Richtigkeit eines konkreten domänenspezifischen Kernels oder seiner Schwellen.

16. Relevanzsemantik

Relevanz ist eine versionierte Funktion eines Modells M :

$$REV_M: X \times P \times C \times T \rightarrow Q$$

mit X als Menge zulässiger UZA-Zielobjekte. Das Ergebnis ist ein Vektor:

$$REV_M(x, p, c, t) = \langle q, u, \sigma, \rho, \delta, e \rangle$$

- q – Relevanzintensität,
- u – epistemische Unsicherheit,
- σ – Stabilität der Bewertung gegenüber kleinen Eingabeänderungen,
- ρ – Reichweite über betroffene Objekte oder Zusammenhänge,
- δ – erwartete zeitliche Dauer,
- e – maschinen- und menschenlesbare Begründung.

Ein Relevanzwert ohne Modellversion, Perspektive, Kontext, Zeit und Begründung ist nicht veröffentlichungsfähig. Mehrere Relevanzmodelle können parallel bestehen. Ihre Ergebnisse werden als verschiedene Relevanzordnungen ausgewiesen und nicht still zu einem Universalwert gemittelt.

17. Orientierung, Gestaltung und Rückwirkung

17.1 Orientierungsraum

$$ORI: Q^* \times RES^* \times Goal \times Constraints \times P \times C \rightarrow O$$

Eine Orientierungsoption enthält Ziel, unterstützende und entgegenstehende Relevanzen, erwartete Wirkungen, betroffene Perspektiven, Ressourcen, Einschränkungen, Unsicherheit, Reversibilität, Zeithorizont und Rückmeldesignale.

Die Synthese kann die Status AVAILABLE, CONTESTED oder INSUFFICIENT_BASIS ausgeben. Eine künstliche Mehrzahl von Optionen ist nicht erforderlich.

17.2 Gestaltung

$$ACT: ORI \times P \times C \times T \rightarrow A$$

Eine Gestaltung verweist auf die ausgewählte Orientierung, Entscheidungsbefugnis, Zustimmungslage, Ressourcenfreigabe, Ausführungsparameter und geplante Beobachtung.

17.3 Simulation und Beobachtung

$$SIM(A, Z_t, M, \Theta) \rightarrow \hat{Z}_{t+1}$$

$$OBS(A, Z_t) \rightarrow Z_{t+1}$$

Simulation und Beobachtung besitzen getrennte Objekttypen und Provenienzketten. Ihre Abweichung lautet:

$$\Delta_{pred} = Dist(\hat{Z}_{t+1}, Z_{t+1})$$

17.4 Rückwirkung

$$FDB: A \times Z_t \times Z_{t+1} \rightarrow Feedback$$

Rückwirkung enthält beobachtete Folgen, Prognoseabweichungen, neu entstandene Beziehungen und Wirkungen, Perspektivänderungen, Relevanzverschiebungen sowie Revisionsvorschläge.

18. Beschreibbarkeit und Emergenz

Die logische Beschreibbarkeit ist zerlegt:

Mit $D_T = TypeComplete(G, Z)$, $D_R = RelationComplete(G, Z)$ und $D_X = TransitionComplete(G, Z)$ gilt:

$$Describeable(G, Z) \Leftrightarrow D_T \wedge D_R \wedge D_X$$

Describe erzeugt daraus einen Report mit Status, drei Abdeckungswerten, typisierten Lücken, Ableitungsspur und Grammatikversion. Die Reporting-Funktion trägt nicht selbst die logische Last.

Emergenz wird in vier Klassen bestimmt:

Klasse	Bedingung	Bedeutung
E0	vollständig beschreibbar, keine strukturelle Neuheit	bekannte Struktur und Grammatik
E1	vollständig beschreibbar, strukturelle Neuheit	neue Konfiguration innerhalb der aktiven Grammatik
E2	unvollständig; ein zulässiger GrammarPatch kann Vollständigkeit herstellen	grammatische Emergenz
E3	unvollständig; innerhalb der Meta-Regeln ist kein ausreichender Patch bildbar	meta-grammatische Grenze

E1 verlangt eine operationalisierte Neuheitsprüfung. E2 verlangt einen konkreten Patchkandidaten. E3 ist ein strenger Grenzstatus und darf nicht allein aus fehlenden Daten abgeleitet werden.

Teil III – Zustandsmaschine und formale Grammatik

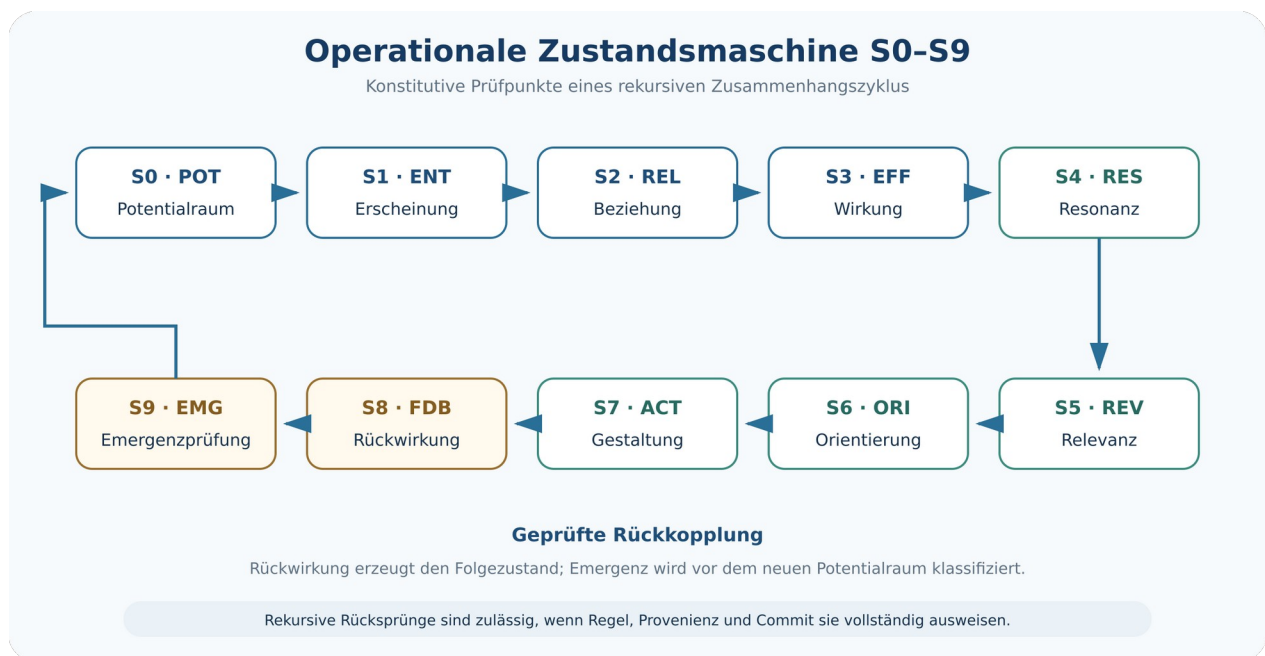
19. Konsolidierte Zustandsmaschine

Die Entwicklung der UZA verwendet drei Darstellungen desselben Grundprozesses:

- Der frühe **Acht-Ebenen-Zyklus** beschreibt Erscheinung, Beziehung, Wirkung, Resonanz, Relevanz, Orientierung, Gestaltung und Rückwirkung.
- Der **semantische Kernzyklus** ergänzt den Potentialraum als Ausgangs- und Rückkehrraum.
- Die **operationale Zustandsmaschine 0.9.1** führt Rückwirkung und anschließende Emergenzanalyse als getrennte Prüfschritte und umfasst S0–S9.

Die normative 0.9.1-Maschine lautet:

$$S0POT \rightarrow S1ENT \rightarrow S2REL \rightarrow S3EFF \rightarrow S4RES \rightarrow S5REV$$

$$S5REV \rightarrow S6ORI \rightarrow S7ACT \rightarrow S8FDB \rightarrow S9EMG \rightarrow S0POT$$


Die Stufen sind konstitutive Prüfpunkte und keine starre lineare Benutzeroberfläche. Rekursive Rücksprünge sind zulässig, sofern eine Regel sie erlaubt und der Commit ihre Herkunft dokumentiert. T11 erzeugt aus Ausführung und Beobachtung zunächst einen rückwirkungsgebundenen Folgezustand. Erst dieser Zustand liefert zusammen mit seinem Vorgänger die vollständige Eingabe für T12. Die Emergenzklassifikation erfolgt deshalb nach FDB und vor Revision, GrammarPatch-Entscheidung und Aktualisierung des Potentialraums. Damit stimmen Zustandsmaschine, T11/T12, Engine-Hauptzyklus, Pseudocode und Demonstrator-Commits überein.

20. Drei Wandeltypen

- **W1 – Variation:** Werte oder Instanzen ändern sich innerhalb bestehender Typen, Beziehungen und Regeln.
- **W2 – Transformation:** Struktur oder Prozess ändert sich durch bereits zulässige Regeln in substantieller Weise.
- **W3 – Emergenz:** Die Veränderung erzeugt strukturelle Neuheit oder eine Grenze der aktiven Grammatik gemäß E1–E3.

Wandeltyp und Emergenzklasse sind verwandt, aber nicht identisch. W3 bezeichnet die Art des Wandels; E0–E3 bezeichnen den Grad der grammatischen Beschreibbarkeit.

21. Transformationsgrammatik T1–T13

T1 – Erscheinung registrieren

Observation × Perspective × Context × Time → ENT

Voraussetzungen: typisierte Beobachtung, aktive Perspektive, aktiver Kontext. Ergebnis: versionierte Erscheinung mit Provenienz und Unsicherheit.

T2 – Potential differenzieren

POT × Perspective × Context → ENT*

Erzeugt explizite Möglichkeitskandidaten. Kandidaten bleiben von beobachteten Erscheinungen unterscheidbar.

Die Isabelle-Referenzsemantik 0.9.1 konkretisiert diese Unterscheidung ohne neuen Kerntyp: Eine durch T1 registrierte Beobachtung trägt keine Potentialquelle, keine Potentialbasis und keinen Analysemarker. Ein T2-Kandidat ist dagegen eine K_ENT-Hypothese mit auflösbarer source_ref, nichtleerer, vollständig referenzierter basis_refs-Menge und Status Applicable. T2 nimmt eine endliche, nichtleere Kandidatenmenge entgegen; Objekt- und Versionsidentitäten müssen innerhalb der Menge eindeutig und gegenüber dem Ausgangszustand frisch sein. Diese Kodierung ist eine operationale Provenienzkonvention, keine Behauptung über die Vollständigkeit des erzeugten Potentialraums.

T3 – Beziehung bilden

ENT × ENT × RelationType × Context × Time → REL

Prüft Endpunkte, Richtung, Typ, Kontext und zeitliche Gültigkeit.

Die Formalrevision T3 konkretisiert diesen Übergang in der integrierten Isabelle-Theorie. Der Ausgangszustand muss state_well_formed sein; CreateRelation und K_REL müssen von der aktiven Grammatik getragen werden. Das neue REL-Objekt erhält frische Objekt- und Versionsidentitäten, zwei verschiedene aktive K_ENT-Endpunkte, vollständige Referenzen, gültige Evidenz- und Zeitstruktur sowie einen vorhandenen Kontext. Relationstyp und Richtung werden in einem typisierten, über die Relations-Version adressierten Relationenkatalog fortgeschrieben. Die

Zielwohlgeformtheit wird aus den Vorbedingungen und der exakten Objektinsertion hergeleitet und nicht vorausgesetzt. Der Isabelle2025-2-Clean-Build einschließlich T4 endete mit Exit-Code 0.

T4 – Wirkungshypothese erzeugen

$REL \times Perspective \times Context \times Time \rightarrow EFF_H$

Erzeugt eine ausdrückliche Hypothese mit offener oder bestimmter Zielstruktur.

Die Formalrevision T4 macht diese Offenheit typisiert und auditierbar. `effect_hypothesis_payload` bindet jede Hypothese an die verursachende Relationsversion, eine Perspektive, einen Kontext und ein Wirkungsprofilmodell; Ziel und V10-Vektor sind jeweils echte option-Werte und werden weder durch Sentinel-IDs noch durch stilles Nullfüllen ersetzt. `effect_input_valid` verlangt eine aktive typisierte `K_REL`-Ursache, begrenzte Unsicherheit, frische Objekt- und Versionsidentitäten, Lifecycle Hypothesis, vollständige Referenzen sowie vorhandene Perspektive, Kontext und Modellversion. Ein bestimmtes Ziel muss aktiv sein; ein offenes Ziel bleibt `None`. Die Zielwohlgeformtheit folgt aus den Vorbedingungen und der exakten Einfügung. Der Isabelle2025-2-Clean-Build mit sieben Theorien endete mit Exit-Code 0. Dieser konditionale Erhaltungssatz belegt weder empirische Richtigkeit noch Vollständigkeit jeder domänenspezifischen Wirkungshypothese.

T5 – Evidenz integrieren

$EFF_H \times Evidence \rightarrow EFF_H \mid EFF_SUPPORTED \mid EFF_V \mid REJECTED$

Die kernelgeprüfte T5-Referenzsemantik aktualisiert kein veröffentlichtes Objekt in-place. Sie konstruiert eine frische Effektversion mit derselben `object_id`, verweist über `previous_version` auf den unveränderten Vorgänger und verschiebt Vorgängerobjekt und Vorgänger-Payload in getrennte Archive. Neue Evidenzquellen müssen aktiv, auflösbar, nicht `Rejected/Archived` und einem anderen logischen Objekt zugeordnet sein. Stützende Evidenz senkt oder erhält Unsicherheit; widersprechende Evidenz erhöht oder erhält sie und führt zu `Rejected`; differenzierende Evidenz muss Ziel oder Wirkungsvektor tatsächlich ändern. Der Betrag jeder Unsicherheitsänderung ist durch die Evidenzstärke begrenzt. Hypothesis darf durch T5 nicht direkt zu `Validated` springen; `Validated` ist nur aus `Supported` bei erfüllter Unsicherheits- und Evidenzschwelle erreichbar. Bereits validierte Effekte werden nicht durch T5 revidiert, sondern fallen in T13 `ReviseObject`. Ein nicht-vakuoser Zeuge führt Hypothesis mit unabhängiger Evidenz tatsächlich zu `Supported`.

T6 – Resonanz klassifizieren

$EFF \times EFF \times Perspective \times Context \rightarrow RES$

Die kernelgeprüfte T6-Referenzsemantik verlangt zwei verschiedene aktive `K_EFF`-Versionen mit auflösbaren typisierten V10-Payloads im selben Kontext, ein vorhandenes Perspektiv- und Modellprofil, begrenzte Unsicherheit, einen positiven Zeithorizont sowie aktive unabhängige Begründungsreferenzen. Sie konstruiert genau ein frisches `K_RES`-Objekt und einen versionierten Resonanz-Payload. Die Zielwohlgeformtheit und Kataloggültigkeit werden aus den Vorbedingungen hergeleitet, nicht vorausgesetzt. Die V10-Summutationsdomäne wird durch

effect_dimension_UNIV exakt als die zehn Dimensionen bewiesen und ist endlich. Der nicht-vakuose Zeuge verwendet $K(\text{Connection}, \text{Stability})=1$ bei sonst 0, erhält $A \rightarrow B=1$ und Amplification, aber $B \rightarrow A=0$ und Neutral; die unsicherheitsbereinigte Stärke beträgt 0,9 und der Ergebnis-Lifecycle ist Validated. Diese Konfiguration beweist gerichtete Ausführbarkeit, nicht die universelle Gültigkeit genau dieses Kernels.

T7 – Relevanz bewerten

$\text{UZAObject} \times \text{Perspective} \times \text{Context} \times \text{Time} \times \text{RelevanceModel} \rightarrow \text{REV}$

Bindet Ergebnis und Begründung an die konkrete Modellversion.

T8 – Orientierung synthetisieren

$\text{REV}^* \times \text{RES}^* \times \text{Goal} \times \text{Constraints} \times \text{Perspective} \times \text{Context} \rightarrow \text{ORI}^*$

Kann mit INSUFFICIENT_BASIS enden.

T9 – Gestaltung auswählen

$\text{ORI} \times \text{Perspective} \times \text{Authority} \times \text{Context} \times \text{Time} \rightarrow \text{ACT_SELECTED}$

Dokumentiert Auswahlgrund, Befugnis, Zustimmung und verworfene Alternativen.

T10 – Gestaltung ausführen

$\text{ACT_SELECTED} \times \text{State} \rightarrow \text{ExecutionRecord} \times \text{State}$

Ausführung ist atomar oder erzeugt einen dokumentierten Teil-/Fehlerstatus.

T11 – Rückwirkung erfassen

$\text{ExecutionRecord} \times \text{Observation}^* \rightarrow \text{FDB} \times \text{State}$

Verknüpft reale Beobachtungen mit der ausgeführten Gestaltung.

T12 – Emergenz analysieren

$\text{State}_t \times \text{State}_{t+1} \times \text{Grammar} \rightarrow \text{EmergenceResult}$

Prüft Neuheit, Beschreibbarkeit, Lücken und mögliche Patches.

T13 – Revidieren

$\text{Deviation} \times \text{Evidence} \times \text{State} \rightarrow \text{State}'$

Erzeugt neue Objektversionen, Relevanzanpassungen, Orientierungsrevisionen oder einen GrammarPatch-Vorschlag.

22. Regelkalkül und Ableitungsprinzipien

Der natürliche Schließungskalkül verwendet typisierte Urteile der Form:

$$\Gamma; G; Z \vdash o p : Z'$$

Dabei enthält Γ Perspektiven, Kontexte, Evidenzen, Modelle und Berechtigungen. G ist die aktive Grammatik, Z der Eingangszustand, op die Operation und Z' ein möglicher Ausgangszustand.

Ein gültiger Ableitungsschritt benötigt:

1. Typkorrektheit aller Eingaben,
2. erfüllte Vorbedingungen der Regel,
3. zulässigen Lifecycle-Übergang,
4. vollständige Provenienz für neu publizierte Objekte,
5. lokale, transaktionale und globale Validierung,
6. atomaren Commit oder vollständigen Rollback.

Konstitutive Stufen dürfen nicht unbegründet übersprungen werden. Eine direkte Operation $ENT \rightarrow ACT$ ist nur dann zulässig, wenn eine zusammengesetzte Makroregel sämtliche Zwischenableitungen erzeugt und im Proof Trace ausweist.

23. Grammatikklassen G0–G4

Klasse	Inhalt	Zulässige Veränderung
G0	primitive Typen, Identitäten und Zeit	nur durch formale Hauptversion
G1	Objektbildungs- und Referenzregeln	validierter kompatibler Patch
G2	Transformationsregeln T1–T13 und Interaktionsregeln	versionierter Regelpatch
G3	domänenspezifische Profile, Modelle und Detektoren	modulare Versionierung
G4	Meta-Regeln für Patchbildung, Freigabe und Migration	besonders kontrollierte Meta-Evolution

Die Klassen verhindern, dass eine Änderung eines Relevanzmodells wie eine Änderung der ontologischen Grundtypen behandelt wird. Je tiefer eine Änderung in G0/G1 eingreift, desto stärker sind Migrations-, Kompatibilitäts- und Beweisanforderungen.

24. Konsistenzbedingungen C1–C10

Code	Bedingung	Prüfkern
C1	Referenzintegrität	jede aktive Referenz zeigt auf ein typkompatibles Objekt oder eine zulässige historische Version
C2	Wirkungskontinuität	jede validierte Wirkung besitzt eine zulässige Herkunft
C3	Perspektivnachvollziehbarkeit	perspektivische Aussagen nennen Perspektive und Version
C4	Kontextbindung	Gültigkeit und Übertragung nennen Zielkontext

Code	Bedingung	Prüfkern
C5	Zeitbindung	Ereignis-, Gültigkeits- und Versionszeit bleiben unterscheidbar
C6	Orientierungsbegründung	jede Orientierung besitzt Relevanz-, Resonanz- und Zielbasis
C7	Gestaltungsherkunft	jede Gestaltung verweist auf Orientierung und Entscheidungsbefugnis
C8	Rückwirkungsnachweis	jede publizierte Rückwirkung verweist auf Ausführung und Beobachtung
C9	Versionskonsistenz	ein Commit verwendet kompatible Grammatik-, Semantik-, Modell- und Engineversionen
C10	historische Rückführbarkeit	jeder Zustand lässt sich über Commits bis zum initialen Zustand zurückführen

25. Meta-Evolution

Der Meta-Evolutionsprozess lautet:

Proposal → Validation → Approval → VersionRelease → Migration

Ein GrammarPatch enthält Auslöser, typisierte Lücken, betroffene Regeln und Typen, vorgeschlagene Erweiterung, Minimalitätsmetrik, Kompatibilitätsanalyse, Migrationsplan, Testfälle, Evidenz, Alternativen und Freigabestatus.

Die minimale Erweiterung wird als Optimierungsziel formuliert:

$$\Delta G_{min} = \underset{\Delta G}{\operatorname{argmin}} \{ \operatorname{Cost}(\Delta G) \mid \operatorname{Describeable}(G \oplus \Delta G, Z) \}$$

Cost ist selbst versioniert und kann strukturelle Größe, semantische Reichweite, Migrationsaufwand, Komplexität und Kompatibilitätsverlust gewichten. Ein Minimalitätsresultat ist ohne ausgewiesene Metrik nicht vollständig.

EvolutionMemory speichert Vorschläge, Annahmen, Ablehnungen, Alternativen, Impact Reports, Evidenzen, Quellfälle, Quellcommits und spätere Bewährung. Dadurch lernt die Architektur aus ihrer eigenen Entwicklung, ohne Regeln stillschweigend umzuschreiben.

25.1 Geltungs-, Scheiter- und Widerlegungsordnung

Die architektonischen RA1–RA12 sind Konformitätsbedingungen. Ein Fall falsifiziert sie nicht wie eine empirische Hypothese; er kann sie erfüllen, verletzen oder außerhalb ihres erklärten Geltungsbereichs liegen. Empirisch prüfbar und widerlegbar sind dagegen konkrete Wirkungs-, Resonanz-, Relevanz-, Dynamik- und Prognosehypothesen einer Modellinstanz.

Die Engine muss folgende terminalen oder revisionspflichtigen Status unterscheiden:

Status	Bedeutung	zulässige Folge
INSUFFICIENT_B ASIS	Evidenz oder Begründungsbasis reicht für die angeforderte Operation nicht aus	Daten ergänzen, Ziel begrenzen oder Operation beenden
MODEL_FALSIFIED	eine vorab dokumentierte Modell- oder Prognoseaussage widerspricht den zugelassenen Beobachtungs- und Fehlerschwellen	Modell revidieren oder verwerfen; ursprüngliches Ergebnis bleibt erhalten
OUT_OF_SCOPE	der Fall lässt sich innerhalb des erklärten Anwendungs- und Auflösungsbereichs nicht verantwortbar modellieren	Geltungsbereich begrenzen oder eigenständiges Theorieprojekt eröffnen
PATCH_REJECTED	ein GrammarPatch verletzt mindestens ein Annahmekriterium	Patch archivieren; aktive Grammatik bleibt unverändert
CORE_CONFLICT	eine erforderliche Änderung widerspricht G0/G1 oder RA1–RA12	kein GrammarPatch; gesonderter CoreRevisionProposal mit neuer Hauptversion
UNRESOLVED_E 3	der Zustand bleibt innerhalb der freigegebenen Meta-Regeln unbeschreibbar	als Grenze erhalten; keine automatische Erweiterung

Eine Abweichung erzeugt niemals automatisch einen Patch. Der ursprüngliche Prognose-, Validierungs- oder Modellfehler bleibt im Commit Ledger sichtbar und darf durch eine spätere Grammatikversion nicht rückwirkend in einen Erfolg umgedeutet werden.

25.2 Patch-Annahme und Abbruchregeln

Ein Patch darf nur freigegeben werden, wenn alle folgenden Bedingungen erfüllt sind:

1. Die auslösende Lücke ist typisiert, reproduzierbar und von bloß fehlenden Daten unterschieden.
2. Eine einfachere Revision von Objekt, Modell, Parameter, Perspektive oder Kontext reicht nicht aus.
3. Der Patch verändert G0/G1 und RA1–RA12 nicht; andernfalls liegt CORE_CONFLICT vor.
4. Kosten, Reichweite und Kompatibilitätsverlust bleiben innerhalb vorab festgelegter Budgets.
5. Positive Fälle, Gegenbeispiele und mindestens ein nicht zur Patchbildung verwendeter Holdout-Fall sind dokumentiert.
6. Der Patch schließt die Ziellücke, ohne zuvor beschreibbare Kontrollfälle zu beschädigen.
7. Migration, Rückrollplan, Versionierung und historisches Replay sind vollständig.
8. Offene Konflikte, Unmapped-Dispositionen und semantische Verschiebungen werden sichtbar ausgewiesen.

Der Patch wird als PATCH_REJECTED beendet, sobald eine Pflichtbedingung scheitert. Wiederholte Patches für denselben Lückentyp lösen eine Komplexitäts- und Geltungsbereichsprüfung aus. Ein Patchstrom ohne abnehmende Lücke oder ohne

generalisierende Holdout-Leistung führt zu OUT_OF_SCOPE oder UNRESOLVED_E3, nicht zu unbegrenzter Meta-Evolution.

25.3 Prüfbarkeit des Universalitätsanspruchs

Der Universalitätsanspruch der UZA ist architektonisch und nicht die Behauptung, jedes Weltphänomen mit einem festen Domänenmodell erklären zu können. Er wird geschwächt oder aufgegeben, wenn wohldefinierte Anwendungsfälle wiederholt nur durch widersprüchliche Kernänderungen, unbeschränkte Patchfolgen oder nicht ausgewiesene Bedeutungsverluste modelliert werden können. Deshalb werden mindestens Repräsentierbarkeit, Verlust, Patchaufwand, Replaytreue und Mehrwert gegenüber einem einfacheren Basismodell vergleichend geprüft.

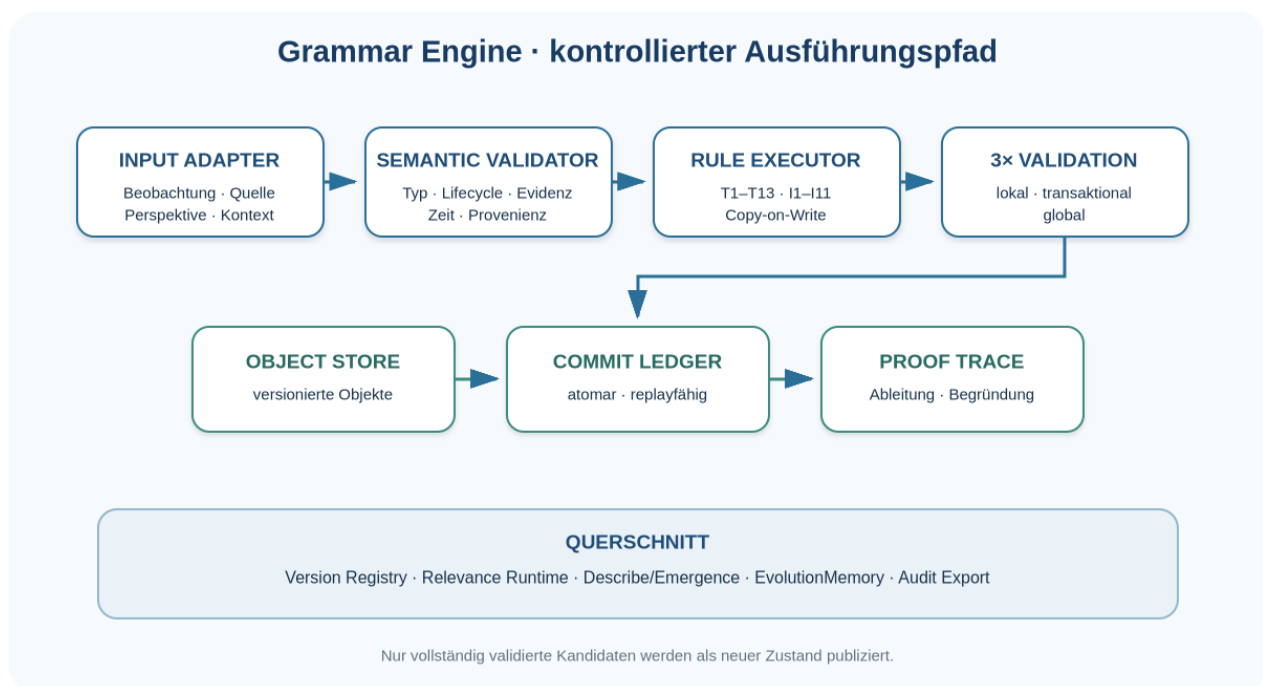
Teil IV – Grammar Engine und operationale Ausführung

26. Engine-Zielbild

Die Grammar Engine führt UZA-Operationen deterministisch relativ zu vollständig versionierten Eingaben aus. Determinismus bedeutet hier:

$$Run(I, V, M, \Theta, D) = Z_n$$

Identische Eingaben I , Versionen V , Modelle M , Parameter Θ und dokumentierte Entscheidungen D müssen dieselbe Zustands- und Commitfolge erzeugen. Diese Reproduzierbarkeit behauptet keinen ontologischen Determinismus der modellierten Wirklichkeit.



27. Komponenten der Grammar Engine

27.1 Input Adapter

Normalisiert Beobachtungen, Quellen, Zeitangaben, Perspektiven und Kontexte in typisierte Eingabeobjekte. Der Adapter darf fehlende Daten markieren, aber keine semantische Sicherheit ergänzen, die in der Quelle nicht vorhanden ist.

27.2 Object Store

Hält sämtliche VersionedObject-Instanzen in einem gemeinsamen, unveränderlichen Objektbestand. Neue Commits erzeugen Copy-on-Write-Sichten. Indizes für Beziehungen, Wirkungen, Perspektiven und Kontexte dienen der Abfragebeschleunigung.

27.3 Semantic Validator

Prüft Typen, Pflichtfelder, Referenzen, Zeitintervalle, Unsicherheit, Lifecycle, Perspektive, Kontext und Evidenzanforderungen.

27.4 Rule Executor

Lädt die anwendbare Regelversion, prüft Vorbedingungen, führt eine reine Zustandsfunktion aus und erzeugt einen Kandidatenzustand mit Proof Trace.

27.5 Relevance Runtime

Verwaltet versionierte Relevanzmodelle, berechnet Vektorergebnisse, Sensitivitätsprofile und Begründungen und verhindert die Vermischung inkompatibler Modelle.

27.6 Orientation Synthesizer

Bildet Orientierungsoptionen aus Relevanzen, Resonanzen, Zielen, Ressourcen, Einschränkungen, Macht- und Verantwortungsprofilen. Er gibt die Grundlage und alle wesentlichen Unsicherheiten aus.

27.7 Simulation Runtime

Erzeugt prognostizierte Zustände in einem getrennten Simulationszweig. Simulationsergebnisse können Orientierung stützen, werden aber niemals als Beobachtung gespeichert.

27.8 Feedback and Revision Engine

Verknüpft Ausführung, Beobachtung, Abweichung und Rückwirkung. Sie erzeugt Revisionen von Wirkungen, Relevanzen, Perspektiven und Orientierungen.

27.9 Emergence Analyzer

Berechnet die drei Beschreibbarkeitsprädikate, erstellt DescriptionResult, prüft strukturelle Neuheit und klassifiziert E0–E3.

27.10 Meta-Evolution Gateway

Nimmt GrammarPatch-Vorschläge entgegen, führt Kompatibilitäts- und Impact-Prüfungen aus und verwaltet Freigabe und Migration.

27.11 Querschnittskomponenten

- **Commit Ledger:** atomare, geordnete Zustandsänderungen.
- **Provenance Store:** Quellen-, Evidenz- und Herkunftsgraph.
- **Version Registry:** Semantik-, Grammatik-, Engine-, Modell- und Transformationsversionen.
- **Integrity Monitor:** globale Invarianten und Zustandsvalidität.
- **Proof Trace:** maschinenlesbare Ableitungs- und Entscheidungswege.
- **EvolutionMemory:** Gedächtnis der Grammatikentwicklung.

- **Audit Exporter:** reproduzierbare Fall-, Zustands- und Beweisexporte.

28. Event-Sourcing und Copy-on-Write

Der veröffentlichte Zustand Z_t bleibt unveränderlich. Eine Operation erzeugt einen Kandidatenzustand Z_{t+1}^* :

$$T_i(Z_t, \theta) \rightarrow Z_{t+1}^*$$

Erst nach erfolgreicher dreistufiger Validierung wird der Commit atomar publiziert:

$$Validate(Z_t, op, Z_{t+1}^*) = true \Rightarrow Commit(Z_{t+1}^*)$$

Bei Fehlschlag wird der Kandidat verworfen und ein Fehlerobjekt erzeugt. Event-Sourcing speichert die Operation und ihre Eingaben; Copy-on-Write speichert nur geänderte oder neue Objektversionen plus Referenz auf den Vorgängerzustand.

29. Commit Ledger

Ein Commit enthält mindestens:

```
commit_id, parent_commit_id, case_id
timestamp, event_time, actor, authority
rule_id, rule_version, operation_id
input_state_id, output_state_id
parameters, documented_decisions
created_objects, revised_objects, retired_objects
evidence_refs, source_refs, perspective_refs, context_refs
semantic_version, grammar_version, engine_version
relevance_model_versions, transform_versions
validation_report, proof_trace, status
```

Atomarität:

$$Commit = SUCCESS \vee ROLLBACK$$

Partielle externe Abläufe wie Transmissionen können mehrere atomare Commits besitzen. Ihr Lifecycle verbindet diese Commits, ohne die Atomarität des einzelnen Zustandswechsels aufzuheben.

30. Drei Validierungsebenen

30.1 Lokale Regelvalidierung

Prüft Operationstyp, Parameter, Objektstatus, Berechtigung, Regelversion und Vorbedingungen.

30.2 Transaktionsvalidierung

Prüft den gesamten Änderungssatz: neu erzeugte und revidierte Objekte, Referenzen, Evidenz, Lifecycle-Übergänge, Konflikte und Rollback-Fähigkeit.

30.3 Globale Zustandsvalidierung

Prüft den Kandidatenzustand gegen C1–C18, Identitäts- und Versionsinvarianten, aktive Grammatik, Containment-Wohlgeformtheit und Interaktionsintegrität.

$$Valid_{commit} = Valid_{local} \wedge Valid_{transaction} \wedge Valid_{global}$$

Der Validierungsbericht enthält jede Prüfung mit PASS, FAIL, NOT_APPLICABLE oder UNDETERMINED. UNDETERMINED kann nur dann publiziert werden, wenn die betroffene Regel dies ausdrücklich zulässt und die verbleibende Unsicherheit dokumentiert ist.

31. Regel-Executor

Der Executor verwendet eine vorbereitete Transaktion:

```
execute(state_id, operation, version_bundle):
    state      := load_immutable_state(state_id)
    rule       := registry.resolve(operation.rule_id, version_bundle.grammar)
    local      := validate_local(state, operation, rule)
    if local.failed: return error_commit(local)

    candidate  := rule.apply(copy_on_write(state), operation)
    transaction:= validate_transaction(state, candidate, operation)
    if transaction.failed: return rollback(transaction)

    global     := validate_global(candidate, version_bundle)
    if global.failed: return rollback(global)

    trace      := build_proof_trace(state, operation, candidate)
    commit     := ledger.append_atomic(candidate, trace, version_bundle)
    return commit.output_state
```

Die Regelanwendung soll als reine Funktion implementiert werden. Datenbank-, Netzwerk- und Freigabeeffekte werden vor oder nach der reinen Transformation über kontrollierte Ports verarbeitet.

32. Fehler- und Konfliktmodell

Fehlerklassen sind TYPE_ERROR, REFERENCE_ERROR, LIFECYCLE_ERROR, CONTEXT_ERROR, PERSPECTIVE_ERROR, EVIDENCE_ERROR, VERSION_ERROR, GRAMMAR_ERROR, VALIDATION_ERROR, MIGRATION_ERROR, TRANSMISSION_ERROR und BOUNDARY_ERROR.

Ein Fehlerobjekt enthält Code, Kategorie, Schwere, Operation, Regel, Zustand, Objektverweise, verletzte Bedingung, Erklärung, zulässige Reparaturoptionen und Zeit. Fehler verändern den publizierten Fachzustand nicht; sie werden in einem getrennten Auditstrom protokolliert.

Ein Konflikt ist kein technischer Fehler. CONFLICT bewahrt zwei oder mehr semantisch oder evidenziell unvereinbare Aussagen, bis eine ausgewiesene Revision, Perspektivdifferenzierung oder Entscheidungsregel angewandt wird.

33. Fünfzehn Mindestanforderungen einer konformen Engine

1. einheitlicher typisierter Object Store,
2. stabile Objekt- und eindeutige Versionsidentitäten,
3. unveränderliche veröffentlichte Zustände,
4. atomare Commits mit Rollback,
5. vollständige Provenienz und Evidence Lineage,
6. explizite Perspektiv-, Kontext- und Dreifachzeitbindung,

7. versionierte Unsicherheit und Lifecycle-Prüfung,
8. dreistufige Validierung,
9. getrennte Simulation und Beobachtung,
10. deterministisches Replay relativ zum Version Bundle,
11. abgeleitetes Describe und E0–E3-Analyse,
12. kontrolliertes GrammarPatch-Gateway,
13. Scale-, Containment- und Interaction-Unterstützung,
14. typisierte partielle Resultate für Transmission und Boundary Operations,
15. vollständiger Audit- und Reproduzierbarkeitsexport.

34. Bereinigter Engine-Hauptzyklus

```

Initialize case and version bundle
Register perspectives, contexts and observations
Create or revise appearances
Create relations
Hypothesize effects
Integrate evidence
Classify resonances
Evaluate perspective-bound relevance
Generate orientations or report insufficient basis
Select and authorize an action
Simulate the predicted state
Execute the selected action
Observe the resulting state
Compare prediction and observation
Record feedback and perspective changes
Analyze structural and grammatical emergence
Revise objects or propose a GrammarPatch
Validate locally, transactionally and globally
Commit atomically
    
```

Teil V – Zusammenhangsdynamik

35. Dynamikmodell

Dynamik analysiert nicht nur Zustände, sondern Trajektorien:

$$\tau(Z) = (Z_{t_0}, Z_{t_1}, \dots, Z_{t_n})$$

Ein DynamicsReport wird aus Historie, Wirkungen, Resonanzen, Rückwirkungen und Zeitfenstern erzeugt:

$$Flow: H(Z) \times W \times Window \times DetectorVersion \rightarrow DynamicsReport$$

Jede Mustererkennung nennt Detektorversion, Eingabefenster, Merkmale, Schwellenwerte, Unsicherheit und Alternativdeutungen. Begriffe wie Attraktor oder Kippunkt erhalten in jeder Domäne eine operative Definition und werden nicht automatisch im strengen Sinn der nichtlinearen Dynamik verwendet.

36. Zehn dynamische Grundmuster

Muster	Operative Lesart	Erforderliche Mindestausgabe
Stabilisierung	Abweichungen nehmen ab; Struktur und Wirkungen werden beständiger	Referenzbereich, Dämpfungsmaß, Zeitraum
Destabilisierung	Abweichungen oder Konfliktwirkungen nehmen zu	Wachstumsmuster, betroffene Dimensionen
Attraktorregion	verschiedene Trajektorien nähern sich einem ähnlichen Zustandsraum	Ähnlichkeitsmetrik, Einzugsbereich, Unsicherheit
Kippunkt	kleine Zusatzänderungen gehen mit qualitativ neuem Verlauf einher	Vorzeichen, Schwelle, Alternativerklärung
Rekursion	Ergebnisse werden wieder zu Eingaben desselben Musters	Rückkopplungspfad, Verzögerung, Verstärkung
Selbstorganisation	koordinierte Struktur entsteht aus lokalen Interaktionen	lokale Regeln, globale Neuheit, E-Klasse
Synchronisation	zuvor getrennte Verläufe gleichen Phase, Takt oder Entscheidung an	Synchronisationsmaß, Kanal, Verzögerung
Drift	langsame gerichtete Verschiebung ohne einzelnen dominanten Sprung	Trend, Stabilität, Perspektivabhängigkeit
Divergenz	Perspektiven, Teilzusammenhänge oder Trajektorien entfernen sich	Distanzfunktion, Treiber, Reichweite
Konvergenz	zuvor verschiedene Verläufe nähern sich	Zielraum, verbleibende Differenzen

36.1 Referenzsemantik der Detektoren

Für eine gewählte, versionierte Metrik m und Fensterfolge $W_1, \dots, W_n$ gelten folgende Mindestentscheidungen:

- **Stabilisierung:** ein robustes Abweichungsmaß fällt über mindestens k aufeinanderfolgende Fenster um die Schwelle ε_s ; Destabilisierung verwendet die entsprechende Zunahme.
- **Drift:** eine robuste Trendsteigung überschreitet ε_d , während kein einzelnes Ereignis mehr als den festgelegten Anteil der Gesamtverschiebung erklärt.
- **Konvergenz/Divergenz:** die Distanz zweier Trajektorien fällt beziehungsweise steigt über mindestens k Fenster; Metrik und Restdistanz werden ausgewiesen.
- **Synchronisation:** die maximale zeitversetzte Korrelation oder ein domänenspezifisches Phasenmaß überschreitet ε_{sync} innerhalb des zugelassenen Lags.
- **Attraktorregion:** mindestens zwei unabhängig initialisierte Trajektorien treten in eine definierte Region ein und verbleiben dort für die Mindestdauer h .
- **Kipppunkt:** ein reproduzierbarer Change-Point, eine qualitative Regimeänderung und Persistenz im Holdout-Fenster liegen gemeinsam vor. Eine bloße Schwellenüberschreitung bleibt Hypothese.
- **Rekursion:** der typisierte Wirkungs- oder Rückkopplungsgraph enthält einen zeitlich konsistenten Zyklus; Verzögerung und Nettoverstärkung werden angegeben.
- **Selbstorganisation:** eine globale neue Koordination ist aus dokumentierten lokalen Interaktionen ableitbar, ohne dass eine einzelne zentrale Operation sie vollständig erzeugt.

Jeder Detektor besitzt `metric_id`, `threshold_profile`, `window_policy`, `minimum_support`, `alternative_tests`, `holdout_policy` und `version`. Ein Resultat ohne diese Angaben bleibt HYPOTHESIS. Diese Referenzsemantik macht die Muster prüfbar, ohne domänenspezifische Metriken fälschlich zu vereinheitlichen.

37. Flow Analyzer und Dynamics Detector

Der Flow Analyzer berechnet Ereignisraten, Wirkungsänderungen, Resonanzdichte, Relevanzverschiebungen, Verzögerungen und Feedbackschleifen. Der Dynamics Detector wendet versionierte Musterdefinitionen an und erzeugt Hypothesen oder validierte Dynamics Events.

Detektorresultate besitzen denselben epistemischen Lifecycle wie andere Wirkungsbehauptungen. Eine einzelne Schwellenüberschreitung kann eine Kipppunkthypothese erzeugen; ein validierter Kipppunkt benötigt zusätzliche Evidenz, alternative Schwellenprüfungen und eine nachvollziehbare qualitative Zustandsänderung.

38. Relevance Feedback Loop

Rückwirkungen verändern nicht nur Sachobjekte, sondern auch die Relevanzmodelle und deren Eingaben:

$$REV_t \rightarrow ORI_t \rightarrow ACT_t \rightarrow FDB_{t+1} \rightarrow REV_{t+1}$$

Die Engine trennt dabei:

- Änderung eines bewerteten Objekts,
- Änderung einer Perspektive,
- Änderung von Modellparametern,
- Änderung des Relevanzmodells selbst.

Nur die ersten drei können innerhalb derselben Modellhauptversion verarbeitet werden. Eine strukturelle Modelländerung erzeugt eine neue Modellversion und einen Impact Report.

39. Perspective Tracker

Der Perspective Tracker erstellt Differenzprofile zwischen p_t und p_{t+1} . Er erfasst neue Beobachtungszugänge, Wissensänderungen, Wertungsverschiebungen, Interessenrevisionen, Erfahrung, Macht, Verantwortung, Freiheit und Handlungsraum. Die Änderungen werden nicht als bloßes „Meinungsrauschen“ verworfen, sondern als mögliche Wirkung des Zusammenhangs analysiert.

40. Complexity Monitor und Zusammenhangskompetenz

Der Complexity Monitor zählt unter anderem aktive Objekttypen, Regelverzweigungen, Abhängigkeitstiefe, offene Konflikte, GrammarPatches, Projektionverluste und Interaktionskanäle. Er warnt vor wachsender Modellkomplexität, ohne Komplexität selbst als Fehler zu behandeln.

Zusammenhangskompetenz wird auf drei Ebenen getrennt:

- $ZK_{Architektur}$ – Coverage, Consistency, Traceability, Extensibility, Minimality, Compatibility und Explainability,
- ZK_{Engine} – Describe, Predict, Revise, Validate, Detect Emergence, Orient, Scale und Interact,
- ZK_{Akteur} – fallgebundene Fähigkeit, Perspektiven, Folgen, Macht, Verantwortung und Gestaltungsmöglichkeiten angemessen einzubeziehen.

Eine mögliche Architekturmetrik lautet:

Dabei stehen Cov , Con , Trc , Ext , Min , Com und Exp für Coverage, Consistency, Traceability, Extensibility, Minimality, Compatibility und Explainability.

$$ZK_{Architektur} = f(Cov, Con, Trc, Ext, Min, Com, Exp) - \lambda Complexity(G)$$

Minimality bewertet, ob die vorhandenen Typen, Regeln und Erweiterungen für die erzielte Abdeckung erforderlich sind. Complexity(G) misst dagegen die tatsächliche strukturelle Last der Grammatik. Beide Größen können korrelieren, sind aber nicht identisch: Eine kleine Grammatik kann unnötige Elemente enthalten, während eine größere Grammatik für einen nachweislich breiteren Geltungsbereich minimal sein kann. Gewichte, Normalisierung und Aggregationsfunktion bleiben versionierte Bewertungsentscheidungen.

Die drei Ebenen werden nicht zu einem einzigen Gesamtscore vermischt.

Teil VI – Skalierung und Interaktion

41. ScaleProfile

Ein ScaleProfile ist analysegebunden:

$$ScaleProfile(Z, A) = \{S_x, T_x, P_x, N_x, R_x\}$$

- S_x (SpatialExtent) beschreibt räumliche oder topologische Reichweite.
- T_x (TemporalExtent) beschreibt den betrachteten Zusammenhangszeitraum.
- P_x (PopulationExtent) beschreibt Umfang und Auswahl beteiligter Einheiten.
- N_x (NestingDepth) beschreibt die Einbettungstiefe.
- R_x (Resolution) beschreibt die Analyseauflösung.

Es gilt:

$$Scale(Z) \neq Resolution(A)$$

Die Regelschemata T1–T13 sind skalenstabil; ihre Instanziierung ist skalenabhängig. Ein Beobachtungsobjekt kann auf Mikroebene eine einzelne Interaktion und auf Makroebene ein aggregiertes Verlaufsmuster darstellen, solange Typ, Aggregationsregel und LossProfile transparent bleiben.

42. ContainmentGraph

Der Einbettungsgraph lautet:

$$ContainmentGraph_t = \{Z_t, R_{contains,t}\}$$

directContains(a,b) bedeutet, dass a unmittelbar in b eingebettet ist. strictContains ist die transitive Hülle. containsEq ergänzt Reflexivität für Identitätsvergleiche.

Der direkte Graph muss azyklisch sein. Mehrfacheinbettung ist zulässig, deshalb ist die Struktur grundsätzlich ein gerichteter azyklischer Graph und kein Baum. Jede Einbettung ist zustands- und versionsgebunden.

43. SUBZ, Projektion und LossProfile

SUBZ(Z_i, Z_j) bezeichnet Z_i als Teilzusammenhang von Z_j . Im Container erscheint nicht automatisch der vollständige Teilzusammenhang, sondern eine Projektion:

$$\pi_j(Z_i, A) = PROJ_{i \rightarrow j}^A$$

Projektionsarten sind Auswahl, Aggregation, Abstraktion und Einbettungsprojektion. Ein Homomorphismus ist ein besonders strukturtreuer Spezialfall, keine allgemeine Voraussetzung.

Das LossProfile dokumentiert Verluste auf mindestens vier Dimensionen:

$$Loss(PROJ) = \langle L_{struct}, L_{sem}, L_{persp}, L_{time}, explanation \rangle$$

Jeder Wert liegt im Einheitsintervall. Die qualitative Erklärung nennt ausgelassene Objekte, zusammengeführte Bedeutungen, nicht sichtbare Perspektiven und zeitliche Verdichtungen.

44. InteractionNetwork und Kopplungsobjekt

$$InteractionNetwork_t = \langle Z_t, C_t \rangle$$

Eine Kopplung zwischen A und B wird beschrieben als:

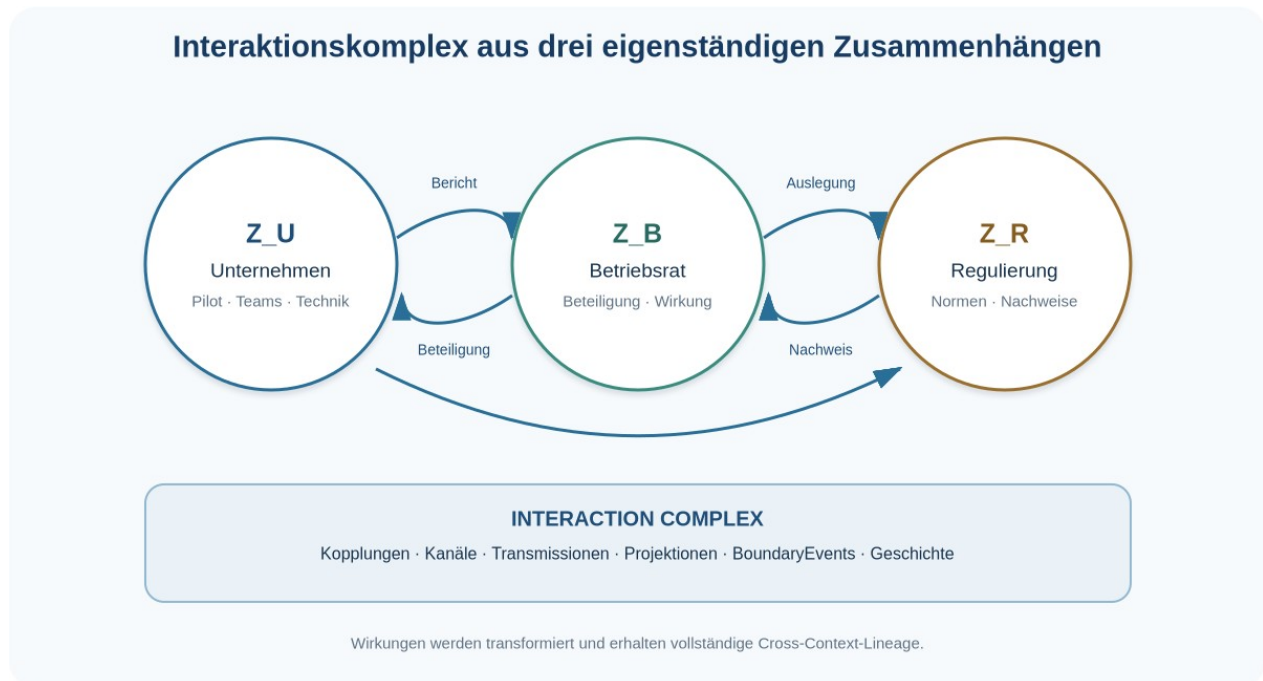
$$C_{AB} = \langle id, A, B, topology, \Pi_{AB}, history \rangle$$

Dabei bündelt Π_{AB} die versionierten Felder direction, channels, functions, strengths, filters, delay und contexts.

Die Stärke ist kanalabhängig:

$$strength: Coup \times Channel \times Time \rightarrow [0, 1]$$

Eine Kopplung kann zugleich kooperative, kompetitive, regulatorische, beobachtende oder austauschbezogene Funktionen tragen. Richtung, Kanal, Filter, Verzögerung und Transformation werden separat versioniert.



45. Kontrollierte Wirkungsübertragung

Eine Quellwirkung wird nicht identisch in den Zielzusammenhang kopiert. Die Kanaltransformation Ψ_c erzeugt ein partielles Resultat:

$$\text{Transmit}(EFF_A, C_{AB}, \text{Channel}, CTX_B, t) \rightarrow \begin{cases} \text{Transformed}(EFF_B^{\text{external}}) \\ \text{Blocked}(\text{reason}) \\ \text{Incompatible} \\ \text{Failed}(\text{error}) \end{cases}$$

Die Prüfung umfasst:

1. Existenz und Aktivität des Kanals,
2. zulässige Richtung,
3. Filterbedingungen,
4. Verzögerung,
5. Verstärkung oder Abschwächung,
6. Bedeutungsänderung,
7. Unsicherheitspropagation,
8. Grammatikkompatibilität,
9. Evidenz und Beobachtbarkeit im Zielkontext.

Die Zielwirkung ist ein neues versioniertes Objekt. Ihre Lineage verweist auf Quellwirkung, Kopplung, Kanal, Transformationsversion und TransmissionEvent.

46. Transmission Lifecycle

SCHEDULED → *DELIVERED* → *OBSERVED* → *VALIDATED*

Zusätzliche Resultate sind *BLOCKED*, *INCOMPATIBLE*, *FAILED* und *CANCELLED*. Ein *TransmissionEvent* enthält Quellwirkung, Kopplung, Kanal, Transformation, geplante und tatsächliche Zeiten, Statushistorie, transformierte Wirkung, Zielbeobachtung und Abweichung.

DELIVERED bedeutet lediglich, dass der Kanal die transformierte Wirkung dem Zielkontext zugeführt hat. Erst *OBSERVED* weist eine Zielbeobachtung aus; *VALIDATED* verlangt die semantischen und evidenziellen Bedingungen des Zielzusammenhangs.

47. Grenztransformationen

47.1 Fusion

$$\text{Fuse}(Z_A, Z_B) \rightarrow \text{Result}(Z_C, \mu_A, \mu_B, \text{BoundaryEvent})$$

Jedes Quellobjekt erhält eine ereignisgebundene Disposition: *Mapped*, *Unified*, *Derived*, *Conflicted* oder *Retired*. Identitätskollisionen werden explizit über *keepSeparate*, *unify*, *deriveNew* oder *conflict* gelöst.

47.2 Fission

$$\text{Split}(Z) \rightarrow \text{Result}(Z_1, \dots, Z_n, \text{Lineage}, \text{BoundaryEvent}), n \geq 2$$

Lineage ordnet jedes relevante Quellobjekt seinen Zielobjekten und Zielzusammenhängen zu. Leere oder verlorene Zuordnungen benötigen eine dokumentierte Disposition.

47.3 Aggregation

$$Aggregate(Z_1, \dots, Z_n, \alpha) \rightarrow Z_M$$

Die Quellzusammenhänge bleiben aktiv. Aggregation erzeugt eine Makrosicht und ist von Fusion zu unterscheiden.

47.4 Embedding und Detachment

Embed erzeugt eine Einbettungsrelation und eine Containerprojektion. Detach beendet oder versioniert die Einbettung, erhält Historie, Lineage und noch aktive Kopplungen. Weitere zulässige Operationen sind Overlap und Decouple.

48. Interaktionskomplex

$$I_t = \{Z_t, \mathbf{C}_t, R_{contains,t}, T_{cross,t}, B_t, H_t\}$$

Der Interaktionskomplex enthält aktive Systemzusammenhänge, Kopplungen, Einbettungen, grenzüberschreitende Transmissionen, BoundaryEvents und Geschichte. Describe_I prüft diesen Komplex als solchen. Eine künstliche Fusion in einen einzigen Zusammenhang ist weder erforderlich noch semantisch neutral.

49. Interaktionsoperationen I1–I11

Code	Operation	Ergebnis
I1	CreateContext	neuer versionierter Systemzusammenhang
I2	EmbedContext	SUBS-Relation und Projektion
I3	DefineCoupling	validiertes Kopplungsobjekt
I4	ModifyCoupling	neue Kopplungsversion und Impact Report
I5	TransmitEffect	TransmissionEvent und partielles Resultat
I6	ProjectContext	PROJ mit LossProfile
I7	MergeContexts	FusionResult und BoundaryEvent
I8	SplitContext	FissionResult, Lineage und BoundaryEvent
I9	AggregateContexts	Makrozusammenhang und Aggregationsregel
I10	DetachContext	beendete Einbettung mit Historie
I11	AnalyzeInteractionEmergence	E0–E3 für den Interaktionskomplex

50. Konsistenzbedingungen C11–C18

Code	Bedingung	Prüfkern
C11	Zusammenhangs-Referenzintegrität	Kopplungen, Events und Projektionen referenzieren gültige Systemzusammenhänge
C12	SUBZ-Wohlgeformtheit	direkte Einbettung ist azyklisch und typisiert
C13	Projektionsaktualität	Projektion nennt Quellzustand, Regelversion und LossProfile
C14	Kopplungsintegrität	Endpunkte, Richtung, Kanäle, Filter und Funktionen sind konsistent
C15	Transmissionsnachvollziehbarkeit	jedes Zielresultat besitzt vollständige Cross-Context-Lineage
C16	Fusionsdispositionsvollständigkeit	jedes relevante Quellobjekt erhält eine Disposition
C17	Fissions-Lineage-Vollständigkeit	jedes relevante Quellobjekt wird zugeordnet oder begründet disponiert
C18	Versionskompatibilität	Grammatik, Transformation und Semantik sind über Grenzen kompatibel oder verwenden einen ausgewiesenen Adapter

51. Interaktions-Emergenz

Interaktions-Emergenz liegt vor, wenn ein neuer Zustand oder Verlauf erst durch die Kopplungsstruktur mehrerer Zusammenhänge erklärbar wird. Die Analyse vergleicht:

- Beschreibbarkeit der Einzelzusammenhänge,
- Beschreibbarkeit der Kopplungen und Transmissionen,
- Beschreibbarkeit des Gesamtverlaufs des Interaktionskomplexes,
- strukturelle Neuheit gegenüber den Einzelverläufen.

Ein Interaktionsmuster kann E1 sein, obwohl alle Einzelzusammenhänge vollständig beschrieben sind. E2 liegt vor, wenn etwa ein neuer Kopplungstyp oder eine neue Cross-Context-Transition benötigt wird. Synchronisationstendenzen gelten zunächst als dynamische Hypothesen und werden erst nach Detektor- und Evidenzprüfung validiert.

Teil VII – Durchgängige Demonstratoren

52. Methodischer Status der Demonstratoren

Die Demonstratoren zeigen die vollständige Anwendung der Spezifikation. Sie sind keine empirischen Studien und keine Belege für einen wirtschaftlichen oder sozialen Nutzen. Beobachtungen, Hypothesen, Simulationen und illustrative Werte werden typisiert getrennt. Ihr Zweck ist der Nachweis architektonischer Durchgängigkeit: Jeder Schritt muss Objekte, Regeln, Versionen, Validierung und Commits erzeugen können.

53. Demonstrator A – KI-Einsatz in einer Organisation

53.1 Fallrahmen

Ein mittelgroßes Dienstleistungsunternehmen plant die Einführung eines generativen Assistenzsystems für interne Recherche, Textentwürfe und Kundenservice. Der Fallkontext CTX-ORG-AI-01 umfasst einen zwölfwöchigen Pilot. Die aktiven Perspektiven sind:

Perspektive	Beobachtungsfokus	Ziele und Interessenhypothese n	Macht/Verantwortung/Freiheit
P-MGT	Kosten, Durchlaufzeit, Qualität, Skalierbarkeit	kürzere Bearbeitung und nutzbarer Geschäftswert	hohe Ressourcenmacht; hohe Ergebnisverantwortung
P-EMP	Arbeitsabläufe, Qualifikation, Belastung, Autonomie	hilfreiche Entlastung und tragfähige Arbeitsgestaltung	mittlere Prozessfreiheit; unmittelbare Folgenbetroffenheit
P-CUST	Antwortqualität, Datenschutz, Vertrauen	verlässliche und verständliche Leistung	geringe interne Macht; hohe Qualitätsbetroffenheit
P-IT	Integration, Sicherheit, Betrieb, Nachvollziehbarkeit	kontrollierbarer und wartbarer Betrieb	hohe technische Zugangskontrolle
P-COMP	Recht, Dokumentation, Risikotragung	regelkonforme Nutzung und prüfbare Verantwortlichkeit	Vetorechte in definierten Risikofällen

Der Fall ist zunächst ein Einzelzusammenhang. Betriebsrat und Regulierung werden im zweiten Demonstrator als eigenständige gekoppelte Zusammenhänge modelliert.

53.2 Beobachtungen und Erscheinungen

Die Eingabe enthält fünf typisierte Beobachtungsgruppen:

- unterschiedliche Bearbeitungszeiten bei wiederkehrenden Rechercheaufgaben,
- hohe Varianz in der Qualität bestehender Textentwürfe,
- begrenzte Erfahrung der Beschäftigten mit generativen Systemen,
- bestehende Freigabe- und Datenschutzprozesse,

- Wunsch nach schnellerer, zugleich nachvollziehbarer Bearbeitung.

T1 erzeugt unter anderem ENT-TASK-RESEARCH, ENT-DRAFT-QUALITY, ENT-SKILL-DIFF, ENT-APPROVAL-PROCESS und ENT-TRACEABILITY-NEED. Jede Erscheinung erhält Quelle, Perspektive, Ereigniszeit, Kontext und Unsicherheit.

53.3 Beziehungen und Wirkungshypothesen

T3 bildet unter anderem folgende Beziehungen:

Relation	Quelle → Ziel	Typ	Status
REL-01	Routine-Recherche → Bearbeitungszeit	conditions	supported
REL-02	Skill-Differenz → Ergebnisvarianz	strengthens	hypothesis
REL-03	Freigabeprozess → Nachvollziehbarkeit	strengthens	supported
REL-04	Zeitdruck → ungeprüfte Übernahme	generates	hypothesis
REL-05	Training → kompetente Nutzung	strengthens	hypothesis

T4 erzeugt daraus Wirkungshypothesen. Beispiel EFF-H-04: Unter hohem Zeitdruck kann ungeprüfte Übernahme die Transparenz, Verantwortung und Lernfähigkeit mindern. Die Zielwerte bleiben zunächst hypothetisch.

53.4 Evidenzintegration

Der Pilot definiert zulässige Evidenzarten: protokollierte Bearbeitungszeiten, qualitätsgeprüfte Stichproben, dokumentierte Korrekturen, Schulungsabschlüsse, Nutzerfeedback, Vorfälleberichte und Freigabeprotokolle. T5 aktualisiert die Wirkungshypothesen nach jedem Pilotfenster.

Eine beobachtete Zeitreduktion darf beispielsweise EFF-H-01 stützen. Sie validiert nicht automatisch die Hypothese einer Qualitätssteigerung. Qualität, Transparenz und Lernfähigkeit benötigen eigene Evidenzpfade.

53.5 Wirkungsprofile

Illustratives, explizit nicht empirisches V10-Profil für die Option „Assistenzsystem mit Training, Quellenanzeige und menschlicher Freigabe“ aus P-EMP:

Dimension	Wert	Begründung im Demonstrator
Verbindung	+1	gemeinsame Review-Praxis schafft zusätzliche Abstimmung
Stabilität	+1	standardisierte Freigabe senkt Prozessvarianz
Offenheit	+2	zusätzliche Recherche- und Entwurfsoptionen

Dimension	Wert	Begründung im Demonstrator
Lernfähigkeit	+2	Training und Korrekturfeedback werden Teil des Ablaufs
Kooperation	+1	Tandem-Reviews fördern Wissensaustausch
Handlungsfähigkeit	+2	Routineaufgaben werden schneller vorbereitbar
Transparenz	+1	Quellen- und Änderungsspuren bleiben sichtbar
Freiheit	+1	Beschäftigte können Vorschläge annehmen, ändern oder verwerfen
Verantwortung	+1	Freigabeverantwortung und Eskalation sind ausgewiesen
Regeneration	0	im Pilot noch keine belastbare Wirkung auf Erholung festgestellt

Dasselbe Maßnahmenpaket erhält aus P-IT und P-COMP andere Werte und Begründungen. Die Engine bewahrt diese Profile getrennt und berechnet keine perspektivlose Durchschnittswahrheit.

53.6 Resonanz und Relevanz

Der Demonstrator klassifiziert unter anderem:

- Verstärkung zwischen Training und kompetenter Nutzung,
- Ergänzung zwischen Quellenanzeige und menschlicher Freigabe,
- produktive Spannung zwischen kurzer Bearbeitungszeit und gründlicher Prüfung,
- hemmende Spannung zwischen breitem Datenzugriff und Datenschutzanforderungen.

Das Relevanzmodell REV-ORG-PILOT-1.2 bewertet jede Wirkung pro Perspektive. Es gibt Intensität, Unsicherheit, Stabilität, Reichweite, Dauer und Begründung aus. Die Sensitivitätsanalyse zeigt, welche Orientierungen von unsicheren Modellgewichten abhängen.

53.7 Orientierungsoptionen

T8 erzeugt vier begründete Optionen:

1. begrenzter Pilot mit Training, Quellenanzeige, Freigabe und Monitoring,
2. technischer Pilot ausschließlich mit synthetischen Daten,
3. Verschiebung bis zur vollständigen Prozess- und Datenschutzklärung,
4. kein breiter Rollout; Nutzung nur für ausgewählte Rechercheaufgaben.

Für jede Option werden erwartete V10-Wirkungen, betroffene Perspektiven, Ressourcen, Einschränkungen, Reversibilität und Rückmeldesignale dokumentiert. Der Demonstrator wählt Option 1, weil sie Lerngewinn ermöglicht, reale Risiken begrenzt und reversibel bleibt. Diese Auswahl ist eine fallgebundene Demonstrationsentscheidung, keine allgemeine UZA-Empfehlung.

53.8 Simulation, Ausführung und Beobachtung

Die Simulation prognostiziert sinkende Bearbeitungszeit, zunächst erhöhte Reviewlast, steigende Lernfähigkeit und eine vorübergehende Differenz zwischen erfahrenen und unerfahrenen Nutzenden. Die Ausführung startet mit zwei Teams, drei Aufgabentypen und einem definierten Eskalationsweg.

Nach dem ersten Fenster zeigt die Beobachtung: Bearbeitungszeit sinkt stärker als prognostiziert; Reviewlast ist ebenfalls höher; Quellenverweise sind lückenhaft; Beschäftigte entwickeln neue Peer-Review-Routinen. Diese Beobachtungen erzeugen FDB-Objekte und neue Beziehungen.

53.9 Emergenzanalyse

Die Peer-Review-Routine ist strukturell neu, aber mit vorhandenen Typen und Übergängen beschreibbar: E1. Ein zusätzliches Muster – gemeinsames Prompt- und Korrekturwissen als eigenständiges kollektives Lernobjekt – ist mit der aktiven Grammatik nur teilweise darstellbar. Der Demonstrator erzeugt deshalb einen E2-Kandidaten mit GrammarPatch CollaborativeLearningArtifact, statt E3 zu behaupten.

53.10 Commitfolge

Commit	Operation	Hauptergebnis	Validierung
C001	InitializeCase	Fall, Version Bundle, Ziel und Kontext	bestanden
C002	RegisterPerspectives	fünf versionierte Perspektiven	bestanden
C003–C007	T1 RegisterEntity	Erscheinungen und Beobachtungen	bestanden
C008–C012	T3 CreateRelation	fünf Ausgangsbeziehungen	bestanden
C013–C017	T4 HypothesizeEffect	Wirkungshypothesen	bestanden
C018	T5 EvidenceUpdate	gestützte und offene Wirkungen	bestanden
C019	T6 RegisterResonance	Resonanznetz	bestanden
C020	T7 EvaluateRelevance	perspektivische Relevanzvektoren	bestanden
C021	T8 GenerateOrientations	vier Optionen	bestanden
C022	T9 SelectAction	begrenzter Pilot	bestanden
C023	Simulate	Prognosezweig	bestanden
C024	T10 Execute	Pilotstart	bestanden

Commit	Operation	Hauptergebnis	Validierung
C025	T11 RecordFeedback	Beobachtungen und Abweichung	bestanden
C026	T12 AnalyzeEmergence	E1 plus E2-Kandidat	bestanden
C027	T13 Revise	Quellenanzeige und Reviewprozess revidiert	bestanden
C028	ProposeGrammarPatch	CollaborativeLearningArtifact	Prüfung offen

54. Demonstrator B – Unternehmen, Betriebsrat und Regulierung

54.1 Drei eigenständige Zusammenhänge

Der erweiterte Demonstrator modelliert:

- Z_U – Unternehmen mit Pilot, Teams, Prozessen und technischen Systemen,
- Z_B – Betriebsrat mit Beteiligungsprozess, Beschäftigtenrückmeldungen und Vereinbarungen,
- Z_R – Regulierung mit Normen, Auslegung, Aufsicht und Berichtspflichten.

Die Zusammenhänge werden nicht zu einem einzigen Modell verschmolzen. Jeder besitzt eigene Perspektiven, Grammatikinstanzen, Zeitverläufe und Relevanzmodelle.

54.2 ScaleProfiles

Zusammenhang	Spatial/ Topological Extent	Temporal Extent	Population	Nesting	Resolution
Z_U	Organisation und technische Infrastruktur	zwölfwöchiger Pilot	zwei Teams plus Funktionen	2	Aufgaben- und Wochenebene
Z_B	betriebliche Vertretungsstruktur	Beteiligungs- und Reviewzyklus	betroffene Beschäftigtengruppen	1	Themen- und Beschlussebene
Z_R	regulatorischer Bezugsraum	gültige und erwartete Anforderungen	regulierte Akteure	0	Norm- und Pflichtenebene

Die Projektion von Z_U in Z_B enthält Arbeitsabläufe, Belastung, Qualifikation, Überwachung und Mitgestaltung; technische Detailparameter werden teilweise abstrahiert. Das LossProfile weist diesen semantischen und strukturellen Verlust aus.

54.3 Kopplungen und Kanäle

Kopplung	Kanal	Richtung	Funktion	Verzögerung
C_UB	Pilotbericht	U → B	Information/Observation	2 Tage
C_BU	Beteiligungsbeschluss	B → U	Regulation/Cooperation	ereignisgebunden
C_RU	Anforderung	R → U	Regulation	0–30 Tage
C_UR	Nachweisbericht	U → R	Observation/ Compliance	periodisch
C_RB	Auslegungsinformation	R → B	Information	variabel

54.4 Beispieltransmission

Quellwirkung in Z_R: Eine neue Nachweisanforderung erhöht die erforderliche Dokumentation. Kanal C_RU/Requirement transformiert sie in Z_U zu einer externen Wirkung auf Transparenz, Verantwortung, Handlungsfähigkeit und Stabilität. Die Übertragung wird geplant, zugestellt, im Complianceprozess beobachtet und erst nach Zielkontextprüfung validiert.

Parallel wird ein Teil der Wirkung über die Unternehmensprojektion an Z_B sichtbar. Die Transformation betont Beschäftigtenfolgen und Beteiligungsrechte. Es entstehen zwei Zielwirkungen mit gemeinsamer Quelllineage, aber unterschiedlichen V10-Profilen.

54.5 Interaktions-Emergenz und Synchronisation

Die drei Zusammenhänge entwickeln zunächst unterschiedliche Takte. Durch periodische Pilotberichte, Beteiligungsreviews und regulatorische Nachweise entsteht ein gemeinsamer Vierwochenrhythmus. Der Dynamics Detector klassifiziert eine Synchronisationstendenz. Diese ist im Interaktionskomplex E1, weil vorhandene Kanal-, Zeit- und Resonanztypen ausreichen.

Ein neu entstehendes gemeinsames Prüfobjekt, das technische Nachweise, Beschäftigtenwirkung und regulatorische Begründung in einer versionierten Struktur verbindet, kann einen E2-Patch CrossContextAssuranceCase auslösen. Der Patch wird nur vorgeschlagen; seine Aufnahme benötigt Meta-Evolutionsprüfung.

54.6 Ergebnis des Demonstrators

Der Durchlauf zeigt die getrennte Identität der drei Zusammenhänge, skalen- und projektionsbewusste Sichtbildung, kontrollierte Wirkungstransmission, kanalabhängige Bedeutungsänderung und Unsicherheit, Interaction Emergence ohne künstliche Fusion, dynamische Synchronisationsanalyse sowie vollständige Cross-Context-Lineage.

Teil VIII – Referenzimplementierungsdesign und Conformance 0.6/0.9.1

55. Technische Architektur

Das Referenzdesign verwendet eine modulare, ereignisbasierte Architektur. Es schreibt keine einzelne Programmiersprache oder Datenbank vor, definiert aber klare Ports:

Modul	Verantwortung	Persistenz/Port
Case Service	Fälle, Ziele, Version Bundles	relational oder dokumentorientiert
Object Service	VersionedObject und Abfragen	Object Store
Graph Index	Beziehungen, Lineage, Containment, Couplings	Graphindex als abgeleitete Sicht
Rule Runtime	T1–T13 und I1–I11	reine Funktionen plus Registry
Validation Service	lokal, transaktional, global	Rule/Invariant Registry
Ledger Service	atomare Commits und Replay	append-only Event Store
Model Runtime	Relevanz, Simulation, Dynamics	versionierte Modellartefakte
Describe Service	Coverage, Gaps, E0–E3	Grammar Registry
Evolution Service	Patches, Impact, Migration	EvolutionMemory
Export Service	Auditpakete und reproduzierbare Runs	signierte/prüfbare Artefakte

Object Store, Ledger und Version Registry sind normative System-of-Record-Komponenten. Graph- und Suchindizes können neu aufgebaut werden und dürfen keine eigenständigen Objektversionen erzeugen.

56. Logisches Datenmodell

56.1 Version Bundle

```
{
  "specification": "UZA-0.9.1-RC",
  "semantic_core": "S-0.3.1",
  "grammar": "G-0.9.1",
  "engine": "REF-0.6-design+conformance-0.1",
  "relevance_models": ["REV-ORG-PILOT-1.2"],
  "dynamics_detectors": ["DYN-0.8.0"],
  "interaction_transforms": ["TRF-0.9.1"],
  "validation_profile": "VAL-0.9.1-strict"
}
```

56.2 VersionedObject

```
{
  "object_id": "EFF-004",
  "version_id": "EFF-004@3",
  "revision": 3,
  "object_type": "EFFECT",
  "payload": {"cause": "REL-004@1", "target": "ENT-TRACE@2"},
  "uncertainty": {"class": "SUPPORTED", "confidence": 0.72},
}
```

```

"lifecycle": "SUPPORTED",
"valid_time": {"from": "2026-08-01T09:00:00Z", "to": null},
"previous_version": "EFF-004@2",
"perspective_refs": ["P-EMP@2"],
"context_refs": ["CTX-ORG-AI-01@1"],
"source_refs": ["SRC-PILOT-W1"],
"evidence_refs": ["EVD-017@1"]
}

```

56.3 Commit

```

{
  "commit_id": "C025",
  "parent_commit_id": "C024",
  "operation": {"id": "OP-025", "rule": "T11", "version": "0.9.1"},
  "input_state": "STATE-024",
  "output_state": "STATE-025",
  "created": ["FDB-001@1"],
  "revised": ["EFF-004@3", "P-EMP@2"],
  "validation_report": "VR-025",
  "proof_trace": "PT-025",
  "version_bundle": "VB-ORG-PILOT-01",
  "status": "SUCCESS"
}

```

57. API-Entwurf

Methode und Pfad	Zweck	Normatives Ergebnis
POST /v1/cases	Fall und Version Bundle anlegen	initialer Zustand und Commit
POST /v1/cases/{id}/objects	typisiertes Objekt registrieren	VersionedObject oder Fehlerobjekt
POST /v1/cases/{id}/operations	T- oder I-Operation ausführen	Commit/rollback mit ValidationReport
GET /v1/states/{id}	unveränderlichen Zustand lesen	Zustand plus Version Bundle
GET /v1/objects/{id}/versions	Objektgeschichte lesen	geordnete Versionen und Provenienz
POST /v1/relevance/evaluate	Relevanzmodell ausführen	REV-Objekte und Sensitivitätsbericht
POST /v1/orientations/generate	Orientierungsraum bilden	ORI-Objekte oder INSUFFICIENT_BASIS
POST /v1/simulations	prognostizierten Zweig erzeugen	getrennte Simulationstrajektorie
POST /v1/observations	reale Beobachtung integrieren	ENT/FDB und Commit
POST /v1/describe	Beschreibbarkeit prüfen	DescriptionResult

Methode und Pfad	Zweck	Normatives Ergebnis
POST /v1/emergence/analyse	E0–E3 bestimmen	EMG-Objekt und Gap-Liste
POST /v1/grammar-patches	Patch vorschlagen	Metaobjekt im Prüfstatus
GET /v1/commits/{id}	Audit und Replaybasis lesen	vollständiger Commit
POST /v1/exports/replay-package	reproduzierbares Paket erzeugen	Eingaben, Versionen, Ledger, Prüfsummen

Schreibende Endpunkte sind idempotent über einen operation_id-Schlüssel. Wiederholung derselben Operation mit identischem Inhalt liefert dasselbe Commitresultat oder einen Identitätskonflikt, aber keinen zweiten fachlichen Commit.

58. Conformance- und Teststrategie

58.1 Unit- und Property-Tests

- Typ- und Lifecycle-Regeln für jeden Objekttyp,
- Idempotenz von Operationen,
- Unveränderlichkeit publizierter Zustände,
- Eindeutigkeit von Objektversionen,
- vollständiger Rollback bei jeder Validierungsstufe,
- C1–C18 als ausführbare Properties,
- Lineage- und Dispositions Vollständigkeit bei Boundary Operations.

58.2 Replay-Tests

Ein Golden Run enthält Eingaben, Version Bundle, dokumentierte Entscheidungen und erwartete Commitfolge. Eine konforme Engine muss dieselben fachlichen Zustände und Proof Traces reproduzieren. Technische Zeitstempel oder interne Speicheradressen sind von der fachlichen Gleichheitsprüfung ausgeschlossen.

58.3 Metamorphe Tests

- Permutation unabhängiger Eingaben verändert das Ergebnis nicht.
- Hinzufügen einer unreferenzierten archivierten Objektversion verändert aktive Analysen nicht.
- Wechsel einer Perspektive verändert nur perspektivisch gebundene Ausgaben und ihre Folgeableitungen.
- Erhöhung der Analyseauflösung erzeugt ein dokumentiertes Loss-/Gain-Profil.
- Blockierte Transmission erzeugt keine externe Zielwirkung.

58.4 Migrations-Tests

Jeder GrammarPatch benötigt Vorher-/Nachher-Fälle, Kompatibilitätsklassifikation, Migrationsabbildung, Rückrollplan und Beweis, dass historische Runs mit ihrem ursprünglichen Version Bundle weiter reproduzierbar bleiben.

58.5 Ausgeführte Golden Runs 0.9.1

Der mitgelieferte Conformance-Harness ist ein kleiner, deterministischer Referenzausschnitt und keine Produktionsimplementierung. Er verwendet ausschließlich versionierte JSON-Eingaben und semantisch kanonisierte JSON-Ausgaben. Ein Replay besteht nur, wenn das berechnete Ergebnis exakt dem gepinnten Golden Output entspricht.

Run	Prüfswertpunkt	Ergebnis	Resultat-SHA-256
GR-A-ORG-AI-0.9.1	End-to-End-Kernlauf, ausdrücklich T11/FDB vor T12/EMG, E2 und zulässiger Patch	bestanden	b971f6a159c494d4822e6d718af3420b31c40ae8b20453882ecf952f575cdf09
GR-B-MULTI-CONTEXT-0.9.1	zwei Transmissionen, vollständige Lineage, LossProfile, PowerShift, Nicht-Fusion und E1	bestanden	3b539589f7837f45c36789f38d1b163408fbb1a3680320e667edd99882e8054d
GR-C-NEGATIVE-GATES-0.9.1	OUT_OF_SCOPE, INSUFFICIENT_BASIS, MODEL_FALSIFIED, CORE_CONFLICT, PATCH_REJECTED und verlustbewusste V10-Migration	bestanden	bf3a95812c04e12c5538255626221434a1ed9580ee3a15ebeatf08c94175437cc

Zusätzlich bestehen drei automatisierte Testgruppen: exakter Replay und normative Reihenfolge; Transmission-Lineage und Nicht-Fusion; negative Geltungs-, Patch- und Migrationsgates. Der Testlauf wurde am 1. August 2026 lokal mit Python ausgeführt. Die Golden Runs belegen die ausführbare Bedeutung des geprüften Ausschnitts, nicht Vollständigkeit, Performance oder Produktionsreife der gesamten Engine.

59. Implementierungsstatus

Das technische Design, das logische Datenmodell, die API, der Regel-Executor und die vollständige Zielarchitektur bleiben spezifiziert. Für 0.9.1 existiert nun zusätzlich ein ausführbarer, deterministischer Conformance-Harness mit drei gepinnten Golden Runs und automatisierten Negativtests. Noch ausstehend sind die vollständige produktive Referenzengine, persistenter Object Store und Commit Ledger, Performance- und Sicherheitsprüfung, die Verknüpfung der Engine-Conformance mit formalisierten Isabelle-Pre-/Postconditions sowie ein unabhängiger Replay durch eine zweite Implementierung. Der korrekte Status ist **Reference Implementation Design 0.6/0.9.1 plus Reference Conformance Harness 0.1**.

Teil IX – Formale Theorie und mathematische Schließung

60. Logische Grundlage

Die primäre Spezifikationssprache ist many-sorted Higher-Order Logic auf Basis einfacher Typentheorie. Zielsystem ist Isabelle/HOL; Lean 4 bleibt eine mögliche alternative Mechanisierung. Eine FOL-Kernreduktion wird als separates Artefakt für geeignete Fragmente entwickelt und ersetzt die HOL-Haupttheorie nicht.

Die Theoriearchitektur wird streng getrennt:

1. primitive Typen und Signatur,
2. Definitionen und abgeleitete Prädikate,
3. statische Axiome,
4. Übergangssystem und Operationsvorbedingungen,
5. Metatheorie, Modelle und Beweise.

Diese Trennung verhindert, dass ein gewünschtes Theorem still als Definition oder Axiom eingebaut wird.

61. Konsolidierte Signaturentscheidungen

- Persistierte Objekte verwenden in der Isabelle2025-2-kernelgeprüften Theorie den Record `uza_object`; `object_kind` unterscheidet die zulässigen Objektarten explizit.
- Objekt- und Versionsidentität sind getrennte Synonyme; Gültigkeit wird über `valid_from_of` und optionales `valid_to_of` modelliert.
- V10-Werte sind ganzzahlig und Relevanz-, Loss- sowie Machtwerte reell; zulässige Wertebereiche werden durch explizite Gültigkeitsprädikate statt durch bereits abgeschlossene typedef-Beweise begrenzt.
- `boundary_event`, `projection`, `coupling` und `transmission` sind eigene Records mit Lifecycle-, Loss-, Lineage- und Vollständigkeitsbedingungen.
- `strict_contains` ist zustandsgebunden als transitiver Abschluss der direkten Einbettung definiert.
- `step : grammar ⇒ snapshot ⇒ operation ⇒ snapshot ⇒ bool` bildet das Übergangssystem und bindet Grammatik, Vor- und Folgezustand ausdrücklich ein.
- `Describe` ist ein abgeleiteter Report aus drei unabhängigen Vollständigkeitsprädikaten.
- Freitext kann Erklärungen tragen, ersetzt aber keine semantischen Identitäten.

Das vollständige ScaleProfile, typisierte Ziele und Gap-/Loss-Taxonomien bleiben normative Bestandteile der 0.9.1-Spezifikation. Ihre vollständige Abbildung in zusätzliche Isabelle-Records gehört zum offenen M4-Ausbau und wird hier nicht als bereits mechanisiert ausgegeben.

Die Datei `UZA_0_9_1.thy` setzt diese Entscheidungen als Isabelle2025-2-kernelgeprüfte Mechanisierungsbasis um. Anders als die 0.9-Signaturskizze enthält sie für die zentralen Begriffe tatsächliche Referenzdefinitionen:

Bereich	Definierter semantischer Kern
Zustand und Übergang	Referenz-, Zeit- und objekttypspezifische Wohlgeformtheit, state_well_formed, step
Evidenzintegrität	evidence_references_complete, evidence_well_formed; auflösbare Evidenz-IDs und nichtleere Evidenz für Supported/Validated
Beschreibbarkeit	type_complete, relation_complete, transition_complete, describe
Emergenz	structural_novelty, repairable, emergence_of für E0–E3
Wirkung und Resonanz	effect_vector_valid, Skalarprodukt, resonance_score, classify_resonance
Relevanz	normalisierte Gewichte, Unsicherheitsabschlag, relevance_score
Macht	separates Fünf-Achsen-Profil und power_shift
Dynamik	Stabilisierung, Destabilisierung, Konvergenz, Divergenz und Drift über endliche Trajektorien
Geltung und Scheitern	applicability_of unterscheidet Applicable, OutOfScope, InsufficientBasis und ModelFalsified
Meta-Evolution	patch_admissible, patch_status, apply_patch, repairable; Kernänderung, Holdout- oder Replayfehler brechen ab

62. Natürlichsprachliches Axiomregister A1–A22

Die folgenden Aussagen bilden das kontrollierte Register. Sie sind in der Isabelle2025-2-kernelgeprüften Theorie 0.9.1 als universell quantifizierte, typisierte Annahmen des Locale uza_model formuliert. Isabelle2025-2 hat die vollständigen Sessions erfolgreich geparkt und typgeprüft. Die Annahmen sind dadurch nicht als Sätze aus einer schwächeren Theorie hergeleitet; ihre gemeinsame Erfüllbarkeit ist jedoch durch Base und zusätzlich durch die reichhaltige Interpretation Rich mechanisch belegt. Base bleibt der minimale Erfüllbarkeitszeuge. Rich enthält zwölf versionierte Objekte und aktiviert mit konkreten Antezedenzen sämtliche objektspezifischen Bedingungen A5–A13 sowie die Interaktionsbedingungen A17–A21.

Axiom	Formalisierter Inhalt
A1	Objektversionen mit gleicher version_id sind identisch.
A2	Versionen desselben Objekts bilden eine wohlfundierte Revisionskette.
A3	Gültigkeitsintervalle sind zeitlich wohlgeformt.
A4	Aktive Objekte referenzieren typkompatible aktive oder zulässig historische Objekte.
A5	Jede Relation besitzt existierende, typkorrekte Endpunkte.
A6	Jede validierte interne Wirkung besitzt eine relationale Herkunft.
A7	Perspektivische Objekte referenzieren eine gültige Perspektivversion.

Axiom	Formalisierter Inhalt
A8	Kontextgebundene Objekte referenzieren einen aktiven oder historisch ausgewiesenen Kontext.
A9	Jede Evidenz-ID löst auf eine aktuelle oder historische Objektversion auf; Objekte im Status Supported oder Validated besitzen zusätzlich eine nichtleere Evidenzmenge.
A10	Jede Relevanz nennt Ziel, Perspektive, Kontext, Zeit und Modellversion.
A11	Jede Orientierung besitzt eine nichtleere, gültige Begründungsbasis oder den Status INSUFFICIENT_BASIS.
A12	Jede Gestaltung verweist auf eine gültige Orientierung und Entscheidungsbefugnis.
A13	Jede Rückwirkung verweist auf Ausführung und Beobachtung.
A14	Ein erfolgreicher step erzeugt einen wohlgeformten Folgezustand.
A15	strict_contains ist irreflexiv.
A16	contains_eq ist antisymmetrisch.
A17	Jede Projektion referenziert eine gültige Einbettung, einen Quellzustand und ein LossProfile.
A18	Jede aktive Kopplung besitzt gültige Endpunkte und mindestens einen gültigen Kanal.
A19	Transmissionstatus folgt einem zulässigen Lifecycle.
A20	Jede transformierte externe Wirkung besitzt vollständige Transmission Lineage.
A21	BoundaryEvents erfüllen je nach Art vollständige Lineage oder Disposition.
A22	Emergenzklassifikation folgt ausschließlich den definierten Beschreibbarkeits- und Neuheitsbedingungen.

63. Konditionale Existenzaxiome der Interaktion

Die Interaktionserweiterung soll keine unnötigen globalen Existenzannahmen erzwingen. Axiome werden konditional formuliert. Beispiel: Nur wenn eine erfolgreiche Transmission vorliegt, muss ein Zielresultat mit vollständiger Lineage existieren. Nur wenn ein FusionResult Succeeded meldet, müssen Zielzusammenhang und Dispositionen vollständig sein.

Dieses Muster ermöglicht Modelle des stabilen Kerns ohne Kopplungen oder BoundaryEvents. Die Interaktionstheorie erweitert solche Modelle nur dann um zusätzliche Objekte, wenn die entsprechenden Operationen vorkommen.

64. Relative Konsistenz und Modell-Expansion

Das zentrale Modellziel lautet:

Zu jedem Modell des stabilen Kerns soll unter expliziten Zusatzbedingungen eine Expansion zu einem Modell der Scale-&-Interaction-Erweiterung konstruiert werden können.

Die geplante Konstruktion ergänzt:

1. eine möglicherweise leere Menge von Systemzusammenhängen,
2. einen azyklischen ContainmentGraph,
3. Kopplungen und Kanäle mit gültigen Endpunkten,
4. TransmissionEvents mit Lifecycle und Lineage,
5. BoundaryEvents mit Disposition und Lineage,
6. InteractionComplex und DescriptionResult.

Gelingt die Expansion aus jedem Kernmodell, dessen Zusatzbedingungen erfüllt sind, folgt relative Konsistenz der Erweiterung relativ zur Basistheorie. Konservativität ist ein stärkeres, optionales Ziel und wird nur für präzise definierte Sprachfragmente behauptet.

Zweiter Modellschritt abgeschlossen. UZA_0_9_1_Base_Model konstruiert weiterhin den minimalen endlichen Erfüllbarkeitszeugen. Ergänzend konstruiert UZA_0_9_1_Rich_Model ein wohlgeformtes Snapshot mit Relation, validierter Wirkung, Resonanz, Relevanz, Orientierung, Handlung und Rückwirkung sowie zwei Kontexten, Projektion, Kopplung, validierter Transmission und erfolgreichem Fusion-Event. Die Lemmata rich_A5_relation_nonvacuous bis rich_A17_A21_interaction_nonvacuous zeigen ausdrücklich, dass die betreffenden Axiome nicht nur durch falsche Antezedenzen erfüllt werden. Rich: uza_model interpretiert erneut A1–A22, und rich_uza_model_exists beweist die Existenz dieser Struktur relativ zu Isabelle/HOL. Offen bleiben die Expansion eines beliebigen Kernmodells und der Konservativitätsnachweis.

65. Beweisziele

Die Beweisreihenfolge lautet:

1. Nichtleerer Zustandsraum mit mindestens einem versionierten Objekt – für den Basismodellzeugen abgeschlossen,
2. Wohlgeformtheit der im Zeugen verwendeten Basisrecords und Summentypen – abgeschlossen,
3. Identitäts- und Versionsinvarianten – für T1 RegisterAppearance konstruktiv erhalten,
4. Azyklizität, Irreflexivität und Antisymmetrie der konkreten Einbettung – für den reichhaltigen Zeugen abgeschlossen; als allgemeine Erhaltungssätze offen,
5. Lifecycle und Lineage einer validierten Transmission – im reichhaltigen Zeugen aktiviert; allgemeine Erhaltung offen,
6. Totalität beziehungsweise typisierte Partialität der Grenzoperationen,
7. Lineage- und Dispositionsvollständigkeit – für den konkreten Fusion-Zeugen aktiviert; allgemeine Sätze offen,
8. Korrektheit der Describe-Aggregation,
9. Exklusivität und, unter Vorbedingungen, Vollständigkeit von E0–E3,
10. Modell-Expansion und relative Konsistenz,
11. optionale Konservativität,
12. FOL-Reduktion ausgewählter Fragmente.

Die Haupttheorie enthält sechs mechanisch geprüfte Haupttheorie-Sentinels: Zustandserhaltung aus A14, Transmission-Lineage aus A20, Korrektheit der Describe-Aggregation, Totalität der vier Emergenzfälle, Abbruch kernverändernder Patches und Wertebereich der Relevanz. Ihre Beweise wurden im erfolgreichen Isabelle2025-2-Session-Build vom Kernel akzeptiert. Die ersten beiden gelten relativ zu den Locale-Annahmen; die übrigen folgen aus den ausführbaren Referenzdefinitionen. Zwei zusätzliche A9-Lemmata sichern die geschlossene Bedeutung ab. Die Rich-Model-Session ergänzt neun explizite Nichtvakuositätszertifikate für A5–A13 und A17–A21 sowie eine zweite vollständige Locale-Interpretation. Die separate Operationssession ergänzt erstmals eine konkrete Kernoperation: Für RegisterAppearance werden Folgezustand, exakte Delta-Eigenschaft, Versionsfrische, Identitätserhaltung, Evidenzintegrität, vollständige Zustandswohlgeformtheit und Refinement zu step ohne vorausgesetzte Zielwohlgeformtheit bewiesen.

Für T3 CreateRelation liegt nun eine vollständige, kernelgeprüfte operationsspezifische Mechanisierung vor. Sie umfasst relation_type, gerichtete oder ungerichtete Katalogmetadaten, zwei verschiedene aktive ENT-Endpunkte, frische REL-Identität, exakte Snapshot- und Katalogdifferenz, Erhaltung von state_well_formed, Identität, Referenzen, Zeit, Kontext und Evidence Integrity sowie das Refinement zu step G s CreateRelation s'. Der reale Build deckte zwei lokale Beweislücken auf und führte zu den expliziten Endpunkt-Referenz- und Katalogtypbeweisen. Der aktuelle T3-Quellhash lautet
a3c801eaf1640624ee7bf41ba91a20d2ba1e000e489ee40f4d3c209b15911ca3.

Für T4 HypothesizeEffect liegt ebenfalls eine vollständige, kernelgeprüfte operationsspezifische Mechanisierung vor. Sie umfasst den typisierten effect_hypothesis_payload, explizite Offenheit von Ziel und Wirkungsvektor, eine aktive K_REL-Ursache, Perspektiv-, Kontext- und Profilmmodellbindung, begrenzte Unsicherheit, frische K_EFF-Identität, exakte Snapshot- und Katalogdifferenz, Erhaltung von state_well_formed, Identität, Katalog, Referenzen, Zeit, Perspektive, Kontext und Evidence Integrity sowie das Refinement zu step G s HypothesizeEffect s'. Der aktuelle T4-Quellhash lautet
2eb8e1d262cb3790f6c446839fb6809de3c2837d20705925507e272c860e2fc4.

Für T5 IntegrateEvidence liegt eine vollständige, kernelgeprüfte operationsspezifische Mechanisierung vor. Sie umfasst Copy-on-Write-Versionierung, vollständiges Objekt- und Payload-Archiv, einen versionierten Evidenz-Ledger, drei Evidenzdispositionen, kontrollierte Unsicherheitsänderung, Lifecycle-Klassifikation ohne Hypothesis → Validated-Sprung, Ausschluss aktueller und unmittelbarer Selbststützung, exakte Zustands- und Audit-Store-Differenzen, Erhaltung von state_well_formed, Identität, Aktiv/Historie-Disjunktheit, Katalog, Referenzen, Zeit, Perspektive, Kontext und Evidence Integrity sowie Refinement zu step G s IntegrateEvidence s'. Der zusätzliche endliche Zeuge aktiviert diese Semantik in einem konkreten Hypothesis → Supported-Lauf. Der T5-Quellhash lautet
4422076e2bd6cb2ff9fe5e2d5e2e135e3bf3c05919a1ca03a42b1faccd5fddc0; der Zeugenhash lautet f5e037e578335431903754153deaedeada461fed8d359a0511e36b59a825a7d.

Für T6 ClassifyResonance liegt eine vollständige, kernelgeprüfte operationsspezifische Mechanisierung vor. Sie umfasst eine gerichtete versionierte Interaktionsmatrix, formal

geschlossene Endlichkeit der zehn V10-Dimensionen, Profilwohlgeformtheit, normalisierten und auf $[-1,1]$ begrenzten Score, totale Fünf-Formen-Klassifikation, unsicherheitsbereinigte Stärke in $[0,1]$, aktive verschiedene Effektoobjekte, unabhängige Begründungsreferenzen, konstruktive Objekt- und Payload-Erzeugung, exaktes Zustandsdelta, Katalogfortschreibung, Erhaltung von state_well_formed, Identität, Effect- und Resonance-Katalog, Referenzen, Evidenz, Zeit, Perspektive und Kontext sowie Refinement zu step G s ClassifyResonance s'. Der nicht-vakuose Zeuge beweist zusätzlich eine echte Asymmetrie $A \rightarrow B \neq B \rightarrow A$. Der T6-Quellhash lautet 3f3583b42f066eaf41208a22bcb53a2d49baed374aa1b41e00a434256741b1cd; der Zeugenhash lautet 494da00eeb17205001ae85090d940a12cd07215ec283d76833aa480126a716ed.

66. Roadmap M0–M6

Phase	Ziel	Aktueller Stand
M0	logische Grundlage wählen	HOL entschieden
M1	Signatur schließen	abgeschlossen für 0.9.1; Isabelle2025-2-Parser und Typsystem bestanden
M2	Signatur, Definitionen, Axiome, Transition und Metatheorie trennen	in der Quelldatei umgesetzt
M3	Basistheorie A1–A22 vollständig in HOL formulieren	Locale-Annahmen einschließlich A9-Evidenzintegrität kernelgeprüft; minimale und reichhaltige Interpretation bestanden
M4	Scale-&-Interaction-Erweiterung formal ergänzen	Records und A17–A21 integriert; konkrete Projektion, Kopplung, validierte Transmission und Fusion im Rich Model kernelgeprüft; allgemeine Erhaltung offen
M5	Erfüllbarkeit, Expansion, relative Konsistenz, optional Konservativität beweisen	gemeinsame Erfüllbarkeit und konkrete Nichtvakuosität von A5–A13/A17–A21 relativ zu HOL belegt; allgemeine Expansion und Konservativität offen
M6	Isabelle-Mechanisierung und optionale FOL-Reduktion	Fresh-Profile-Build mit elf Theorien bestanden; T1–T6 einschließlich nicht-vakuoosen T5-/T6-Zeugen abgeschlossen; T7–T13 und FOL-Reduktion offen

67. Mathematischer Netto-Status

Bereich	Reife	Aussage
Ontologische Geschlossenheit	sehr hoch	Typ- und Abhängigkeitsarchitektur konsolidiert

Bereich	Reife	Aussage
Semantische Präzision	sehr hoch	Gültigkeit, Unsicherheit, Perspektive und Describe getrennt
Grammatik/Engine	0.9.1-RC	spezifiziert; begrenzter Conformance-Harness und Golden Runs bestanden
Scale & Interaction	0.9-RC	Operationen, Lineage und Bedingungen spezifiziert
Dynamik	spezifikatorisch integriert	Detektoren domänenspezifisch; HOL offen
HOL-Theorie	Isabelle2025-2 Kernel-Checked 0.9.1 für T1–T6	zentrale Semantik, A1–A22, Modellzeugen und T1–T6-Erhaltung kernelgeprüft; T5/T6 mit nicht-vakuosen Ausführungszeugen; T6-Richtungsasymmetrie konkret bewiesen; T7–T13 offen
Modelltheorie/ Beweistheorie	Rich Witness + T1–T6 Preservations Kernel-Checked	gemeinsame Erfüllbarkeit und konkrete Nichtvakuosität relativ zu HOL, T1–T6-Erhaltung und konkrete T5-/T6-Läufe belegt; allgemeine Expansion und Konservativität offen

Die konzeptionelle Expansion kann zugunsten der formalen Schließung pausieren. Haupttheorie, minimales Basismodell, reichhaltiger Modellzeuge, die konkreten T1–T6-Theorien und die nicht-vakuosen T5-/T6-Ausführungszeugen wurden gemeinsam in einer Isabelle2025-2-Fresh-Profile-Session gebaut. A9 ist referenziell geschlossen; Rich zeigt konkrete Nichtvakuosität von A5–A13 und A17–A21. T1 bis T6 erhalten die jeweils ausgewiesenen Invarianten in konstruierten Folgezuständen. T6 schließt zusätzlich die bislang nur verbal behauptete Richtungsabhängigkeit durch einen operationalen Kernel und einen asymmetrischen Zeugen. Der nächste Operationsschritt ist T7 EvaluateRelevance; allgemeine Modell-Expansion und Konservativität bleiben eigenständige Beweisziele.

Teil X – Gesamteinordnung

68. Abschließende Definition

Die Universelle Zusammenhangsarchitektur ist eine versionierte, typisierte und semantisch validierte Grammatik zur reproduzierbaren Erfassung, Analyse, Orientierung, Gestaltung und kontrollierten Weiterentwicklung komplexer, dynamischer, verschachtelter und interagierender Zusammenhänge.

Sie verbindet strukturelle Modellierung, perspektivische Bewertung, dynamische Analyse, operationale Ausführbarkeit und kontrollierte Meta-Evolution. Ihr besonderer Beitrag liegt in der durchgehenden Traceability vom Eingangssignal über Beziehung, Wirkung, Resonanz, Relevanz, Orientierung und Gestaltung bis zur beobachteten Rückwirkung und möglichen Grammatikerweiterung.

Die UZA trennt systematisch:

- Beobachtung, Hypothese und Evidenzstatus,
- Beziehung, Wirkung und Resonanz,
- Relevanz, Orientierung und Entscheidung,
- Simulation und Beobachtung,
- Ereignis-, Zusammenhangs- und Versionszeit,
- Zustand, Prozess, Potential und Geschichte,
- Maßstab und Analyseauflösung,
- Teilzusammenhang, Projektion und Container,
- Kopplung, Transmission und Fusion,
- Spezifikation, Implementierung und mathematischen Beweisstatus.

Damit bietet sie eine gemeinsame formale Sprache für sehr verschiedene Anwendungsfelder, ohne deren Domänenwissen zu vereinheitlichen. Ihre Regeln bleiben gleichartig; Daten, Perspektiven, Evidenzstandards, Relevanzmodelle und Ziele werden fallspezifisch eingebracht.

Anhang A – Verbindliche Artefakt- und Statusmatrix

Artefakt	Inhalt	Status	Nächster objektiver Abschluss
Reference Architecture	Kern, Typen, Axiome, Invarianten	0.9.1 konsolidiert	Änderungen nur begründet/versioniert
Semantic Core	Bedeutung, Unsicherheit, Perspektiven, Describe	0.9.1-RC	unabhängige Domänen-Conformance
Formal Rule Calculus	T1–T13 und Ableitungsprinzipien	0.9.1-RC	vollständige mechanisierte Pre-/Postconditions
Grammar Engine	Store, Ledger, Executor, Validation, Evolution	0.9.1-RC-Spezifikation	vollständiger Referenzcode
Reference Implementation	Datenmodell, API, Tests, Demonstratoren	Design 0.6/0.9.1	produktive Engine und unabhängiger Replay
Reference Conformance Harness	ausführbarer Kern, drei Golden Runs und Negativgates	0.1; lokal bestanden	zweite unabhängige Implementierung
Dynamics Layer	Flow, zehn Muster, Tracker und Monitor	0.9.1 spezifiziert; Referenzsemantik für fünf Muster	Kalibrierung und Testkorpus je Domäne
Scale & Interaction	ScaleProfile, SUBZ/PROJ, I1–I11, C11–C18	0.9.1-RC	vollständige Engine- und Beweisinstanziierung
HOL Theory	Signatur, zentrale Definitionen, A1–A22, Modellzeugen sowie konkrete T1–T6-Semantik und T5-/T6-Ausführungszeugen	Isabelle2025-2 Kernel-Checked; Fresh-Profile-Build mit elf Theorien, Exit-Code 0	T7 EvaluateRelevance mechanisieren; anschließend T8–T13
Axiomatics A1–A22	Basis- und Interaktionsaxiome	Locale-Annahmen; gemeinsame Erfüllbarkeit und konkrete Nichtvakuosität von A5–A13/A17–A21 relativ zu HOL belegt	allgemeine Expansion
Model Theory	Basismodell, Rich Model, Expansion, relative Konsistenz und Konservativität	minimaler und reichhaltiger expliziter Modellzeuge kernelgeprüft	Modell-Expansion und Konservativitätsanalyse
FOL Core Reduction	geeignetes Kernfragment	geplant	Signatur- und Übersetzungsspezifikation

Artefakt	Inhalt	Status	Nächster objektiver Abschluss
UZA 1.0	integrierte Referenzfreigabe	nicht freigegeben	formale und technische Gates erfüllen

Anhang B – Version Bundle und Reproduzierbarkeitsmanifest

Ein vollständiges Reproduzierbarkeitsmanifest enthält:

```
case_id
initial_state_hash
ordered_input_artifacts + content hashes
specification_version
semantic_core_version
grammar_version
engine_version
all relevance model versions
all dynamics detector versions
all channel transform versions
validation profile version
documented human decisions and authorities
ordered commit ledger
final_state_hash
proof trace bundle
export timestamp and exporter version
```

Fachliche Reproduzierbarkeit ist erfüllt, wenn der Replaylauf dieselben fachlichen Objektversionen, Zustandsbeziehungen, Validierungsergebnisse und Emergenzklassifikationen erzeugt.

Nichtfachliche Metadaten wie neue Exportzeitstempel dürfen abweichen.

Für die 0.9.1-Referenzläufe ist folgende Suite gepinnt:

Suite	Runner	Fälle	Ergebnis
UZA-0.9.1-golden	reference-conformance-0.1	3	3 bestanden, 0 fehlgeschlagen

Die Fixture- und Resultat-Prüfsummen werden mit kanonischer UTF-8-JSON-Darstellung und SHA-256 berechnet. Das maschinenlesbare results/summary.json ist Bestandteil des Conformance-Pakets.

Anhang C – Vollständiger Referenz-Pseudocode

```

function run_case(case_input, version_bundle):
    registry.assert_compatible(version_bundle)
    state = initialize_case(case_input.identity, version_bundle)

    state = commit_batch(state, register_contexts(case_input.contexts))
    state = commit_batch(state, register_perspectives(case_input.perspectives))

    for observation in case_input.observations:
        state = execute_T1(state, observation)

    candidate_relations = relation_builder.propose(state)
    for relation in candidate_relations:
        state = execute_T3(state, relation)

    for relation in state.active_relations:
        hypotheses = effect_builder.hypothesize(relation, state)
        for hypothesis in hypotheses:
            state = execute_T4(state, hypothesis)

    for evidence in case_input.evidence_stream:
        targets = evidence_router.resolve_targets(evidence, state)
        for effect in targets:
            state = execute_T5(state, effect, evidence)

    resonance_candidates = resonance_analyzer.pairs(state.active_effects)
    for pair in resonance_candidates:
        state = execute_T6(state, pair)

    for model in version_bundle.relevance_models:
        for target in model.target_scope(state):
            state = execute_T7(state, target, model)

    orientation_result = execute_T8(
        state,
        state.active_relevances,
        state.active_resonances,
        case_input.goal,
        case_input.constraints
    )
    state = orientation_result.state

    if orientation_result.status == INSUFFICIENT_BASIS:
        return publish_analysis_without_action(state)

    selected = decision_port.select(
        orientation_result.options,
        case_input.authority,
        case_input.documented_decisions
    )
    state = execute_T9(state, selected)

    predicted_branch = simulation_runtime.simulate(
        state,
        selected,
        version_bundle.models,
        case_input.simulation_parameters
    )
    state = commit_simulation_reference(state, predicted_branch)

    execution = execute_T10(state, selected)
    state = execution.state

    observations = observation_port.collect(execution.record)
    state = execute_T11(state, execution.record, observations)

    deviation = compare(predicted_branch.final_state, state)
    emergence = execute_T12(state.parent, state, state.active_grammar)
    state = emergence.state

    revisions = revision_engine.propose(deviation, emergence, state)
    for revision in revisions.object_revisions:
        state = execute_T13(state, revision)

    for gap in revisions.grammar_gaps:

```

```

        patch = meta_evolution.propose_minimal_patch(gap, state)
        state = commit_patch_proposal(state, patch)

    return audit_exporter.publish(state)

function transmit_effect(state, effect, coupling, channel, scheduled_at):
    assert active(effect, state)
    assert active(coupling, state)
    assert channel in coupling.channels
    assert direction_allows(coupling, effect.context, channel.target)

    event = schedule_transmission(effect, coupling, channel, scheduled_at)
    state = commit(state, event)

    result = channel.transform.apply(
        effect,
        coupling.source_context,
        coupling.target_context,
        scheduled_at + channel.delay
    )

    match result:
        Transformed(target_effect):
            target_effect.lineage = {
                source_effect: effect.version_id,
                coupling: coupling.version_id,
                channel: channel.version_id,
                transform: channel.transform.version_id,
                transmission_event: event.version_id
            }
            return commit_delivery(state, event, target_effect)
        Blocked(reason):
            return commit_blocked(state, event, reason)
        Incompatible(report):
            return commit_incompatible(state, event, report)
        Failed(error):
            return commit_failed(state, event, error)

```

Anhang D – UZA HOL Theory 0.9.1: T1–T6 Kernel-Checked

D.0 Leistungsstand auf einen Blick

Diese Übersicht steht bewusst vor dem Quelltext. Sie trennt bereits mechanisch Geleistetes von Teilresultaten, vorbereiteten Kandidaten und offenen Zielen. Der Status abgeschlossen wird nur vergeben, wenn Quelltext, tatsächlicher Isabelle2025-2-Build mit Exit-Code 0 und ein gepinnter SHA-256-Nachweis vorliegen. Für T5 und T6 kommen nicht-vakuose Ausführungszeugen hinzu; der T6-Zeuge belegt ausdrücklich gerichtete Asymmetrie. Teilweise abgeschlossen bezeichnet echten Kernelbeweisanschritt, dessen allgemeine Fassung noch offen ist. Mechanization Candidate bezeichnet ausgearbeiteten Isabelle-Quelltext ohne behaupteten Kernelabschluss.

Bereich	Status	Bereits geleistet und belegt	Noch offen
Haupttheorie	abgeschlossen für 0.9.1	Signatur, Referenzsemantik, A1–A22, sechs Sentinels; Hauptsession Exit-Code 0	Ableitung der Locale-Annahmen aus einer schwächeren Theorie
A9 Evidence Integrity	abgeschlossen	auflösbare Evidence-IDs, nichtleere Evidenz für Supported/Validated, zwei kernelgeprüfte Lemmata	keine offene A9-Lücke im spezifizierten Umfang
Minimaler Modellzeuge	abgeschlossen	Base: uza_model, explicit_uza_model_exists	bewusst keine Nichtvakuosität der objektspezifischen Bedingungen
Reichhaltiger Modellzeuge	abgeschlossen	zwölf Objekte; A5–A13 und A17–A21 nichtvakuos; Rich: uza_model	Expansion beliebiger Kernmodelle
T1 Register Appearance	abgeschlossen	konstruktive Transition, exaktes Delta, Identität, Evidence Integrity, Bindungen, state_well_formed, Refinement zu step	Vollständigkeit oder Produktivengine-Äquivalenz
T2 Differentiate Potential	abgeschlossen	endliche nichtleere Kandidatenmenge, Trennung von Beobachtung/Kandidat, exaktes Delta, Identität, Evidence Integrity, Bindungen, state_well_formed, Refinement zu step	Vollständigkeit und domänenspezifische Güte der Kandidatenerzeugung

Bereich	Status	Bereits geleistet und belegt	Noch offen
T3 Create Relation	abgeschlossen	aktive verschiedene ENT-Endpunkte, frische REL-Identität, exaktes Delta, Katalog-, Zustands-, Identitäts-, Referenz-, Zeit-, Kontext- und Evidence-Integrity-Erhaltung sowie Refinement zu step; Clean-Build Exit 0	Vollständigkeit der Relationstaxonomie und Produktivengine-Äquivalenz
T4 Hypothesize Effect	abgeschlossen	aktive typisierte REL-Ursache; explizit optionales Ziel und optionaler Wirkungsvektor; Perspektiv-, Kontext- und Profilmodellbindung; begrenzte Unsicherheit; frische K_EFF-Identität; exaktes Delta; Katalog-, Zustands-, Identitäts-, Referenz-, Zeit-, Perspektiv-, Kontext- und Evidence-Integrity-Erhaltung sowie Refinement zu step; Clean-Build Exit 0	Vollständigkeit der Wirkungstaxonomie, konkreter nicht-vakuoser T4-Eingabezeuge und Produktivengine-Äquivalenz

Bereich	Status	Bereits geleistet und belegt	Noch offen
T5 Integrate Evidence	abgeschlossen	Copy-on-Write-Effektversion, vollständiges Objekt- und Payload-Archiv, Evidenz-Ledger, drei Dispositionen, kontrollierte Unsicherheit, Lifecycle ohne Hypothesis → Validated-Sprung, Ausschluss von Selbststützung, exaktes Delta, Zustands-, Identitäts-, Katalog-, Archiv-, Referenz-, Zeit-, Perspektiv-, Kontext- und Evidence-Integrity-Erhaltung, Refinement zu step; nicht-vakuoser Hypothesis → Supported-Zeuge; Clean-Build Exit 0	empirische Güte und Vollständigkeit der Evidenzklassifikation sowie Produktivengine-Äquivalenz

Bereich	Status	Bereits geleistet und belegt	Noch offen
T6 Classify Resonance	abgeschlossen	gerichtetes versioniertes Kernelprofil; formal endliche V10-Domäne; normalisierter begrenzter Score; totale Formklassifikation; Unsicherheitsdämpfung; aktive verschiedene Effekte; unabhängige Begründung; konstruktive Objekt-/Payload-Erzeugung; exaktes Delta; Zustands-, Identitäts-, Effect-/Resonance-Katalog-, Referenz-, Evidenz-, Zeit-, Perspektiv- und Kontexterhaltung; step-Refinement; nicht-vakuoser Asymmetriezeuge; Fresh-Profile-Build Exit 0	empirische Herleitung der Kernel und Schwellen, Vollständigkeit der Formen sowie Produktivengine-Äquivalenz
T7–T13	offen	Operationsnamen und natürliche Spezifikation vorhanden	konstruktive Semantik, nicht-vakuaue Zeugen und Erhaltungsbeweise je Operation

Bereich	Status	Bereits geleistet und belegt	Noch offen
Modell-Expansion und relative Konsistenz	teilweise abgeschlossen	zwei konkrete Modelle belegen gemeinsame Erfüllbarkeit relativ zu HOL	allgemeines Expansionslemma und relativer Konsistenzsatz
Konservativität und FOL-Reduktion	offen	Ziel und Trennung der Artefakte festgelegt	Beweise und Übersetzung
Produktive Engine	offen	Conformance-Harness mit 3/3 Golden Runs	unabhängige Implementierung, Produktions- und Sicherheitsnachweise

D.1 Haupttheorie

Die folgende Quelldatei enthält Signatur, zentrale Referenzsemantik, A1–A22 als universell quantifizierte, typisierte Locale-Annahmen und sechs Haupttheorie-Sentinels. A9 ist nun über `evidence_references_complete` und `evidence_well_formed` operational geschlossen: Jede Evidence-ID muss auf eine aktuelle oder historische Objektversion auflösen; Supported und Validated verlangen zusätzlich eine nichtleere Evidenzmenge.

`unresolved_evidence_reference_rejected` und `A9_upgraded_evidence_resolves` sichern beide Richtungen als kernelgeprüfte Lemmata ab. Die Evidenzbedingung ist Bestandteil von `state_well_formed` und der Describe-Komponente `relation_complete`.

Ein struktureller Preflight prüfte Theoriehülle, Klammern, die vollständige Axiomnummerierung, zentrale Definitionsnamen, die sechs Sentinels, beide A9-Lemmata und die Sessiondatei. Zusätzlich verarbeitete der integrierte Isabelle2025-2-Clean-Build die Haupttheorie vollständig. Der Build endete mit Exit-Code 0; der Hauptquelltext ist mit SHA-256 `21dcfb175d6e754adf20042b941cf4a23329831d0165ebaf2023b0048213ed24` bezeichnet. Dieser Build allein ist noch kein Beweis der Locale-Annahmen aus einer schwächeren Theorie; konkrete Erfüllbarkeit wird durch die nachfolgenden Modellzeugen belegt.

```
theory UZA_0_9_1
  imports Complex_Main
begin

section \<open>Primitive identifiers and enumerations\<close>

type_synonym object_id = nat
type_synonym version_id = nat
type_synonym perspective_id = nat
type_synonym context_id = nat
type_synonym model_id = nat
type_synonym grammar_id = nat
type_synonym authority_id = nat
type_synonym channel_id = nat
type_synonym coupling_id = nat
type_synonym effect_id = nat
type_synonym patch_id = nat
type_synonym operation_id = nat
type_synonym time_point = nat

datatype object_kind =
  K_ENT | K_REL | K_EFF | K_RES | K_REV | K_ORI | K_ACT | K_FDB | K_EMG
  | K_PERSP | K_CTX | K_META | K_CONFLICT
```

```

datatype lifecycle_status =
    Draft | Hypothesis | Supported | Validated | Revised | Rejected | Archived

datatype emergence_level = E0 | E1 | E2 | E3

datatype description_status = DescriptionComplete | DescriptionIncomplete

datatype analysis_status =
    Applicable | InsufficientBasis | ModelFalsified | OutOfScope
    | PatchRejected | CoreConflict | UnresolvedE3

datatype operation =
    RegisterAppearance | DifferentiatePotential | CreateRelation
    | HypothesizeEffect | IntegrateEvidence | ClassifyResonance
    | EvaluateRelevance | GenerateOrientation | SelectAction | ExecuteAction
    | RecordFeedback | AnalyzeEmergence | ReviseObject
    | DefineCoupling | InitiateTransmission | ApplyBoundary

datatype trans_status =
    TransScheduled | TransDelivered | TransObserved | TransValidated
    | TransBlocked | TransIncompatible | TransFailed | TransCancelled

datatype boundary_kind = Fusion | Fission | Aggregation | Embedding | Detachment

datatype effect_dimension =
    Connection | Stability | Openness | Learning | Cooperation
    | Agency | Transparency | Freedom | Responsibility | Regeneration

datatype power_dimension =
    ResourcePower | AccessControl | DecisionAuthority | RiskBearing
    | CorrectionCapacity

datatype resonance_form =
    Amplification | Complement | ProductiveTension | Inhibition | Neutral

section \<open>Versioned objects and snapshots\<close>

record uza_object =
    object_id_of :: object_id
    version_id_of :: version_id
    kind_of :: object_kind
    lifecycle_of :: lifecycle_status
    previous_version_of :: "version_id option"
    references_of :: "version_id set"
    evidence_of :: "version_id set"
    perspective_of :: "perspective_id option"
    context_of :: "context_id option"
    model_of :: "model_id option"
    authority_of :: "authority_id option"
    valid_from_of :: time_point
    valid_to_of :: "time_point option"
    source_ref_of :: "version_id option"
    target_ref_of :: "version_id option"
    effect_origin_of :: "version_id option"
    basis_refs_of :: "version_id set"
    execution_ref_of :: "version_id option"
    observation_refs_of :: "version_id set"
    object_status_of :: "analysis_status option"

record loss_profile =
    structural_loss_of :: real
    semantic_loss_of :: real
    perspective_loss_of :: real
    temporal_loss_of :: real
    loss_explanation_of :: string

record projection =
    projection_id_of :: nat
    contained_context_of :: context_id
    container_context_of :: context_id
    source_snapshot_of :: nat
    projection_loss_of :: loss_profile

record coupling =
    coupling_id_of :: coupling_id
    coupling_source_of :: context_id
    coupling_target_of :: context_id
    coupling_channels_of :: "channel_id set"

```



```

record transmission =
  transmission_id_of :: nat
  transmission_coupling_of :: coupling_id
  transmission_channel_of :: channel_id
  transmission_source_effect_of :: effect_id
  transmission_target_effect_of :: "effect_id option"
  prior_trans_status_of :: "trans_status option"
  trans_status_of :: trans_status
  transmission_lineage_complete_of :: bool

record boundary_event =
  boundary_id_of :: nat
  boundary_kind_of :: boundary_kind
  boundary_succeeded_of :: bool
  boundary_lineage_complete_of :: bool
  boundary_disposition_complete_of :: bool

record snapshot =
  snapshot_id_of :: nat
  objects_of :: "uza_object set"
  historical_versions_of :: "version_id set"
  perspectives_of :: "perspective_id set"
  contexts_of :: "context_id set"
  models_of :: "model_id set"
  authorities_of :: "authority_id set"
  contains_edges_of :: "(context_id * context_id) set"
  projections_of :: "projection set"
  couplings_of :: "coupling set"
  transmissions_of :: "transmission set"
  boundaries_of :: "boundary_event set"

record grammar =
  grammar_id_of :: grammar_id
  supported_kinds_of :: "object_kind set"
  allowed_operations_of :: "operation set"
  patch_budget_of :: nat

record transition =
  transition_before_of :: snapshot
  transition_operation_of :: operation
  transition_after_of :: snapshot

record grammar_patch =
  patch_id_of :: patch_id
  added_kinds_of :: "object_kind set"
  added_operations_of :: "operation set"
  patch_cost_of :: nat
  changes_core_of :: bool
  holdout_passed_of :: bool
  replay_preserved_of :: bool
  migration_defined_of :: bool
  rollback_defined_of :: bool
  unresolved_conflicts_of :: nat

record interaction_complex =
  baseline_of :: snapshot
  current_of :: snapshot
  trace_of :: "transition list"
  candidate_patches_of :: "grammar_patch set"

record description_result =
  dr_status_of :: description_status
  dr_type_complete_of :: bool
  dr_relation_complete_of :: bool
  dr_transition_complete_of :: bool

section \<open>Object, time, reference, and state semantics\<close>

definition version_ids :: "snapshot \<Rightarrow> version_id set" where
  "version_ids s = version_id_of ` objects_of s"

definition unique_versions :: "snapshot \<Rightarrow> bool" where
  "unique_versions s \<longlefttrightarrow> inj_on version_id_of (objects_of s)"

definition present_ref :: "snapshot \<Rightarrow> version_id option \<Rightarrow> bool" where
  "present_ref s r \<longlefttrightarrow>
    (case r of None \<Rightarrow> False | Some v \<Rightarrow> v \<in> version_ids s \<union>
    historical_versions_of s)"
    
```

```

definition references_complete :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "references_complete s obj \<longlefttrightarrow>
    references_of obj \<subteq> version_ids s \<union> historical_versions_of s"

definition evidence_references_complete :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "evidence_references_complete s obj \<longlefttrightarrow>
    evidence_of obj \<subteq> version_ids s \<union> historical_versions_of s"

definition evidence_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "evidence_well_formed s obj \<longlefttrightarrow>
    evidence_references_complete s obj \<and>
    (lifecycle_of obj \<in> {Supported, Validated}) \<longrightarrow> evidence_of obj \<noteq> {})"

lemma unresolved_evidence_reference_rejected:
    assumes "v \<in> evidence_of obj"
    "v \<notin> version_ids s \<union> historical_versions_of s"
    shows "\<not> evidence_well_formed s obj"
    using assms
    unfolding evidence_well_formed_def evidence_references_complete_def
    by blast

definition time_well_formed :: "uza_object \<Rightarrow> bool" where
    "time_well_formed obj \<longlefttrightarrow>
    (case valid_to_of obj of None \<Rightarrow> True | Some t \<Rightarrow> valid_from_of obj \<le> t)"

definition relation_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "relation_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_REL \<longrightarrow>
    present_ref s (source_ref_of obj) \<and> present_ref s (target_ref_of obj))"

definition effect_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "effect_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_EFF \<and> lifecycle_of obj = Validated \<longrightarrow>
    present_ref s (effect_origin_of obj))"

definition perspective_bound_kind :: "object_kind \<Rightarrow> bool" where
    "perspective_bound_kind k \<longlefttrightarrow> k \<in> {K_ENT, K_EFF, K_RES, K_REV, K_ORI, K_ACT}"

definition context_bound_kind :: "object_kind \<Rightarrow> bool" where
    "context_bound_kind k \<longlefttrightarrow> k \<in> {K_ENT, K_REL, K_EFF, K_RES, K_REV, K_ORI, K_ACT,
    K_FDB}"

definition perspective_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "perspective_well_formed s obj \<longlefttrightarrow>
    (perspective_bound_kind (kind_of obj) \<longrightarrow>
    (case perspective_of obj of None \<Rightarrow> False | Some p \<Rightarrow> p \<in>
    perspectives_of s))"

definition context_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "context_well_formed s obj \<longlefttrightarrow>
    (context_bound_kind (kind_of obj) \<longrightarrow>
    (case context_of obj of None \<Rightarrow> False | Some c \<Rightarrow> c \<in> contexts_of s))"

definition relevance_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "relevance_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_REV \<longrightarrow>
    perspective_well_formed s obj \<and> context_well_formed s obj \<and>
    (case model_of obj of None \<Rightarrow> False | Some m \<Rightarrow> m \<in> models_of s) \<and>
    references_of obj \<noteq> {})"

definition orientation_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "orientation_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_ORI \<longrightarrow>
    basis_refs_of obj \<noteq> {} \<or> object_status_of obj = Some InsufficientBasis)"

definition action_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "action_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_ACT \<longrightarrow>
    basis_refs_of obj \<noteq> {} \<and>
    (case authority_of obj of None \<Rightarrow> False | Some a \<Rightarrow> a \<in> authorities_of
    s))"

definition feedback_well_formed :: "snapshot \<Rightarrow> uza_object \<Rightarrow> bool" where
    "feedback_well_formed s obj \<longlefttrightarrow>
    (kind_of obj = K_FDB \<longrightarrow>
    present_ref s (execution_ref_of obj) \<and> observation_refs_of obj \<noteq> {})"

definition loss_well_formed :: "loss_profile \<Rightarrow> bool" where
    "loss_well_formed l \<longlefttrightarrow>

```

```

0 \<le> structural_loss_of l \<and> structural_loss_of l \<le> 1 \<and>
0 \<le> semantic_loss_of l \<and> semantic_loss_of l \<le> 1 \<and>
0 \<le> perspective_loss_of l \<and> perspective_loss_of l \<le> 1 \<and>
0 \<le> temporal_loss_of l \<and> temporal_loss_of l \<le> 1"

definition projection_well_formed :: "snapshot \<Rightarrow> projection \<Rightarrow> bool" where
    "projection_well_formed s p \<longleftarrowtrightrightarrow>
    contained_context_of p \<in> contexts_of s \<and>
    container_context_of p \<in> contexts_of s \<and>
    loss_well_formed (projection_loss_of p)"

definition coupling_well_formed :: "snapshot \<Rightarrow> coupling \<Rightarrow> bool" where
    "coupling_well_formed s c \<longleftarrowtrightrightarrow>
    coupling_source_of c \<in> contexts_of s \<and>
    coupling_target_of c \<in> contexts_of s \<and>
    coupling_channels_of c \<noteq> {}"

fun allowed_trans_transition :: "trans_status option \<Rightarrow> trans_status \<Rightarrow> bool"
where
    "allowed_trans_transition None TransScheduled = True"
| "allowed_trans_transition (Some TransScheduled) TransDelivered = True"
| "allowed_trans_transition (Some TransScheduled) TransBlocked = True"
| "allowed_trans_transition (Some TransScheduled) TransIncompatible = True"
| "allowed_trans_transition (Some TransScheduled) TransCancelled = True"
| "allowed_trans_transition (Some TransDelivered) TransObserved = True"
| "allowed_trans_transition (Some TransDelivered) TransFailed = True"
| "allowed_trans_transition (Some TransObserved) TransValidated = True"
| "allowed_trans_transition (Some TransObserved) TransFailed = True"
| "allowed_trans_transition _ _ = False"

definition transmission_well_formed :: "snapshot \<Rightarrow> transmission \<Rightarrow> bool" where
    "transmission_well_formed s t \<longleftarrowtrightrightarrow>
    (\<exists>c\<in>couplings_of s.
    coupling_id_of c = transmission_coupling_of t \<and>
    transmission_channel_of t \<in> coupling_channels_of c) \<and>
    allowed_trans_transition (prior_trans_status_of t) (trans_status_of t) \<and>
    (trans_status_of t = TransValidated \<longrightrightarrow>
    transmission_target_effect_of t \<noteq> None \<and> transmission_lineage_complete_of t)"

definition boundary_well_formed :: "boundary_event \<Rightarrow> bool" where
    "boundary_well_formed b \<longleftarrowtrightrightarrow>
    (boundary_succeeded_of b \<longrightrightarrow>
    (case boundary_kind_of b of
    Fusion \<Rightarrow> boundary_disposition_complete_of b
    | Fission \<Rightarrow> boundary_lineage_complete_of b
    | Aggregation \<Rightarrow> boundary_lineage_complete_of b
    | Embedding \<Rightarrow> boundary_lineage_complete_of b
    | Detachment \<Rightarrow> boundary_lineage_complete_of b))"

definition state_well_formed :: "grammar \<Rightarrow> snapshot \<Rightarrow> bool" where
    "state_well_formed G s \<longleftarrowtrightrightarrow>
    unique_versions s \<and>
    (\<forall>obj\<in>objects_of s.
    kind_of obj \<in> supported_kinds_of G \<and>
    references_complete s obj \<and> evidence_well_formed s obj \<and>
    time_well_formed obj \<and>
    relation_well_formed s obj \<and> effect_well_formed s obj \<and>
    perspective_well_formed s obj \<and> context_well_formed s obj \<and>
    relevance_well_formed s obj \<and> orientation_well_formed s obj \<and>
    action_well_formed s obj \<and> feedback_well_formed s obj) \<and>
    (\<forall>p\<in>projections_of s. projection_well_formed s p) \<and>
    (\<forall>c\<in>couplings_of s. coupling_well_formed s c) \<and>
    (\<forall>t\<in>transmissions_of s. transmission_well_formed s t) \<and>
    (\<forall>b\<in>boundaries_of s. boundary_well_formed b)"

definition immutable_preserved :: "snapshot \<Rightarrow> snapshot \<Rightarrow> bool" where
    "immutable_preserved s s' \<longleftarrowtrightrightarrow>
    (\<forall>obj\<in>objects_of s. \<forall>obj'\<in>objects_of s'.
    version_id_of obj = version_id_of obj' \<longrightrightarrow> obj = obj')"

definition step :: "grammar \<Rightarrow> snapshot \<Rightarrow> operation \<Rightarrow> snapshot
\<Rightarrow> bool" where
    "step G s oper s' \<longleftarrowtrightrightarrow>
    state_well_formed G s \<and> state_well_formed G s' \<and>
    oper \<in> allowed_operations_of G \<and> immutable_preserved s s'"

definition revision_edges_of :: "snapshot \<Rightarrow> (version_id * version_id) set" where
    "revision_edges_of s =
    {(version_id_of obj, p) | obj p. obj \<in> objects_of s \<and> previous_version_of obj = Some p}"

```

```

definition strict_contains :: "snapshot \<Rightarrow> context_id \<Rightarrow> context_id \<Rightarrow>
bool" where
    "strict_contains s a b \<longleftarrow> (a,b) \<in> trancl (contains_edges_of s)"

definition contains_eq :: "snapshot \<Rightarrow> context_id \<Rightarrow> context_id \<Rightarrow>
bool" where
    "contains_eq s a b \<longleftarrow> (a = b \<or> strict_contains s a b)"

section \<open>V10, resonance, relevance, power, and dynamics reference semantics\<close>

type_synonym effect_vector = "effect_dimension \<Rightarrow> int"
type_synonym relevance_weights = "effect_dimension \<Rightarrow> real"
type_synonym power_profile = "power_dimension \<Rightarrow> real"

definition effect_vector_valid :: "effect_vector \<Rightarrow> bool" where
    "effect_vector_valid v \<longleftarrow> (\<forall>d. -3 \<le> v d \<and> v d \<le> 3)"

definition v10_dot :: "effect_vector \<Rightarrow> effect_vector \<Rightarrow> int" where
    "v10_dot a b = sum (\<lambda>d. a d * b d) UNIV"

definition resonance_score :: "effect_vector \<Rightarrow> effect_vector \<Rightarrow> real" where
    "resonance_score a b = of_int (v10_dot a b) / 90"

definition classify_resonance :: "effect_vector \<Rightarrow> effect_vector \<Rightarrow>
resonance_form" where
    "classify_resonance a b =
        (let r = resonance_score a b in
         if r \<ge> 0.5 then Amplification
         else if r \<ge> 0.15 then Complement
         else if r \<le> -0.5 then Inhibition
         else if r \<le> -0.15 then ProductiveTension
         else Neutral)"

definition clamp01 :: "real \<Rightarrow> real" where
    "clamp01 x = max 0 (min 1 x)"

definition weight_mass :: "relevance_weights \<Rightarrow> real" where
    "weight_mass w = sum (\<lambda>d. abs (w d)) UNIV"

definition normalized_effect_component :: "effect_vector \<Rightarrow> effect_dimension \<Rightarrow>
real" where
    "normalized_effect_component v d = (of_int (v d) + 3) / 6"

definition relevance_weights_valid :: "relevance_weights \<Rightarrow> bool" where
    "relevance_weights_valid w \<longleftarrow> (\<forall>d. 0 \<le> w d) \<and> weight_mass w > 0"

definition relevance_score :: "relevance_weights \<Rightarrow> effect_vector \<Rightarrow> real
\<Rightarrow> real" where
    "relevance_score w v uncertainty =
        clamp01
        ((if weight_mass w = 0 then 0.5
          else sum (\<lambda>d. w d * normalized_effect_component v d) UNIV / weight_mass w)
         * (1 - clamp01 uncertainty))"

definition power_profile_valid :: "power_profile \<Rightarrow> bool" where
    "power_profile_valid p \<longleftarrow> (\<forall>d. 0 \<le> p d \<and> p d \<le> 1)"

definition power_shift :: "power_profile \<Rightarrow> power_profile \<Rightarrow> power_profile" where
    "power_shift before after = (\<lambda>d. after d - before d)"

definition applicability_of :: "bool \<Rightarrow> bool \<Rightarrow> real \<Rightarrow> real
\<Rightarrow> analysis_status" where
    "applicability_of in_scope sufficient_basis observed_error falsification_limit =
        (if \<not> in_scope then OutOfScope
         else if \<not> sufficient_basis then InsufficientBasis
         else if observed_error > falsification_limit then ModelFalsified
         else Applicable)"

fun successive_deltas :: "real list \<Rightarrow> real list" where
    "successive_deltas [] = []"
  | "successive_deltas [x] = []"
  | "successive_deltas (x # y # xs) = abs (y - x) # successive_deltas (y # xs)"

fun nonincreasing :: "real list \<Rightarrow> bool" where
    "nonincreasing [] = True"
  | "nonincreasing [x] = True"
  | "nonincreasing (x # y # xs) = (y \<le> x \<and> nonincreasing (y # xs))"
    
```

```

fun nondecreasing :: "real list \ $\rightarrow$  bool" where
  "nondecreasing [] = True"
| "nondecreasing [x] = True"
| "nondecreasing (x # y # xs) = (x \ $\leq$  y \ $\wedge$  nondecreasing (y # xs))"

definition stabilizing :: "real list \ $\rightarrow$  bool" where
  "stabilizing xs \ $\longleftrightarrow$  length xs \ $\geq$  3 \ $\wedge$  nonincreasing (successive_deltas xs)"

definition destabilizing :: "real list \ $\rightarrow$  bool" where
  "destabilizing xs \ $\longleftrightarrow$  length xs \ $\geq$  3 \ $\wedge$  nondecreasing (successive_deltas xs)"

definition converging :: "real list \ $\rightarrow$  real \ $\rightarrow$  bool" where
  "converging distances epsilon \ $\longleftrightarrow$ 
    distances \ $\neq$  [] \ $\wedge$  nonincreasing distances \ $\wedge$  last distances \ $\leq$  epsilon"

definition diverging :: "real list \ $\rightarrow$  real \ $\rightarrow$  bool" where
  "diverging distances epsilon \ $\longleftrightarrow$ 
    distances \ $\neq$  [] \ $\wedge$  nondecreasing distances \ $\wedge$  last distances \ $\geq$  epsilon"

definition drifting :: "real list \ $\rightarrow$  real \ $\rightarrow$  bool" where
  "drifting xs epsilon \ $\longleftrightarrow$ 
    length xs \ $\geq$  2 \ $\wedge$  abs (last xs - hd xs) \ $\geq$  epsilon \ $\wedge$ 
    (\forall d \in set (successive_deltas xs). d < abs (last xs - hd xs))"

section \ $\langle$ Describe, patch, and emergence semantics\ $\rangle$ 

definition type_complete :: "grammar \ $\rightarrow$  interaction_complex \ $\rightarrow$  bool" where
  "type_complete G I \ $\longleftrightarrow$ 
    (\forall obj \in objects_of (current_of I). kind_of obj \in supported_kinds_of G)"

definition relation_complete :: "grammar \ $\rightarrow$  interaction_complex \ $\rightarrow$  bool" where
  "relation_complete G I \ $\longleftrightarrow$ 
    (\forall obj \in objects_of (current_of I).
      references_complete (current_of I) obj \ $\wedge$ 
      evidence_well_formed (current_of I) obj \ $\wedge$ 
      relation_well_formed (current_of I) obj \ $\wedge$ 
      effect_well_formed (current_of I) obj)"

definition transition_complete :: "grammar \ $\rightarrow$  interaction_complex \ $\rightarrow$  bool" where
  "transition_complete G I \ $\longleftrightarrow$ 
    (\forall tr \in set (trace_of I).
      step G (transition_before_of tr) (transition_operation_of tr)
      (transition_after_of tr))"

definition describeable :: "grammar \ $\rightarrow$  interaction_complex \ $\rightarrow$  bool" where
  "describeable G I \ $\longleftrightarrow$ 
    type_complete G I \ $\wedge$  relation_complete G I \ $\wedge$  transition_complete G I"

definition describe :: "grammar \ $\rightarrow$  interaction_complex \ $\rightarrow$  description_result"
where
  "describe G I =
    \lparr dr_status_of =
      (if describeable G I then DescriptionComplete else DescriptionIncomplete),
      dr_type_complete_of = type_complete G I,
      dr_relation_complete_of = relation_complete G I,
      dr_transition_complete_of = transition_complete G I \rparr"

definition relation_shape :: "snapshot \ $\rightarrow$  (object_kind * object_kind) set" where
  "relation_shape s =
    {(kind_of a, kind_of b) | r a b.
      r \in objects_of s \ $\wedge$  kind_of r = K_REL \ $\wedge$ 
      a \in objects_of s \ $\wedge$  b \in objects_of s \ $\wedge$ 
      source_ref_of r = Some (version_id_of a) \ $\wedge$ 
      target_ref_of r = Some (version_id_of b)}"

definition structural_novelty :: "interaction_complex \ $\rightarrow$  bool" where
  "structural_novelty I \ $\longleftrightarrow$ 
    relation_shape (current_of I) - relation_shape (baseline_of I) \ $\neq$  {} \or
    kind_of ` objects_of (current_of I) - kind_of ` objects_of (baseline_of I) \ $\neq$  {}"

definition patch_admissible :: "grammar \ $\rightarrow$  grammar_patch \ $\rightarrow$  bool" where
  "patch_admissible G p \ $\longleftrightarrow$ 
    \not changes_core_of p \ $\wedge$  patch_cost_of p \ $\leq$  patch_budget_of G \ $\wedge$ 
    holdout_passed_of p \ $\wedge$  replay_preserved_of p \ $\wedge$ 
    migration_defined_of p \ $\wedge$  rollback_defined_of p \ $\wedge$ 
    unresolved_conflicts_of p = 0"

definition patch_status :: "grammar \ $\rightarrow$  grammar_patch \ $\rightarrow$  analysis_status" where
  "patch_status G p =

```

```

        (if changes_core_of p then CoreConflict
        else if patch_admissible G p then Applicable
        else PatchRejected)"

definition apply_patch :: "grammar \<Rightarrow> grammar_patch \<Rightarrow> grammar" where
    "apply_patch G p =
        G\<lparr> supported_kinds_of := supported_kinds_of G \<union> added_kinds_of p,
        allowed_operations_of := allowed_operations_of G \<union> added_operations_of p \<rparr>"

definition repairable :: "grammar \<Rightarrow> interaction_complex \<Rightarrow> bool" where
    "repairable G I \<longlefttrightarrow>
        (\<exists>p\<in>candidate_patches_of I.
        patch_admissible G p \<and> describeable (apply_patch G p) I)"

definition emergence_of :: "grammar \<Rightarrow> interaction_complex \<Rightarrow> emergence_level"
where
    "emergence_of G I =
        (if describeable G I then
            (if structural_novelty I then E1 else E0)
            else if repairable G I then E2 else E3)"

section \<open>A1-A22 as a locale for models of UZA 0.9.1\<close>

locale uza_model =
    fixes States :: "snapshot set"
    and G :: grammar
    and classified_emergence :: "interaction_complex \<Rightarrow> emergence_level"
    assumes A1_version_identity:
        "\<And>s o1 o2. \<lbbr>s \<in> States; o1 \<in> objects_of s; o2 \<in> objects_of s;
        version_id_of o1 = version_id_of o2\<rbbr> \<Longrightarrow> o1 = o2"
    and A2_revision_well_founded:
        "\<And>s. s \<in> States \<Longrightarrow> wf (revision_edges_of s)"
    and A3_time_well_formed:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    time_well_formed obj"
    and A4_reference_integrity:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    references_complete s obj"
    and A5_relation_endpoints:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    relation_well_formed s obj"
    and A6_effect_origin:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    effect_well_formed s obj"
    and A7_perspective_binding:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    perspective_well_formed s obj"
    and A8_context_binding:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    context_well_formed s obj"
    and A9_evidence_integrity:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr>
        \<Longrightarrow> evidence_well_formed s obj"
    and A10_relevance_binding:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    relevance_well_formed s obj"
    and A11_orientation_basis:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    orientation_well_formed s obj"
    and A12_action_authority:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    action_well_formed s obj"
    and A13_feedback_origin:
        "\<And>s obj. \<lbbr>s \<in> States; obj \<in> objects_of s\<rbbr> \<Longrightarrow>
    feedback_well_formed s obj"
    and A14_step_preserves_well_formedness:
        "\<And>s oper s'. \<lbbr>s \<in> States; step G s oper s'\<rbbr> \<Longrightarrow>
    state_well_formed G s'"
    and A15_strict_contains_irreflexive:
        "\<And>s c. s \<in> States \<Longrightarrow> \<not> strict_contains s c c"
    and A16_contains_eq_antisymmetric:
        "\<And>s a b. \<lbbr>s \<in> States; contains_eq s a b; contains_eq s b a\<rbbr>
        \<Longrightarrow> a = b"
    and A17_projection_integrity:
        "\<And>s p. \<lbbr>s \<in> States; p \<in> projections_of s\<rbbr> \<Longrightarrow>
    projection_well_formed s p"
    and A18_coupling_integrity:
        "\<And>s c. \<lbbr>s \<in> States; c \<in> couplings_of s\<rbbr> \<Longrightarrow>
    coupling_well_formed s c"

```

```

    and A19_transmission_lifecycle:
        "\<And>s t. \<lbrakk>s \<in> States; t \<in> transmissions_of s\<rbrakk> \<Longrightarrow>
transmission_well_formed s t"
    and A20_external_effect_lineage:
        "\<And>s t. \<lbrakk>s \<in> States; t \<in> transmissions_of s; trans_status_of t =
TransValidated\<rbrakk>
        \<Longrightarrow> transmission_target_effect_of t \<noteq> None \<and>
transmission_lineage_complete_of t"
    and A21_boundary_completeness:
        "\<And>s b. \<lbrakk>s \<in> States; b \<in> boundaries_of s\<rbrakk> \<Longrightarrow>
boundary_well_formed b"
    and A22_emergence_definition:
        "\<And>I. current_of I \<in> States \<Longrightarrow> classified_emergence I = emergence_of G I"
begin

theorem A14_state_result:
    assumes "s \<in> States" and "step G s oper s'"
    shows "state_well_formed G s'"
    using assms A14_step_preserves_well_formedness by blast

theorem A20_validated_transmission_has_lineage:
    assumes "s \<in> States" "t \<in> transmissions_of s" "trans_status_of t = TransValidated"
    shows "transmission_target_effect_of t \<noteq> None \<and> transmission_lineage_complete_of t"
    using assms A20_external_effect_lineage by blast

lemma A9_upgraded_evidence_resolves:
    assumes "s \<in> States" "obj \<in> objects_of s"
    "lifecycle_of obj \<in> {Supported, Validated}"
    shows
        "evidence_of obj \<noteq> {} \<and>
        evidence_of obj \<subseq> version_ids s \<union> historical_versions_of s"
proof -
    have "evidence_well_formed s obj"
    using assms(1,2) A9_evidence_integrity by blast
    then show ?thesis
    using assms(3)
    unfolding evidence_well_formed_def evidence_references_complete_def
    by blast
qed

end

theorem describe_status_iff:
    "dr_status_of (describe G I) = DescriptionComplete \<longleftarrow> describeable G I"
    by (simp add: describe_def)

theorem emergence_total:
    "emergence_of G I \<in> {E0, E1, E2, E3}"
    by (auto simp: emergence_of_def)

theorem core_change_rejected:
    assumes "changes_core_of p"
    shows "\<not> patch_admissible G p \<and> patch_status G p = CoreConflict"
    using assms by (simp add: patch_admissible_def patch_status_def)

theorem relevance_bounded:
    "0 \<le> relevance_score w v uncertainty \<and> relevance_score w v uncertainty \<le> 1"
    unfolding relevance_score_def clamp01_def
    by auto

end
    
```

D.2 Expliziter minimaler Basismodellzeuge

Die zweite Theorie UZA_0_9_1_Base_Model konstruiert `base_object`, `base_snapshot`, `base_grammar` und die nichtleere Zustandsmenge `base_states`. Der Zustand enthält ein versioniertes Metaobjekt, erfüllt einschließlich A9 `state_well_formed` und besitzt mit `RegisterAppearance` einen zulässigen Selbstübergang. Die Interpretation `Base: uza_model` weist sämtliche Locale-Annahmen A1–A22 für diese konkrete Struktur nach. Das abschließende Theorem `explicit_uza_model_exists` beweist die Existenz eines nichtleeren `uza_model` mit mindestens einem Objekt. Dieser Zeuge ist ausdrücklich minimal: Da sein einziges Objekt

K_META und Draft ist und die Erweiterungsmengen leer sind, werden die meisten objekttypspezifischen und Interaktionsbedingungen noch vakuos erfüllt.

Der integrierte Isabelle2025-2-Clean-Build verarbeitete die Basismodelltheorie vollständig und endete mit Exit-Code 0. Der aktuelle, auf lokalen Theorieimport umgestellte Modell Quelltext besitzt SHA-256 322d2a8fc31dbca1ea6c7cc2463b24c891701a79eebb761a94f5256a62632a4b. Damit ist die gemeinsame Erfüllbarkeit von A1–A22 relativ zu Isabelle/HOL mechanisch belegt. Der Zeuge selbst bleibt absichtlich minimal; seine frühere Nichtvakuositätsgrenze wird durch den zusätzlichen Rich-Model-Zeugen in D.4 adressiert. Nicht daraus folgt die Ableitung von A1–A22 aus einer schwächeren Theorie.

```
theory UZA_0_9_1_Base_Model
  imports UZA_0_9_1
begin

section \<open>An explicit nonempty witness model\<close>

text \<open>
  This theory gives a concrete, finite interpretation of the locale
  @{locale uza_model}. The witness is deliberately minimal: it contains one
  state, one versioned meta-object, and one permitted self-transition. It is a
  satisfiability witness, while most kind-specific obligations remain vacuous.
  Thus locale satisfiability is no longer merely presumed.
\<close>

definition base_object :: uza_object where
  "base_object =
    \<lparr> object_id_of = 0,
      version_id_of = 0,
      kind_of = K_META,
      lifecycle_of = Draft,
      previous_version_of = None,
      references_of = {},
      evidence_of = {},
      perspective_of = None,
      context_of = None,
      model_of = None,
      authority_of = None,
      valid_from_of = 0,
      valid_to_of = None,
      source_ref_of = None,
      target_ref_of = None,
      effect_origin_of = None,
      basis_refs_of = {},
      execution_ref_of = None,
      observation_refs_of = {},
      object_status_of = None \<rparr>"

definition base_snapshot :: snapshot where
  "base_snapshot =
    \<lparr> snapshot_id_of = 0,
      objects_of = {base_object},
      historical_versions_of = {},
      perspectives_of = {},
      contexts_of = {},
      models_of = {},
      authorities_of = {},
      contains_edges_of = {},
      projections_of = {},
      couplings_of = {},
      transmissions_of = {},
      boundaries_of = {} \<rparr>"

definition base_grammar :: grammar where
  "base_grammar =
    \<lparr> grammar_id_of = 0,
      supported_kinds_of = {K_META},
      allowed_operations_of = {RegisterAppearance},
      patch_budget_of = 0 \<rparr>"

definition base_states :: "snapshot set" where
  "base_states = {base_snapshot}"
```



```

definition base_classified_emergence ::
  "interaction_complex \ $\rightarrow$  emergence_level" where
  "base_classified_emergence I = emergence_of base_grammar I"

lemma base_object_simps [simp]:
  "object_id_of base_object = 0"
  "version_id_of base_object = 0"
  "kind_of base_object = K_META"
  "lifecycle_of base_object = Draft"
  "previous_version_of base_object = None"
  "references_of base_object = {}"
  "evidence_of base_object = {}"
  "perspective_of base_object = None"
  "context_of base_object = None"
  "model_of base_object = None"
  "authority_of base_object = None"
  "valid_from_of base_object = 0"
  "valid_to_of base_object = None"
  "source_ref_of base_object = None"
  "target_ref_of base_object = None"
  "effect_origin_of base_object = None"
  "basis_refs_of base_object = {}"
  "execution_ref_of base_object = None"
  "observation_refs_of base_object = {}"
  "object_status_of base_object = None"
  by (simp_all add: base_object_def)

lemma base_snapshot_simps [simp]:
  "snapshot_id_of base_snapshot = 0"
  "objects_of base_snapshot = {base_object}"
  "historical_versions_of base_snapshot = {}"
  "perspectives_of base_snapshot = {}"
  "contexts_of base_snapshot = {}"
  "models_of base_snapshot = {}"
  "authorities_of base_snapshot = {}"
  "contains_edges_of base_snapshot = {}"
  "projections_of base_snapshot = {}"
  "couplings_of base_snapshot = {}"
  "transmissions_of base_snapshot = {}"
  "boundaries_of base_snapshot = {}"
  by (simp_all add: base_snapshot_def)

lemma base_grammar_simps [simp]:
  "grammar_id_of base_grammar = 0"
  "supported_kinds_of base_grammar = {K_META}"
  "allowed_operations_of base_grammar = {RegisterAppearance}"
  "patch_budget_of base_grammar = 0"
  by (simp_all add: base_grammar_def)

lemma base_states_nonempty:
  "base_states \ $\neq$  {}"
  by (simp add: base_states_def)

lemma base_objects_nonempty:
  "objects_of base_snapshot \ $\neq$  {}"
  by (simp add: base_snapshot_def)

lemma base_state_well_formed:
  "state_well_formed base_grammar base_snapshot"
  unfolding state_well_formed_def
  by (auto simp: unique_versions_def version_ids_def
    references_complete_def evidence_well_formed_def
    evidence_references_complete_def time_well_formed_def relation_well_formed_def
    effect_well_formed_def perspective_well_formed_def perspective_bound_kind_def
    context_well_formed_def context_bound_kind_def relevance_well_formed_def
    orientation_well_formed_def action_well_formed_def feedback_well_formed_def
    base_object_def base_snapshot_def base_grammar_def)

lemma base_self_step:
  "step base_grammar base_snapshot RegisterAppearance base_snapshot"
  unfolding step_def
  by (simp add: immutable_preserved_def base_state_well_formed)

interpretation Base: uza_model
  base_states base_grammar base_classified_emergence
proof
  fix s o1 o2
  assume "s \ $\in$  base_states" "o1 \ $\in$  objects_of s" "o2 \ $\in$  objects_of s"

```

```

"version_id_of o1 = version_id_of o2"
then show "o1 = o2"
  by (simp add: base_states_def base_snapshot_def)
next
fix s
  assume "s \<in> base_states"
  then show "wf (revision_edges_of s)"
    by (simp add: base_states_def revision_edges_of_def base_snapshot_def
      base_object_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "time_well_formed obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      time_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "references_complete s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      references_complete_def version_ids_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "relation_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      relation_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "effect_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      effect_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "perspective_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      perspective_well_formed_def perspective_bound_kind_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "context_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      context_well_formed_def context_bound_kind_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "evidence_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      evidence_well_formed_def evidence_references_complete_def version_ids_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "relevance_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      relevance_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "orientation_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      orientation_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "action_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      action_well_formed_def)
next
fix s obj
  assume "s \<in> base_states" "obj \<in> objects_of s"
  then show "feedback_well_formed s obj"
    by (simp add: base_states_def base_snapshot_def base_object_def
      feedback_well_formed_def)
next
fix s oper s'
  assume "s \<in> base_states" "step base_grammar s oper s'"
  then show "state_well_formed base_grammar s'"

```

```

        by (simp add: step_def)
    next
        fix s c
        assume "s \<in> base_states"
        then show "\<not> strict_contains s c c"
        by (simp add: base_states_def strict_contains_def base_snapshot_def)
    next
        fix s a b
        assume "s \<in> base_states" "contains_eq s a b" "contains_eq s b a"
        then show "a = b"
        by (simp add: base_states_def contains_eq_def strict_contains_def
            base_snapshot_def)
    next
        fix s p
        assume "s \<in> base_states" "p \<in> projections_of s"
        then show "projection_well_formed s p"
        by (simp add: base_states_def base_snapshot_def)
    next
        fix s c
        assume "s \<in> base_states" "c \<in> couplings_of s"
        then show "coupling_well_formed s c"
        by (simp add: base_states_def base_snapshot_def)
    next
        fix s t
        assume "s \<in> base_states" "t \<in> transmissions_of s"
        then show "transmission_well_formed s t"
        by (simp add: base_states_def base_snapshot_def)
    next
        fix s t
        assume "s \<in> base_states" "t \<in> transmissions_of s"
        "trans_status_of t = TransValidated"
        then show "transmission_target_effect_of t \<noteq> None \&and>
            transmission_lineage_complete_of t"
        by (simp add: base_states_def base_snapshot_def)
    next
        fix s b
        assume "s \<in> base_states" "b \<in> boundaries_of s"
        then show "boundary_well_formed b"
        by (simp add: base_states_def base_snapshot_def)
    next
        fix I :: interaction_complex
        assume "current_of I \<in> base_states"
        show "base_classified_emergence I = emergence_of base_grammar I"
        by (simp add: base_classified_emergence_def)
qed

theorem explicit_uza_model_exists:
  "\<exists>States G classify.
    States \<noteq> {} \&and>
    (\<exists>s\<in>States. objects_of s \<noteq> {})) \&and>
    uza_model States G classify"
  using base_states_nonempty base_objects_nonempty Base.uza_model_axioms
  by (intro exI[of _ base_states] exI[of _ base_grammar]
    exI[of _ base_classified_emergence])
    (auto simp: base_states_def)

end

```

D.3 Konkrete T1-Semantik und Erhaltung für RegisterAppearance

Die dritte Theorie UZA_0_9_1_Register_Appearance schließt erstmals die Lücke zwischen dem generischen Operationsnamen und einer konstruktiven Zustandssemantik.

appearance_input_valid verlangt einen wohlgeformten Ausgangszustand, eine zulässige Grammatikoperation, den unterstützten Typ K_ENT, frische Objekt- und Versionsidentitäten, vollständige allgemeine Referenzen, evidence_well_formed, gültige Zeit sowie vorhandene Perspektiv- und Kontextbindungen. Eine registrierte Beobachtung besitzt keine Potentialquelle, keine Potentialbasis und keinen Analysemarker; dadurch bleibt sie von den in D.5 definierten T2-Kandidaten unterscheidbar. add_appearance erzeugt den Folgezustand durch genau eine Objektinsertion und eine fortgeschriebene Snapshot-ID. register_appearance_step verbindet

Vorbedingung und Konstruktion; die Wohlgeformtheit des Zielzustands ist ausdrücklich **keine** Vorbedingung.

Isabelle beweist anschließend die exakte Zustandsdifferenz, die Eindeutigkeit der Versionsidentitäten, die Unveränderlichkeit sämtlicher bereits vorhandener Versionen, die Erhaltung von Referenzen, Evidence Integrity, Zeit, Perspektive und Kontext, die vollständige `state_well_formed`-Eigenschaft des konstruierten Folgezustands und schließlich `register_appearance_refines_generic_step`. Das zusätzliche Theorem `register_appearance_preserves_evidence_integrity` weist A9 ausdrücklich für alle Objekte des Zielzustands nach. Der integrierte Isabelle2025-2-Clean-Build verarbeitete die T1-Theorie vollständig und endete mit Exit-Code 0. Der aktuelle Operations Quelltext besitzt SHA-256 `9b8cc6167b99070a47f63b4e63bd5a783a000f6ec84f7620945bee224f116cd0`.

Damit ist Aufgabe 4 für T1 bis T6 abgeschlossen. T5 besitzt einen kernelgeprüften nicht-vakuen Evidenzintegrationszeugen; T6 einen kernelgeprüften nicht-vakuen Richtungszeugen. T7–T13 bleiben offen. Ebenfalls nicht bewiesen sind Produktivengine-Äquivalenz, allgemeine Modell-Expansion, Konservativität oder semantische Angemessenheit in jeder Anwendungsdomäne.

```
theory UZA_0_9_1_Register_Appearance
  imports UZA_0_9_1
begin

section \<open>T1 RegisterAppearance: concrete operation semantics\<close>

text \<open>
  This theory gives the first core operation an exact, non-circular transition
  semantics. The source state is required to be well formed. The target state
  is constructed by adding exactly one fresh appearance object; target
  well-formedness is derived rather than assumed.
\<close>

definition appearance_input_valid ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> uza_object \<Rightarrow> nat \<Rightarrow> bool" where
  "appearance_input_valid G s new_obj next_snapshot_id \<longleftarrowrightarrow>
    state_well_formed G s \<and>
    RegisterAppearance \<in> allowed_operations_of G \<and>
    K_ENT \<in> supported_kinds_of G \<and>
    kind_of new_obj = K_ENT \<and>
    lifecycle_of new_obj \<in> {Draft, Hypothesis} \<and>
    object_id_of new_obj \<notin> object_id_of ` objects_of s \<and>
    version_id_of new_obj \<notin> version_ids s \<union> historical_versions_of s \<and>
    previous_version_of new_obj = None \<and>
    source_ref_of new_obj = None \<and>
    basis_refs_of new_obj = {} \<and>
    object_status_of new_obj = None \<and>
    references_complete s new_obj \<and>
    evidence_well_formed s new_obj \<and>
    time_well_formed new_obj \<and>
    (\<exists>p\<in>perspectives_of s. perspective_of new_obj = Some p) \<and>
    (\<exists>c\<in>contexts_of s. context_of new_obj = Some c) \<and>
    snapshot_id_of s < next_snapshot_id"

definition add_appearance ::
  "snapshot \<Rightarrow> uza_object \<Rightarrow> nat \<Rightarrow> snapshot" where
  "add_appearance s new_obj next_snapshot_id =
    s\<lparr> snapshot_id_of := next_snapshot_id,
    objects_of := insert new_obj (objects_of s) \<rparr>"

definition register_appearance_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> uza_object \<Rightarrow> nat \<Rightarrow> snapshot
  \<Rightarrow> bool" where
  "register_appearance_step G s new_obj next_snapshot_id s' \<longleftarrowrightarrow>
    appearance_input_valid G s new_obj next_snapshot_id \<and>
    s' = add_appearance s new_obj next_snapshot_id"

subsection \<open>Exact update and frame properties\<close>

lemma add_appearance_objects [simp]:
```

```

"objects_of (add_appearance s new_obj next_snapshot_id) =
  insert new_obj (objects_of s)"
by (simp add: add_appearance_def)

lemma add_appearance_snapshot_id [simp]:
  "snapshot_id_of (add_appearance s new_obj next_snapshot_id) = next_snapshot_id"
  by (simp add: add_appearance_def)

lemma add_appearance_background [simp]:
  "historical_versions_of (add_appearance s new_obj next_snapshot_id) =
    historical_versions_of s"
  "perspectives_of (add_appearance s new_obj next_snapshot_id) = perspectives_of s"
  "contexts_of (add_appearance s new_obj next_snapshot_id) = contexts_of s"
  "models_of (add_appearance s new_obj next_snapshot_id) = models_of s"
  "authorities_of (add_appearance s new_obj next_snapshot_id) = authorities_of s"
  "contains_edges_of (add_appearance s new_obj next_snapshot_id) = contains_edges_of s"
  "projections_of (add_appearance s new_obj next_snapshot_id) = projections_of s"
  "couplings_of (add_appearance s new_obj next_snapshot_id) = couplings_of s"
  "transmissions_of (add_appearance s new_obj next_snapshot_id) = transmissions_of s"
  "boundaries_of (add_appearance s new_obj next_snapshot_id) = boundaries_of s"
  by (simp_all add: add_appearance_def)

lemma add_appearance_version_ids [simp]:
  "version_ids (add_appearance s new_obj next_snapshot_id) =
    insert (version_id_of new_obj) (version_ids s)"
  by (auto simp: version_ids_def)

lemma appearance_input_fresh_object:
  assumes valid: "appearance_input_valid G s new_obj next_snapshot_id"
  shows "new_obj \<notin> objects_of s"
  using valid
  by (auto simp: appearance_input_valid_def)

lemma register_appearance_exact_delta:
  assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
  shows
    "objects_of s' = insert new_obj (objects_of s) \<and>
      new_obj \<notin> objects_of s \<and>
      snapshot_id_of s' = next_snapshot_id"
  using concrete appearance_input_fresh_object
  by (auto simp: register_appearance_step_def)

subsection \<open>Monotonicity of object conditions\<close>

lemma add_appearance_present_ref_mono:
  assumes "present_ref s ref"
  shows "present_ref (add_appearance s new_obj next_snapshot_id) ref"
  using assms
  by (cases ref) (auto simp: present_ref_def)

lemma add_appearance_references_mono:
  assumes "references_complete s obj"
  shows "references_complete (add_appearance s new_obj next_snapshot_id) obj"
  using assms
  by (auto simp: references_complete_def)

lemma add_appearance_evidence_mono:
  assumes "evidence_well_formed s obj"
  shows "evidence_well_formed (add_appearance s new_obj next_snapshot_id) obj"
  using assms
  by (auto simp: evidence_well_formed_def evidence_references_complete_def)

lemma add_appearance_relation_mono:
  assumes "relation_well_formed s obj"
  shows "relation_well_formed (add_appearance s new_obj next_snapshot_id) obj"
  using assms add_appearance_present_ref_mono
  unfolding relation_well_formed_def
  by blast

lemma add_appearance_effect_mono:
  assumes "effect_well_formed s obj"
  shows "effect_well_formed (add_appearance s new_obj next_snapshot_id) obj"
  using assms add_appearance_present_ref_mono
  unfolding effect_well_formed_def
  by blast

lemma add_appearance_perspective_iff [simp]:
  "perspective_well_formed (add_appearance s new_obj next_snapshot_id) obj
    \<longleftarrowrightarrow> perspective_well_formed s obj"

```

```

unfolding perspective_well_formed_def
by (simp only: add_appearance_background(2))

lemma add_appearance_context_iff [simp]:
  "context_well_formed (add_appearance s new_obj next_snapshot_id) obj
   \<longlefttrightarrow> context_well_formed s obj"
  unfolding context_well_formed_def
  by (simp only: add_appearance_background(3))

lemma add_appearance_relevance_iff [simp]:
  "relevance_well_formed (add_appearance s new_obj next_snapshot_id) obj
   \<longlefttrightarrow> relevance_well_formed s obj"
  unfolding relevance_well_formed_def
  by (simp only: add_appearance_perspective_iff add_appearance_context_iff
    add_appearance_background(4))

lemma add_appearance_orientation_iff [simp]:
  "orientation_well_formed (add_appearance s new_obj next_snapshot_id) obj
   \<longlefttrightarrow> orientation_well_formed s obj"
  by (simp add: orientation_well_formed_def)

lemma add_appearance_action_iff [simp]:
  "action_well_formed (add_appearance s new_obj next_snapshot_id) obj
   \<longlefttrightarrow> action_well_formed s obj"
  unfolding action_well_formed_def
  by (simp only: add_appearance_background(5))

lemma add_appearance_feedback_mono:
  assumes "feedback_well_formed s obj"
  shows "feedback_well_formed (add_appearance s new_obj next_snapshot_id) obj"
  using assms add_appearance_present_ref_mono
  unfolding feedback_well_formed_def
  by blast

subsection \<open>Fresh identity and new-object validity\<close>

lemma add_appearance_preserves_unique_versions:
  assumes valid: "appearance_input_valid G s new_obj next_snapshot_id"
  shows "unique_versions (add_appearance s new_obj next_snapshot_id)"
proof -
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
  and fresh: "version_id_of new_obj \<notin> version_ids s"
  unfolding appearance_input_valid_def state_well_formed_def unique_versions_def
  by auto
  show ?thesis
  unfolding unique_versions_def
proof (rule inj_onI)
  fix left right
  assume left_mem: "left \<in> objects_of (add_appearance s new_obj next_snapshot_id)"
  and right_mem: "right \<in> objects_of (add_appearance s new_obj next_snapshot_id)"
  and same_version: "version_id_of left = version_id_of right"
  show "left = right"
  proof (cases "left = new_obj")
    case True
    show ?thesis
    proof (cases "right = new_obj")
      case True
      then show ?thesis using \<open>left = new_obj\<close> by simp
    next
    case False
    then have right_old: "right \<in> objects_of s" using right_mem by simp
    have new_version_old: "version_id_of new_obj \<in> version_ids s"
    unfolding version_ids_def
    using same_version \<open>left = new_obj\<close> right_old
    by (metis imageI)
    have contradiction: False
    using fresh new_version_old by blast
    show ?thesis using contradiction by blast
  qed
next
  case False
  then have left_old: "left \<in> objects_of s" using left_mem by simp
  show ?thesis
  proof (cases "right = new_obj")
    case True
    have new_version_old: "version_id_of new_obj \<in> version_ids s"
    unfolding version_ids_def
    using same_version left_old True
    by (metis imageI)

```

```

        have contradiction: False
        using fresh new_version_old by blast
        show ?thesis using contradiction by blast
    next
        case False
        then have right_old: "right \<in> objects_of s" using right_mem by simp
        show ?thesis
            using inj_onD[OF source_unique same_version left_old right_old] .
qed
qed
qed
qed

lemma appearance_input_new_object_conditions:
  assumes valid: "appearance_input_valid G s new_obj next_snapshot_id"
  shows
    "kind_of new_obj \<in> supported_kinds_of G \<and>
    references_complete (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    evidence_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    time_well_formed new_obj \<and>
    relation_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    effect_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    perspective_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    context_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    relevance_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    orientation_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    action_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    feedback_well_formed (add_appearance s new_obj next_snapshot_id) new_obj"
proof -
  from valid have supported: "kind_of new_obj \<in> supported_kinds_of G"
  and kind: "kind_of new_obj = K_ENT"
  and refs: "references_complete s new_obj"
  and evidence: "evidence_well_formed s new_obj"
  and time: "time_well_formed new_obj"
  and perspective:
    "perspective_well_formed (add_appearance s new_obj next_snapshot_id) new_obj"
  and context_ok:
    "context_well_formed (add_appearance s new_obj next_snapshot_id) new_obj"
  unfolding appearance_input_valid_def perspective_well_formed_def
    context_well_formed_def perspective_bound_kind_def context_bound_kind_def
  by auto
  have refs_target:
    "references_complete (add_appearance s new_obj next_snapshot_id) new_obj"
  using add_appearance_references_mono[OF refs] .
  have evidence_target:
    "evidence_well_formed (add_appearance s new_obj next_snapshot_id) new_obj"
  using add_appearance_evidence_mono[OF evidence] .
  have typed_conditions:
    "relation_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    effect_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    relevance_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    orientation_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    action_well_formed (add_appearance s new_obj next_snapshot_id) new_obj \<and>
    feedback_well_formed (add_appearance s new_obj next_snapshot_id) new_obj"
  using kind
  by (simp add: relation_well_formed_def effect_well_formed_def
    relevance_well_formed_def orientation_well_formed_def
    action_well_formed_def feedback_well_formed_def)
  show ?thesis
    using supported refs_target evidence_target time perspective context_ok typed_conditions
    by blast
qed

subsection \<open>Operation-specific preservation theorems\<close>

theorem register_appearance_preserves_state_well_formed:
  assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
  shows "state_well_formed G s'"
proof -
  from concrete have valid: "appearance_input_valid G s new_obj next_snapshot_id"
  and target: "s' = add_appearance s new_obj next_snapshot_id"
  by (auto simp: register_appearance_step_def)
  from valid have source_wf: "state_well_formed G s"
  by (simp add: appearance_input_valid_def)
  have unique: "unique_versions (add_appearance s new_obj next_snapshot_id)"
  using add_appearance_preserves_unique_versions[OF valid] .
  have all_objects:
    "\<forall>obj\<in>objects_of (add_appearance s new_obj next_snapshot_id).
    kind_of obj \<in> supported_kinds_of G \<and>"

```

```

references_complete (add_appearance s new_obj next_snapshot_id) obj \<and>
evidence_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
time_well_formed obj \<and>
relation_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
effect_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
perspective_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
context_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
relevance_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
orientation_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
action_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
feedback_well_formed (add_appearance s new_obj next_snapshot_id) obj"
proof (intro ballI)
  fix obj
  assume obj_mem:
    "obj \<in> objects_of (add_appearance s new_obj next_snapshot_id)"
  consider (new) "obj = new_obj" | (old) "obj \<in> objects_of s"
  using obj_mem by auto
  then show
    "kind_of obj \<in> supported_kinds_of G \<and>
    references_complete (add_appearance s new_obj next_snapshot_id) obj \<and>
    evidence_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    time_well_formed obj \<and>
    relation_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    effect_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    perspective_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    context_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    relevance_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    orientation_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    action_well_formed (add_appearance s new_obj next_snapshot_id) obj \<and>
    feedback_well_formed (add_appearance s new_obj next_snapshot_id) obj"
  proof cases
    case new
    then show ?thesis
      using appearance_input_new_object_conditions[OF valid] by simp
  next
    case old
    from source_wf old have source_conditions:
      "kind_of obj \<in> supported_kinds_of G \<and>
      references_complete s obj \<and> evidence_well_formed s obj \<and>
      time_well_formed obj \<and>
      relation_well_formed s obj \<and> effect_well_formed s obj \<and>
      perspective_well_formed s obj \<and> context_well_formed s obj \<and>
      relevance_well_formed s obj \<and> orientation_well_formed s obj \<and>
      action_well_formed s obj \<and> feedback_well_formed s obj"
    unfolding state_well_formed_def by blast
    from source_conditions have supported: "kind_of obj \<in> supported_kinds_of G"
    and refs: "references_complete s obj"
    and evidence: "evidence_well_formed s obj"
    and time: "time_well_formed obj"
    and relation: "relation_well_formed s obj"
    and effect: "effect_well_formed s obj"
    and perspective: "perspective_well_formed s obj"
    and context_ok: "context_well_formed s obj"
    and relevance: "relevance_well_formed s obj"
    and orientation: "orientation_well_formed s obj"
    and action: "action_well_formed s obj"
    and feedback: "feedback_well_formed s obj"
    by blast+
    have refs_target:
      "references_complete (add_appearance s new_obj next_snapshot_id) obj"
      using add_appearance_references_mono[OF refs] .
    have evidence_target:
      "evidence_well_formed (add_appearance s new_obj next_snapshot_id) obj"
      using add_appearance_evidence_mono[OF evidence] .
    have relation_target:
      "relation_well_formed (add_appearance s new_obj next_snapshot_id) obj"
      using add_appearance_relation_mono[OF relation] .
    have effect_target:
      "effect_well_formed (add_appearance s new_obj next_snapshot_id) obj"
      using add_appearance_effect_mono[OF effect] .
    have feedback_target:
      "feedback_well_formed (add_appearance s new_obj next_snapshot_id) obj"
      using add_appearance_feedback_mono[OF feedback] .
    show ?thesis
      using supported_refs_target_evidence_target_time_relation_target_effect_target_perspective
        context_ok_relevance_orientation_action_feedback_target
      by simp
qed
qed

```



```

from source_wf have source_secondary:
  "(\<forall>p\<in>projections_of s. projection_well_formed s p) \<and>
   (\<forall>c\<in>couplings_of s. coupling_well_formed s c) \<and>
   (\<forall>t\<in>transmissions_of s. transmission_well_formed s t) \<and>
   (\<forall>b\<in>boundaries_of s. boundary_well_formed b)"
  unfolding state_well_formed_def by blast
have target_secondary:
  "(\<forall>p\<in>projections_of (add_appearance s new_obj next_snapshot_id).
   projection_well_formed (add_appearance s new_obj next_snapshot_id) p) \<and>
   (\<forall>c\<in>couplings_of (add_appearance s new_obj next_snapshot_id).
   coupling_well_formed (add_appearance s new_obj next_snapshot_id) c) \<and>
   (\<forall>t\<in>transmissions_of (add_appearance s new_obj next_snapshot_id).
   transmission_well_formed (add_appearance s new_obj next_snapshot_id) t) \<and>
   (\<forall>b\<in>boundaries_of (add_appearance s new_obj next_snapshot_id).
   boundary_well_formed b)"
  using source_secondary
  by (simp add: projection_well_formed_def coupling_well_formed_def
    transmission_well_formed_def)
show ?thesis
  unfolding target_state_well_formed_def
  using unique_all_objects target_secondary by blast
qed

theorem register_appearance_preserves_identity:
  assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
  shows "unique_versions s' \<and> immutable_preserved s s'"
proof -
  from concrete have valid: "appearance_input_valid G s new_obj next_snapshot_id"
  and target: "s' = add_appearance s new_obj next_snapshot_id"
  by (auto simp: register_appearance_step_def)
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
  and fresh: "version_id_of new_obj \<notin> version_ids s"
  unfolding appearance_input_valid_def state_well_formed_def unique_versions_def
  by auto
  have immutable: "immutable_preserved s s'"
  unfolding immutable_preserved_def target
  proof (intro ballI allI impI)
    fix old_obj target_obj
    assume old_mem: "old_obj \<in> objects_of s"
    and target_mem:
      "target_obj \<in> objects_of (add_appearance s new_obj next_snapshot_id)"
    and same_version: "version_id_of old_obj = version_id_of target_obj"
    show "old_obj = target_obj"
    proof (cases "target_obj = new_obj")
      case True
      have new_version_old: "version_id_of new_obj \<in> version_ids s"
      unfolding version_ids_def
      using old_mem same_version True
      by (metis imageI)
      have contradiction: False
      using fresh new_version_old by blast
      show ?thesis using contradiction by blast
    next
      case False
      then have target_old: "target_obj \<in> objects_of s" using target_mem by simp
      show ?thesis
        using inj_onD[OF source_unique same_version old_mem target_old] .
    qed
  qed
  have target_unique: "unique_versions s'"
  unfolding target
  using add_appearance_preserves_unique_versions[OF valid] .
  show ?thesis using target_unique immutable by blast
qed

theorem register_appearance_preserves_references_time_perspective_context:
  assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
  shows
    "\<forall>obj\<in>objects_of s'.
     references_complete s' obj \<and>
     evidence_well_formed s' obj \<and>
     time_well_formed obj \<and>
     perspective_well_formed s' obj \<and>
     context_well_formed s' obj"
  using register_appearance_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def
  by blast

theorem register_appearance_preserves_evidence_integrity:

```

```

assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
shows "\<forall>obj\<in>objects_of s'. evidence_well_formed s' obj"
using register_appearance_preserves_state_well_formed[OF concrete]
unfolding state_well_formed_def
by blast

theorem register_appearance_refines_generic_step:
  assumes concrete: "register_appearance_step G s new_obj next_snapshot_id s'"
  shows "step G s RegisterAppearance s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
  and allowed: "RegisterAppearance \<in> allowed_operations_of G"
  by (auto simp: register_appearance_step_def appearance_input_valid_def)
  have target_wf: "state_well_formed G s'"
  using register_appearance_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
  using register_appearance_preserves_identity[OF concrete] by blast
  show ?thesis
  unfolding step_def
  using source_wf target_wf allowed immutable by blast
qed
end

```

D.4 Reichhaltiger nicht-vakuoser Modellzeuge

Die vierte Theorie UZA_0_9_1_Rich_Model ergänzt den minimalen Erfüllbarkeitszeugen durch ein endliches, semantisch belegtes Snapshot mit zwölf versionierten Objekten. Enthalten sind Metaobjekt, Perspektiven- und Kontextobjekte, Erscheinung, Relation, validierte Wirkung, Resonanz, validierte Relevanz, Orientierung, validierte Handlung und Rückwirkung. Zusätzlich enthält der Zustand zwei Kontexte, eine verlustfreie Projektion, eine Kopplung mit Kanal, eine validierte Transmission mit Zielwirkung und vollständiger Lineage sowie ein erfolgreiches Fusion-BoundaryEvent mit vollständiger Disposition.

Die Lemmata rich_A5_relation_nonvacuous, rich_A6_effect_nonvacuous, rich_A7_A8_bindings_nonvacuous, rich_A9_evidence_nonvacuous, rich_A10_relevance_nonvacuous, rich_A11_orientation_nonvacuous, rich_A12_action_nonvacuous, rich_A13_feedback_nonvacuous und rich_A17_A21_interaction_nonvacuous weisen für jede betroffene Bedingungsgruppe ausdrücklich erfüllte Antezedenzen und die zugehörige Wohlgeformtheit nach. Dadurch werden A5–A13 und A17–A21 nicht mehr nur durch leere Objektmengen oder unzutreffende Objekttypen erfüllt. rich_state_well_formed beweist den gesamten Zustand, Rich: uza_model interpretiert erneut A1–A22, und rich_uza_model_exists belegt die Existenz eines solchen nichtvakuosen Modells relativ zu Isabelle/HOL.

Der integrierte Isabelle2025-2-Clean-Build verarbeitete UZA_0_9_1_Rich_Model vollständig und endete mit Exit-Code 0. Der aktuelle Quelltext besitzt SHA-256

3a3671b918d9639607fb6152dedd039cdf6145b0f040db8edf6ba296b8dc9a8c. Damit ist das Rich-Witness-Gate abgeschlossen. Nicht daraus folgen Expansion jedes beliebigen Kernmodells, Konservativität, semantische Angemessenheit für jede Domäne oder Produktivengine-Äquivalenz.

```

theory UZA_0_9_1_Rich_Model
  imports UZA_0_9_1_Base_Model
begin

section \<open>A finite non-vacuous witness model\<close>

text \<open>
  The minimal base model proves joint satisfiability of A1--A22 but leaves most
  kind-specific antecedents false. This second finite interpretation activates

```

```

every object-specific obligation A5--A13 and the interaction obligations
A17--A21. It contains an appearance, a relation with resolved endpoints, a
validated effect with origin and evidence, resonance, relevance, orientation,
action, feedback, two contexts, a projection, a coupling, a validated
transmission, and a successful fusion boundary event.
\<close>

definition rich_meta :: uza_object where
    "rich_meta = base_object"

definition rich_perspective_obj :: uza_object where
    "rich_perspective_obj =
        base_object\<lparr>object_id_of := 1, version_id_of := 1,
        kind_of := K_PERSP\<rparr>"

definition rich_context_source_obj :: uza_object where
    "rich_context_source_obj =
        base_object\<lparr>object_id_of := 2, version_id_of := 2,
        kind_of := K_CTX\<rparr>"

definition rich_entity :: uza_object where
    "rich_entity =
        base_object\<lparr>object_id_of := 3, version_id_of := 3,
        kind_of := K_ENT, lifecycle_of := Supported,
        references_of := {0}, evidence_of := {0},
        perspective_of := Some 10, context_of := Some 20\<rparr>"

definition rich_relation :: uza_object where
    "rich_relation =
        base_object\<lparr>object_id_of := 4, version_id_of := 4,
        kind_of := K_REL, lifecycle_of := Supported,
        references_of := {3}, evidence_of := {3}, context_of := Some 20,
        source_ref_of := Some 3, target_ref_of := Some 3\<rparr>"

definition rich_effect :: uza_object where
    "rich_effect =
        base_object\<lparr>object_id_of := 5, version_id_of := 5,
        kind_of := K_EFF, lifecycle_of := Validated,
        references_of := {4}, evidence_of := {3,4},
        perspective_of := Some 10, context_of := Some 20,
        effect_origin_of := Some 4\<rparr>"

definition rich_resonance :: uza_object where
    "rich_resonance =
        base_object\<lparr>object_id_of := 6, version_id_of := 6,
        kind_of := K_RES, lifecycle_of := Supported,
        references_of := {5}, evidence_of := {5},
        perspective_of := Some 10, context_of := Some 20\<rparr>"

definition rich_relevance :: uza_object where
    "rich_relevance =
        base_object\<lparr>object_id_of := 7, version_id_of := 7,
        kind_of := K_REV, lifecycle_of := Validated,
        references_of := {5,6}, evidence_of := {5},
        perspective_of := Some 10, context_of := Some 20,
        model_of := Some 30\<rparr>"

definition rich_orientation :: uza_object where
    "rich_orientation =
        base_object\<lparr>object_id_of := 8, version_id_of := 8,
        kind_of := K_ORI, lifecycle_of := Supported,
        references_of := {7}, evidence_of := {7},
        perspective_of := Some 10, context_of := Some 20,
        basis_refs_of := {7}\<rparr>"

definition rich_action :: uza_object where
    "rich_action =
        base_object\<lparr>object_id_of := 9, version_id_of := 9,
        kind_of := K_ACT, lifecycle_of := Validated,
        references_of := {8}, evidence_of := {8},
        perspective_of := Some 10, context_of := Some 20,
        authority_of := Some 40, basis_refs_of := {8}\<rparr>"

definition rich_feedback :: uza_object where
    "rich_feedback =
        base_object\<lparr>object_id_of := 10, version_id_of := 10,
        kind_of := K_FDB, lifecycle_of := Supported,
        references_of := {3,9}, evidence_of := {3}, context_of := Some 20,
        execution_ref_of := Some 9, observation_refs_of := {3}\<rparr>"
    
```

```

definition rich_context_target_obj :: uza_object where
    "rich_context_target_obj =
        base_object\<lparr>object_id_of := 11, version_id_of := 11,
        kind_of := K_CTX\<rparr>"

definition rich_objects :: "uza_object set" where
    "rich_objects =
        {rich_meta, rich_perspective_obj, rich_context_source_obj, rich_entity,
        rich_relation, rich_effect, rich_resonance, rich_relevance,
        rich_orientation, rich_action, rich_feedback, rich_context_target_obj}"

definition rich_loss :: loss_profile where
    "rich_loss =
        \<lparr>structural_loss_of = 0, semantic_loss_of = 0,
        perspective_loss_of = 0, temporal_loss_of = 0,
        loss_explanation_of = 'identity witness'\<rparr>"

definition rich_projection :: projection where
    "rich_projection =
        \<lparr>projection_id_of = 1, contained_context_of = 20,
        container_context_of = 21, source_snapshot_of = 1,
        projection_loss_of = rich_loss\<rparr>"

definition rich_coupling :: coupling where
    "rich_coupling =
        \<lparr>coupling_id_of = 50, coupling_source_of = 20,
        coupling_target_of = 21, coupling_channels_of = {60}\<rparr>"

definition rich_transmission :: transmission where
    "rich_transmission =
        \<lparr>transmission_id_of = 70, transmission_coupling_of = 50,
        transmission_channel_of = 60, transmission_source_effect_of = 5,
        transmission_target_effect_of = Some 15,
        prior_trans_status_of = Some TransObserved,
        trans_status_of = TransValidated,
        transmission_lineage_complete_of = True\<rparr>"

definition rich_boundary :: boundary_event where
    "rich_boundary =
        \<lparr>boundary_id_of = 80, boundary_kind_of = Fusion,
        boundary_succeeded_of = True, boundary_lineage_complete_of = True,
        boundary_disposition_complete_of = True\<rparr>"

definition rich_snapshot :: snapshot where
    "rich_snapshot =
        \<lparr>snapshot_id_of = 1,
        objects_of = rich_objects,
        historical_versions_of = {},
        perspectives_of = {10},
        contexts_of = {20,21},
        models_of = {30},
        authorities_of = {40},
        contains_edges_of = {(20,21)},
        projections_of = {rich_projection},
        couplings_of = {rich_coupling},
        transmissions_of = {rich_transmission},
        boundaries_of = {rich_boundary}\<rparr>"

definition rich_grammar :: grammar where
    "rich_grammar =
        \<lparr>grammar_id_of = 1,
        supported_kinds_of =
            {K_META, K_PERSP, K_CTX, K_ENT, K_REL, K_EFF, K_RES, K_REV,
            K_ORI, K_ACT, K_FDB},
        allowed_operations_of = {RegisterAppearance},
        patch_budget_of = 0\<rparr>"

definition rich_states :: "snapshot set" where
    "rich_states = {rich_snapshot}"

definition rich_classified_emergence ::
    "interaction_complex \<Rightarrow> emergence_level" where
    "rich_classified_emergence I = emergence_of rich_grammar I"

lemmas rich_object_defs =
    rich_meta_def rich_perspective_obj_def rich_context_source_obj_def
    rich_entity_def rich_relation_def rich_effect_def rich_resonance_def
    rich_relevance_def rich_orientation_def rich_action_def rich_feedback_def
    
```

```

rich_context_target_obj_def

lemma rich_snapshot_simps [simp]:
  "snapshot_id_of rich_snapshot = 1"
  "objects_of rich_snapshot = rich_objects"
  "historical_versions_of rich_snapshot = {}"
  "perspectives_of rich_snapshot = {10}"
  "contexts_of rich_snapshot = {20,21}"
  "models_of rich_snapshot = {30}"
  "authorities_of rich_snapshot = {40}"
  "contains_edges_of rich_snapshot = {(20,21)}"
  "projections_of rich_snapshot = {rich_projection}"
  "couplings_of rich_snapshot = {rich_coupling}"
  "transmissions_of rich_snapshot = {rich_transmission}"
  "boundaries_of rich_snapshot = {rich_boundary}"
  by (simp_all add: rich_snapshot_def)

lemma rich_grammar_simps [simp]:
  "grammar_id_of rich_grammar = 1"
  "supported_kinds_of rich_grammar =
    {K_META, K_PERSP, K_CTX, K_ENT, K_REL, K_EFF, K_RES, K_REV,
     K_ORI, K_ACT, K_FDB}"
  "allowed_operations_of rich_grammar = {RegisterAppearance}"
  "patch_budget_of rich_grammar = 0"
  by (simp_all add: rich_grammar_def)

lemma rich_version_ids [simp]:
  "version_ids rich_snapshot = {0,1,2,3,4,5,6,7,8,9,10,11}"
  by (simp add: version_ids_def rich_snapshot_def rich_objects_def
    rich_object_defs base_object_def)

lemma rich_revision_edges [simp]:
  "revision_edges_of rich_snapshot = {}"
  by (auto simp: revision_edges_of_def rich_snapshot_def rich_objects_def
    rich_object_defs base_object_def)

lemma rich_unique_versions:
  "unique_versions rich_snapshot"
  unfolding unique_versions_def inj_on_def
  by (auto simp: rich_objects_def rich_object_defs base_object_def)

lemma rich_singleton_trancl [simp]:
  "trancl {(20::context_id,21)} = {(20,21)}"
proof (rule equalityI)
  show "trancl {(20::context_id,21)} \<subseq> {(20,21)}"
  proof
    fix edge
    assume "edge \<in> trancl {(20::context_id,21)}"
    then obtain x y where edge: "edge = (x,y)"
    by (cases edge) auto
    have "(x,y) \<in> trancl {(20,21)}"
    using \<open>edge \<in> trancl {(20::context_id,21)}\<close>
    unfolding edge by simp
    then have "x = 20 \<and> y = 21"
    by (induction rule: trancl_induct) auto
    then show "edge \<in> {(20,21)}"
    by (simp add: edge)
  qed
  show "{(20,21)} \<subseq> trancl {(20::context_id,21)}"
  by auto
qed

lemma rich_contains_trancl [simp]:
  "trancl (contains_edges_of rich_snapshot) = {(20,21)}"
  by simp

lemma rich_state_well_formed:
  "state_well_formed rich_grammar rich_snapshot"
  unfolding state_well_formed_def
  using rich_unique_versions
  by (auto simp: present_ref_def
    references_complete_def evidence_well_formed_def
    evidence_references_complete_def time_well_formed_def
    relation_well_formed_def effect_well_formed_def
    perspective_well_formed_def perspective_bound_kind_def
    context_well_formed_def context_bound_kind_def
    relevance_well_formed_def orientation_well_formed_def
    action_well_formed_def feedback_well_formed_def
    projection_well_formed_def loss_well_formed_def)

```

```

coupling_well_formed_def transmission_well_formed_def
boundary_well_formed_def rich_objects_def
rich_projection_def rich_loss_def rich_coupling_def
rich_transmission_def rich_boundary_def rich_object_defs base_object_def)

lemma rich_self_step:
  "step rich_grammar rich_snapshot RegisterAppearance rich_snapshot"
proof -
  have allowed:
    "RegisterAppearance \<in> allowed_operations_of rich_grammar"
  by (simp add: rich_grammar_def)
  have immutable:
    "immutable_preserved rich_snapshot rich_snapshot"
  using rich_unique_versions
  unfolding immutable_preserved_def unique_versions_def
  by (meson inj_onD)
  show ?thesis
  unfolding step_def
  using rich_state_well_formed allowed immutable by blast
qed

subsection \<open>Explicit non-vacuity certificates\<close>

lemma rich_A5_relation_nonvacuous:
  "rich_relation \<in> objects_of rich_snapshot \<and>
  kind_of rich_relation = K_REL \<and>
  present_ref rich_snapshot (source_ref_of rich_relation) \<and>
  present_ref rich_snapshot (target_ref_of rich_relation)"
  by (simp add: rich_objects_def rich_relation_def
    base_object_def present_ref_def)

lemma rich_A6_effect_nonvacuous:
  "rich_effect \<in> objects_of rich_snapshot \<and>
  kind_of rich_effect = K_EFF \<and> lifecycle_of rich_effect = Validated \<and>
  present_ref rich_snapshot (effect_origin_of rich_effect)"
  by (simp add: rich_objects_def rich_effect_def
    base_object_def present_ref_def)

lemma rich_A7_A8_bindings_nonvacuous:
  "kind_of rich_entity = K_ENT \<and>
  perspective_of rich_entity = Some 10 \<and> 10 \<in> perspectives_of rich_snapshot \<and>
  context_of rich_entity = Some 20 \<and> 20 \<in> contexts_of rich_snapshot"
  by (simp add: rich_entity_def base_object_def)

lemma rich_A9_evidence_nonvacuous:
  "lifecycle_of rich_effect = Validated \<and>
  evidence_of rich_effect \<noteq> {} \<and>
  evidence_references_complete rich_snapshot rich_effect"
  by (simp add: rich_effect_def base_object_def
    evidence_references_complete_def)

lemma rich_A10_relevance_nonvacuous:
  "kind_of rich_relevance = K_REV \<and>
  references_of rich_relevance \<noteq> {} \<and>
  model_of rich_relevance = Some 30 \<and> 30 \<in> models_of rich_snapshot \<and>
  relevance_well_formed rich_snapshot rich_relevance"
  by (simp add: rich_relevance_def base_object_def
    relevance_well_formed_def perspective_well_formed_def
    perspective_bound_kind_def context_well_formed_def context_bound_kind_def)

lemma rich_A11_orientation_nonvacuous:
  "kind_of rich_orientation = K_ORI \<and>
  basis_refs_of rich_orientation \<noteq> {} \<and>
  orientation_well_formed rich_snapshot rich_orientation"
  by (simp add: rich_orientation_def base_object_def orientation_well_formed_def)

lemma rich_A12_action_nonvacuous:
  "kind_of rich_action = K_ACT \<and>
  basis_refs_of rich_action \<noteq> {} \<and>
  authority_of rich_action = Some 40 \<and> 40 \<in> authorities_of rich_snapshot \<and>
  action_well_formed rich_snapshot rich_action"
  by (simp add: rich_action_def base_object_def action_well_formed_def)

lemma rich_A13_feedback_nonvacuous:
  "kind_of rich_feedback = K_FDB \<and>
  execution_ref_of rich_feedback = Some 9 \<and>
  observation_refs_of rich_feedback \<noteq> {} \<and>
  feedback_well_formed rich_snapshot rich_feedback"
  by (simp add: rich_feedback_def base_object_def feedback_well_formed_def)

```

```

        present_ref_def)

lemma rich_A17_A21_interaction_nonvacuous:
  "rich_projection \<in> projections_of rich_snapshot \<and>
  projection_well_formed rich_snapshot rich_projection \<and>
  rich_coupling \<in> couplings_of rich_snapshot \<and>
  coupling_well_formed rich_snapshot rich_coupling \<and>
  rich_transmission \<in> transmissions_of rich_snapshot \<and>
  transmission_well_formed rich_snapshot rich_transmission \<and>
  trans_status_of rich_transmission = TransValidated \<and>
  transmission_target_effect_of rich_transmission \<noteq> None \<and>
  transmission_lineage_complete_of rich_transmission \<and>
  rich_boundary \<in> boundaries_of rich_snapshot \<and>
  boundary_well_formed rich_boundary"
  by (simp add: rich_projection_def projection_well_formed_def
    rich_loss_def loss_well_formed_def rich_coupling_def coupling_well_formed_def
    rich_transmission_def transmission_well_formed_def
    rich_boundary_def boundary_well_formed_def)

subsection \<open>Locale interpretation\<close>

interpretation Rich: uza_model
  rich_states rich_grammar rich_classified_emergence
proof
  fix s o1 o2
  assume state: "s \<in> rich_states"
  and o1: "o1 \<in> objects_of s" and o2: "o2 \<in> objects_of s"
  and version: "version_id_of o1 = version_id_of o2"
  have wf: "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  have "unique_versions s"
  using wf unfolding state_well_formed_def by blast
  then show "o1 = o2"
  using o1 o2 version unfolding unique_versions_def by (meson inj_onD)
next
  fix s
  assume "s \<in> rich_states"
  then show "wf (revision_edges_of s)"
  by (simp add: rich_states_def)
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  then show "time_well_formed obj"
  using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  then show "references_complete s obj"
  using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  then show "relation_well_formed s obj"
  using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  then show "effect_well_formed s obj"
  using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)
  then show "perspective_well_formed s obj"
  using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
  using state by (simp add: rich_states_def rich_state_well_formed)

```

```

    then show "context_well_formed s obj"
      using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "evidence_well_formed s obj"
    using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "relevance_well_formed s obj"
    using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "orientation_well_formed s obj"
    using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "action_well_formed s obj"
    using obj unfolding state_well_formed_def by blast
next
  fix s obj
  assume state: "s \<in> rich_states" and obj: "obj \<in> objects_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "feedback_well_formed s obj"
    using obj unfolding state_well_formed_def by blast
next
  fix s oper s'
  assume "s \<in> rich_states" "step rich_grammar s oper s'"
  then show "state_well_formed rich_grammar s'"
    by (simp add: step_def)
next
  fix s c
  assume "s \<in> rich_states"
  then show "\<not> strict_contains s c c"
    by (auto simp: rich_states_def strict_contains_def)
next
  fix s a b
  assume state: "s \<in> rich_states"
  and ab: "contains_eq s a b" and ba: "contains_eq s b a"
  show "a = b"
    using state ab ba
    by (auto simp: rich_states_def contains_eq_def strict_contains_def)
next
  fix s p
  assume state: "s \<in> rich_states" and member: "p \<in> projections_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "projection_well_formed s p"
    using member unfolding state_well_formed_def by blast
next
  fix s c
  assume state: "s \<in> rich_states" and member: "c \<in> couplings_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "coupling_well_formed s c"
    using member unfolding state_well_formed_def by blast
next
  fix s t
  assume state: "s \<in> rich_states" and member: "t \<in> transmissions_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "transmission_well_formed s t"
    using member unfolding state_well_formed_def by blast
next
  fix s t
  assume state: "s \<in> rich_states" and member: "t \<in> transmissions_of s"
  and validated: "trans_status_of t = TransValidated"

```



```

have wf: "transmission_well_formed s t"
proof -
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show ?thesis
    using member unfolding state_well_formed_def by blast
qed
show "transmission_target_effect_of t \<noteq> None \<and>
      transmission_lineage_complete_of t"
  using wf validated unfolding transmission_well_formed_def by blast
next
fix s b
  assume state: "s \<in> rich_states" and member: "b \<in> boundaries_of s"
  have "state_well_formed rich_grammar s"
    using state by (simp add: rich_states_def rich_state_well_formed)
  then show "boundary_well_formed b"
    using member unfolding state_well_formed_def by blast
next
fix I :: interaction_complex
  assume "current_of I \<in> rich_states"
  show "rich_classified_emergence I = emergence_of rich_grammar I"
    by (simp add: rich_classified_emergence_def)
qed

theorem rich_uza_model_exists:
  "\<exists>States G classify s.
    uza_model States G classify \<and> s \<in> States \<and>
    (\<exists>r\<in>objects_of s. kind_of r = K_REL) \<and>
    (\<exists>e\<in>objects_of s. kind_of e = K_EFF \<and> lifecycle_of e = Validated) \<and>
    (\<exists>v\<in>objects_of s. kind_of v = K_REV) \<and>
    (\<exists>ori\<in>objects_of s. kind_of ori = K_ORI) \<and>
    (\<exists>a\<in>objects_of s. kind_of a = K_ACT) \<and>
    (\<exists>f\<in>objects_of s. kind_of f = K_FDB) \<and>
    projections_of s \<noteq> {} \<and> couplings_of s \<noteq> {} \<and>
    transmissions_of s \<noteq> {} \<and> boundaries_of s \<noteq> {}"
proof (intro exI[of _ rich_states] exI[of _ rich_grammar]
  exI[of _ rich_classified_emergence] exI[of _ rich_snapshot])
  show "uza_model rich_states rich_grammar rich_classified_emergence \<and>
    rich_snapshot \<in> rich_states \<and>
    (\<exists>r\<in>objects_of rich_snapshot. kind_of r = K_REL) \<and>
    (\<exists>e\<in>objects_of rich_snapshot.
      kind_of e = K_EFF \<and> lifecycle_of e = Validated) \<and>
    (\<exists>v\<in>objects_of rich_snapshot. kind_of v = K_REV) \<and>
    (\<exists>ori\<in>objects_of rich_snapshot. kind_of ori = K_ORI) \<and>
    (\<exists>a\<in>objects_of rich_snapshot. kind_of a = K_ACT) \<and>
    (\<exists>f\<in>objects_of rich_snapshot. kind_of f = K_FDB) \<and>
    projections_of rich_snapshot \<noteq> {} \<and>
    couplings_of rich_snapshot \<noteq> {} \<and>
    transmissions_of rich_snapshot \<noteq> {} \<and>
    boundaries_of rich_snapshot \<noteq> {}"
  proof (intro conjI)
    show "uza_model rich_states rich_grammar rich_classified_emergence"
      by (rule Rich.uza_model_axioms)
  next
    show "rich_snapshot \<in> rich_states"
      by (simp add: rich_states_def)
  next
    show "\<exists>r\<in>objects_of rich_snapshot. kind_of r = K_REL"
      by (intro bexI[of _ rich_relation])
      (simp_all add: rich_objects_def rich_relation_def base_object_def)
  next
    show "\<exists>e\<in>objects_of rich_snapshot.
      kind_of e = K_EFF \<and> lifecycle_of e = Validated"
      by (intro bexI[of _ rich_effect])
      (simp_all add: rich_objects_def rich_effect_def base_object_def)
  next
    show "\<exists>v\<in>objects_of rich_snapshot. kind_of v = K_REV"
      by (intro bexI[of _ rich_relevance])
      (simp_all add: rich_objects_def rich_relevance_def base_object_def)
  next
    show "\<exists>ori\<in>objects_of rich_snapshot. kind_of ori = K_ORI"
      by (intro bexI[of _ rich_orientation])
      (simp_all add: rich_objects_def rich_orientation_def base_object_def)
  next
    show "\<exists>a\<in>objects_of rich_snapshot. kind_of a = K_ACT"
      by (intro bexI[of _ rich_action])
      (simp_all add: rich_objects_def rich_action_def base_object_def)
  next
    show "\<exists>f\<in>objects_of rich_snapshot. kind_of f = K_FDB"

```

```

    by (intro bexI[of _ rich_feedback])
      (simp_all add: rich_objects_def rich_feedback_def base_object_def)
  next
    show "projections_of rich_snapshot \<noteq> {}"
    by simp
  next
    show "couplings_of rich_snapshot \<noteq> {}"
    by simp
  next
    show "transmissions_of rich_snapshot \<noteq> {}"
    by simp
  next
    show "boundaries_of rich_snapshot \<noteq> {}"
    by simp
qed
qed
end

```

D.5 Konkrete T2-Semantik und Erhaltung für DifferentiatePotential

Die fünfte Theorie UZA_0_9_1_Differentiate_Potential konkretisiert T2 als konstruktive Mengentransition. `potential_input_valid` verlangt einen wohlgeformten Ausgangszustand, eine zulässige Operation, eine endliche und nichtleere Kandidatenmenge, paarweise eindeutige Objekt- und Versionsidentitäten sowie Frische gegenüber dem Ausgangszustand. Jeder Kandidat ist eine K_ENT-Hypothese mit Status Applicable, auflösbarer `source_ref`, nichtleerer `basis_refs`-Menge, vollständigen Referenzen, gültiger Evidenz- und Zeitstruktur sowie vorhandener Perspektiv- und Kontextbindung. `potential_candidate_marker` und `observed_appearance_marker` sind disjunkt; `potential_candidate_not_observed` beweist die Unterscheidbarkeit ausdrücklich.

`add_potential_candidates` erzeugt den Folgezustand ausschließlich durch Vereinigung der bisherigen Objektmenge mit der frischen Kandidatenmenge und Fortschreibung der Snapshot-ID. Isabelle beweist `differentiate_potential_exact_delta`, die explizite Kandidateneigenschaft, eindeutige Versionsidentitäten, Unveränderlichkeit aller vorhandenen Versionen, Erhaltung von Referenzen, Evidence Integrity, Zeit, Perspektive und Kontext, vollständige `state_well_formed`-Erhaltung und `differentiate_potential_refines_generic_step`. Die Zielwohlgeformtheit ist keine Vorbedingung.

Der integrierte Isabelle2025-2-Clean-Build verarbeitete UZA_0_9_1_Differentiate_Potential vollständig und endete mit Exit-Code 0. Der aktuelle T2-Quelltext besitzt SHA-256 `ef483b279d0581752bfff655db6672fa57823dac348ace29268e68e486a40f9b`. Damit ist Aufgabe 4 auch für T2 abgeschlossen. Nicht bewiesen sind die Vollständigkeit aller möglichen Kandidaten oder ihre domänenspezifische Güte.

```

theory UZA_0_9_1_Differentiate_Potential
  imports UZA_0_9_1_Register_Appearance
begin

section \<open>T2 DifferentiatePotential: concrete operation semantics\<close>

text \<open>
  T2 materialises a finite, non-empty family of explicit possibility candidates.
  Candidates are not observations: an observed T1 appearance has no source,
  basis, or analysis marker, whereas a T2 candidate is a Hypothesis with an
  existing source, a non-empty resolved basis, and status Applicable.

  The target state is constructed by inserting exactly the candidate set and
  advancing the snapshot identifier. Target well-formedness is proved, not
  assumed.
\<close>

definition observed_appearance_marker :: "uza_object \<Rightarrow> bool" where

```

```

"observed_appearance_marker obj \<longlefttrightarrow>
  kind_of obj = K_ENT \<and>
  source_ref_of obj = None \<and>
  basis_refs_of obj = {} \<and>
  object_status_of obj = None"

definition potential_candidate_marker :: "uza_object \<Rightarrow> bool" where
  "potential_candidate_marker obj \<longlefttrightarrow>
    kind_of obj = K_ENT \<and>
    lifecycle_of obj = Hypothesis \<and>
    source_ref_of obj \<noteq> None \<and>
    basis_refs_of obj \<noteq> {} \<and>
    object_status_of obj = Some Applicable"

lemma potential_candidate_not_observed:
  assumes "potential_candidate_marker obj"
  shows "\<not> observed_appearance_marker obj"
  using assms
  by (auto simp: potential_candidate_marker_def observed_appearance_marker_def)

definition potential_input_valid ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> uza_object set \<Rightarrow> nat \<Rightarrow> bool"
where
  "potential_input_valid G s candidates next_snapshot_id \<longlefttrightarrow>
    state_well_formed G s \<and>
    DifferentiatePotential \<in> allowed_operations_of G \<and>
    K_ENT \<in> supported_kinds_of G \<and>
    finite candidates \<and>
    candidates \<noteq> {} \<and>
    inj_on object_id_of candidates \<and>
    inj_on version_id_of candidates \<and>
    object_id_of ` candidates \<inter> object_id_of ` objects_of s = {} \<and>
    version_id_of ` candidates \<inter>
      (version_ids s \<union> historical_versions_of s) = {} \<and>
    (\<forall>candidate\<in>candidates.
      potential_candidate_marker candidate \<and>
      previous_version_of candidate = None \<and>
      references_complete s candidate \<and>
      evidence_well_formed s candidate \<and>
      time_well_formed candidate \<and>
      (\<exists>source. source_ref_of candidate = Some source \<and>
        source \<in> basis_refs_of candidate) \<and>
      basis_refs_of candidate \<subteq> references_of candidate \<and>
      (\<exists>p\<in>perspectives_of s. perspective_of candidate = Some p) \<and>
      (\<exists>c\<in>contexts_of s. context_of candidate = Some c)) \<and>
    snapshot_id_of s < next_snapshot_id"

definition add_potential_candidates ::
  "snapshot \<Rightarrow> uza_object set \<Rightarrow> nat \<Rightarrow> snapshot" where
  "add_potential_candidates s candidates next_snapshot_id =
    s\<lparr> snapshot_id_of := next_snapshot_id,
      objects_of := objects_of s \<union> candidates \<rpar>"

definition differentiate_potential_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> uza_object set \<Rightarrow> nat \<Rightarrow> snapshot
  \<Rightarrow> bool" where
  "differentiate_potential_step G s candidates next_snapshot_id s' \<longlefttrightarrow>
    potential_input_valid G s candidates next_snapshot_id \<and>
    s' = add_potential_candidates s candidates next_snapshot_id"

subsection \<open>Exact update and candidate separation\<close>

lemma add_potential_candidates_objects [simp]:
  "objects_of (add_potential_candidates s candidates next_snapshot_id) =
    objects_of s \<union> candidates"
  by (simp add: add_potential_candidates_def)

lemma add_potential_candidates_snapshot_id [simp]:
  "snapshot_id_of (add_potential_candidates s candidates next_snapshot_id) =
    next_snapshot_id"
  by (simp add: add_potential_candidates_def)

lemma add_potential_candidates_background [simp]:
  "historical_versions_of (add_potential_candidates s candidates next_snapshot_id) =
    historical_versions_of s"
  "perspectives_of (add_potential_candidates s candidates next_snapshot_id) = perspectives_of s"
  "contexts_of (add_potential_candidates s candidates next_snapshot_id) = contexts_of s"
  "models_of (add_potential_candidates s candidates next_snapshot_id) = models_of s"
  "authorities_of (add_potential_candidates s candidates next_snapshot_id) = authorities_of s"

```

```

"contains_edges_of (add_potential_candidates s candidates next_snapshot_id) = contains_edges_of s"
"projections_of (add_potential_candidates s candidates next_snapshot_id) = projections_of s"
"couplings_of (add_potential_candidates s candidates next_snapshot_id) = couplings_of s"
"transmissions_of (add_potential_candidates s candidates next_snapshot_id) = transmissions_of s"
"boundaries_of (add_potential_candidates s candidates next_snapshot_id) = boundaries_of s"
by (simp_all add: add_potential_candidates_def)

lemma add_potential_candidates_version_ids [simp]:
  "version_ids (add_potential_candidates s candidates next_snapshot_id) =
    version_ids s \ $\cup$  version_ids candidates"
  by (auto simp: version_ids_def)

lemma potential_input_candidates_disjoint:
  assumes valid: "potential_input_valid G s candidates next_snapshot_id"
  shows "objects_of s \ $\cap$  candidates = {}"

proof (rule ccontr)
  assume "objects_of s \ $\cap$  candidates \ $\neq$  {}"
  then obtain obj where old: "obj \ $\in$  objects_of s" and candidate: "obj \ $\in$  candidates"
  by blast
  from valid have fresh_ids:
    "object_id_of ` candidates \ $\cap$  object_id_of ` objects_of s = {}"
  unfolding potential_input_valid_def by blast
  have "object_id_of obj \ $\in$  object_id_of ` candidates \ $\cap$  object_id_of ` objects_of s"
  using old candidate by blast
  then show False using fresh_ids by blast
qed

lemma differentiate_potential_exact_delta:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows
    "objects_of s' = objects_of s \ $\cup$  candidates \ $\wedge$ 
    objects_of s \ $\cap$  candidates = {} \ $\wedge$ 
    finite candidates \ $\wedge$  candidates \ $\neq$  {} \ $\wedge$ 
    snapshot_id_of s' = next_snapshot_id"
  using concrete potential_input_candidates_disjoint
  by (auto simp: differentiate_potential_step_def potential_input_valid_def)

lemma differentiate_potential_candidates_are_explicit:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows
    "candidates \ $\subseteq$  objects_of s' \ $\wedge$ 
    ( $\forall$  candidate \ $\in$  candidates.
      potential_candidate_marker candidate \ $\wedge$ 
       $\neg$  observed_appearance_marker candidate)"
  using concrete potential_candidate_not_observed
  by (auto simp: differentiate_potential_step_def potential_input_valid_def)

subsection \ $\langle$ Monotonicity of state-relative conditions $\rangle$ 

lemma add_potential_present_ref_mono:
  assumes "present_ref s ref"
  shows "present_ref (add_potential_candidates s candidates next_snapshot_id) ref"
  using assms
  by (cases ref) (auto simp: present_ref_def)

lemma add_potential_references_mono:
  assumes "references_complete s obj"
  shows "references_complete (add_potential_candidates s candidates next_snapshot_id) obj"
  using assms
  by (auto simp: references_complete_def)

lemma add_potential_evidence_mono:
  assumes "evidence_well_formed s obj"
  shows "evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
  using assms
  by (auto simp: evidence_well_formed_def evidence_references_complete_def)

lemma add_potential_relation_mono:
  assumes "relation_well_formed s obj"
  shows "relation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
  using assms add_potential_present_ref_mono
  unfolding relation_well_formed_def
  by blast

lemma add_potential_effect_mono:
  assumes "effect_well_formed s obj"
  shows "effect_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"

```

```

using assms add_potential_present_ref_mono
unfolding effect_well_formed_def
by blast

lemma add_potential_perspective_iff [simp]:
  "perspective_well_formed (add_potential_candidates s candidates next_snapshot_id) obj
   \<longleftarrowrightarrow> perspective_well_formed s obj"
  unfolding perspective_well_formed_def
  by (simp only: add_potential_candidates_background(2))

lemma add_potential_context_iff [simp]:
  "context_well_formed (add_potential_candidates s candidates next_snapshot_id) obj
   \<longleftarrowrightarrow> context_well_formed s obj"
  unfolding context_well_formed_def
  by (simp only: add_potential_candidates_background(3))

lemma add_potential_relevance_iff [simp]:
  "relevance_well_formed (add_potential_candidates s candidates next_snapshot_id) obj
   \<longleftarrowrightarrow> relevance_well_formed s obj"
  unfolding relevance_well_formed_def
  by (simp only: add_potential_perspective_iff add_potential_context_iff
    add_potential_candidates_background(4))

lemma add_potential_orientation_iff [simp]:
  "orientation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj
   \<longleftarrowrightarrow> orientation_well_formed s obj"
  by (simp add: orientation_well_formed_def)

lemma add_potential_action_iff [simp]:
  "action_well_formed (add_potential_candidates s candidates next_snapshot_id) obj
   \<longleftarrowrightarrow> action_well_formed s obj"
  unfolding action_well_formed_def
  by (simp only: add_potential_candidates_background(5))

lemma add_potential_feedback_mono:
  assumes "feedback_well_formed s obj"
  shows "feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
  using assms add_potential_present_ref_mono
  unfolding feedback_well_formed_def
  by blast

subsection \<open>Fresh identity and candidate validity\<close>

lemma add_potential_preserves_unique_versions:
  assumes valid: "potential_input_valid G s candidates next_snapshot_id"
  shows "unique_versions (add_potential_candidates s candidates next_snapshot_id)"
proof -
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
    and candidate_unique: "inj_on version_id_of candidates"
    and fresh:
      "version_id_of ` candidates \<inter> (version_ids s \<union> historical_versions_of s) = {}"
    unfolding potential_input_valid_def state_well_formed_def unique_versions_def
    by auto
  have cross_fresh: "version_id_of ` candidates \<inter> version_id_of ` objects_of s = {}"
    using fresh unfolding version_ids_def by blast
  show ?thesis
    unfolding unique_versions_def
  proof (rule inj_onI)
    fix left right
    assume left_mem:
      "left \<in> objects_of (add_potential_candidates s candidates next_snapshot_id)"
    and right_mem:
      "right \<in> objects_of (add_potential_candidates s candidates next_snapshot_id)"
    and same_version: "version_id_of left = version_id_of right"
    consider
      (old_old) "left \<in> objects_of s" "right \<in> objects_of s"
    | (candidate_candidate) "left \<in> candidates" "right \<in> candidates"
    | (old_candidate) "left \<in> objects_of s" "right \<in> candidates"
    | (candidate_old) "left \<in> candidates" "right \<in> objects_of s"
    using left_mem right_mem by auto
    then show "left = right"
  proof cases
    case old_old
      show ?thesis using inj_onD[OF source_unique same_version old_old] .
    next
      case candidate_candidate
      show ?thesis using inj_onD[OF candidate_unique same_version candidate_candidate] .
    next
      case old_candidate
  end
end

```

```

    have candidate_image: "version_id_of right \<in> version_id_of ` candidates"
      using old_candidate by blast
    have old_image: "version_id_of right \<in> version_id_of ` objects_of s"
      using old_candidate same_version by (metis imageI)
    have "version_id_of right \<in>
      version_id_of ` candidates \<inter> version_id_of ` objects_of s"
      using candidate_image old_image by blast
    then show ?thesis using cross_fresh by blast
  next
  case candidate_old
  have candidate_image: "version_id_of left \<in> version_id_of ` candidates"
    using candidate_old by blast
  have old_image: "version_id_of left \<in> version_id_of ` objects_of s"
    using candidate_old same_version by (metis imageI)
  have "version_id_of left \<in>
    version_id_of ` candidates \<inter> version_id_of ` objects_of s"
    using candidate_image old_image by blast
  then show ?thesis using cross_fresh by blast
qed
qed
qed

lemma potential_input_new_candidate_conditions:
  assumes valid: "potential_input_valid G s candidates next_snapshot_id"
  and member: "candidate \<in> candidates"
  shows
    "kind_of candidate \<in> supported_kinds_of G \<and>
    references_complete (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    time_well_formed candidate \<and>
    relation_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    effect_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    perspective_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    context_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    relevance_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    orientation_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    action_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate"
proof -
  from valid member have supported: "kind_of candidate \<in> supported_kinds_of G"
  and marker: "potential_candidate_marker candidate"
  and refs: "references_complete s candidate"
  and evidence: "evidence_well_formed s candidate"
  and time: "time_well_formed candidate"
  and perspective:
    "perspective_well_formed
    (add_potential_candidates s candidates next_snapshot_id) candidate"
  and context_ok:
    "context_well_formed
    (add_potential_candidates s candidates next_snapshot_id) candidate"
  unfolding potential_input_valid_def potential_candidate_marker_def
  perspective_well_formed_def context_well_formed_def
  perspective_bound_kind_def context_bound_kind_def
  by auto
  from marker have kind: "kind_of candidate = K_ENT"
  unfolding potential_candidate_marker_def by blast
  have refs_target:
    "references_complete (add_potential_candidates s candidates next_snapshot_id) candidate"
    using add_potential_references_mono[OF refs] .
  have evidence_target:
    "evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate"
    using add_potential_evidence_mono[OF evidence] .
  have typed_conditions:
    "relation_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    effect_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    relevance_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    orientation_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    action_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate \<and>
    feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) candidate"
    using kind
    by (simp add: relation_well_formed_def effect_well_formed_def
      relevance_well_formed_def orientation_well_formed_def
      action_well_formed_def feedback_well_formed_def)
  show ?thesis
    using supported refs_target evidence_target time perspective context_ok typed_conditions
    by blast
qed

subsection \<open>Operation-specific preservation theorems\<close>

```

```

theorem differentiate_potential_preserves_state_well_formed:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows "state_well_formed G s'"
proof -
  from concrete have valid: "potential_input_valid G s candidates next_snapshot_id"
  and target: "s' = add_potential_candidates s candidates next_snapshot_id"
  by (auto simp: differentiate_potential_step_def)
  from valid have source_wf: "state_well_formed G s"
  by (simp add: potential_input_valid_def)
  have unique:
    "unique_versions (add_potential_candidates s candidates next_snapshot_id)"
  using add_potential_preserves_unique_versions[OF valid] .
  have all_objects:
    "\<forall>obj\<in>objects_of (add_potential_candidates s candidates next_snapshot_id).
      kind_of obj \<in> supported_kinds_of G \<and>
      references_complete (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      time_well_formed obj \<and>
      relation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      effect_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      perspective_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      context_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      relevance_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      orientation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      action_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
      feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
  proof (intro ballI)
    fix obj
    assume obj_mem:
      "obj \<in> objects_of (add_potential_candidates s candidates next_snapshot_id)"
    consider (old) "obj \<in> objects_of s" | (candidate) "obj \<in> candidates"
    using obj_mem by auto
    then show
      "kind_of obj \<in> supported_kinds_of G \<and>
        references_complete (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        time_well_formed obj \<and>
        relation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        effect_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        perspective_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        context_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        relevance_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        orientation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        action_well_formed (add_potential_candidates s candidates next_snapshot_id) obj \<and>
        feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
  proof cases
    case candidate
    then show ?thesis
      using potential_input_new_candidate_conditions[OF valid] by blast
  next
    case old
    from source_wf old have source_conditions:
      "kind_of obj \<in> supported_kinds_of G \<and>
        references_complete s obj \<and> evidence_well_formed s obj \<and>
        time_well_formed obj \<and>
        relation_well_formed s obj \<and> effect_well_formed s obj \<and>
        perspective_well_formed s obj \<and> context_well_formed s obj \<and>
        relevance_well_formed s obj \<and> orientation_well_formed s obj \<and>
        action_well_formed s obj \<and> feedback_well_formed s obj"
    unfolding state_well_formed_def by blast
    from source_conditions have supported: "kind_of obj \<in> supported_kinds_of G"
    and refs: "references_complete s obj"
    and evidence: "evidence_well_formed s obj"
    and time: "time_well_formed obj"
    and relation: "relation_well_formed s obj"
    and effect: "effect_well_formed s obj"
    and perspective: "perspective_well_formed s obj"
    and context_ok: "context_well_formed s obj"
    and relevance: "relevance_well_formed s obj"
    and orientation: "orientation_well_formed s obj"
    and action: "action_well_formed s obj"
    and feedback: "feedback_well_formed s obj"
    by blast+
    have refs_target:
      "references_complete (add_potential_candidates s candidates next_snapshot_id) obj"
    using add_potential_references_mono[OF refs] .
    have evidence_target:

```

```

        "evidence_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
        using add_potential_evidence_mono[OF evidence] .
    have relation_target:
        "relation_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
        using add_potential_relation_mono[OF relation] .
    have effect_target:
        "effect_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
        using add_potential_effect_mono[OF effect] .
    have feedback_target:
        "feedback_well_formed (add_potential_candidates s candidates next_snapshot_id) obj"
        using add_potential_feedback_mono[OF feedback] .
    show ?thesis
        using supported refs_target evidence_target time relation_target effect_target perspective
            context_ok relevance orientation action feedback_target
            by simp
qed
qed
from source_wf have source_secondary:
    "(\<forall>p\<in>projections_of s. projection_well_formed s p) \<and>
    (\<forall>c\<in>couplings_of s. coupling_well_formed s c) \<and>
    (\<forall>t\<in>transmissions_of s. transmission_well_formed s t) \<and>
    (\<forall>b\<in>boundaries_of s. boundary_well_formed b)"
    unfolding state_well_formed_def by blast
have target_secondary:
    "(\<forall>p\<in>projections_of (add_potential_candidates s candidates next_snapshot_id).
    projection_well_formed (add_potential_candidates s candidates next_snapshot_id) p) \<and>
    (\<forall>c\<in>couplings_of (add_potential_candidates s candidates next_snapshot_id).
    coupling_well_formed (add_potential_candidates s candidates next_snapshot_id) c) \<and>
    (\<forall>t\<in>transmissions_of (add_potential_candidates s candidates next_snapshot_id).
    transmission_well_formed (add_potential_candidates s candidates next_snapshot_id) t) \<and>
    (\<forall>b\<in>boundaries_of (add_potential_candidates s candidates next_snapshot_id).
    boundary_well_formed b)"
    using source_secondary
    by (simp add: projection_well_formed_def coupling_well_formed_def
        transmission_well_formed_def)
show ?thesis
    unfolding target state_well_formed_def
    using unique all_objects target_secondary by blast
qed

theorem differentiate_potential_preserves_identity:
    assumes concrete:
        "differentiate_potential_step G s candidates next_snapshot_id s'"
    shows "unique_versions s' \<and> immutable_preserved s s'"
proof -
    from concrete have valid: "potential_input_valid G s candidates next_snapshot_id"
        and target: "s' = add_potential_candidates s candidates next_snapshot_id"
        by (auto simp: differentiate_potential_step_def)
    from valid have source_unique: "inj_on version_id_of (objects_of s)"
        and fresh:
            "version_id_of ` candidates \<inter> (version_ids s \<union> historical_versions_of s) = {}"
        unfolding potential_input_valid_def state_well_formed_def unique_versions_def
        by auto
    have cross_fresh: "version_id_of ` candidates \<inter> version_id_of ` objects_of s = {}"
        using fresh unfolding version_ids_def by blast
    have immutable: "immutable_preserved s s'"
        unfolding immutable_preserved_def target
    proof (intro ballI allI impI)
        fix old_obj target_obj
        assume old_mem: "old_obj \<in> objects_of s"
        and target_mem:
            "target_obj \<in> objects_of
            (add_potential_candidates s candidates next_snapshot_id)"
        and same_version: "version_id_of old_obj = version_id_of target_obj"
        consider (old) "target_obj \<in> objects_of s" | (candidate) "target_obj \<in> candidates"
        using target_mem by auto
        then show "old_obj = target_obj"
    proof cases
        case old
        show ?thesis using inj_onD[OF source_unique same_version old_mem old] .
    next
        case candidate
        have candidate_image:
            "version_id_of target_obj \<in> version_id_of ` candidates"
            using candidate by blast
        have old_image:
            "version_id_of target_obj \<in> version_id_of ` objects_of s"
            using old_mem same_version by (metis imageI)
        have "version_id_of target_obj \<in>

```



```

        version_id_of ` candidates \<inter> version_id_of ` objects_of s"
        using candidate_image old_image by blast
        then show ?thesis using cross_fresh by blast
    qed
qed
have target_unique: "unique_versions s'"
  unfolding target
  using add_potential_preserves_unique_versions[OF valid] .
show ?thesis using target_unique immutable by blast
qed

theorem differentiate_potential_preserves_references_time_perspective_context:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows
    "\<forall>obj\<in>objects_of s'.
      references_complete s' obj \<and>
      evidence_well_formed s' obj \<and>
      time_well_formed obj \<and>
      perspective_well_formed s' obj \<and>
      context_well_formed s' obj"
  using differentiate_potential_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def
  by blast

theorem differentiate_potential_preserves_evidence_integrity:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows "\<forall>obj\<in>objects_of s'. evidence_well_formed s' obj"
  using differentiate_potential_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def
  by blast

theorem differentiate_potential_refines_generic_step:
  assumes concrete:
    "differentiate_potential_step G s candidates next_snapshot_id s'"
  shows "step G s DifferentiatePotential s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
    and allowed: "DifferentiatePotential \<in> allowed_operations_of G"
    by (auto simp: differentiate_potential_step_def potential_input_valid_def)
  have target_wf: "state_well_formed G s'"
    using differentiate_potential_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
    using differentiate_potential_preserves_identity[OF concrete] by blast
  show ?thesis
    unfolding step_def
    using source_wf target_wf allowed immutable by blast
qed
end

```

D.6 Konkrete T3-Semantik und Erhaltung für CreateRelation

Die sechste Theorie UZA_0_9_1_Create_Relation erweitert die integrierte Theorie konservativ um `relation_type` sowie einen typisierten `relation_catalog`. Der Katalog ordnet jeder erzeugten Relations-Version Typ und Richtung zu. Diese Sidecar-Kodierung erhält den Basistyp `snapshot` unverändert und macht zugleich sichtbar, dass eine spätere Hauptversion die Relationsmetadaten gegebenenfalls in einen allgemeinen Objekt-Payload integrieren sollte.

`relation_input_valid` verlangt einen wohlgeformten Ausgangszustand, einen wohlgeformten Ausgangskatalog, die zugelassene Operation `CreateRelation`, den unterstützten Typ `K_REL`, frische Objekt- und Versionsidentitäten, zwei verschiedene aktive `K_ENT`-Endpunkte, vollständige Endpunktreferenzen, gültige Evidenz- und Zeitstruktur, einen vorhandenen Kontext sowie eine fortgeschriebene Snapshot-ID. `add_relation` verändert ausschließlich Snapshot-ID und Objektmenge. `extend_relation_catalog` ergänzt ausschließlich den neuen `version_id`-Schlüssel. Die Zielwohlgeformtheit ist keine Vorbedingung.

Die kernelgeprüfte Theorie enthält `create_relation_exact_delta`, `create_relation_typed_endpoints`, `create_relation_preserves_state_well_formed`, `create_relation_preserves_identity`, `create_relation_preserves_catalog`, `create_relation_preserves_references_time_context`, `create_relation_preserves_evidence_integrity` und `create_relation_refines_generic_step`. Der reale Build ergänzte das Hilfslemma `active_ent_endpoint_present_ref` und präzisierte zwei lokale Beweisschritte. Der finale Quelltext umfasst 580 Zeilen, enthält weder `sorry` noch `oops` noch `admit` und besitzt SHA-256

a3c801eaf1640624ee7bf41ba91a20d2ba1e000e489ee40f4d3c209b15911ca3.

Verbindliche integrierte Sessiondatei und ausgeführter Buildbefehl:

```
session UZA_0_9_1 = HOL +
  options [document = false]
  theories
    UZA_0_9_1
    UZA_0_9_1_Base_Model
    UZA_0_9_1_Register_Appearance
    UZA_0_9_1_Rich_Model
    UZA_0_9_1_Differentiate_Potential
    UZA_0_9_1_Create_Relation
    UZA_0_9_1_Hypothesize_Effect
```

```
USER_HOME=<pinned-user-home> Isabelle2025-2/bin/isabelle build -c -v -j 1 -D . UZA_0_9_1
```

Vollständiger kernelgeprüfter Isabelle/HOL-Quelltext:

```
theory UZA_0_9_1_Create_Relation
  imports UZA_0_9_1_Differentiate_Potential
begin

section \<open>T3 CreateRelation: conservative typed operation semantics\<close>

text \<open>
  T3 creates exactly one fresh K_REL object between two distinct active K_ENT
  endpoints. The already kernel-checked base snapshot type is not changed.
  Relation type and direction are therefore persisted in a conservative typed
  side catalog keyed by the relation version identifier. Target-state
  well-formedness is derived rather than assumed.
\<close>

datatype relation_type =
  RelConnects | RelConditions | RelReinforces | RelInhibits
  | RelContradicts | RelGenerates | RelTransforms

type_synonym relation_catalog = "version_id \<righttharpoonup> (relation_type \<times> bool)"

definition active_ent_endpoint :: "snapshot \<Rightarrow> version_id \<Rightarrow> bool" where
  "active_ent_endpoint s v \<longlefttrightarrow>
    (\<exists>obj\<in>objects_of s.
      version_id_of obj = v \<and>
      kind_of obj = K_ENT \<and>
      lifecycle_of obj \<notin> {Revised, Rejected, Archived})"
```

```
definition relation_catalog_well_formed ::
  "snapshot \<Rightarrow> relation_catalog \<Rightarrow> bool" where
  "relation_catalog_well_formed s catalog \<longlefttrightarrow>
    (\<forall>v metadata. catalog v = Some metadata \<longrightarrow>
      (\<exists>r\<in>objects_of s. version_id_of r = v \<and> kind_of r = K_REL))"
```

```
definition relation_input_valid ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> relation_catalog \<Rightarrow> uza_object \<Rightarrow>
    relation_type \<Rightarrow> bool \<Rightarrow> version_id \<Rightarrow> version_id \<Rightarrow>
    nat \<Rightarrow> bool" where
  "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id
\<longlefttrightarrow>
  state_well_formed G s \<and>
  relation_catalog_well_formed s catalog \<and>
  CreateRelation \<in> allowed_operations_of G \<and>
  K_REL \<in> supported_kinds_of G \<and>
  kind_of new_rel = K_REL \<and>
  lifecycle_of new_rel \<in> {Draft, Hypothesis} \<and>
  object_id_of new_rel \<notin> object_id_of ` objects_of s \<and>
  version_id_of new_rel \<notin> version_ids s \<union> historical_versions_of s \<and>
```

```

catalog (version_id_of new_rel) = None \<and>
previous_version_of new_rel = None \<and>
source_ref_of new_rel = Some src \<and>
target_ref_of new_rel = Some tgt \<and>
src \<noteq> tgt \<and>
active_ent_endpoint s src \<and>
active_ent_endpoint s tgt \<and>
{src, tgt} \<subteq> references_of new_rel \<and>
references_complete s new_rel \<and>
evidence_well_formed s new_rel \<and>
time_well_formed new_rel \<and>
(\<exists>c\<in>contexts_of s. context_of new_rel = Some c) \<and>
perspective_of new_rel = None \<and>
model_of new_rel = None \<and>
authority_of new_rel = None \<and>
effect_origin_of new_rel = None \<and>
basis_refs_of new_rel = {} \<and>
execution_ref_of new_rel = None \<and>
observation_refs_of new_rel = {} \<and>
object_status_of new_rel = None \<and>
snapshot_id_of s < next_snapshot_id"

definition add_relation ::
  "snapshot \<Rightarrow> uza_object \<Rightarrow> nat \<Rightarrow> snapshot" where
  "add_relation s new_rel next_snapshot_id =
    s\<lparr> snapshot_id_of := next_snapshot_id,
    objects_of := insert new_rel (objects_of s) \<rparr>"

definition extend_relation_catalog ::
  "relation_catalog \<Rightarrow> uza_object \<Rightarrow> relation_type \<Rightarrow> bool
  \<Rightarrow>
  relation_catalog" where
  "extend_relation_catalog catalog new_rel rtype directed =
    catalog(version_id_of new_rel \<mapsto> (rtype, directed))"

definition create_relation_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> relation_catalog \<Rightarrow> uza_object \<Rightarrow>
  relation_type \<Rightarrow> bool \<Rightarrow> version_id \<Rightarrow> version_id \<Rightarrow>
  nat \<Rightarrow> snapshot \<Rightarrow> relation_catalog \<Rightarrow> bool" where
  "create_relation_step G s catalog new_rel rtype directed src tgt
    next_snapshot_id s' catalog' \<longlefttrightarrow>
    relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id \<and>
    s' = add_relation s new_rel next_snapshot_id \<and>
    catalog' = extend_relation_catalog catalog new_rel rtype directed"

subsection \<open>Exact update and frame properties\<close>

lemma add_relation_objects [simp]:
  "objects_of (add_relation s new_rel next_snapshot_id) =
    insert new_rel (objects_of s)"
  by (simp add: add_relation_def)

lemma add_relation_snapshot_id [simp]:
  "snapshot_id_of (add_relation s new_rel next_snapshot_id) = next_snapshot_id"
  by (simp add: add_relation_def)

lemma add_relation_background [simp]:
  "historical_versions_of (add_relation s new_rel next_snapshot_id) =
    historical_versions_of s"
  "perspectives_of (add_relation s new_rel next_snapshot_id) = perspectives_of s"
  "contexts_of (add_relation s new_rel next_snapshot_id) = contexts_of s"
  "models_of (add_relation s new_rel next_snapshot_id) = models_of s"
  "authorities_of (add_relation s new_rel next_snapshot_id) = authorities_of s"
  "contains_edges_of (add_relation s new_rel next_snapshot_id) = contains_edges_of s"
  "projections_of (add_relation s new_rel next_snapshot_id) = projections_of s"
  "couplings_of (add_relation s new_rel next_snapshot_id) = couplings_of s"
  "transmissions_of (add_relation s new_rel next_snapshot_id) = transmissions_of s"
  "boundaries_of (add_relation s new_rel next_snapshot_id) = boundaries_of s"
  by (simp_all add: add_relation_def)

lemma add_relation_version_ids [simp]:
  "version_ids (add_relation s new_rel next_snapshot_id) =
    insert (version_id_of new_rel) (version_ids s)"
  by (auto simp: version_ids_def)

lemma active_ent_endpoint_present_ref:
  assumes "active_ent_endpoint s v"
  shows "present_ref s (Some v)"
  using assms

```

```

by (auto simp: active_ent_endpoint_def present_ref_def version_ids_def)

lemma relation_input_fresh_object:
  assumes valid:
    "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
  shows "new_rel \<notin> objects_of s"
  using valid
  by (auto simp: relation_input_valid_def)

lemma create_relation_exact_delta:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
      next_snapshot_id s' catalog"
  shows
    "objects_of s' = insert new_rel (objects_of s) \<and>
      new_rel \<notin> objects_of s \<and>
      snapshot_id_of s' = next_snapshot_id \<and>
      catalog' (version_id_of new_rel) = Some (rtype, directed)"
  using concrete relation_input_fresh_object
  by (auto simp: create_relation_step_def extend_relation_catalog_def)

lemma create_relation_typed_endpoints:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
      next_snapshot_id s' catalog"
  shows
    "src \<noteq> tgt \<and>
      active_ent_endpoint s src \<and>
      active_ent_endpoint s tgt \<and>
      source_ref_of new_rel = Some src \<and>
      target_ref_of new_rel = Some tgt"
  using concrete
  by (auto simp: create_relation_step_def relation_input_valid_def)

subsection \<open>Monotonicity of object conditions\<close>

lemma add_relation_present_ref_mono:
  assumes "present_ref s ref"
  shows "present_ref (add_relation s new_rel next_snapshot_id) ref"
  using assms
  by (cases ref) (auto simp: present_ref_def)

lemma add_relation_references_mono:
  assumes "references_complete s obj"
  shows "references_complete (add_relation s new_rel next_snapshot_id) obj"
  using assms
  by (auto simp: references_complete_def)

lemma add_relation_evidence_mono:
  assumes "evidence_well_formed s obj"
  shows "evidence_well_formed (add_relation s new_rel next_snapshot_id) obj"
  using assms
  by (auto simp: evidence_well_formed_def evidence_references_complete_def)

lemma add_relation_relation_mono:
  assumes "relation_well_formed s obj"
  shows "relation_well_formed (add_relation s new_rel next_snapshot_id) obj"
  using assms add_relation_present_ref_mono
  unfolding relation_well_formed_def
  by blast

lemma add_relation_effect_mono:
  assumes "effect_well_formed s obj"
  shows "effect_well_formed (add_relation s new_rel next_snapshot_id) obj"
  using assms add_relation_present_ref_mono
  unfolding effect_well_formed_def
  by blast

lemma add_relation_perspective_iff [simp]:
  "perspective_well_formed (add_relation s new_rel next_snapshot_id) obj
    \<longleftarrowrightarrow> perspective_well_formed s obj"
  unfolding perspective_well_formed_def
  by (simp only: add_relation_background(2))

lemma add_relation_context_iff [simp]:
  "context_well_formed (add_relation s new_rel next_snapshot_id) obj
    \<longleftarrowrightarrow> context_well_formed s obj"
  unfolding context_well_formed_def
  by (simp only: add_relation_background(3))

```

```

lemma add_relation_relevance_iff [simp]:
  "relevance_well_formed (add_relation s new_rel next_snapshot_id) obj
   \<longleftarrowrightarrow> relevance_well_formed s obj"
  unfolding relevance_well_formed_def
  by (simp only: add_relation_perspective_iff add_relation_context_iff
    add_relation_background(4))

lemma add_relation_orientation_iff [simp]:
  "orientation_well_formed (add_relation s new_rel next_snapshot_id) obj
   \<longleftarrowrightarrow> orientation_well_formed s obj"
  by (simp add: orientation_well_formed_def)

lemma add_relation_action_iff [simp]:
  "action_well_formed (add_relation s new_rel next_snapshot_id) obj
   \<longleftarrowrightarrow> action_well_formed s obj"
  unfolding action_well_formed_def
  by (simp only: add_relation_background(5))

lemma add_relation_feedback_mono:
  assumes "feedback_well_formed s obj"
  shows "feedback_well_formed (add_relation s new_rel next_snapshot_id) obj"
  using assms add_relation_present_ref_mono
  unfolding feedback_well_formed_def
  by blast

subsection \<open>Fresh identity and new-relation validity\<close>

lemma add_relation_preserves_unique_versions:
  assumes valid:
    "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
  shows "unique_versions (add_relation s new_rel next_snapshot_id)"
proof -
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
    and fresh: "version_id_of new_rel \<notin> version_ids s"
    unfolding relation_input_valid_def state_well_formed_def unique_versions_def
    by auto
  show ?thesis
    unfolding unique_versions_def
  proof (rule inj_onI)
    fix left right
    assume left_mem: "left \<in> objects_of (add_relation s new_rel next_snapshot_id)"
    and right_mem: "right \<in> objects_of (add_relation s new_rel next_snapshot_id)"
    and same_version: "version_id_of left = version_id_of right"
    show "left = right"
    proof (cases "left = new_rel")
      case True
      show ?thesis
      proof (cases "right = new_rel")
        case True
        then show ?thesis using \<open>left = new_rel\<close> by simp
      next
        case False
        then have right_old: "right \<in> objects_of s" using right_mem by simp
        have "version_id_of new_rel \<in> version_ids s"
          unfolding version_ids_def
          using same_version \<open>left = new_rel\<close> right_old
          by (metis imageI)
        then show ?thesis using fresh by blast
      qed
    next
      case False
      then have left_old: "left \<in> objects_of s" using left_mem by simp
      show ?thesis
      proof (cases "right = new_rel")
        case True
        have "version_id_of new_rel \<in> version_ids s"
          unfolding version_ids_def
          using same_version left_old True
          by (metis imageI)
        then show ?thesis using fresh by blast
      next
        case False
        then have right_old: "right \<in> objects_of s" using right_mem by simp
        show ?thesis
          using inj_onD[OF source_unique same_version left_old right_old] .
      qed
    qed
  qed

```

qed

```
lemma relation_input_new_object_conditions:
  assumes valid:
    "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
  shows
    "kind_of new_rel \<in> supported_kinds_of G \<and>
     references_complete (add_relation s new_rel next_snapshot_id) new_rel \<and>
     evidence_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     time_well_formed new_rel \<and>
     relation_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     effect_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     perspective_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     context_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     relevance_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     orientation_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     action_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     feedback_well_formed (add_relation s new_rel next_snapshot_id) new_rel"
```

```
proof -
  from valid have supported: "kind_of new_rel \<in> supported_kinds_of G"
  and kind: "kind_of new_rel = K_REL"
  and refs: "references_complete s new_rel"
  and evidence: "evidence_well_formed s new_rel"
  and time: "time_well_formed new_rel"
  and src_ref: "source_ref_of new_rel = Some src"
  and tgt_ref: "target_ref_of new_rel = Some tgt"
  and src_active: "active_ent_endpoint s src"
  and tgt_active: "active_ent_endpoint s tgt"
  and context_ok: "context_well_formed s new_rel"
  unfolding relation_input_valid_def context_well_formed_def
    context_bound_kind_def
  by auto
  have src_present: "present_ref s (Some src)"
  using active_ent_endpoint_present_ref[OF src_active] .
  have tgt_present: "present_ref s (Some tgt)"
  using active_ent_endpoint_present_ref[OF tgt_active] .
  have relation: "relation_well_formed s new_rel"
  unfolding relation_well_formed_def
  using kind src_ref tgt_ref src_present tgt_present by simp
  have refs_target:
    "references_complete (add_relation s new_rel next_snapshot_id) new_rel"
  using add_relation_references_mono[OF refs] .
  have evidence_target:
    "evidence_well_formed (add_relation s new_rel next_snapshot_id) new_rel"
  using add_relation_evidence_mono[OF evidence] .
  have relation_target:
    "relation_well_formed (add_relation s new_rel next_snapshot_id) new_rel"
  using add_relation_relation_mono[OF relation] .
  have typed_conditions:
    "effect_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     perspective_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     relevance_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     orientation_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     action_well_formed (add_relation s new_rel next_snapshot_id) new_rel \<and>
     feedback_well_formed (add_relation s new_rel next_snapshot_id) new_rel"
  using kind
  by (simp add: effect_well_formed_def perspective_well_formed_def
    perspective_bound_kind_def relevance_well_formed_def
    orientation_well_formed_def action_well_formed_def
    feedback_well_formed_def)
  show ?thesis
  using supported refs_target evidence_target time relation_target context_ok
    typed_conditions
  by simp
```

qed

subsection \<open>Operation-specific preservation theorems\<close>

```
theorem create_relation_preserves_state_well_formed:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows "state_well_formed G s'"
proof -
  from concrete have valid:
    "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
  and target: "s' = add_relation s new_rel next_snapshot_id"
  by (auto simp: create_relation_step_def)
  from valid have source_wf: "state_well_formed G s"
```

```

    by (simp add: relation_input_valid_def)
    have unique: "unique_versions (add_relation s new_rel next_snapshot_id)"
    using add_relation_preserves_unique_versions[OF valid] .
    have all_objects:
    "\<forall>obj\<in>objects_of (add_relation s new_rel next_snapshot_id).
    kind_of obj \<in> supported_kinds_of G \<and>
    references_complete (add_relation s new_rel next_snapshot_id) obj \<and>
    evidence_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    time_well_formed obj \<and>
    relation_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    effect_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    perspective_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    context_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    relevance_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    orientation_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    action_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    feedback_well_formed (add_relation s new_rel next_snapshot_id) obj"
    proof (intro ballI)
    fix obj
    assume obj_mem: "obj \<in> objects_of (add_relation s new_rel next_snapshot_id)"
    consider (new) "obj = new_rel" | (old) "obj \<in> objects_of s"
    using obj_mem by auto
    then show
    "kind_of obj \<in> supported_kinds_of G \<and>
    references_complete (add_relation s new_rel next_snapshot_id) obj \<and>
    evidence_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    time_well_formed obj \<and>
    relation_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    effect_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    perspective_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    context_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    relevance_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    orientation_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    action_well_formed (add_relation s new_rel next_snapshot_id) obj \<and>
    feedback_well_formed (add_relation s new_rel next_snapshot_id) obj"
    proof cases
    case new
    then show ?thesis
    using relation_input_new_object_conditions[OF valid] by simp
    next
    case old
    from source_wf old have source_conditions:
    "kind_of obj \<in> supported_kinds_of G \<and>
    references_complete s obj \<and> evidence_well_formed s obj \<and>
    time_well_formed obj \<and>
    relation_well_formed s obj \<and> effect_well_formed s obj \<and>
    perspective_well_formed s obj \<and> context_well_formed s obj \<and>
    relevance_well_formed s obj \<and> orientation_well_formed s obj \<and>
    action_well_formed s obj \<and> feedback_well_formed s obj"
    unfolding state_well_formed_def by blast
    from source_conditions have supported: "kind_of obj \<in> supported_kinds_of G"
    and refs: "references_complete s obj"
    and evidence: "evidence_well_formed s obj"
    and time: "time_well_formed obj"
    and relation: "relation_well_formed s obj"
    and effect: "effect_well_formed s obj"
    and perspective: "perspective_well_formed s obj"
    and context_ok: "context_well_formed s obj"
    and relevance: "relevance_well_formed s obj"
    and orientation: "orientation_well_formed s obj"
    and action: "action_well_formed s obj"
    and feedback: "feedback_well_formed s obj"
    by blast+
    have refs_target:
    "references_complete (add_relation s new_rel next_snapshot_id) obj"
    using add_relation_references_mono[OF refs] .
    have evidence_target:
    "evidence_well_formed (add_relation s new_rel next_snapshot_id) obj"
    using add_relation_evidence_mono[OF evidence] .
    have relation_target:
    "relation_well_formed (add_relation s new_rel next_snapshot_id) obj"
    using add_relation_relation_mono[OF relation] .
    have effect_target:
    "effect_well_formed (add_relation s new_rel next_snapshot_id) obj"
    using add_relation_effect_mono[OF effect] .
    have feedback_target:
    "feedback_well_formed (add_relation s new_rel next_snapshot_id) obj"
    using add_relation_feedback_mono[OF feedback] .
    show ?thesis

```

```

        using supported refs_target evidence_target time relation_target effect_target
        perspective context_ok relevance orientation action feedback_target
    by simp
qed
qed
from source_wf have source_secondary:
  "(\<forall>p\<in>projections_of s. projection_well_formed s p) \<and>
   (\<forall>c\<in>couplings_of s. coupling_well_formed s c) \<and>
   (\<forall>t\<in>transmissions_of s. transmission_well_formed s t) \<and>
   (\<forall>b\<in>boundaries_of s. boundary_well_formed b)"
  unfolding state_well_formed_def by blast
have target_secondary:
  "(\<forall>p\<in>projections_of (add_relation s new_rel next_snapshot_id).
   projection_well_formed (add_relation s new_rel next_snapshot_id) p) \<and>
   (\<forall>c\<in>couplings_of (add_relation s new_rel next_snapshot_id).
   coupling_well_formed (add_relation s new_rel next_snapshot_id) c) \<and>
   (\<forall>t\<in>transmissions_of (add_relation s new_rel next_snapshot_id).
   transmission_well_formed (add_relation s new_rel next_snapshot_id) t) \<and>
   (\<forall>b\<in>boundaries_of (add_relation s new_rel next_snapshot_id).
   boundary_well_formed b)"
  using source_secondary
  by (simp add: projection_well_formed_def coupling_well_formed_def
      transmission_well_formed_def)
show ?thesis
  unfolding target_state_well_formed_def
  using unique_all_objects target_secondary by blast
qed

theorem create_relation_preserves_identity:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows "unique_versions s' \<and> immutable_preserved s s'"
proof -
  from concrete have valid:
    "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
    and target: "s' = add_relation s new_rel next_snapshot_id"
    by (auto simp: create_relation_step_def)
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
    and fresh: "version_id_of new_rel \<notin> version_ids s"
    unfolding relation_input_valid_def state_well_formed_def unique_versions_def
    by auto
  have immutable: "immutable_preserved s s'"
    unfolding immutable_preserved_def target
  proof (intro ballI allI impI)
    fix old_obj target_obj
    assume old_mem: "old_obj \<in> objects_of s"
    and target_mem:
      "target_obj \<in> objects_of (add_relation s new_rel next_snapshot_id)"
    and same_version: "version_id_of old_obj = version_id_of target_obj"
    show "old_obj = target_obj"
    proof (cases "target_obj = new_rel")
      case True
      have "version_id_of new_rel \<in> version_ids s"
        unfolding version_ids_def
        using old_mem same_version True
        by (metis imageI)
      then show ?thesis using fresh by blast
    next
      case False
      then have target_old: "target_obj \<in> objects_of s" using target_mem by simp
      show ?thesis
        using inj_onD[OF source_unique same_version old_mem target_old] .
    qed
  qed
  have target_unique: "unique_versions s'"
    unfolding target
    using add_relation_preserves_unique_versions[OF valid] .
  show ?thesis using target_unique immutable by blast
qed

theorem create_relation_preserves_catalog:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows
    "relation_catalog_well_formed s' catalog' \<and>
     catalog' (version_id_of new_rel) = Some (rtype, directed)"
proof -

```



```

from concrete have valid:
  "relation_input_valid G s catalog new_rel rtype directed src tgt next_snapshot_id"
  and target: "s' = add_relation s new_rel next_snapshot_id"
  and catalog_target:
    "catalog' = extend_relation_catalog catalog new_rel rtype directed"
  by (auto simp: create_relation_step_def)
from valid have source_catalog: "relation_catalog_well_formed s catalog"
  by (simp add: relation_input_valid_def)
have target_catalog: "relation_catalog_well_formed s' catalog'"
  unfolding relation_catalog_well_formed_def target catalog_target
  extend_relation_catalog_def
  using source_catalog valid
  unfolding relation_catalog_well_formed_def relation_input_valid_def
  by auto
show ?thesis
  using target_catalog catalog_target
  by (simp add: extend_relation_catalog_def)
qed

```

```

theorem create_relation_preserves_references_time_context:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows
    "\<forall>obj\<in>objects_of s'.
     references_complete s' obj \<and>
     evidence_well_formed s' obj \<and>
     time_well_formed obj \<and>
     context_well_formed s' obj"
  using create_relation_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def

```

Beweisstatus dieser D.6-Fassung: kernelgeprüft. Der integrierte saubere und verbose Isabelle2025-2-Build verarbeitete am 8. August 2026 alle sieben Theorien zu 100 % und endete mit Exit-Code 0. Für T3 sind damit Parser-, Typ- und Kernelakzeptanz des gepinnten Quellstands belegt. Der Nachweis umfasst die im Quelltext formulierten Erhaltungs- und Refinementsätze; er beweist weder die Vollständigkeit aller möglichen Relationstypen noch allgemeine Konservativität oder Produktivengine-Äquivalenz.

```

  by blast

theorem create_relation_preserves_evidence_integrity:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows "\<forall>obj\<in>objects_of s'. evidence_well_formed s' obj"
  using create_relation_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def
  by blast

theorem create_relation_refines_generic_step:
  assumes concrete:
    "create_relation_step G s catalog new_rel rtype directed src tgt
     next_snapshot_id s' catalog'"
  shows "step G s CreateRelation s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
    and allowed: "CreateRelation \<in> allowed_operations_of G"
    by (auto simp: create_relation_step_def relation_input_valid_def)
  have target_wf: "state_well_formed G s'"
    using create_relation_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
    using create_relation_preserves_identity[OF concrete] by blast
  show ?thesis
    unfolding step_def
    using source_wf target_wf allowed immutable by blast
qed
end

```

D.7 Konkrete T4-Semantik und Erhaltung für HypothesizeEffect

Die siebte Theorie UZA_0_9_1_Hypothesize_Effect importiert T3 und erweitert den unveränderten Snapshot-Kerntyp um einen versionierten effect_catalog. Der Katalog ordnet jeder erzeugten EFF-Version einen effect_hypothesis_payload zu: verursachende Relationsversion, optionales Ziel, optionaler effect_vector, Unsicherheit und Profilmodell. option bildet offene Ziel- oder Vektorstruktur explizit ab; es gibt weder Sentinel-IDs noch stilles Zero-Fill.

effect_input_valid verlangt einen wohlgeformten Ausgangszustand und Katalog, die zugelassene Operation HypothesizeEffect, den unterstützten Typ K_EFF, eine aktive typisierte K_REL-Ursache, gegebenenfalls ein aktives Ziel, eine vorhandene Perspektive, einen vorhandenen Kontext und ein vorhandenes Profilmodell, Unsicherheit im geschlossenen Intervall [0,1], frische Objekt- und Versionsidentitäten, Lifecycle Hypothesis, vollständige Referenzen, gültige Evidenz- und Zeitstruktur sowie eine fortgeschriebene Snapshot-ID. add_effect_hypothesis verändert ausschließlich Snapshot-ID und Objektmenge; extend_effect_catalog ergänzt ausschließlich den neuen Versionsschlüssel. Die Zielwohlgeformtheit ist keine Vorbedingung.

Die kernelgeprüfte Theorie enthält hypothesize_effect_exact_delta, hypothesize_effect_origin_and_openness, hypothesize_effect_preserves_state_well_formed, hypothesize_effect_preserves_identity, hypothesize_effect_preserves_catalog, hypothesize_effect_preserves_references_time_perspective_context, hypothesize_effect_preserves_evidence_integrity und hypothesize_effect_refines_generic_step. Der finale Quelltext umfasst 718 Zeilen, enthält weder sorry noch oops noch admit und besitzt SHA-256 2eb8e1d262cb3790f6c446839fb6809de3c2837d20705925507e272c860e2fc4.

```
theory UZA_0_9_1_Hypothesize_Effect
  imports UZA_0_9_1_Create_Relation
  begin

  section \<open>T4 HypothesizeEffect: typed effect-hypothesis semantics\<close>

  text \<open>
    T4 creates exactly one fresh K_EFF object from one active, typed K_REL
    origin. Target and V10 profile may each remain explicitly open. Their
    openness is represented by option types rather than sentinel values or
    silent zero filling. Perspective, context, time, uncertainty, profile
    model, references, and provenance are checked before construction.

    The kernel-checked base snapshot type remains unchanged. The effect
    profile and uncertainty are persisted in a conservative typed side catalog
    keyed by the effect version identifier. Target-state well-formedness is
    derived and is not an input assumption.
  \<close>

  record effect_hypothesis_payload =
    hypothesis_cause_of :: version_id
    hypothesis_target_of :: "version_id option"
    hypothesis_vector_of :: "effect_vector option"
    hypothesis_uncertainty_of :: real
    hypothesis_profile_model_of :: model_id

  type_synonym effect_catalog =
    "version_id \<rightarrow> effect_hypothesis_payload"

  definition active_relation_origin ::
    "snapshot \<Rightarrow> version_id \<Rightarrow> bool" where
    "active_relation_origin s v \<longleftarrow>
      (\<exists>rel\<in>objects_of s.
        version_id_of rel = v \<and>
        kind_of rel = K_REL \<and>
        lifecycle_of rel \<notin> {Revised, Rejected, Archived})"
```

```

definition active_effect_target ::
  "snapshot \<Rightarrow> effect_hypothesis_payload \<Rightarrow> bool" where
  "active_effect_target s v \<longleftarrowtrirightarrow>
    (\<exists>obj\<in>objects_of s.
      version_id_of obj = v \<and>
      lifecycle_of obj \<notin> {Revised, Rejected, Archived})"

definition effect_payload_valid ::
  "uza_object \<Rightarrow> effect_hypothesis_payload \<Rightarrow> bool" where
  "effect_payload_valid eff payload \<longleftarrowtrirightarrow>
    kind_of eff = K_EFF \<and>
    effect_origin_of eff = Some (hypothesis_cause_of payload) \<and>
    source_ref_of eff = Some (hypothesis_cause_of payload) \<and>
    target_ref_of eff = hypothesis_target_of payload \<and>
    model_of eff = Some (hypothesis_profile_model_of payload) \<and>
    0 \<le> hypothesis_uncertainty_of payload \<and>
    hypothesis_uncertainty_of payload \<le> 1 \<and>
    (case hypothesis_vector_of payload of
      None \<Rightarrow> True
    | Some vector \<Rightarrow> effect_vector_valid vector)"

definition effect_catalog_well_formed ::
  "snapshot \<Rightarrow> effect_catalog \<Rightarrow> bool" where
  "effect_catalog_well_formed s catalog \<longleftarrowtrirightarrow>
    (\<forall>v payload. catalog v = Some payload \<longrightarrow>
      (\<exists>eff\<in>objects_of s.
        version_id_of eff = v \<and> effect_payload_valid eff payload))"

definition effect_target_refs ::
  "version_id \<Rightarrow> version_id option \<Rightarrow> version_id set" where
  "effect_target_refs cause target =
    insert cause (case target of None \<Rightarrow> {} | Some v \<Rightarrow> {v})"

definition effect_input_valid ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> relation_catalog \<Rightarrow>
    effect_catalog \<Rightarrow> uza_object \<Rightarrow> effect_hypothesis_payload
    \<Rightarrow> nat \<Rightarrow> bool" where
  "effect_input_valid G s relation_catalog effect_catalog new_eff payload
    next_snapshot_id \<longleftarrowtrirightarrow>
    state_well_formed G s \<and>
    relation_catalog_well_formed s relation_catalog \<and>
    effect_catalog_well_formed s effect_catalog \<and>
    HypothesizeEffect \<in> allowed_operations_of G \<and>
    K_EFF \<in> supported_kinds_of G \<and>
    kind_of new_eff = K_EFF \<and>
    lifecycle_of new_eff = Hypothesis \<and>
    object_id_of new_eff \<notin> object_id_of ` objects_of s \<and>
    version_id_of new_eff \<notin>
      version_ids s \<union> historical_versions_of s \<and>
    effect_catalog (version_id_of new_eff) = None \<and>
    previous_version_of new_eff = None \<and>
    active_relation_origin s (hypothesis_cause_of payload) \<and>
    relation_catalog (hypothesis_cause_of payload) \<noteq> None \<and>
    effect_origin_of new_eff = Some (hypothesis_cause_of payload) \<and>
    source_ref_of new_eff = Some (hypothesis_cause_of payload) \<and>
    target_ref_of new_eff = hypothesis_target_of payload \<and>
    (case hypothesis_target_of payload of
      None \<Rightarrow> True
    | Some target \<Rightarrow> active_effect_target s target) \<and>
    effect_target_refs (hypothesis_cause_of payload)
      (hypothesis_target_of payload) \<subsepeq> references_of new_eff \<and>
    basis_refs_of new_eff =
      effect_target_refs (hypothesis_cause_of payload)
        (hypothesis_target_of payload) \<and>
    references_complete s new_eff \<and>
    evidence_well_formed s new_eff \<and>
    time_well_formed new_eff \<and>
    (\<exists>p\<in>perspectives_of s. perspective_of new_eff = Some p) \<and>
    (\<exists>c\<in>contexts_of s. context_of new_eff = Some c) \<and>
    hypothesis_profile_model_of payload \<in> models_of s \<and>
    model_of new_eff = Some (hypothesis_profile_model_of payload) \<and>
    authority_of new_eff = None \<and>
    execution_ref_of new_eff = None \<and>
    observation_refs_of new_eff = {} \<and>
    object_status_of new_eff = Some Applicable \<and>
    effect_payload_valid new_eff payload \<and>
    snapshot_id_of s < next_snapshot_id"

definition add_effect_hypothesis ::
  "snapshot \<Rightarrow> uza_object \<Rightarrow> nat \<Rightarrow> snapshot" where

```

```

"add_effect_hypothesis s new_eff next_snapshot_id =
  s\<lparr> snapshot_id_of := next_snapshot_id,
  objects_of := insert new_eff (objects_of s) \<rparr>"

definition extend_effect_catalog ::
  "effect_catalog \<Rightarrow> uza_object \<Rightarrow> effect_hypothesis_payload
  \<Rightarrow> effect_catalog" where
  "extend_effect_catalog catalog new_eff payload =
    catalog(version_id_of new_eff \<mapsto> payload)"

definition hypothesize_effect_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> relation_catalog \<Rightarrow>
  effect_catalog \<Rightarrow> uza_object \<Rightarrow> effect_hypothesis_payload
  \<Rightarrow> nat \<Rightarrow> snapshot \<Rightarrow> effect_catalog \<Rightarrow> bool" where
  "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
    next_snapshot_id s' effect_catalog' \<longlefttrightarrow>
    effect_input_valid G s relation_catalog effect_catalog new_eff payload
    next_snapshot_id \<and>
    s' = add_effect_hypothesis s new_eff next_snapshot_id \<and>
    effect_catalog' = extend_effect_catalog effect_catalog new_eff payload"

subsection \<open>Exact update and semantic input facts\<close>

lemma add_effect_objects [simp]:
  "objects_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    insert new_eff (objects_of s)"
  by (simp add: add_effect_hypothesis_def)

lemma add_effect_snapshot_id [simp]:
  "snapshot_id_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    next_snapshot_id"
  by (simp add: add_effect_hypothesis_def)

lemma add_effect_background [simp]:
  "historical_versions_of
    (add_effect_hypothesis s new_eff next_snapshot_id) =
    historical_versions_of s"
  "perspectives_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    perspectives_of s"
  "contexts_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    contexts_of s"
  "models_of (add_effect_hypothesis s new_eff next_snapshot_id) = models_of s"
  "authorities_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    authorities_of s"
  "contains_edges_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    contains_edges_of s"
  "projections_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    projections_of s"
  "couplings_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    couplings_of s"
  "transmissions_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    transmissions_of s"
  "boundaries_of (add_effect_hypothesis s new_eff next_snapshot_id) =
    boundaries_of s"
  by (simp_all add: add_effect_hypothesis_def)

lemma add_effect_version_ids [simp]:
  "version_ids (add_effect_hypothesis s new_eff next_snapshot_id) =
    insert (version_id_of new_eff) (version_ids s)"
  by (auto simp: version_ids_def)

lemma active_relation_origin_present_ref:
  assumes "active_relation_origin s v"
  shows "present_ref s (Some v)"
  using assms
  by (auto simp: active_relation_origin_def present_ref_def version_ids_def)

lemma active_effect_target_present_ref:
  assumes "active_effect_target s v"
  shows "present_ref s (Some v)"
  using assms
  by (auto simp: active_effect_target_def present_ref_def version_ids_def)

lemma effect_input_fresh_object:
  assumes valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
    next_snapshot_id"
  shows "new_eff \<notin> objects_of s"
  using valid
  by (auto simp: effect_input_valid_def)

```

```

lemma hypothesize_effect_exact_delta:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id s' effect_catalog'"
  shows
    "objects_of s' = insert new_eff (objects_of s) \<and>
     new_eff \<notin> objects_of s \<and>
     snapshot_id_of s' = next_snapshot_id \<and>
     effect_catalog' (version_id_of new_eff) = Some payload"
  using concrete effect_input_fresh_object
  by (auto simp: hypothesize_effect_step_def extend_effect_catalog_def)

lemma hypothesize_effect_origin_and_openness:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id s' effect_catalog'"
  shows
    "active_relation_origin s (hypothesis_cause_of payload) \<and>
     relation_catalog (hypothesis_cause_of payload) \<noteq> None \<and>
     effect_origin_of new_eff = Some (hypothesis_cause_of payload) \<and>
     source_ref_of new_eff = Some (hypothesis_cause_of payload) \<and>
     target_ref_of new_eff = hypothesis_target_of payload \<and>
     (case hypothesis_target_of payload of
       None \<Rightarrow> True
     | Some target \<Rightarrow> active_effect_target s target) \<and>
     (case hypothesis_vector_of payload of
       None \<Rightarrow> True
     | Some vector \<Rightarrow> effect_vector_valid vector) \<and>
     0 \<le> hypothesis_uncertainty_of payload \<and>
     hypothesis_uncertainty_of payload \<le> 1"
  using concrete
  by (auto simp: hypothesize_effect_step_def effect_input_valid_def
    effect_payload_valid_def split: option.splits)

subsection \<open>Monotonicity of inherited state conditions\<close>

lemma add_effect_present_ref_mono:
  assumes "present_ref s ref"
  shows "present_ref (add_effect_hypothesis s new_eff next_snapshot_id) ref"
  using assms
  by (cases ref) (auto simp: present_ref_def)

lemma add_effect_references_mono:
  assumes "references_complete s obj"
  shows "references_complete
    (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using assms
  by (auto simp: references_complete_def)

lemma add_effect_evidence_mono:
  assumes "evidence_well_formed s obj"
  shows "evidence_well_formed
    (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using assms
  by (auto simp: evidence_well_formed_def evidence_references_complete_def)

lemma add_effect_relation_mono:
  assumes "relation_well_formed s obj"
  shows "relation_well_formed
    (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using assms add_effect_present_ref_mono
  unfolding relation_well_formed_def
  by blast

lemma add_effect_effect_mono:
  assumes "effect_well_formed s obj"
  shows "effect_well_formed
    (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using assms add_effect_present_ref_mono
  unfolding effect_well_formed_def
  by blast

lemma add_effect_perspective_iff [simp]:
  "perspective_well_formed
    (add_effect_hypothesis s new_eff next_snapshot_id) obj
    \<longleftarrowrightarrow> perspective_well_formed s obj"
  unfolding perspective_well_formed_def
  by (simp only: add_effect_background(2))

```

```

lemma add_effect_context_iff [simp]:
  "context_well_formed (add_effect_hypothesis s new_eff next_snapshot_id) obj
   \<longlefttrightarrow> context_well_formed s obj"
  unfolding context_well_formed_def
  by (simp only: add_effect_background(3))

lemma add_effect_relevance_iff [simp]:
  "relevance_well_formed
   (add_effect_hypothesis s new_eff next_snapshot_id) obj
   \<longlefttrightarrow> relevance_well_formed s obj"
  unfolding relevance_well_formed_def
  by (simp only: add_effect_perspective_iff add_effect_context_iff
    add_effect_background(4))

lemma add_effect_orientation_iff [simp]:
  "orientation_well_formed
   (add_effect_hypothesis s new_eff next_snapshot_id) obj
   \<longlefttrightarrow> orientation_well_formed s obj"
  by (simp add: orientation_well_formed_def)

lemma add_effect_action_iff [simp]:
  "action_well_formed (add_effect_hypothesis s new_eff next_snapshot_id) obj
   \<longlefttrightarrow> action_well_formed s obj"
  unfolding action_well_formed_def
  by (simp only: add_effect_background(5))

lemma add_effect_feedback_mono:
  assumes "feedback_well_formed s obj"
  shows "feedback_well_formed
   (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using assms add_effect_present_ref_mono
  unfolding feedback_well_formed_def
  by blast

subsection \<open>Fresh identity and new-effect validity\<close>

lemma add_effect_preserves_unique_versions:
  assumes valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id"
  shows "unique_versions
   (add_effect_hypothesis s new_eff next_snapshot_id)"
proof -
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
  and fresh: "version_id_of new_eff \<notin> version_ids s"
  unfolding effect_input_valid_def state_well_formed_def unique_versions_def
  by auto
  show ?thesis
  unfolding unique_versions_def
proof (rule inj_onI)
  fix left right
  assume left_mem:
    "left \<in> objects_of (add_effect_hypothesis s new_eff next_snapshot_id)"
  and right_mem:
    "right \<in> objects_of (add_effect_hypothesis s new_eff next_snapshot_id)"
  and same_version: "version_id_of left = version_id_of right"
  show "left = right"
proof (cases "left = new_eff")
  case True
  show ?thesis
proof (cases "right = new_eff")
  case True
  then show ?thesis using \<open>left = new_eff\<close> by simp
next
  case False
  then have right_old: "right \<in> objects_of s" using right_mem by simp
  have "version_id_of new_eff \<in> version_ids s"
  unfolding version_ids_def
  using same_version \<open>left = new_eff\<close> right_old
  by (metis imageI)
  then show ?thesis using fresh by blast
qed
next
  case False
  then have left_old: "left \<in> objects_of s" using left_mem by simp
  show ?thesis
proof (cases "right = new_eff")
  case True
  have "version_id_of new_eff \<in> version_ids s"
  unfolding version_ids_def

```

```

        using same_version left_old True
        by (metis imageI)
        then show ?thesis using fresh by blast
    next
    case False
    then have right_old: "right \<in> objects_of s" using right_mem by simp
    show ?thesis
        using inj_onD[OF source_unique same_version left_old right_old] .
qed
qed
qed
qed

lemma effect_input_new_object_conditions:
  assumes valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id"
  shows
    "kind_of new_eff \<in> supported_kinds_of G \<and>
     references_complete
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     evidence_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     time_well_formed new_eff \<and>
     relation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     effect_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     perspective_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     context_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     relevance_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     orientation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     action_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     feedback_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff"
proof -
  from valid have supported: "kind_of new_eff \<in> supported_kinds_of G"
  and kind: "kind_of new_eff = K_EFF"
  and lifecycle: "lifecycle_of new_eff = Hypothesis"
  and refs: "references_complete s new_eff"
  and evidence: "evidence_well_formed s new_eff"
  and time: "time_well_formed new_eff"
  and perspective: "perspective_well_formed s new_eff"
  and context_ok: "context_well_formed s new_eff"
  unfolding effect_input_valid_def perspective_well_formed_def
    perspective_bound_kind_def context_well_formed_def context_bound_kind_def
  by auto
  have refs_target:
    "references_complete
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff"
  using add_effect_references_mono[OF refs] .
  have evidence_target:
    "evidence_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff"
  using add_effect_evidence_mono[OF evidence] .
  have typed_conditions:
    "relation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     effect_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     relevance_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     orientation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     action_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff \<and>
     feedback_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) new_eff"
  using kind lifecycle
  by (simp add: relation_well_formed_def effect_well_formed_def
    relevance_well_formed_def orientation_well_formed_def
    action_well_formed_def feedback_well_formed_def)
  show ?thesis
    using supported refs_target evidence_target time perspective context_ok
    typed_conditions

```

```

    by simp
qed

subsection \<open>Operation-specific preservation theorems\<close>

theorem hypothesize_effect_preserves_state_well_formed:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id s' effect_catalog'"
  shows "state_well_formed G s'"
proof -
  from concrete have valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id"
  and target: "s' = add_effect_hypothesis s new_eff next_snapshot_id"
  by (auto simp: hypothesize_effect_step_def)
  from valid have source_wf: "state_well_formed G s"
  by (simp add: effect_input_valid_def)
  have unique:
    "unique_versions (add_effect_hypothesis s new_eff next_snapshot_id)"
  using add_effect_preserves_unique_versions[OF valid] .
  have all_objects:
    "\<forall>obj\<in>objects_of (add_effect_hypothesis s new_eff next_snapshot_id).
     kind_of obj \<in> supported_kinds_of G \<and>
     references_complete
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     evidence_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     time_well_formed obj \<and>
     relation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     effect_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     perspective_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     context_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     relevance_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     orientation_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     action_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
     feedback_well_formed
     (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  proof (intro ballI)
    fix obj
    assume obj_mem:
      "obj \<in> objects_of (add_effect_hypothesis s new_eff next_snapshot_id)"
    consider (new) "obj = new_eff" | (old) "obj \<in> objects_of s"
    using obj_mem by auto
    then show
      "kind_of obj \<in> supported_kinds_of G \<and>
       references_complete
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       evidence_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       time_well_formed obj \<and>
       relation_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       effect_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       perspective_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       context_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       relevance_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       orientation_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       action_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj \<and>
       feedback_well_formed
       (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  proof cases
    case new
    then show ?thesis
      using effect_input_new_object_conditions[OF valid] by simp
  next
    case old

```



```

from source_wf old have source_conditions:
  "kind_of obj \<in> supported_kinds_of G \<and>
  references_complete s obj \<and> evidence_well_formed s obj \<and>
  time_well_formed obj \<and> relation_well_formed s obj \<and>
  effect_well_formed s obj \<and> perspective_well_formed s obj \<and>
  context_well_formed s obj \<and> relevance_well_formed s obj \<and>
  orientation_well_formed s obj \<and> action_well_formed s obj \<and>
  feedback_well_formed s obj"
  unfolding state_well_formed_def by blast
from source_conditions have supported:
  "kind_of obj \<in> supported_kinds_of G"
  and refs: "references_complete s obj"
  and evidence: "evidence_well_formed s obj"
  and time: "time_well_formed obj"
  and relation: "relation_well_formed s obj"
  and effect: "effect_well_formed s obj"
  and perspective: "perspective_well_formed s obj"
  and context_ok: "context_well_formed s obj"
  and relevance: "relevance_well_formed s obj"
  and orientation: "orientation_well_formed s obj"
  and action: "action_well_formed s obj"
  and feedback: "feedback_well_formed s obj"
  by blast+
have refs_target:
  "references_complete
  (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using add_effect_references_mono[OF refs] .
have evidence_target:
  "evidence_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using add_effect_evidence_mono[OF evidence] .
have relation_target:
  "relation_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using add_effect_relation_mono[OF relation] .
have effect_target:
  "effect_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using add_effect_effect_mono[OF effect] .
have feedback_target:
  "feedback_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) obj"
  using add_effect_feedback_mono[OF feedback] .
show ?thesis
  using supported refs_target evidence_target time relation_target
  effect_target perspective context_ok relevance orientation action
  feedback_target
  by simp
qed
qed
from source_wf have source_secondary:
  "(\<forall>p\<in>projections_of s. projection_well_formed s p) \<and>
  (\<forall>c\<in>couplings_of s. coupling_well_formed s c) \<and>
  (\<forall>t\<in>transmissions_of s. transmission_well_formed s t) \<and>
  (\<forall>b\<in>boundaries_of s. boundary_well_formed b)"
  unfolding state_well_formed_def by blast
have target_secondary:
  "(\<forall>p\<in>projections_of
  (add_effect_hypothesis s new_eff next_snapshot_id).
  projection_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) p) \<and>
  (\<forall>c\<in>couplings_of
  (add_effect_hypothesis s new_eff next_snapshot_id).
  coupling_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) c) \<and>
  (\<forall>t\<in>transmissions_of
  (add_effect_hypothesis s new_eff next_snapshot_id).
  transmission_well_formed
  (add_effect_hypothesis s new_eff next_snapshot_id) t) \<and>
  (\<forall>b\<in>boundaries_of
  (add_effect_hypothesis s new_eff next_snapshot_id).
  boundary_well_formed b)"
  using source_secondary
  by (simp add: projection_well_formed_def coupling_well_formed_def
    transmission_well_formed_def)
show ?thesis
  unfolding target state_well_formed_def
  using unique_all_objects target_secondary by blast
qed

```

```

theorem hypothesize_effect_preserves_identity:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id s' effect_catalog'"
  shows "unique_versions s' \<and> immutable_preserved s s'"
proof -
  from concrete have valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id"
  and target: "s' = add_effect_hypothesis s new_eff next_snapshot_id"
  by (auto simp: hypothesize_effect_step_def)
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
  and fresh: "version_id_of new_eff \<notin> version_ids s"
  unfolding effect_input_valid_def state_well_formed_def unique_versions_def
  by auto
  have immutable: "immutable_preserved s s'"
  unfolding immutable_preserved_def target
  proof (intro ballI allI impI)
    fix old_obj target_obj
    assume old_mem: "old_obj \<in> objects_of s"
    and target_mem:
      "target_obj \<in>
       objects_of (add_effect_hypothesis s new_eff next_snapshot_id)"
    and same_version:
      "version_id_of old_obj = version_id_of target_obj"
    show "old_obj = target_obj"
    proof (cases "target_obj = new_eff")
      case True
      have "version_id_of new_eff \<in> version_ids s"
      unfolding version_ids_def
      using old_mem same_version True
      by (metis imageI)
      then show ?thesis using fresh by blast
    next
      case False
      then have target_old: "target_obj \<in> objects_of s"
      using target_mem by simp
      show ?thesis
      using inj_onD[OF source_unique same_version old_mem target_old] .
    qed
  qed
  have target_unique: "unique_versions s'"
  unfolding target
  using add_effect_preserves_unique_versions[OF valid] .
  show ?thesis using target_unique immutable by blast
qed

theorem hypothesize_effect_preserves_catalog:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id s' effect_catalog'"
  shows
    "effect_catalog_well_formed s' effect_catalog' \<and>
     effect_catalog' (version_id_of new_eff) = Some payload"
proof -
  from concrete have valid:
    "effect_input_valid G s relation_catalog effect_catalog new_eff payload
     next_snapshot_id"
  and target: "s' = add_effect_hypothesis s new_eff next_snapshot_id"
  and catalog_target:
    "effect_catalog' = extend_effect_catalog effect_catalog new_eff payload"
  by (auto simp: hypothesize_effect_step_def)
  from valid have source_catalog:
    "effect_catalog_well_formed s effect_catalog"
  and payload_valid: "effect_payload_valid new_eff payload"
  and fresh_key: "effect_catalog (version_id_of new_eff) = None"
  by (auto simp: effect_input_valid_def)
  have target_catalog: "effect_catalog_well_formed s' effect_catalog'"
  unfolding effect_catalog_well_formed_def target catalog_target
  extend_effect_catalog_def
  using source_catalog payload_valid fresh_key
  unfolding effect_catalog_well_formed_def
  by auto
  show ?thesis
  using target_catalog catalog_target
  by (simp add: extend_effect_catalog_def)
qed

theorem hypothesize_effect_preserves_references_time_context:
  assumes concrete:

```

```

    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
      next_snapshot_id s' effect_catalog'"
shows
  "\<forall>obj\<in>objects_of s'.
    references_complete s' obj \<and>
    evidence_well_formed s' obj \<and>
    time_well_formed obj \<and>
    perspective_well_formed s' obj \<and>
    context_well_formed s' obj"
using hypothesize_effect_preserves_state_well_formed[OF concrete]
unfolding state_well_formed_def
by blast

theorem hypothesize_effect_preserves_evidence_integrity:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
      next_snapshot_id s' effect_catalog'"
  shows "\<forall>obj\<in>objects_of s'. evidence_well_formed s' obj"
  using hypothesize_effect_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def
  by blast

theorem hypothesize_effect_refines_generic_step:
  assumes concrete:
    "hypothesize_effect_step G s relation_catalog effect_catalog new_eff payload
      next_snapshot_id s' effect_catalog'"
  shows "step G s HypothesizeEffect s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
  and allowed: "HypothesizeEffect \<in> allowed_operations_of G"
  by (auto simp: hypothesize_effect_step_def effect_input_valid_def)
  have target_wf: "state_well_formed G s'"
  using hypothesize_effect_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
  using hypothesize_effect_preserves_identity[OF concrete] by blast
  show ?thesis
  unfolding step_def
  using source_wf target_wf allowed immutable by blast
qed

end

```

Beweisstatus dieser D.7-Fassung: kernelgeprüft. Der ausgeführte Clean-Build startete am 8. August 2026 um 19:19:25 CEST und endete um 19:19:55 CEST. Alle sieben Theorien wurden zu 100 % verarbeitet; die Session benötigte 25,284 s, der Gesamtlauf 29 s und endete mit Exit-Code 0. Der T4-Quellhash lautet 2eb8e1d262cb3790f6c446839fb6809de3c2837d20705925507e272c860e2fc4; ROOT besitzt SHA-256 210a567ba560a9c2ed66f1ccdaa8ec355a2b4dad8504afc4708369f1fe3ed7b2; das unveränderte Buildlog besitzt SHA-256 7a49a99e15790d7324def03c5078600e4cb75bd2715b9439380fe711e85daa06. Belegt sind Parser-, Typ- und Kernelakzeptanz der formulierten konditionalen Semantik sowie ihrer Erhaltungs- und Refinementsätze. Nicht belegt sind empirische Richtigkeit, Vollständigkeit der Wirkungstaxonomie, Existenz gültiger Eingaben in jeder Domäne, allgemeine Konservativität oder Produktivengine-Äquivalenz.

D.8 Konkrete T5-Semantik und Erhaltung für IntegrateEvidence

T5 schließt die erste lifecycle-verändernde Kernoperation. Die Semantik ist konstruktiv: Aus einem wohlgeformten Ausgangszustand, einem aktiven Effect-Katalog, disjunkter Aktiv/Historie-Struktur, konsistenten Objekt- und Payload-Archiven sowie einem Evidenz-Ledger wird der Nachzustand berechnet. Der Vorgänger bleibt als vollständiges unverändertes Objekt auffindbar; das aktive Objekt erhält eine frische `version_id` bei gleicher `object_id`. Damit wird Copy-on-Write erstmals nicht nur als Engine-Prinzip, sondern als beweisbare Operationssemantik realisiert.

Die Assessment-Semantik unterscheidet `EvidenceSupports`, `EvidenceDifferentiates` und `EvidenceContradicts`. Evidenzreferenzen müssen nichtleer, aktiv und auflösbar sein und dürfen weder die neue noch die unmittelbare Vorgängerversion zur Selbststützung verwenden. Unsicherheit bleibt in $[0,1]$, ihre absolute Änderung ist durch `evidence_strength` begrenzt. Lifecycle-Fortschreibung ist abgeleitet: Hypothesis plus stützende Evidenz ergibt `Supported`; `Supported` kann nur bei erfüllter Evidenz- und Unsicherheitsschwelle `Validated` werden; widersprechende Evidenz ergibt `Rejected`; differenzierende Evidenz muss eine inhaltliche Ziel- oder Vektoränderung tragen.

```
theory UZA_0_9_1_Integrate_Evidence
  imports UZA_0_9_1_Hypothesize_Effect
begin

section \<open>T5 IntegrateEvidence: versioned evidence integration\<close>

text \<open>
  T5 does not mutate an active effect version. It replaces one active
  K_EFF hypothesis by one fresh version of the same logical object, archives
  the complete predecessor object and its effect payload, accumulates typed
  evidence, and derives the successor lifecycle from the evidence
  disposition. The construction therefore makes copy-on-write, provenance,
  uncertainty movement, and the prohibition of self-support explicit.

  The operation is deliberately restricted to Hypothesis and Supported.
  Evidence applied to an already Validated effect belongs to T13
  ReviseObject. This boundary prevents T5 from silently serving as a general
  revision operator.
\<close>

datatype evidence_disposition =
  EvidenceSupports | EvidenceDifferentiates | EvidenceContradicts

record evidence_assessment =
  evidence_subject_of :: object_id
  evidence_prior_version_of :: version_id
  evidence_refs_of :: "version_id set"
  evidence_disposition_of :: evidence_disposition
  evidence_strength_of :: real
  evidence_post_uncertainty_of :: real
  evidence_post_target_of :: "version_id option"
  evidence_post_vector_of :: "effect_vector option"
  evidence_assessment_model_of :: model_id
  validation_uncertainty_max_of :: real
  validation_evidence_min_of :: nat

type_synonym object_archive = "version_id \<rightarrow> uza_object"
type_synonym effect_payload_archive =
  "version_id \<rightarrow> effect_hypothesis_payload"
type_synonym evidence_ledger =
  "version_id \<rightarrow> evidence_assessment"

definition active_history_disjoint :: "snapshot \<rightarrow> bool" where
  "active_history_disjoint s \<longleftarrow>
    version_ids s \<inter> historical_versions_of s = {}"

definition object_archive_well_formed ::
  "snapshot \<rightarrow> object_archive \<rightarrow> bool" where
```

```

"object_archive_well_formed s archive \<longleftarrow\rightarrow
  (\<forall>v obj. archive v = Some obj \<longrightarrow\rightarrow
    version_id_of obj = v \<and>
    v \<in> historical_versions_of s \<and>
    v \<notin> version_ids s)"

definition resolved_version_object ::
  "snapshot \<Rightarrow\rightarrow object_archive \<Rightarrow\rightarrow version_id \<Rightarrow\rightarrow
    uza_object \<Rightarrow\rightarrow bool" where
  "resolved_version_object s archive v obj \<longleftarrow\rightarrow
    (obj \<in> objects_of s \<and> version_id_of obj = v) \<or>
    archive v = Some obj"

definition effect_payload_archive_well_formed ::
  "object_archive \<Rightarrow\rightarrow effect_payload_archive \<Rightarrow\rightarrow bool" where
  "effect_payload_archive_well_formed archive payload_archive \<longleftarrow\rightarrow
    (\<forall>v payload. payload_archive v = Some payload \<longrightarrow\rightarrow
      (\<exists>eff. archive v = Some eff \<and>
        effect_payload_valid eff payload))"

definition evidence_entry_well_formed ::
  "snapshot \<Rightarrow\rightarrow object_archive \<Rightarrow\rightarrow version_id \<Rightarrow\rightarrow
    evidence_assessment \<Rightarrow\rightarrow bool" where
  "evidence_entry_well_formed s archive v assessment \<longleftarrow\rightarrow
    (\<exists>eff. resolved_version_object s archive v eff \<and>
      kind_of eff = K_EFF \<and>
      evidence_subject_of assessment = object_id_of eff \<and>
      previous_version_of eff = Some (evidence_prior_version_of assessment) \<and>
      evidence_refs_of assessment \<noteq> {} \<and>
      evidence_refs_of assessment \<subteq> evidence_of eff \<and>
      evidence_refs_of assessment
        \<subteq> version_ids s \<union> historical_versions_of s \<and>
      v \<notin> evidence_refs_of assessment \<and>
      evidence_prior_version_of assessment
        \<notin> evidence_refs_of assessment \<and>
      0 \<le> evidence_strength_of assessment \<and>
      evidence_strength_of assessment \<le> 1 \<and>
      0 \<le> evidence_post_uncertainty_of assessment \<and>
      evidence_post_uncertainty_of assessment \<le> 1 \<and>
      0 \<le> validation_uncertainty_max_of assessment \<and>
      validation_uncertainty_max_of assessment \<le> 1 \<and>
      0 < validation_evidence_min_of assessment \<and>
      evidence_assessment_model_of assessment \<in> models_of s)"

definition evidence_ledger_well_formed ::
  "snapshot \<Rightarrow\rightarrow object_archive \<Rightarrow\rightarrow evidence_ledger \<Rightarrow\rightarrow bool" where
  "evidence_ledger_well_formed s archive ledger \<longleftarrow\rightarrow
    (\<forall>v assessment. ledger v = Some assessment \<longrightarrow\rightarrow
      evidence_entry_well_formed s archive v assessment)"

definition active_evidence_source ::
  "snapshot \<Rightarrow\rightarrow object_id \<Rightarrow\rightarrow version_id \<Rightarrow\rightarrow bool" where
  "active_evidence_source s subject v \<longleftarrow\rightarrow
    (\<exists>ev\<in>objects_of s.
      version_id_of ev = v \<and>
      object_id_of ev \<noteq> subject \<and>
      lifecycle_of ev \<notin> {Rejected, Archived})"

definition admissible_post_target ::
  "snapshot \<Rightarrow\rightarrow object_id \<Rightarrow\rightarrow version_id option \<Rightarrow\rightarrow bool" where
  "admissible_post_target s subject target \<longleftarrow\rightarrow
    (case target of
      None \<Rightarrow\rightarrow True
    | Some v \<Rightarrow\rightarrow
      (\<exists>obj\<in>objects_of s.
        version_id_of obj = v \<and>
        object_id_of obj \<noteq> subject \<and>
        lifecycle_of obj \<notin> {Revised, Rejected, Archived}))"

definition assessment_bounds :: "evidence_assessment \<Rightarrow\rightarrow bool" where
  "assessment_bounds assessment \<longleftarrow\rightarrow
    0 \<le> evidence_strength_of assessment \<and>
    evidence_strength_of assessment \<le> 1 \<and>
    0 \<le> evidence_post_uncertainty_of assessment \<and>
    evidence_post_uncertainty_of assessment \<le> 1 \<and>
    0 \<le> validation_uncertainty_max_of assessment \<and>
    validation_uncertainty_max_of assessment \<le> 1 \<and>
    0 < validation_evidence_min_of assessment"

definition assessment_change_valid ::

```

```

"effect_hypothesis_payload \<Rightarrow> evidence_assessment \<Rightarrow> bool" where
"assessment_change_valid prior assessment \<longleftarrowrightarrow>
  abs (evidence_post_uncertainty_of assessment -
    hypothesis_uncertainty_of prior)
  \<le> evidence_strength_of assessment \<and>
  (case evidence_disposition_of assessment of
    EvidenceSupports \<Rightarrow>
      evidence_post_target_of assessment = hypothesis_target_of prior \<and>
      evidence_post_vector_of assessment = hypothesis_vector_of prior \<and>
      evidence_post_uncertainty_of assessment
        \<le> hypothesis_uncertainty_of prior
    | EvidenceDifferentiates \<Rightarrow>
      evidence_post_target_of assessment \<noteq> hypothesis_target_of prior \<or>
      evidence_post_vector_of assessment \<noteq> hypothesis_vector_of prior
    | EvidenceContradicts \<Rightarrow>
      evidence_post_target_of assessment = hypothesis_target_of prior \<and>
      evidence_post_vector_of assessment = hypothesis_vector_of prior \<and>
      hypothesis_uncertainty_of prior
        \<le> evidence_post_uncertainty_of assessment)"

definition validation_ready ::
  "uza_object \<Rightarrow> evidence_assessment \<Rightarrow> bool" where
  "validation_ready old_eff assessment \<longleftarrowrightarrow>
    evidence_post_uncertainty_of assessment
    \<le> validation_uncertainty_max_of assessment \<and>
    card (evidence_of old_eff \<union> evidence_refs_of assessment)
    \<ge> validation_evidence_min_of assessment"

definition integrated_lifecycle ::
  "uza_object \<Rightarrow> evidence_assessment \<Rightarrow> lifecycle_status" where
  "integrated_lifecycle old_eff assessment =
    (case evidence_disposition_of assessment of
      EvidenceSupports \<Rightarrow>
        (if lifecycle_of old_eff = Hypothesis then Supported
          else if validation_ready old_eff assessment then Validated
          else Supported)
    | EvidenceDifferentiates \<Rightarrow> lifecycle_of old_eff
    | EvidenceContradicts \<Rightarrow> Rejected)"

definition integrated_effect_payload ::
  "effect_hypothesis_payload \<Rightarrow> evidence_assessment \<Rightarrow>
    effect_hypothesis_payload" where
  "integrated_effect_payload prior assessment =
    prior\<lparr>
      hypothesis_target_of := evidence_post_target_of assessment,
      hypothesis_vector_of := evidence_post_vector_of assessment,
      hypothesis_uncertainty_of := evidence_post_uncertainty_of assessment
    \<rparr>"

definition option_ref_set :: "version_id option \<Rightarrow> version_id set" where
  "option_ref_set target = (case target of None \<Rightarrow> {} | Some v \<Rightarrow> {v})"

definition integrated_effect_object ::
  "uza_object \<Rightarrow> evidence_assessment \<Rightarrow> version_id \<Rightarrow>
    time_point \<Rightarrow> uza_object" where
  "integrated_effect_object old_eff assessment new_version integration_time =
    old_eff\<lparr>
      version_id_of := new_version,
      lifecycle_of := integrated_lifecycle old_eff assessment,
      previous_version_of := Some (version_id_of old_eff),
      references_of := references_of old_eff
        \<union> evidence_refs_of assessment
        \<union> {version_id_of old_eff}
        \<union> option_ref_set (evidence_post_target_of assessment),
      evidence_of := evidence_of old_eff \<union> evidence_refs_of assessment,
      valid_from_of := integration_time,
      valid_to_of := None,
      target_ref_of := evidence_post_target_of assessment,
      basis_refs_of := basis_refs_of old_eff
        \<union> evidence_refs_of assessment
        \<union> {version_id_of old_eff}
        \<union> option_ref_set (evidence_post_target_of assessment),
      object_status_of :=
        (if evidence_disposition_of assessment = EvidenceContradicts
          then Some ModelFalsified else Some Applicable)
    \<rparr>"

definition replace_effect_version ::
  "snapshot \<Rightarrow> uza_object \<Rightarrow> uza_object \<Rightarrow> nat
    \<Rightarrow> snapshot" where

```

```

"replace_effect_version s old_eff new_eff next_snapshot_id =
  s\<lparr>
    snapshot_id_of := next_snapshot_id,
    objects_of := insert new_eff (objects_of s - {old_eff}),
    historical_versions_of :=
      insert (version_id_of old_eff) (historical_versions_of s)
  \<rparr>"

definition replace_active_effect_payload ::
  "effect_catalog \<Rightarrow> version_id \<Rightarrow> version_id \<Rightarrow>
    effect_hypothesis_payload \<Rightarrow> effect_catalog" where
  "replace_active_effect_payload catalog old_version new_version payload =
    (catalog(old_version := None))(new_version \<mapsto> payload)"

definition extend_object_archive ::
  "object_archive \<Rightarrow> uza_object \<Rightarrow> object_archive" where
  "extend_object_archive archive old_eff =
    archive(version_id_of old_eff \<mapsto> old_eff)"

definition extend_effect_payload_archive ::
  "effect_payload_archive \<Rightarrow> uza_object \<Rightarrow>
    effect_hypothesis_payload \<Rightarrow> effect_payload_archive" where
  "extend_effect_payload_archive archive old_eff prior =
    archive(version_id_of old_eff \<mapsto> prior)"

definition extend_evidence_ledger ::
  "evidence_ledger \<Rightarrow> version_id \<Rightarrow> evidence_assessment
    \<Rightarrow> evidence_ledger" where
  "extend_evidence_ledger ledger new_version assessment =
    ledger(new_version \<mapsto> assessment)"

definition integrate_evidence_input_valid ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> effect_catalog \<Rightarrow>
    object_archive \<Rightarrow> effect_payload_archive \<Rightarrow>
    evidence_ledger \<Rightarrow> uza_object \<Rightarrow>
    effect_hypothesis_payload \<Rightarrow> evidence_assessment \<Rightarrow>
    version_id \<Rightarrow> time_point \<Rightarrow> nat \<Rightarrow> bool" where
  "integrate_evidence_input_valid G s catalog object_archive payload_archive
    ledger old_eff prior assessment new_version integration_time
    next_snapshot_id \<longlefttrightarrow>
    state_well_formed G s \<and>
    active_history_disjoint s \<and>
    effect_catalog_well_formed s catalog \<and>
    object_archive_well_formed s object_archive \<and>
    effect_payload_archive_well_formed object_archive payload_archive \<and>
    evidence_ledger_well_formed s object_archive ledger \<and>
    IntegrateEvidence \<in> allowed_operations_of G \<and>
    old_eff \<in> objects_of s \<and>
    kind_of old_eff = K_EFF \<and>
    lifecycle_of old_eff \<in> {Hypothesis, Supported} \<and>
    catalog (version_id_of old_eff) = Some prior \<and>
    effect_payload_valid old_eff prior \<and>
    present_ref s (effect_origin_of old_eff) \<and>
    object_archive (version_id_of old_eff) = None \<and>
    payload_archive (version_id_of old_eff) = None \<and>
    new_version \<notin> version_ids s \<union> historical_versions_of s \<and>
    catalog new_version = None \<and>
    object_archive new_version = None \<and>
    payload_archive new_version = None \<and>
    ledger new_version = None \<and>
    evidence_subject_of assessment = object_id_of old_eff \<and>
    evidence_prior_version_of assessment = version_id_of old_eff \<and>
    evidence_refs_of assessment \<noteq> {} \<and>
    (\<forall>v\<in>evidence_refs_of assessment.
      active_evidence_source s (object_id_of old_eff) v) \<and>
    evidence_assessment_model_of assessment \<in> models_of s \<and>
    assessment_bounds assessment \<and>
    assessment_change_valid prior assessment \<and>
    admissible_post_target s (object_id_of old_eff)
    (evidence_post_target_of assessment) \<and>
    (case evidence_post_vector_of assessment of
      None \<Rightarrow> True
    | Some vector \<Rightarrow> effect_vector_valid vector) \<and>
    valid_from_of old_eff \<le> integration_time \<and>
    snapshot_id_of s < next_snapshot_id"

definition integrate_evidence_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> effect_catalog \<Rightarrow>
    object_archive \<Rightarrow> effect_payload_archive \<Rightarrow>
    evidence_ledger \<Rightarrow> uza_object \<Rightarrow>

```

```

effect_hypothesis_payload \<Rightarrow> evidence_assessment \<Rightarrow>
version_id \<Rightarrow> time_point \<Rightarrow> nat \<Rightarrow>
snapshot \<Rightarrow> effect_catalog \<Rightarrow> object_archive \<Rightarrow>
effect_payload_archive \<Rightarrow> evidence_ledger \<Rightarrow> bool" where
"integrate_evidence_step G s catalog object_archive payload_archive ledger
  old_eff prior assessment new_version integration_time next_snapshot_id
  s' catalog' object_archive' payload_archive' ledger' \<longlefttrightarrow>
integrate_evidence_input_valid G s catalog object_archive payload_archive
  ledger old_eff prior assessment new_version integration_time
  next_snapshot_id \<and>
(let new_eff = integrated_effect_object old_eff assessment new_version
  integration_time;
  new_payload = integrated_effect_payload prior assessment
  in s' = replace_effect_version s old_eff new_eff next_snapshot_id \<and>
  catalog' = replace_active_effect_payload catalog
    (version_id_of old_eff) new_version new_payload \<and>
  object_archive' = extend_object_archive object_archive old_eff \<and>
  payload_archive' = extend_effect_payload_archive payload_archive
    old_eff prior \<and>
  ledger' = extend_evidence_ledger ledger new_version assessment)"

subsection \<open>Constructor equations and exact delta\<close>

lemma replace_effect_simps [simp]:
  "snapshot_id_of (replace_effect_version s old_eff new_eff next_id) = next_id"
  "objects_of (replace_effect_version s old_eff new_eff next_id) =
    insert new_eff (objects_of s - {old_eff})"
  "historical_versions_of
    (replace_effect_version s old_eff new_eff next_id) =
    insert (version_id_of old_eff) (historical_versions_of s)"
  "perspectives_of (replace_effect_version s old_eff new_eff next_id) =
    perspectives_of s"
  "contexts_of (replace_effect_version s old_eff new_eff next_id) = contexts_of s"
  "models_of (replace_effect_version s old_eff new_eff next_id) = models_of s"
  "authorities_of (replace_effect_version s old_eff new_eff next_id) =
    authorities_of s"
  "contains_edges_of (replace_effect_version s old_eff new_eff next_id) =
    contains_edges_of s"
  "projections_of (replace_effect_version s old_eff new_eff next_id) =
    projections_of s"
  "couplings_of (replace_effect_version s old_eff new_eff next_id) =
    couplings_of s"
  "transmissions_of (replace_effect_version s old_eff new_eff next_id) =
    transmissions_of s"
  "boundaries_of (replace_effect_version s old_eff new_eff next_id) =
    boundaries_of s"
  by (simp_all add: replace_effect_version_def)

lemma integrated_effect_simps [simp]:
  "object_id_of (integrated_effect_object old assessment new_v at_time) =
    object_id_of old"
  "version_id_of (integrated_effect_object old assessment new_v at_time) = new_v"
  "kind_of (integrated_effect_object old assessment new_v at_time) = kind_of old"
  "lifecycle_of (integrated_effect_object old assessment new_v at_time) =
    integrated_lifecycle old assessment"
  "previous_version_of
    (integrated_effect_object old assessment new_v at_time) =
    Some (version_id_of old)"
  "references_of (integrated_effect_object old assessment new_v at_time) =
    references_of old \<union> evidence_refs_of assessment \<union>
    {version_id_of old} \<union>
    option_ref_set (evidence_post_target_of assessment)"
  "evidence_of (integrated_effect_object old assessment new_v at_time) =
    evidence_of old \<union> evidence_refs_of assessment"
  "perspective_of (integrated_effect_object old assessment new_v at_time) =
    perspective_of old"
  "context_of (integrated_effect_object old assessment new_v at_time) =
    context_of old"
  "model_of (integrated_effect_object old assessment new_v at_time) = model_of old"
  "source_ref_of (integrated_effect_object old assessment new_v at_time) =
    source_ref_of old"
  "target_ref_of (integrated_effect_object old assessment new_v at_time) =
    evidence_post_target_of assessment"
  "effect_origin_of (integrated_effect_object old assessment new_v at_time) =
    effect_origin_of old"
  "valid_from_of (integrated_effect_object old assessment new_v at_time) = at_time"
  "valid_to_of (integrated_effect_object old assessment new_v at_time) = None"
  "basis_refs_of (integrated_effect_object old assessment new_v at_time) =
    basis_refs_of old \<union> evidence_refs_of assessment \<union>
    {version_id_of old} \<union>

```



```

    option_ref_set (evidence_post_target_of assessment)"
    by (simp_all add: integrated_effect_object_def)

lemma integrated_payload_simps [simp]:
  "hypothesis_cause_of (integrated_effect_payload prior assessment) =
    hypothesis_cause_of prior"
  "hypothesis_target_of (integrated_effect_payload prior assessment) =
    evidence_post_target_of assessment"
  "hypothesis_vector_of (integrated_effect_payload prior assessment) =
    evidence_post_vector_of assessment"
  "hypothesis_uncertainty_of (integrated_effect_payload prior assessment) =
    evidence_post_uncertainty_of assessment"
  "hypothesis_profile_model_of (integrated_effect_payload prior assessment) =
    hypothesis_profile_model_of prior"
  by (simp_all add: integrated_effect_payload_def)

lemma integrate_evidence_exact_delta:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
      old_eff prior assessment new_version integration_time next_snapshot_id
      s' catalog' object_archive' payload_archive' ledger'"
  defines "new_eff \ integrated_effect_object old_eff assessment new_version
    integration_time"
    and "new_payload \ integrated_effect_payload prior assessment"
  shows
    "objects_of s' = insert new_eff (objects_of s - {old_eff}) \<in> objects_of s' \

```

```

subsection \open>Lifecycle, uncertainty, and provenance properties\<close>

theorem integrate_evidence_lifecycle_classification:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
     old_eff prior assessment new_version integration_time next_snapshot_id
     s' catalog' object_archive' payload_archive' ledger'"
  shows
    "(evidence_disposition_of assessment = EvidenceContradicts
     \<longrightarrow> integrated_lifecycle old_eff assessment = Rejected) \<and>
     (evidence_disposition_of assessment = EvidenceDifferentiates
     \<longrightarrow> integrated_lifecycle old_eff assessment = lifecycle_of old_eff) \<and>
     (evidence_disposition_of assessment = EvidenceSupports \<and>
     lifecycle_of old_eff = Hypothesis
     \<longrightarrow> integrated_lifecycle old_eff assessment = Supported) \<and>
     (integrated_lifecycle old_eff assessment = Validated
     \<longrightarrow> lifecycle_of old_eff = Supported \<and>
     evidence_disposition_of assessment = EvidenceSupports \<and>
     validation_ready old_eff assessment)"
  proof -
    from concrete have source_lifecycle:
      "lifecycle_of old_eff \<in> {Hypothesis, Supported}"
    by (auto simp: integrate_evidence_step_def
        integrate_evidence_input_valid_def)
    show ?thesis
      using source_lifecycle
      by (cases "evidence_disposition_of assessment";
          auto simp: integrated_lifecycle_def split: if_splits)
  qed

theorem integrate_evidence_uncertainty_controlled:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
     old_eff prior assessment new_version integration_time next_snapshot_id
     s' catalog' object_archive' payload_archive' ledger'"
  shows
    "0 \<le> evidence_post_uncertainty_of assessment \<and>
     evidence_post_uncertainty_of assessment \<le> 1 \<and>
     abs (evidence_post_uncertainty_of assessment -
          hypothesis_uncertainty_of prior)
     \<le> evidence_strength_of assessment \<and>
     (evidence_disposition_of assessment = EvidenceSupports
     \<longrightarrow> evidence_post_uncertainty_of assessment
     \<le> hypothesis_uncertainty_of prior) \<and>
     (evidence_disposition_of assessment = EvidenceContradicts
     \<longrightarrow> hypothesis_uncertainty_of prior
     \<le> evidence_post_uncertainty_of assessment)"
  using concrete
  unfolding integrate_evidence_step_def integrate_evidence_input_valid_def
  assessment_bounds_def assessment_change_valid_def
  by (cases "evidence_disposition_of assessment"; auto)

theorem integrate_evidence_lineage_and_no_self_support:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
     old_eff prior assessment new_version integration_time next_snapshot_id
     s' catalog' object_archive' payload_archive' ledger'"
  defines "new_eff \<equiv> integrated_effect_object old_eff assessment new_version
           integration_time"
  shows
    "previous_version_of new_eff = Some (version_id_of old_eff) \<and>
     evidence_refs_of assessment \<noteq> {} \<and>
     evidence_refs_of assessment \<subsetq> evidence_of new_eff \<and>
     new_version \<notin> evidence_refs_of assessment \<and>
     version_id_of old_eff \<notin> evidence_refs_of assessment \<and>
     (\<forall>v\<in>evidence_refs_of assessment.
      \<exists>ev\<in>objects_of s.
      version_id_of ev = v \<and>
      object_id_of ev \<noteq> object_id_of old_eff \<and>
      lifecycle_of ev \<notin> {Rejected, Archived})"
  proof -
    from concrete have valid:
      "integrate_evidence_input_valid G s catalog object_archive payload_archive
       ledger old_eff prior assessment new_version integration_time
       next_snapshot_id"
    by (simp add: integrate_evidence_step_def)
    from valid have refs_nonempty: "evidence_refs_of assessment \<noteq> {}"
    and active:
      "\<forall>v\<in>evidence_refs_of assessment.
       active_evidence_source s (object_id_of old_eff) v"

```

```

and fresh:
  "new_version \<notin> version_ids s \<union> historical_versions_of s"
  and old_mem: "old_eff \<in> objects_of s"
  and source_wf: "state_well_formed G s"
  unfolding integrate_evidence_input_valid_def by blast+
  have source_unique: "inj_on version_id_of (objects_of s)"
  using source_wf unfolding state_well_formed_def unique_versions_def by blast
  have prior_not: "version_id_of old_eff \<notin> evidence_refs_of assessment"
  proof
    assume prior_in: "version_id_of old_eff \<in> evidence_refs_of assessment"
    from active prior_in obtain ev where ev_mem: "ev \<in> objects_of s"
      and same_version: "version_id_of ev = version_id_of old_eff"
      and different_object: "object_id_of ev \<noteq> object_id_of old_eff"
    unfolding active_evidence_source_def by blast
    have "ev = old_eff"
      using inj_onD[OF source_unique same_version ev_mem old_mem] .
    then show False using different_object by simp
  qed
  have new_not: "new_version \<notin> evidence_refs_of assessment"
  proof
    assume "new_version \<in> evidence_refs_of assessment"
    then obtain ev where "ev \<in> objects_of s" "version_id_of ev = new_version"
      using active_unfolding_active_evidence_source_def by blast
    then have "new_version \<in> version_ids s"
      unfolding version_ids_def by blast
    then show False using fresh by blast
  qed
  show ?thesis
    unfolding new_eff_def
    using refs_nonempty active prior_not new_not
    by (auto simp: active_evidence_source_def)
qed

subsection \<open>Reference-universe monotonicity\<close>

lemma replace_effect_version_universe_mono:
  assumes old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  shows
    "version_ids s \<union> historical_versions_of s \<subteq>
      version_ids (replace_effect_version s old_eff new_eff next_id)
      \<union> historical_versions_of
        (replace_effect_version s old_eff new_eff next_id)"
  proof
    fix v
    assume v_mem: "v \<in> version_ids s \<union> historical_versions_of s"
    show "v \<in> version_ids (replace_effect_version s old_eff new_eff next_id)
      \<union> historical_versions_of
        (replace_effect_version s old_eff new_eff next_id)"
    proof (cases "v = version_id_of old_eff")
      case is_old: True
      then show ?thesis by simp
    next
      case not_old: False
      show ?thesis
        proof (cases "v \<in> historical_versions_of s")
          case in_history: True
          then show ?thesis by simp
        next
          case not_in_history: False
          with v_mem obtain obj where obj_mem: "obj \<in> objects_of s"
            and obj_v: "version_id_of obj = v"
            unfolding version_ids_def by blast
          have "obj \<noteq> old_eff"
            proof
              assume "obj = old_eff"
              then have "v = version_id_of old_eff" using obj_v by simp
              then show False using not_old by contradiction
            qed
          then have "obj \<in> objects_of s - {old_eff}" using obj_mem by blast
          then have "v \<in> version_ids
            (replace_effect_version s old_eff new_eff next_id)"
            unfolding version_ids_def using obj_v by auto
          then show ?thesis by blast
        qed
      qed
    qed
  qed

lemma replace_effect_present_ref_mono:

```

```

assumes old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  and present: "present_ref s ref"
shows "present_ref
  (replace_effect_version s old_eff new_eff next_id) ref"
using replace_effect_version_universe_mono[OF old_mem fresh] present
unfolding present_ref_def
by (cases ref; auto)

lemma replace_effect_references_mono:
  assumes old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  and refs: "references_complete s obj"
shows "references_complete
  (replace_effect_version s old_eff new_eff next_id) obj"
using replace_effect_version_universe_mono[OF old_mem fresh] refs
unfolding references_complete_def by blast

lemma replace_effect_evidence_mono:
  assumes old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  and evidence: "evidence_well_formed s obj"
shows "evidence_well_formed
  (replace_effect_version s old_eff new_eff next_id) obj"
using replace_effect_version_universe_mono[OF old_mem fresh] evidence
unfolding evidence_well_formed_def evidence_references_complete_def by blast

lemma replace_effect_preserves_active_history_disjoint:
  assumes source_disjoint: "active_history_disjoint s"
  and source_unique: "unique_versions s"
  and old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
shows "active_history_disjoint
  (replace_effect_version s old_eff new_eff next_id)"
proof -
  have old_version_active: "version_id_of old_eff \<in> version_ids s"
  unfolding version_ids_def using old_mem by blast
  have versions_different:
    "version_id_of old_eff \<noteq> version_id_of new_eff"
  proof
    assume same: "version_id_of old_eff = version_id_of new_eff"
    from fresh have "version_id_of new_eff \<notin> version_ids s" by blast
    with old_version_active same show False by simp
  qed
  show ?thesis
  using source_disjoint source_unique old_mem fresh versions_different
  unfolding active_history_disjoint_def unique_versions_def version_ids_def
  replace_effect_version_def
  by (auto simp: inj_on_def)
qed

lemma evidence_entry_replace_mono:
  assumes old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  and old_not_archived: "archive (version_id_of old_eff) = None"
  and entry: "evidence_entry_well_formed s archive v assessment"
shows "evidence_entry_well_formed
  (replace_effect_version s old_eff new_eff next_id)
  (extend_object_archive archive old_eff) v assessment"
proof -
  let ?target = "replace_effect_version s old_eff new_eff next_id"
  let ?archive = "extend_object_archive archive old_eff"
  from entry obtain eff where resolved:
    "resolved_version_object s archive v eff"
  and remaining:
    "kind_of eff = K_EFF \<and>
    evidence_subject_of assessment = object_id_of eff \<and>
    previous_version_of eff = Some (evidence_prior_version_of assessment) \<and>
    evidence_refs_of assessment \<noteq> {} \<and>
    evidence_refs_of assessment \<subsetq> evidence_of eff \<and>
    evidence_refs_of assessment
      \<subsetq> version_ids s \<union> historical_versions_of s \<and>
    v \<notin> evidence_refs_of assessment \<and>
    evidence_prior_version_of assessment
      \<notin> evidence_refs_of assessment \<and>"

```

```

    0 \<le> evidence_strength_of assessment \<and>
    evidence_strength_of assessment \<le> 1 \<and>
    0 \<le> evidence_post_uncertainty_of assessment \<and>
    evidence_post_uncertainty_of assessment \<le> 1 \<and>
    0 \<le> validation_uncertainty_max_of assessment \<and>
    validation_uncertainty_max_of assessment \<le> 1 \<and>
    0 < validation_evidence_min_of assessment \<and>
    evidence_assessment_model_of assessment \<in> models_of s"
    unfolding evidence_entry_well_formed_def by blast
have resolved_target:
    "resolved_version_object ?target ?archive v eff"
proof (cases "eff \<in> objects_of s \<and> version_id_of eff = v")
  case active: True
  show ?thesis
  proof (cases "eff = old_eff")
    case same_old: True
    from active have eff_version: "version_id_of eff = v" by blast
    have "?archive v = Some eff"
      unfolding extend_object_archive_def
      using same_old eff_version by simp
    then show ?thesis
      unfolding resolved_version_object_def by blast
  next
  case different_old: False
  with active have "eff \<in> objects_of s - {old_eff}" by blast
  with active show ?thesis
    unfolding resolved_version_object_def by auto
qed
next
case not_active: False
with resolved have archived: "archive v = Some eff"
  unfolding resolved_version_object_def by blast
have key_different: "v \<noteq> version_id_of old_eff"
proof
  assume "v = version_id_of old_eff"
  with archived old_not_archived show False by simp
qed
show ?thesis
  unfolding resolved_version_object_def extend_object_archive_def
  using archived key_different by simp
qed
have refs_target:
    "evidence_refs_of assessment
    \<subsetq> version_ids ?target \<union> historical_versions_of ?target"
  using remaining replace_effect_version_universe_mono[OF old_mem fresh]
  by blast
show ?thesis
  unfolding evidence_entry_well_formed_def
proof (rule exI[of _ eff])
  show "resolved_version_object ?target ?archive v eff \<and>
    kind_of eff = K_EFF \<and>
    evidence_subject_of assessment = object_id_of eff \<and>
    previous_version_of eff = Some (evidence_prior_version_of assessment) \<and>
    evidence_refs_of assessment \<noteq> {} \<and>
    evidence_refs_of assessment \<subsetq> evidence_of eff \<and>
    evidence_refs_of assessment
      \<subsetq> version_ids ?target \<union> historical_versions_of ?target \<and>
    v \<notin> evidence_refs_of assessment \<and>
    evidence_prior_version_of assessment
      \<notin> evidence_refs_of assessment \<and>
    0 \<le> evidence_strength_of assessment \<and>
    evidence_strength_of assessment \<le> 1 \<and>
    0 \<le> evidence_post_uncertainty_of assessment \<and>
    evidence_post_uncertainty_of assessment \<le> 1 \<and>
    0 \<le> validation_uncertainty_max_of assessment \<and>
    validation_uncertainty_max_of assessment \<le> 1 \<and>
    0 < validation_evidence_min_of assessment \<and>
    evidence_assessment_model_of assessment \<in> models_of ?target"
  using resolved_target remaining refs_target
  by (simp only: replace_effect_simps; blast)
qed
qed

subsection \<open>Successor state and audit-store preservation\<close>

lemma integrate_evidence_payload_valid:
  assumes valid:
    "integrate_evidence_input_valid G s catalog object_archive payload_archive
    ledger old_eff prior assessment new_version integration_time
    next_snapshot_id"

```

```

shows "effect_payload_valid
  (integrated_effect_object old_eff assessment new_version integration_time)
  (integrated_effect_payload prior assessment)"
using valid
unfolding integrate_evidence_input_valid_def assessment_bounds_def
  effect_payload_valid_def
by (cases "evidence_post_vector_of assessment"; auto)

lemma integrate_evidence_new_object_conditions:
  assumes valid:
    "integrate_evidence_input_valid G s catalog object_archive payload_archive
      ledger old_eff prior assessment new_version integration_time
      next_snapshot_id"
  defines "new_eff \<equiv> integrated_effect_object old_eff assessment new_version
    integration_time"
  shows
    "kind_of new_eff \<in> supported_kinds_of G \<and>
      references_complete
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      evidence_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      time_well_formed new_eff \<and>
      relation_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      effect_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      perspective_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      context_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      relevance_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      orientation_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      action_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff \<and>
      feedback_well_formed
        (replace_effect_version s old_eff new_eff next_snapshot_id) new_eff"
proof -
  let ?target = "replace_effect_version s old_eff new_eff next_snapshot_id"
  from valid have source_wf: "state_well_formed G s"
  and old_mem: "old_eff \<in> objects_of s"
  and old_kind: "kind_of old_eff = K_EFF"
  and old_lifecycle: "lifecycle_of old_eff \<in> {Hypothesis, Supported}"
  and fresh: "new_version \<notin> version_ids s \<union> historical_versions_of s"
  and origin: "present_ref s (effect_origin_of old_eff)"
  and evidence_nonempty: "evidence_refs_of assessment \<noteq> {}"
  and evidence_active:
    "\<forall>v \<in> evidence_refs_of assessment.
      active_evidence_source s (object_id_of old_eff) v"
  and target_ok:
    "admissible_post_target s (object_id_of old_eff)
      (evidence_post_target_of assessment)"
  unfolding integrate_evidence_input_valid_def by blast+
  from source_wf old_mem have old_conditions:
    "kind_of old_eff \<in> supported_kinds_of G \<and>
      references_complete s old_eff \<and>
      evidence_well_formed s old_eff \<and>
      time_well_formed old_eff \<and>
      perspective_well_formed s old_eff \<and>
      context_well_formed s old_eff"
  unfolding state_well_formed_def by blast
  have universe_mono:
    "version_ids s \<union> historical_versions_of s \<subsepeq>
      version_ids ?target \<union> historical_versions_of ?target"
  using replace_effect_version_universe_mono[OF old_mem]
  unfolding new_eff_def using fresh by simp
  have evidence_refs_resolve:
    "evidence_refs_of assessment
      \<subsepeq> version_ids ?target \<union> historical_versions_of ?target"
proof
  fix v
  assume "v \<in> evidence_refs_of assessment"
  then obtain ev where ev_mem: "ev \<in> objects_of s"
  and ev_v: "version_id_of ev = v"
  and different: "object_id_of ev \<noteq> object_id_of old_eff"
  using evidence_active unfolding active_evidence_source_def by blast
  have "ev \<noteq> old_eff" using different by auto
  then have "ev \<in> objects_of s - {old_eff}" using ev_mem by blast
  then have "v \<in> version_ids ?target"

```

```

    unfolding version_ids_def new_eff_def using ev_v by auto
    then show "v \<in> version_ids ?target \<union> historical_versions_of ?target"
    by blast
qed
have target_ref_resolves:
  "option_ref_set (evidence_post_target_of assessment)
  \<subseq> version_ids ?target \<union> historical_versions_of ?target"
proof (cases "evidence_post_target_of assessment")
  case None
  then show ?thesis by (simp add: option_ref_set_def)
next
case (Some v)
  then obtain obj where obj_mem: "obj \<in> objects_of s"
  and obj_v: "version_id_of obj = v"
  and different: "object_id_of obj \<noteq> object_id_of old_eff"
  using target_ok unfolding admissible_post_target_def by auto
  have "obj \<noteq> old_eff" using different by auto
  then have "obj \<in> objects_of s - {old_eff}" using obj_mem by blast
  then have "v \<in> version_ids ?target"
  unfolding version_ids_def new_eff_def using obj_v by auto
  with Some show ?thesis by (simp add: option_ref_set_def)
qed
have prior_resolves:
  "version_id_of old_eff
  \<in> version_ids ?target \<union> historical_versions_of ?target"
  by simp
from old_conditions have old_refs_source:
  "references_of old_eff
  \<subseq> version_ids s \<union> historical_versions_of s"
  unfolding references_complete_def by blast
have old_refs_target:
  "references_of old_eff
  \<subseq> version_ids ?target \<union> historical_versions_of ?target"
  using old_refs_source universe_mono by blast
have combined_refs_target:
  "references_of old_eff \<union> evidence_refs_of assessment \<union>
  {version_id_of old_eff} \<union>
  option_ref_set (evidence_post_target_of assessment)
  \<subseq> version_ids ?target \<union> historical_versions_of ?target"
  using old_refs_target evidence_refs_resolve target_ref_resolves
  prior_resolves by blast
have new_refs_eq:
  "references_of new_eff =
  references_of old_eff \<union> evidence_refs_of assessment \<union>
  {version_id_of old_eff} \<union>
  option_ref_set (evidence_post_target_of assessment)"
  unfolding new_eff_def by simp
have refs_new: "references_complete ?target new_eff"
  unfolding references_complete_def
  using combined_refs_target new_refs_eq by simp
have evidence_new: "evidence_well_formed ?target new_eff"
proof -
  from old_conditions have old_evidence_source:
    "evidence_of old_eff
    \<subseq> version_ids s \<union> historical_versions_of s"
    unfolding evidence_well_formed_def evidence_references_complete_def
    by blast
  have old_evidence_target:
    "evidence_of old_eff
    \<subseq> version_ids ?target \<union> historical_versions_of ?target"
    using old_evidence_source universe_mono by blast
  have new_evidence_eq:
    "evidence_of new_eff =
    evidence_of old_eff \<union> evidence_refs_of assessment"
    unfolding new_eff_def by simp
  have complete:
    "evidence_of new_eff
    \<subseq> version_ids ?target \<union> historical_versions_of ?target"
    using old_evidence_target evidence_refs_resolve new_evidence_eq by blast
  have upgraded_nonempty:
    "lifecycle_of new_eff \<in> {Supported, Validated}
    \<longrightarrow> evidence_of new_eff \<noteq> {}"
    using evidence_nonempty
    unfolding new_eff_def integrated_effect_object_def
    by auto
  show ?thesis
    unfolding evidence_well_formed_def evidence_references_complete_def
    using complete upgraded_nonempty by blast
qed
have origin_target: "present_ref ?target (effect_origin_of new_eff)"

```

```

    unfolding new_eff_def
    using replace_effect_present_ref_mono[OF old_mem _ origin] fresh by simp
have perspective_new: "perspective_well_formed ?target new_eff"
proof -
  from old_conditions have old_perspective:
    "perspective_well_formed s old_eff" by blast
  show ?thesis
  using old_perspective
  unfolding perspective_well_formed_def new_eff_def
  by (simp only: integrated_effect_simps replace_effect_simps)
qed
have context_new: "context_well_formed ?target new_eff"
proof -
  from old_conditions have old_context:
    "context_well_formed s old_eff" by blast
  show ?thesis
  using old_context
  unfolding context_well_formed_def new_eff_def
  by (simp only: integrated_effect_simps replace_effect_simps)
qed
have other_typed:
  "time_well_formed new_eff \<and>
  relation_well_formed ?target new_eff \<and>
  effect_well_formed ?target new_eff \<and>
  relevance_well_formed ?target new_eff \<and>
  orientation_well_formed ?target new_eff \<and>
  action_well_formed ?target new_eff \<and>
  feedback_well_formed ?target new_eff"
proof -
  have new_kind: "kind_of new_eff = K_EFF"
  using old_kind unfolding new_eff_def by simp
  have new_time: "time_well_formed new_eff"
  unfolding time_well_formed_def new_eff_def by simp
  have new_relation: "relation_well_formed ?target new_eff"
  using new_kind unfolding relation_well_formed_def by simp
  have new_effect: "effect_well_formed ?target new_eff"
  unfolding effect_well_formed_def
  using new_kind origin_target by blast
  have new_relevance: "relevance_well_formed ?target new_eff"
  using new_kind unfolding relevance_well_formed_def by simp
  have new_orientation: "orientation_well_formed ?target new_eff"
  using new_kind unfolding orientation_well_formed_def by simp
  have new_action: "action_well_formed ?target new_eff"
  using new_kind unfolding action_well_formed_def by simp
  have new_feedback: "feedback_well_formed ?target new_eff"
  using new_kind unfolding feedback_well_formed_def by simp
  show ?thesis
  using new_time new_relation new_effect new_relevance new_orientation
    new_action new_feedback by blast
qed
show ?thesis
  using old_conditions refs_new evidence_new perspective_new context_new
    other_typed
  unfolding new_eff_def by simp
qed

lemma replace_effect_preserves_unique_versions:
  assumes source_unique: "unique_versions s"
  and old_mem: "old_eff \<in> objects_of s"
  and fresh: "version_id_of new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  shows "unique_versions
    (replace_effect_version s old_eff new_eff next_snapshot_id)"
proof -
  from source_unique have inj: "inj_on version_id_of (objects_of s)"
  unfolding unique_versions_def .
  show ?thesis
  unfolding unique_versions_def
proof (rule inj_onI)
  fix left right
  assume left_mem:
    "left \<in> objects_of
    (replace_effect_version s old_eff new_eff next_snapshot_id)"
  and right_mem:
    "right \<in> objects_of
    (replace_effect_version s old_eff new_eff next_snapshot_id)"
  and same: "version_id_of left = version_id_of right"
  consider (left_new) "left = new_eff" | (left_old) "left \<in> objects_of s - {old_eff}"
  using left_mem by auto
  then show "left = right"

```



```

proof cases
  case left_new
  show ?thesis
  proof (cases "right = new_eff")
    case True
    with left_new show ?thesis by simp
  next
    case False
    with right_mem have right_old: "right \<in> objects_of s"
    by auto
    have "version_id_of new_eff \<in> version_ids s"
    unfolding version_ids_def using same left_new right_old by blast
    then show ?thesis using fresh by blast
  qed
next
  case left_old
  show ?thesis
  proof (cases "right = new_eff")
    case True
    have left_version: "version_id_of left \<in> version_ids s"
    unfolding version_ids_def using left_old by blast
    have "version_id_of left = version_id_of new_eff"
    using same True by simp
    with left_version have "version_id_of new_eff \<in> version_ids s"
    by simp
    then show ?thesis using fresh by blast
  next
    case False
    with right_mem have right_old: "right \<in> objects_of s" by auto
    show ?thesis
    using inj_onD[OF inj same] left_old right_old by blast
  qed
qed
qed
qed
qed

theorem integrate_evidence_preserves_state_well_formed:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
    old_eff prior assessment new_version integration_time next_snapshot_id
    s' catalog' object_archive' payload_archive' ledger'"
  shows "state_well_formed G s'"
proof -
  let ?new_eff = "integrated_effect_object old_eff assessment new_version
  integration_time"
  let ?target = "replace_effect_version s old_eff ?new_eff next_snapshot_id"
  from concrete have valid:
    "integrate_evidence_input_valid G s catalog object_archive payload_archive
    ledger old_eff prior assessment new_version integration_time
    next_snapshot_id"
  and target: "s' = ?target"
  by (auto simp: integrate_evidence_step_def Let_def)
  from valid have source_wf: "state_well_formed G s"
  and old_mem: "old_eff \<in> objects_of s"
  and fresh: "new_version \<notin> version_ids s \<union> historical_versions_of s"
  unfolding integrate_evidence_input_valid_def by blast+
  have unique: "unique_versions ?target"
  using replace_effect_preserves_unique_versions old_mem fresh source_wf
  unfolding state_well_formed_def by simp
  have new_conditions:
    "kind_of ?new_eff \<in> supported_kinds_of G \<and>
    references_complete ?target ?new_eff \<and>
    evidence_well_formed ?target ?new_eff \<and>
    time_well_formed ?new_eff \<and>
    relation_well_formed ?target ?new_eff \<and>
    effect_well_formed ?target ?new_eff \<and>
    perspective_well_formed ?target ?new_eff \<and>
    context_well_formed ?target ?new_eff \<and>
    relevance_well_formed ?target ?new_eff \<and>
    orientation_well_formed ?target ?new_eff \<and>
    action_well_formed ?target ?new_eff \<and>
    feedback_well_formed ?target ?new_eff"
  using integrate_evidence_new_object_conditions[OF valid] by simp
  have old_object_conditions:
    "\<forall>obj \<in> objects_of s - {old_eff}.
    kind_of obj \<in> supported_kinds_of G \<and>
    references_complete ?target obj \<and>
    evidence_well_formed ?target obj \<and>
    time_well_formed obj \<and>
    relation_well_formed ?target obj \<and>"

```

```

effect_well_formed ?target obj \<and>
perspective_well_formed ?target obj \<and>
context_well_formed ?target obj \<and>
relevance_well_formed ?target obj \<and>
orientation_well_formed ?target obj \<and>
action_well_formed ?target obj \<and>
feedback_well_formed ?target obj"
proof (intro ballI)
  fix obj
  assume obj_mem: "obj \<in> objects_of s - {old_eff}"
  from source_wf obj_mem have source_conditions:
    "kind_of obj \<in> supported_kinds_of G \<and>
     references_complete s obj \<and> evidence_well_formed s obj \<and>
     time_well_formed obj \<and> relation_well_formed s obj \<and>
     effect_well_formed s obj \<and> perspective_well_formed s obj \<and>
     context_well_formed s obj \<and> relevance_well_formed s obj \<and>
     orientation_well_formed s obj \<and> action_well_formed s obj \<and>
     feedback_well_formed s obj"
  unfolding state_well_formed_def by blast
  have present_mono: "\<And>ref. present_ref s ref \<Longrightarrow> present_ref ?target ref"
  using replace_effect_present_ref_mono[OF old_mem _] fresh by simp
  have refs_target: "references_complete ?target obj"
  using replace_effect_references_mono[OF old_mem _]
    source_conditions fresh by simp
  have evidence_target: "evidence_well_formed ?target obj"
  using replace_effect_evidence_mono[OF old_mem _]
    source_conditions fresh by simp
  have typed_target:
    "relation_well_formed ?target obj \<and>
     effect_well_formed ?target obj \<and>
     feedback_well_formed ?target obj"
  using source_conditions present_mono
  unfolding relation_well_formed_def effect_well_formed_def
    feedback_well_formed_def
  by blast
  from source_conditions have source_perspective:
    "perspective_well_formed s obj"
  and source_context: "context_well_formed s obj"
  and source_relevance: "relevance_well_formed s obj"
  and source_orientation: "orientation_well_formed s obj"
  and source_action: "action_well_formed s obj"
  by blast+
  have perspective_target: "perspective_well_formed ?target obj"
  using source_perspective
  unfolding perspective_well_formed_def
  by (simp only: replace_effect_simps)
  have context_target: "context_well_formed ?target obj"
  using source_context
  unfolding context_well_formed_def
  by (simp only: replace_effect_simps)
  have relevance_target: "relevance_well_formed ?target obj"
  unfolding relevance_well_formed_def
  proof
    assume kind: "kind_of obj = K_REV"
    from source_relevance kind have source_model:
      "case model_of obj of None \<Rightarrow> False
       | Some m \<Rightarrow> m \<in> models_of s"
    and source_refs: "references_of obj \<noteq> {}"
    unfolding relevance_well_formed_def by blast+
    have target_model:
      "case model_of obj of None \<Rightarrow> False
       | Some m \<Rightarrow> m \<in> models_of ?target"
    using source_model by (simp only: replace_effect_simps)
    show "perspective_well_formed ?target obj \<and>
      context_well_formed ?target obj \<and>
      (case model_of obj of None \<Rightarrow> False
       | Some m \<Rightarrow> m \<in> models_of ?target) \<and>
      references_of obj \<noteq> {}"
    using perspective_target context_target target_model source_refs by blast
  qed
  have orientation_target: "orientation_well_formed ?target obj"
  using source_orientation unfolding orientation_well_formed_def by simp
  have action_target: "action_well_formed ?target obj"
  using source_action unfolding action_well_formed_def
  by (simp only: replace_effect_simps)
  have stable_target:
    "perspective_well_formed ?target obj \<and>
     context_well_formed ?target obj \<and>
     relevance_well_formed ?target obj \<and>
     orientation_well_formed ?target obj \<and>

```

```

        action_well_formed ?target obj"
    using perspective_target context_target relevance_target
        orientation_target action_target by blast
show "kind_of obj \<in> supported_kinds_of G \<and>
    references_complete ?target obj \<and>
    evidence_well_formed ?target obj \<and>
    time_well_formed obj \<and>
    relation_well_formed ?target obj \<and>
    effect_well_formed ?target obj \<and>
    perspective_well_formed ?target obj \<and>
    context_well_formed ?target obj \<and>
    relevance_well_formed ?target obj \<and>
    orientation_well_formed ?target obj \<and>
    action_well_formed ?target obj \<and>
    feedback_well_formed ?target obj"
    using source_conditions refs_target evidence_target typed_target
        stable_target by blast
qed
have all_objects:
    "\<forall>obj\<in>objects_of ?target.
    kind_of obj \<in> supported_kinds_of G \<and>
    references_complete ?target obj \<and>
    evidence_well_formed ?target obj \<and>
    time_well_formed obj \<and>
    relation_well_formed ?target obj \<and>
    effect_well_formed ?target obj \<and>
    perspective_well_formed ?target obj \<and>
    context_well_formed ?target obj \<and>
    relevance_well_formed ?target obj \<and>
    orientation_well_formed ?target obj \<and>
    action_well_formed ?target obj \<and>
    feedback_well_formed ?target obj"
    using new_conditions old_object_conditions by auto
from source_wf have secondary:
    "(\<forall>p\<in>projections_of ?target. projection_well_formed ?target p) \<and>
    (\<forall>c\<in>couplings_of ?target. coupling_well_formed ?target c) \<and>
    (\<forall>t\<in>transmissions_of ?target. transmission_well_formed ?target t) \<and>
    (\<forall>b\<in>boundaries_of ?target. boundary_well_formed b)"
    unfolding state_well_formed_def projection_well_formed_def
        coupling_well_formed_def transmission_well_formed_def
    by simp
show ?thesis
    unfolding target state_well_formed_def
    using unique_all_objects secondary by blast
qed

theorem integrate_evidence_preserves_identity:
    assumes concrete:
        "integrate_evidence_step G s catalog object_archive payload_archive ledger
        old_eff prior assessment new_version integration_time next_snapshot_id
        s' catalog' object_archive' payload_archive' ledger'"
    shows "unique_versions s' \<and> immutable_preserved s s'"
proof -
    let ?new_eff = "integrated_effect_object old_eff assessment new_version
        integration_time"
    from concrete have valid:
        "integrate_evidence_input_valid G s catalog object_archive payload_archive
        ledger old_eff prior assessment new_version integration_time
        next_snapshot_id"
    and target:
        "s' = replace_effect_version s old_eff ?new_eff next_snapshot_id"
    by (auto simp: integrate_evidence_step_def Let_def)
    from valid have source_wf: "state_well_formed G s"
    and old_mem: "old_eff \<in> objects_of s"
    and fresh: "new_version \<notin> version_ids s \<union> historical_versions_of s"
    unfolding integrate_evidence_input_valid_def by blast+
    have unique: "unique_versions s'"
    unfolding target
    using replace_effect_preserves_unique_versions old_mem fresh source_wf
    unfolding state_well_formed_def by simp
    have immutable: "immutable_preserved s s'"
    unfolding immutable_preserved_def target
    proof (intro ballI allI impI)
        fix before after
        assume before_mem: "before \<in> objects_of s"
        and after_mem:
            "after \<in> objects_of
            (replace_effect_version s old_eff ?new_eff next_snapshot_id)"
        and same: "version_id_of before = version_id_of after"
        show "before = after"
    end
end

```

```

proof (cases "after = ?new_eff")
  case True
  have "new_version \<in> version_ids s"
    unfolding version_ids_def using before_mem same True by auto
  then show ?thesis using fresh by blast
next
  case False
  then have after_old: "after \<in> objects_of s" using after_mem by auto
  from source_wf have inj: "inj_on version_id_of (objects_of s)"
    unfolding state_well_formed_def unique_versions_def by blast
  show ?thesis using inj_onD[OF inj same before_mem after_old] .
qed
qed
show ?thesis using unique immutable by blast
qed

theorem integrate_evidence_preserves_audit_stores:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
    old_eff prior assessment new_version integration_time next_snapshot_id
    s' catalog' object_archive' payload_archive' ledger'"
  shows
    "active_history_disjoint s' \<and>
    object_archive_well_formed s' object_archive' \<and>
    effect_payload_archive_well_formed object_archive' payload_archive' \<and>
    evidence_ledger_well_formed s' object_archive' ledger'"
proof -
  let ?new_eff = "integrated_effect_object old_eff assessment new_version
  integration_time"
  from concrete have valid:
    "integrate_evidence_input_valid G s catalog object_archive payload_archive
    ledger old_eff prior assessment new_version integration_time
    next_snapshot_id"
  and target:
    "s' = replace_effect_version s old_eff ?new_eff next_snapshot_id"
  and object_archive_target:
    "object_archive' = extend_object_archive object_archive old_eff"
  and payload_archive_target:
    "payload_archive' = extend_effect_payload_archive payload_archive
    old_eff prior"
  and ledger_target:
    "ledger' = extend_evidence_ledger ledger new_version assessment"
  by (auto simp: integrate_evidence_step_def Let_def)
  from valid have source_wf: "state_well_formed G s"
  and source_disjoint: "active_history_disjoint s"
  and source_object_archive:
    "object_archive_well_formed s object_archive"
  and source_payload_archive:
    "effect_payload_archive_well_formed object_archive payload_archive"
  and source_ledger:
    "evidence_ledger_well_formed s object_archive ledger"
  and old_mem: "old_eff \<in> objects_of s"
  and old_archive_none:
    "object_archive (version_id_of old_eff) = None"
  and payload_old_none:
    "payload_archive (version_id_of old_eff) = None"
  and fresh:
    "new_version \<notin> version_ids s \<union> historical_versions_of s"
  and ledger_new_none: "ledger new_version = None"
  and assessment_bounds: "assessment_bounds assessment"
  and assessment_model:
    "evidence_assessment_model_of assessment \<in> models_of s"
  and old_kind: "kind_of old_eff = K_EFF"
  and payload_valid: "effect_payload_valid old_eff prior"
  unfolding integrate_evidence_input_valid_def by blast+
  from source_wf have source_unique: "unique_versions s"
  unfolding state_well_formed_def by blast
  have fresh_eff:
    "version_id_of ?new_eff
    \<notin> version_ids s \<union> historical_versions_of s"
  using fresh by simp
  have target_disjoint: "active_history_disjoint s'"
  unfolding target
  using replace_effect_preserves_active_history_disjoint
  [OF source_disjoint source_unique old_mem fresh_eff] .
  have target_object_archive:
    "object_archive_well_formed s' object_archive'"
proof -
  have archived_properties:
    "\<forall>v obj. object_archive' v = Some obj \<longrightarrow>"

```

```

    version_id_of obj = v \<and> v \<in> historical_versions_of s'"
proof (intro allI impI)
  fix v obj
  assume mapped: "object_archive' v = Some obj"
  show "version_id_of obj = v \<and> v \<in> historical_versions_of s'"
  proof (cases "v = version_id_of old_eff")
    case True
    with mapped object_archive_target target show ?thesis
      unfolding extend_object_archive_def by simp
  next
    case False
    with mapped object_archive_target have source_mapped:
      "object_archive v = Some obj"
      unfolding extend_object_archive_def by simp
    from source_object_archive source_mapped have
      "version_id_of obj = v \<and> v \<in> historical_versions_of s"
      unfolding object_archive_well_formed_def by blast
    with target show ?thesis by simp
  qed
qed
show ?thesis
  unfolding object_archive_well_formed_def
  using archived_properties target_disjoint
  unfolding active_history_disjoint_def by blast
qed
have target_payload_archive:
  "effect_payload_archive_well_formed object_archive' payload_archive'"
proof (unfold effect_payload_archive_well_formed_def, intro allI impI)
  fix v stored_payload
  assume mapped: "payload_archive' v = Some stored_payload"
  show "\<exists>eff. object_archive' v = Some eff \<and>
    effect_payload_valid eff stored_payload"
  proof (cases "v = version_id_of old_eff")
    case True
    with mapped payload_archive_target object_archive_target
      payload_valid show ?thesis
      unfolding extend_effect_payload_archive_def extend_object_archive_def
      by auto
  next
    case False
    with mapped payload_archive_target have source_mapped:
      "payload_archive v = Some stored_payload"
      unfolding extend_effect_payload_archive_def by simp
    from source_payload_archive source_mapped obtain eff where
      source_object: "object_archive v = Some eff"
      and valid_payload: "effect_payload_valid eff stored_payload"
      unfolding effect_payload_archive_well_formed_def by blast
    have "object_archive' v = Some eff"
      unfolding object_archive_target extend_object_archive_def
      using source_object False by simp
    with valid_payload show ?thesis by blast
  qed
qed
have target_ledger:
  "evidence_ledger_well_formed s' object_archive' ledger'"
proof (unfold evidence_ledger_well_formed_def, intro allI impI)
  fix v stored_assessment
  assume mapped: "ledger' v = Some stored_assessment"
  show "evidence_entry_well_formed s' object_archive' v stored_assessment"
  proof (cases "v = new_version")
    case True
    with mapped ledger_target have assessment_eq:
      "stored_assessment = assessment"
      unfolding extend_evidence_ledger_def by simp
    have target_wf: "state_well_formed G s'"
      using integrate_evidence_preserves_state_well_formed[OF concrete] .
    have exact:
      "objects_of s' = insert ?new_eff (objects_of s - {old_eff})"
      using concrete
      unfolding integrate_evidence_step_def Let_def by auto
    have new_mem: "?new_eff \<in> objects_of s'" using exact by simp
    have evidence_refs_in_new:
      "evidence_refs_of assessment \<subseq> evidence_of ?new_eff"
      by simp
    from target_wf new_mem have new_evidence_complete:
      "evidence_of ?new_eff
        \<subseq> version_ids s' \<union> historical_versions_of s'"
      unfolding state_well_formed_def evidence_well_formed_def
      evidence_references_complete_def
      by blast
  qed

```

```

from target_wf new_mem have evidence_complete:
  "evidence_refs_of assessment
   \<subseq> version_ids s' \<union> historical_versions_of s'"
  using evidence_refs_in_new new_evidence_complete by blast
from integrate_evidence_lineage_and_no_self_support[OF concrete]
have evidence_nonempty: "evidence_refs_of assessment \<noteq> {}"
  and new_not_self: "new_version \<notin> evidence_refs_of assessment"
  and prior_not_self:
    "version_id_of old_eff \<notin> evidence_refs_of assessment"
  by blast+
from valid have subject:
  "evidence_subject_of assessment = object_id_of old_eff"
  and prior_version:
    "evidence_prior_version_of assessment = version_id_of old_eff"
  unfolding integrate_evidence_input_valid_def by blast+
have resolved:
  "resolved_version_object s' object_archive' new_version ?new_eff"
  unfolding resolved_version_object_def using new_mem by simp
show ?thesis
  unfolding True assessment_eq evidence_entry_well_formed_def
  using resolved evidence_nonempty evidence_complete new_not_self
  prior_not_self subject prior_version evidence_refs_in_new
  assessment_bounds old_kind
  assessment_model
  unfolding assessment_bounds_def target
  by auto
next
case False
with mapped ledger_target have source_mapped:
  "ledger v = Some stored_assessment"
  unfolding extend_evidence_ledger_def by simp
from source_ledger source_mapped have source_entry:
  "evidence_entry_well_formed s object_archive v stored_assessment"
  unfolding evidence_ledger_well_formed_def by blast
have transferred:
  "evidence_entry_well_formed
   (replace_effect_version s old_eff ?new_eff next_snapshot_id)
   (extend_object_archive object_archive old_eff) v stored_assessment"
  using evidence_entry_replace_mono[OF old_mem fresh_eff
    old_archive_none source_entry] .
show ?thesis
  unfolding target object_archive_target using transferred .
qed
qed
show ?thesis
  using target_disjoint target_object_archive target_payload_archive
  target_ledger by blast
qed

theorem integrate_evidence_preserves_active_catalog:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
     old_eff prior assessment new_version integration_time next_snapshot_id
     s' catalog' object_archive' payload_archive' ledger'"
  shows "effect_catalog_well_formed s' catalog'"
proof -
  let ?new_eff = "integrated_effect_object old_eff assessment new_version
    integration_time"
  let ?new_payload = "integrated_effect_payload prior assessment"
  from concrete have valid:
    "integrate_evidence_input_valid G s catalog object_archive payload_archive
     ledger old_eff prior assessment new_version integration_time
     next_snapshot_id"
  and target:
    "s' = replace_effect_version s old_eff ?new_eff next_snapshot_id"
  and catalog_target:
    "catalog' = replace_active_effect_payload catalog
     (version_id_of old_eff) new_version ?new_payload"
  by (auto simp: integrate_evidence_step_def Let_def)
from valid have source_catalog: "effect_catalog_well_formed s catalog"
  and old_mem: "old_eff \<in> objects_of s"
  and fresh:
    "new_version \<notin> version_ids s \<union> historical_versions_of s"
  unfolding integrate_evidence_input_valid_def by blast+
have new_payload_valid: "effect_payload_valid ?new_eff ?new_payload"
  using integrate_evidence_payload_valid[OF valid] .
have new_mem: "?new_eff \<in> objects_of s'"
  unfolding target by simp
show ?thesis
  unfolding effect_catalog_well_formed_def

```

```

proof (intro allI impI)
  fix v stored_payload
  assume mapped: "catalog' v = Some stored_payload"
  show "\<exists>eff\<in>objects_of s'.
    version_id_of eff = v \<and> effect_payload_valid eff stored_payload"
  proof (cases "v = new_version")
    case True
    with mapped catalog_target have payload_eq:
      "stored_payload = ?new_payload"
    unfolding replace_active_effect_payload_def by simp
    show ?thesis
    proof (rule bexI[of _ ?new_eff])
      show "version_id_of ?new_eff = v \<and>
        effect_payload_valid ?new_eff stored_payload"
      using new_payload_valid True payload_eq by simp
      show "?new_eff \<in> objects_of s'" using new_mem .
    qed
  qed
next
case False
have old_key_different: "v \<noteq> version_id_of old_eff"
proof
  assume "v = version_id_of old_eff"
  with mapped catalog_target False show False
  unfolding replace_active_effect_payload_def by simp
qed
from mapped catalog_target False old_key_different have source_mapped:
  "catalog v = Some stored_payload"
unfolding replace_active_effect_payload_def by simp
from source_catalog source_mapped obtain eff where eff_mem:
  "eff \<in> objects_of s"
and eff_version: "version_id_of eff = v"
and eff_payload: "effect_payload_valid eff stored_payload"
unfolding effect_catalog_well_formed_def by blast
have eff_not_old: "eff \<noteq> old_eff"
using eff_version old_key_different by auto
have "eff \<in> objects_of s'"
unfolding target using eff_mem eff_not_old by auto
with eff_version eff_payload show ?thesis by blast
qed
qed
qed

theorem integrate_evidence_preserves_references_time_context:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
      old_eff prior assessment new_version integration_time next_snapshot_id
      s' catalog' object_archive' payload_archive' ledger'"
  shows
    "\<forall>obj\<in>objects_of s'.
      references_complete s' obj \<and>
      evidence_well_formed s' obj \<and>
      time_well_formed obj \<and>
      perspective_well_formed s' obj \<and>
      context_well_formed s' obj"
  using integrate_evidence_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def by blast

theorem integrate_evidence_refines_generic_step:
  assumes concrete:
    "integrate_evidence_step G s catalog object_archive payload_archive ledger
      old_eff prior assessment new_version integration_time next_snapshot_id
      s' catalog' object_archive' payload_archive' ledger'"
  shows "step G s IntegrateEvidence s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
  and allowed: "IntegrateEvidence \<in> allowed_operations_of G"
  by (auto simp: integrate_evidence_step_def integrate_evidence_input_valid_def)
  have target_wf: "state_well_formed G s'"
  using integrate_evidence_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
  using integrate_evidence_preserves_identity[OF concrete] by blast
  show ?thesis
  unfolding step_def using source_wf target_wf allowed immutable by blast
qed
end

```

Beweisstatus dieser D.8-Fassung: kernelgeprüft. Der gezielte Isabelle2025-2-Clean-Build verarbeitete die vollständige neunteilige Session und endete mit Exit-Code 0. T5 enthält 1.564 Quellzeilen und keine Vorkommen von sorry, oops, axiomatization oder quick_and_dirty. Der Quellhash lautet 4422076e2bd6cb2ff9fe5e2d5e2e135e3bf3c05919a1ca03a42b1faccd5fddc0. Belegt sind die formulierten konstruktiven Delta-, Lifecycle-, Unsicherheits-, Lineage-, Archiv-, Katalog-, Zustandserhaltungs- und Refinementsätze. Nicht belegt sind empirische Richtigkeit einer Evidenzbewertung, Vollständigkeit zulässiger Evidenzarten, allgemeine Konservativität oder Produktivengine-Äquivalenz.

D.9 Nicht-vakuoser Ausführungszeuge für T5

Der Zeuge verhindert, dass T5 lediglich als konditionales Schema mit unerfüllten Vorbedingungen bestehen bleibt. Ein endlicher Ausgangszustand enthält eine aktive Erscheinung als unabhängige Evidenzquelle, eine typisierte Relationsursache und eine K_EFF-Hypothese. Die konkrete Assessment-Instanz stützt die Hypothese. Der mechanisch konstruierte Nachzustand enthält Version 4 desselben Effektoobjekts mit Lifecycle Supported, previous_version = Some 3 und Evidenzmenge {1}; Version 3 und ihr Payload liegen in den getrennten Archiven, der aktive Katalog verweist auf Version 4 und der Evidenz-Ledger hält das Assessment.

```
theory UZA_0_9_1_Integrate_Evidence_Witness
  imports UZA_0_9_1_Integrate_Evidence UZA_0_9_1_Base_Model
begin

section \<open>A non-vacuous executable witness for T5\<close>

text \<open>
  This finite witness activates the substantive T5 antecedents. A typed
  relation supports one K_EFF hypothesis; an independent active appearance is
  then integrated as supporting evidence. The constructed successor is a
  fresh Supported version, the predecessor and its payload are archived, and
  the evidence ledger records a non-self-referential assessment. Thus the T5
  preservation theorems are exercised by an actual state transition rather
  than only by a conditional schema.
\<close>

definition t5_evidence_object :: uza_object where
  "t5_evidence_object =
    base_object\<lparr>
      object_id_of := 1,
      version_id_of := 1,
      kind_of := K_ENT,
      lifecycle_of := Draft,
      perspective_of := Some 10,
      context_of := Some 20,
      object_status_of := Some Applicable
    \<rparr>"

definition t5_origin_relation :: uza_object where
  "t5_origin_relation =
    base_object\<lparr>
      object_id_of := 2,
      version_id_of := 2,
      kind_of := K_REL,
      lifecycle_of := Hypothesis,
      references_of := {1},
      context_of := Some 20,
      source_ref_of := Some 1,
      target_ref_of := Some 1,
      object_status_of := Some Applicable
    \<rparr>"

definition t5_effect_hypothesis :: uza_object where
  "t5_effect_hypothesis =
    base_object\<lparr>
      object_id_of := 3,
      version_id_of := 3,
      kind_of := K_EFF,
      lifecycle_of := Hypothesis,
      references_of := {2},
      perspective_of := Some 10,
      context_of := Some 20,
      model_of := Some 30,
      source_ref_of := Some 2,
      effect_origin_of := Some 2,
      basis_refs_of := {2},
      object_status_of := Some Applicable
    \<rparr>"

definition t5_witness_objects :: "uza_object set" where
  "t5_witness_objects =
    {t5_evidence_object, t5_origin_relation, t5_effect_hypothesis}"
```

```

definition t5_witness_source :: snapshot where
  "t5_witness_source =
    \<lparr>
      snapshot_id_of = 1,
      objects_of = t5_witness_objects,
      historical_versions_of = {},
      perspectives_of = {10},
      contexts_of = {20},
      models_of = {30},
      authorities_of = {},
      contains_edges_of = {},
      projections_of = {},
      couplings_of = {},
      transmissions_of = {},
      boundaries_of = {}
    \<rparr>"

definition t5_witness_grammar :: grammar where
  "t5_witness_grammar =
    \<lparr>
      grammar_id_of = 5,
      supported_kinds_of = {K_ENT, K_REL, K_EFF},
      allowed_operations_of = {IntegrateEvidence},
      patch_budget_of = 0
    \<rparr>"

definition t5_prior_payload :: effect_hypothesis_payload where
  "t5_prior_payload =
    \<lparr>
      hypothesis_cause_of = 2,
      hypothesis_target_of = None,
      hypothesis_vector_of = None,
      hypothesis_uncertainty_of = 3 / 4,
      hypothesis_profile_model_of = 30
    \<rparr>"

definition t5_assessment :: evidence_assessment where
  "t5_assessment =
    \<lparr>
      evidence_subject_of = 3,
      evidence_prior_version_of = 3,
      evidence_refs_of = {1},
      evidence_disposition_of = EvidenceSupports,
      evidence_strength_of = 1 / 2,
      evidence_post_uncertainty_of = 1 / 2,
      evidence_post_target_of = None,
      evidence_post_vector_of = None,
      evidence_assessment_model_of = 30,
      validation_uncertainty_max_of = 1 / 4,
      validation_evidence_min_of = 2
    \<rparr>"

definition t5_active_catalog :: effect_catalog where
  "t5_active_catalog = Map.empty(3 \<mapsto> t5_prior_payload)"

definition t5_object_archive :: object_archive where
  "t5_object_archive = Map.empty"

definition t5_payload_archive :: effect_payload_archive where
  "t5_payload_archive = Map.empty"

definition t5_evidence_ledger :: evidence_ledger where
  "t5_evidence_ledger = Map.empty"

definition t5_successor_effect :: uza_object where
  "t5_successor_effect =
    integrated_effect_object t5_effect_hypothesis t5_assessment 4 2"

definition t5_witness_target :: snapshot where
  "t5_witness_target =
    replace_effect_version t5_witness_source t5_effect_hypothesis
      t5_successor_effect 2"

definition t5_target_catalog :: effect_catalog where
  "t5_target_catalog =
    replace_active_effect_payload t5_active_catalog 3 4
      (integrated_effect_payload t5_prior_payload t5_assessment)"

definition t5_target_object_archive :: object_archive where

```

```

    "t5_target_object_archive =
      extend_object_archive t5_object_archive t5_effect_hypothesis"

definition t5_target_payload_archive :: effect_payload_archive where
    "t5_target_payload_archive =
      extend_effect_payload_archive t5_payload_archive t5_effect_hypothesis
      t5_prior_payload"

definition t5_target_evidence_ledger :: evidence_ledger where
    "t5_target_evidence_ledger =
      extend_evidence_ledger t5_evidence_ledger 4 t5_assessment"

lemmas t5_object_defs =
    t5_evidence_object_def t5_origin_relation_def t5_effect_hypothesis_def

lemma t5_witness_source_simps [simp]:
    "snapshot_id_of t5_witness_source = 1"
    "objects_of t5_witness_source = t5_witness_objects"
    "historical_versions_of t5_witness_source = {}"
    "perspectives_of t5_witness_source = {10}"
    "contexts_of t5_witness_source = {20}"
    "models_of t5_witness_source = {30}"
    "authorities_of t5_witness_source = {}"
    "contains_edges_of t5_witness_source = {}"
    "projections_of t5_witness_source = {}"
    "couplings_of t5_witness_source = {}"
    "transmissions_of t5_witness_source = {}"
    "boundaries_of t5_witness_source = {}"
    by (simp_all add: t5_witness_source_def)

lemma t5_witness_version_ids [simp]:
    "version_ids t5_witness_source = {1,2,3}"
    by (auto simp: version_ids_def t5_witness_objects_def t5_object_defs
        base_object_def)

lemma t5_witness_source_well_formed:
    "state_well_formed t5_witness_grammar t5_witness_source"
    unfolding state_well_formed_def
    by (auto simp: unique_versions_def inj_on_def
        t5_witness_grammar_def t5_witness_objects_def t5_object_defs
        base_object_def present_ref_def references_complete_def
        evidence_well_formed_def evidence_references_complete_def
        time_well_formed_def relation_well_formed_def effect_well_formed_def
        perspective_well_formed_def perspective_bound_kind_def
        context_well_formed_def context_bound_kind_def relevance_well_formed_def
        orientation_well_formed_def action_well_formed_def
        feedback_well_formed_def)

lemma t5_witness_catalog_well_formed:
    "effect_catalog_well_formed t5_witness_source t5_active_catalog"
    unfolding effect_catalog_well_formed_def
    by (auto simp: t5_active_catalog_def t5_prior_payload_def
        t5_witness_objects_def t5_object_defs base_object_def
        effect_payload_valid_def)

lemma t5_witness_input_valid:
    "integrate_evidence_input_valid t5_witness_grammar t5_witness_source
      t5_active_catalog t5_object_archive t5_payload_archive
      t5_evidence_ledger t5_effect_hypothesis t5_prior_payload
      t5_assessment 4 2 2"
    unfolding integrate_evidence_input_valid_def
    using t5_witness_source_well_formed t5_witness_catalog_well_formed
    by (auto simp: active_history_disjoint_def object_archive_well_formed_def
        effect_payload_archive_well_formed_def evidence_ledger_well_formed_def
        t5_witness_grammar_def t5_object_archive_def t5_payload_archive_def
        t5_evidence_ledger_def t5_active_catalog_def t5_witness_objects_def
        t5_object_defs base_object_def t5_prior_payload_def t5_assessment_def
        effect_payload_valid_def present_ref_def active_evidence_source_def
        assessment_bounds_def assessment_change_valid_def
        admissible_post_target_def effect_vector_valid_def)

theorem t5_witness_concrete_step:
    "integrate_evidence_step t5_witness_grammar t5_witness_source
      t5_active_catalog t5_object_archive t5_payload_archive
      t5_evidence_ledger t5_effect_hypothesis t5_prior_payload
      t5_assessment 4 2 2 t5_witness_target t5_target_catalog
      t5_target_object_archive t5_target_payload_archive
      t5_target_evidence_ledger"
    unfolding integrate_evidence_step_def Let_def t5_witness_target_def
    t5_successor_effect_def t5_target_catalog_def

```

```

    t5_target_object_archive_def t5_target_payload_archive_def
    t5_target_evidence_ledger_def
    using t5_witness_input_valid
    by (simp add: t5_effect_hypothesis_def base_object_def)

theorem t5_witness_is_nonvacuous:
  "lifecycle_of t5_effect_hypothesis = Hypothesis \<and>
  evidence_refs_of t5_assessment = {1} \<and>
  lifecycle_of t5_successor_effect = Supported \<and>
  previous_version_of t5_successor_effect = Some 3 \<and>
  evidence_of t5_successor_effect = {1} \<and>
  version_id_of t5_successor_effect = 4 \<and>
  object_id_of t5_successor_effect = object_id_of t5_effect_hypothesis \<and>
  4 \<notin> evidence_refs_of t5_assessment \<and>
  3 \<notin> evidence_refs_of t5_assessment"
  by (simp add: t5_successor_effect_def t5_effect_hypothesis_def
    t5_assessment_def base_object_def integrated_lifecycle_def
    validation_ready_def)

theorem t5_witness_target_certificates:
  "state_well_formed t5_witness_grammar t5_witness_target \<and>
  active_history_disjoint t5_witness_target \<and>
  effect_catalog_well_formed t5_witness_target t5_target_catalog \<and>
  object_archive_well_formed t5_witness_target
    t5_target_object_archive \<and>
  effect_payload_archive_well_formed t5_target_object_archive
    t5_target_payload_archive \<and>
  evidence_ledger_well_formed t5_witness_target
    t5_target_object_archive t5_target_evidence_ledger \<and>
  step t5_witness_grammar t5_witness_source IntegrateEvidence
    t5_witness_target"
  using integrate_evidence_preserves_state_well_formed
    [OF t5_witness_concrete_step]
    integrate_evidence_preserves_audit_stores
    [OF t5_witness_concrete_step]
    integrate_evidence_preserves_active_catalog
    [OF t5_witness_concrete_step]
    integrate_evidence_refines_generic_step
    [OF t5_witness_concrete_step]
  by blast

theorem t5_witness_exact_audit_delta:
  "historical_versions_of t5_witness_target = {3} \<and>
  t5_target_object_archive 3 = Some t5_effect_hypothesis \<and>
  t5_target_payload_archive 3 = Some t5_prior_payload \<and>
  t5_target_catalog 3 = None \<and>
  t5_target_catalog 4 =
    Some (integrated_effect_payload t5_prior_payload t5_assessment) \<and>
  t5_target_evidence_ledger 4 = Some t5_assessment"
  by (simp add: t5_witness_target_def t5_successor_effect_def
    t5_target_object_archive_def t5_object_archive_def
    extend_object_archive_def t5_target_payload_archive_def
    t5_payload_archive_def extend_effect_payload_archive_def
    t5_target_catalog_def t5_active_catalog_def
    replace_active_effect_payload_def t5_target_evidence_ledger_def
    t5_evidence_ledger_def extend_evidence_ledger_def
    t5_effect_hypothesis_def base_object_def)

end

```

Beweisstatus dieser D.9-Fassung: kernelgeprüft und nicht-vakuos im ausgewiesenen Umfang. Der Zeugenhash lautet

f5e037e578335431903754153deaedeadd461fed8d359a0511e36b59a825a7d. Bewiesen sind die tatsächliche Erfüllung der T5-Eingabebedingungen, der konkrete IntegrateEvidence-Schritt, Hypothesis → Supported, die exakte Audit-Differenz, Zielwohlgeformtheit und Refinement zur generischen step-Relation. Der Zeuge ist ein Existenz- und Ausführbarkeitsbeleg für genau diese endliche Konfiguration, kein empirischer Universalitätsbeweis.

D.10 Gerichtete T6-Semantik und Erhaltung für ClassifyResonance

T6 operationalisiert die in §15 geforderte Richtung der Resonanz. Ein `resonance_model_profile` bindet eine Modell-ID, eine Matrix $K: \text{effect_dimension} \rightarrow \text{effect_dimension} \rightarrow \text{real}$, vier geordnete Formschwellen und eine Validierungsgrenze für Unsicherheit. Der Score summiert $K(\text{source}, \text{target}) \cdot V_{\text{source}}(\text{source}) \cdot V_{\text{target}}(\text{target})$, normalisiert durch $9 \cdot \sum |K|$ und klemmt das Resultat auf $[-1, 1]$. Weil Quell- und Zieldimension in K verschiedene Argumentpositionen besitzen, ist Symmetrie weder vorausgesetzt noch impliziert.

Die Operation akzeptiert nur zwei verschiedene aktive Effektoobjekte mit typisierten V10-Vektoren im selben deklarierten Kontext. Perspektive, Kontext und Profilmodell müssen im Snapshot vorhanden sein. Begründungsreferenzen sind nichtleer, aktiv, von den beiden Effekten verschieden und vollständig auflösbar. Das Ergebnisobjekt und der Payload werden ausschließlich aus Request und Quellzustand konstruiert. Die Beweise leiten Objektfrische, exaktes Delta, Score- und Stärkengrenzen, totale Klassifikation, Source-/Target-Integrität, Payloadgültigkeit, Zielwohlgeformtheit, Identität, Katalogerhaltung und generisches step-Refinement her.

```
theory UZA_0_9_1_Classify_Resonance
  imports UZA_0_9_1_Integrate_Evidence
begin

section \<open>T6 ClassifyResonance: directed, model-bound resonance semantics\<close>

text \<open>
  T6 creates exactly one fresh K_RES object from an ordered pair of active
  effects. Direction is semantic, not merely documentary: a versioned
  interaction kernel maps source dimensions to target dimensions. Thus the
  score for A-to-B need not equal the score for B-to-A.

  Perspective, context, model, temporal horizon, rationale, uncertainty,
  thresholds, form, score, and strength are persisted in a typed side
  catalog. The result object and payload are constructed from the request;
  target-state well-formedness is derived and is not an input assumption.
\<close>

type_synonym resonance_kernel =
  "effect_dimension \<Rightarrow> effect_dimension \<Rightarrow> real"

record resonance_model_profile =
  resonance_profile_model_of :: model_id
  resonance_kernel_of :: resonance_kernel
  amplification_cutoff_of :: real
  complement_cutoff_of :: real
  productive_tension_cutoff_of :: real
  inhibition_cutoff_of :: real
  resonance_validation_uncertainty_max_of :: real

record resonance_request =
  requested_resonance_object_id_of :: object_id
  requested_resonance_version_id_of :: version_id
  requested_source_effect_of :: version_id
  requested_target_effect_of :: version_id
  requested_source_vector_of :: effect_vector
  requested_target_vector_of :: effect_vector
  requested_resonance_profile_of :: resonance_model_profile
  requested_resonance_perspective_of :: perspective_id
  requested_resonance_context_of :: context_id
  requested_resonance_horizon_of :: nat
  requested_resonance_uncertainty_of :: real
  requested_resonance_rationale_refs_of :: "version_id set"
  requested_resonance_rationale_of :: string
  requested_resonance_valid_from_of :: time_point

record resonance_payload =
  classified_source_effect_of :: version_id
  classified_target_effect_of :: version_id
  classified_source_vector_of :: effect_vector
```

```

classified_target_vector_of :: effect_vector
classified_resonance_profile_of :: resonance_model_profile
classified_resonance_perspective_of :: perspective_id
classified_resonance_context_of :: context_id
classified_resonance_form_of :: resonance_form
classified_resonance_score_of :: real
classified_resonance_strength_of :: real
classified_resonance_horizon_of :: nat
classified_resonance_uncertainty_of :: real
classified_resonance_rationale_refs_of :: "version_id set"
classified_resonance_rationale_of :: string

type_synonym resonance_catalog = "version_id \ $\rightarrow$  resonance_payload"

lemma effect_dimension_UNIV:
  "(UNIV :: effect_dimension set) =
   {Connection, Stability, Openness, Learning, Cooperation,
    Agency, Transparency, Freedom, Responsibility, Regeneration}"
proof (rule set_eqI)
  fix dimension :: effect_dimension
  show
    "dimension \ $\in$  UNIV  $\longleftrightarrow$ 
     dimension \ $\in$ 
     {Connection, Stability, Openness, Learning, Cooperation,
      Agency, Transparency, Freedom, Responsibility, Regeneration}"
  by (case_tac dimension) simp_all
qed

lemma effect_dimensions_finite [simp]:
  "finite (UNIV :: effect_dimension set)"
  by (simp add: effect_dimension_UNIV)

definition resonance_kernel_mass :: "resonance_kernel \ $\rightarrow$  real" where
  "resonance_kernel_mass interaction_kernel =
   sum (\ $\lambda$ source_dimension.
     sum (\ $\lambda$ target_dimension.
       abs (interaction_kernel source_dimension target_dimension)) UNIV) UNIV"

definition resonance_profile_well_formed ::
  "resonance_model_profile \ $\rightarrow$  bool" where
  "resonance_profile_well_formed profile  $\longleftrightarrow$ 
   ( $\forall$ source_dimension target_dimension.
    -1  $\leq$  resonance_kernel_of profile source_dimension target_dimension  $\wedge$ 
     resonance_kernel_of profile source_dimension target_dimension  $\leq$  1)  $\wedge$ 
    resonance_kernel_mass (resonance_kernel_of profile)  $>$  0  $\wedge$ 
    -1  $\leq$  inhibition_cutoff_of profile  $\wedge$ 
    inhibition_cutoff_of profile  $<$  productive_tension_cutoff_of profile  $\wedge$ 
    productive_tension_cutoff_of profile  $<$  0  $\wedge$ 
    0  $<$  complement_cutoff_of profile  $\wedge$ 
    complement_cutoff_of profile  $<$  amplification_cutoff_of profile  $\wedge$ 
    amplification_cutoff_of profile  $\leq$  1  $\wedge$ 
    0  $\leq$  resonance_validation_uncertainty_max_of profile  $\wedge$ 
    resonance_validation_uncertainty_max_of profile  $\leq$  1"

definition clamp11 :: "real \ $\rightarrow$  real" where
  "clamp11 value = max (-1) (min 1 value)"

definition directional_resonance_raw_score ::
  "resonance_model_profile \ $\rightarrow$  effect_vector  $\rightarrow$ 
   effect_vector  $\rightarrow$  real" where
  "directional_resonance_raw_score profile source_vector target_vector =
   (let mass = resonance_kernel_mass (resonance_kernel_of profile) in
    if mass = 0 then 0
    else
     sum (\ $\lambda$ source_dimension.
       sum (\ $\lambda$ target_dimension.
         resonance_kernel_of profile source_dimension target_dimension *
          of_int (source_vector source_dimension *
                 target_vector target_dimension)) UNIV) UNIV /
    (9 * mass))"

definition directional_resonance_score ::
  "resonance_model_profile \ $\rightarrow$  effect_vector  $\rightarrow$ 
   effect_vector  $\rightarrow$  real" where
  "directional_resonance_score profile source_vector target_vector =
   clamp11
    (directional_resonance_raw_score profile source_vector target_vector)"

definition classify_directional_score ::
  "resonance_model_profile \ $\rightarrow$  real  $\rightarrow$  resonance_form" where

```

```

"classify_directional_score profile score =
  (if score \<ge> amplification_cutoff_of profile then Amplification
   else if score \<ge> complement_cutoff_of profile then Complement
   else if score \<le> inhibition_cutoff_of profile then Inhibition
   else if score \<le> productive_tension_cutoff_of profile
     then ProductiveTension
   else Neutral)"

definition uncertainty_adjusted_resonance_strength ::
  "real \<Rightarrow> real \<Rightarrow> real" where
  "uncertainty_adjusted_resonance_strength score uncertainty =
    clamp01 (abs score * (1 - clamp01 uncertainty))"

definition effect_validated_at :: "snapshot \<Rightarrow> version_id \<Rightarrow> bool" where
  "effect_validated_at s version \<longlefttrightarrow>
    (\<exists>effect\<in>objects_of s.
     version_id_of effect = version \<and>
     kind_of effect = K_EFF \<and>
     lifecycle_of effect = Validated)"

definition active_classifiable_effect ::
  "snapshot \<Rightarrow> effect_catalog \<Rightarrow> version_id \<Rightarrow>
    effect_vector \<Rightarrow> context_id \<Rightarrow> bool" where
  "active_classifiable_effect s effect_catalog version vector context \<longlefttrightarrow>
    (\<exists>effect\<in>objects_of s.
     version_id_of effect = version \<and>
     kind_of effect = K_EFF \<and>
     lifecycle_of effect \<in> {Supported, Validated} \<and>
     context_of effect = Some context \<and>
     (\<exists>payload.
      effect_catalog version = Some payload \<and>
      hypothesis_vector_of payload = Some vector \<and>
      effect_payload_valid effect payload)))"

definition distinct_active_effect_objects ::
  "snapshot \<Rightarrow> version_id \<Rightarrow> version_id \<Rightarrow> bool" where
  "distinct_active_effect_objects s source target \<longlefttrightarrow>
    (\<exists>source_effect\<in>objects_of s. \<exists>target_effect\<in>objects_of s.
     version_id_of source_effect = source \<and>
     version_id_of target_effect = target \<and>
     object_id_of source_effect \<noteq> object_id_of target_effect)"

definition active_resonance_rationale ::
  "snapshot \<Rightarrow> version_id \<Rightarrow> version_id \<Rightarrow>
    version_id set \<Rightarrow> bool" where
  "active_resonance_rationale s source target rationale_refs \<longlefttrightarrow>
    rationale_refs \<noteq> {} \<and>
    rationale_refs \<inter> {source, target} = {} \<and>
    (\<forall>version\<in>rationale_refs.
     \<exists>obj\<in>objects_of s.
     version_id_of obj = version \<and>
     lifecycle_of obj \<notin> {Revised, Rejected, Archived})"

definition requested_directional_score :: "resonance_request \<Rightarrow> real" where
  "requested_directional_score request =
    directional_resonance_score
      (requested_resonance_profile_of request)
      (requested_source_vector_of request)
      (requested_target_vector_of request)"

definition requested_resonance_form ::
  "resonance_request \<Rightarrow> resonance_form" where
  "requested_resonance_form request =
    classify_directional_score
      (requested_resonance_profile_of request)
      (requested_directional_score request)"

definition requested_resonance_strength :: "resonance_request \<Rightarrow> real" where
  "requested_resonance_strength request =
    uncertainty_adjusted_resonance_strength
      (requested_directional_score request)
      (requested_resonance_uncertainty_of request)"

definition classified_resonance_lifecycle ::
  "snapshot \<Rightarrow> resonance_request \<Rightarrow> lifecycle_status" where
  "classified_resonance_lifecycle s request =
    (if effect_validated_at s (requested_source_effect_of request) \<and>
     effect_validated_at s (requested_target_effect_of request) \<and>
     requested_resonance_uncertainty_of request \<le>
     resonance_validation_uncertainty_max_of

```

```

        (requested_resonance_profile_of request)
    then Validated else Supported)"

definition requested_resonance_references ::
    "resonance_request \<Rightarrow> version_id set" where
    "requested_resonance_references request =
        insert (requested_source_effect_of request)
            (insert (requested_target_effect_of request)
                (requested_resonance_rationale_refs_of request))"

definition classified_resonance_object ::
    "snapshot \<Rightarrow> resonance_request \<Rightarrow> uza_object" where
    "classified_resonance_object s request =
        \<lparr>
            object_id_of = requested_resonance_object_id_of request,
            version_id_of = requested_resonance_version_id_of request,
            kind_of = K_RES,
            lifecycle_of = classified_resonance_lifecycle s request,
            previous_version_of = None,
            references_of = requested_resonance_references request,
            evidence_of = requested_resonance_rationale_refs_of request,
            perspective_of = Some (requested_resonance_perspective_of request),
            context_of = Some (requested_resonance_context_of request),
            model_of = Some
                (resonance_profile_model_of (requested_resonance_profile_of request)),
            authority_of = None,
            valid_from_of = requested_resonance_valid_from_of request,
            valid_to_of = None,
            source_ref_of = Some (requested_source_effect_of request),
            target_ref_of = Some (requested_target_effect_of request),
            effect_origin_of = None,
            basis_refs_of =
                {requested_source_effect_of request, requested_target_effect_of request},
            execution_ref_of = None,
            observation_refs_of = {},
            object_status_of = Some Applicable
        \<rparr>"

definition classified_resonance_payload ::
    "resonance_request \<Rightarrow> resonance_payload" where
    "classified_resonance_payload request =
        \<lparr>
            classified_source_effect_of = requested_source_effect_of request,
            classified_target_effect_of = requested_target_effect_of request,
            classified_source_vector_of = requested_source_vector_of request,
            classified_target_vector_of = requested_target_vector_of request,
            classified_resonance_profile_of = requested_resonance_profile_of request,
            classified_resonance_perspective_of =
                requested_resonance_perspective_of request,
            classified_resonance_context_of = requested_resonance_context_of request,
            classified_resonance_form_of = requested_resonance_form request,
            classified_resonance_score_of = requested_directional_score request,
            classified_resonance_strength_of = requested_resonance_strength request,
            classified_resonance_horizon_of = requested_resonance_horizon_of request,
            classified_resonance_uncertainty_of =
                requested_resonance_uncertainty_of request,
            classified_resonance_rationale_refs_of =
                requested_resonance_rationale_refs_of request,
            classified_resonance_rationale_of = requested_resonance_rationale_of request
        \<rparr>"

definition resonance_payload_valid ::
    "uza_object \<Rightarrow> resonance_payload \<Rightarrow> bool" where
    "resonance_payload_valid resonance payload \<longlefttrightarrow>
        kind_of resonance = K_RES \<and>
        lifecycle_of resonance \<in> {Supported, Validated} \<and>
        classified_source_effect_of payload \<noteq> classified_target_effect_of payload \<and>
        source_ref_of resonance = Some (classified_source_effect_of payload) \<and>
        target_ref_of resonance = Some (classified_target_effect_of payload) \<and>
        basis_refs_of resonance =
            {classified_source_effect_of payload, classified_target_effect_of payload} \<and>
        references_of resonance =
            insert (classified_source_effect_of payload)
                (insert (classified_target_effect_of payload)
                    (classified_resonance_rationale_refs_of payload)) \<and>
        evidence_of resonance = classified_resonance_rationale_refs_of payload \<and>
        perspective_of resonance =
            Some (classified_resonance_perspective_of payload) \<and>
        context_of resonance = Some (classified_resonance_context_of payload) \<and>
        model_of resonance = Some
    "
```



```

    (resonance_profile_model_of (classified_resonance_profile_of payload)) \<and>
    resonance_profile_well_formed (classified_resonance_profile_of payload) \<and>
    effect_vector_valid (classified_source_vector_of payload) \<and>
    effect_vector_valid (classified_target_vector_of payload) \<and>
    classified_resonance_score_of payload =
        directional_resonance_score
            (classified_resonance_profile_of payload)
            (classified_source_vector_of payload)
            (classified_target_vector_of payload) \<and>
    classified_resonance_form_of payload =
        classify_directional_score
            (classified_resonance_profile_of payload)
            (classified_resonance_score_of payload) \<and>
    classified_resonance_strength_of payload =
        uncertainty_adjusted_resonance_strength
            (classified_resonance_score_of payload)
            (classified_resonance_uncertainty_of payload) \<and>
    0 \<le> classified_resonance_strength_of payload \<and>
    classified_resonance_strength_of payload \<le> 1 \<and>
    classified_resonance_horizon_of payload > 0 \<and>
    0 \<le> classified_resonance_uncertainty_of payload \<and>
    classified_resonance_uncertainty_of payload \<le> 1 \<and>
    classified_resonance_rationale_refs_of payload \<noteq> {} \<and>
    classified_resonance_rationale_of payload \<noteq> [] \<and>
    object_status_of resonance = Some Applicable"

definition resonance_catalog_well_formed ::
    "snapshot \<Rightarrow> resonance_catalog \<Rightarrow> bool" where
    "resonance_catalog_well_formed s catalog \<longleftarrowrightarrow>
        (\<forall>version payload. catalog version = Some payload \<longrightarrow>
            (\<exists>resonance\<in>objects_of s.
                version_id_of resonance = version \<and>
                resonance_payload_valid resonance payload \<and>
                references_complete s resonance \<and>
                evidence_well_formed s resonance))"

definition classify_resonance_input_valid ::
    "grammar \<Rightarrow> snapshot \<Rightarrow> effect_catalog \<Rightarrow>
        resonance_catalog \<Rightarrow> resonance_request \<Rightarrow> nat \<Rightarrow> bool" where
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
        next_snapshot_id \<longleftarrowrightarrow>
        state_well_formed G s \<and>
        active_history_disjoint s \<and>
        effect_catalog_well_formed s effect_catalog \<and>
        resonance_catalog_well_formed s resonance_catalog \<and>
        ClassifyResonance \<in> allowed_operations_of G \<and>
        K_RES \<in> supported_kinds_of G \<and>
        requested_resonance_object_id_of request \<notin>
            object_id_of ` objects_of s \<and>
        requested_resonance_version_id_of request \<notin>
            version_ids s \<union> historical_versions_of s \<and>
        resonance_catalog (requested_resonance_version_id_of request) = None \<and>
        requested_source_effect_of request \<noteq> requested_target_effect_of request \<and>
        distinct_active_effect_objects s
            (requested_source_effect_of request)
            (requested_target_effect_of request) \<and>
        active_classifiable_effect s effect_catalog
            (requested_source_effect_of request)
            (requested_source_vector_of request)
            (requested_resonance_context_of request) \<and>
        active_classifiable_effect s effect_catalog
            (requested_target_effect_of request)
            (requested_target_vector_of request)
            (requested_resonance_context_of request) \<and>
        effect_vector_valid (requested_source_vector_of request) \<and>
        effect_vector_valid (requested_target_vector_of request) \<and>
        resonance_profile_well_formed (requested_resonance_profile_of request) \<and>
        resonance_profile_model_of (requested_resonance_profile_of request)
            \<in> models_of s \<and>
        requested_resonance_perspective_of request \<in> perspectives_of s \<and>
        requested_resonance_context_of request \<in> contexts_of s \<and>
        requested_resonance_horizon_of request > 0 \<and>
        0 \<le> requested_resonance_uncertainty_of request \<and>
        requested_resonance_uncertainty_of request \<le> 1 \<and>
        active_resonance_rationale s
            (requested_source_effect_of request)
            (requested_target_effect_of request)
            (requested_resonance_rationale_refs_of request) \<and>
        requested_resonance_rationale_of request \<noteq> [] \<and>
        snapshot_id_of s < next_snapshot_id"
```

```

definition add_classified_resonance ::
  "snapshot \<Rightarrow> resonance_request \<Rightarrow> nat \<Rightarrow> snapshot" where
  "add_classified_resonance s request next_snapshot_id =
    s\<lparr>
      snapshot_id_of := next_snapshot_id,
      objects_of := insert (classified_resonance_object s request) (objects_of s)
    \<rparr>"

definition extend_resonance_catalog ::
  "resonance_catalog \<Rightarrow> resonance_request \<Rightarrow> resonance_catalog" where
  "extend_resonance_catalog catalog request =
    catalog(requested_resonance_version_id_of request \<mapsto>
      classified_resonance_payload request)"

definition classify_resonance_step ::
  "grammar \<Rightarrow> snapshot \<Rightarrow> effect_catalog \<Rightarrow>
    resonance_catalog \<Rightarrow> resonance_request \<Rightarrow> nat \<Rightarrow>
    snapshot \<Rightarrow> resonance_catalog \<Rightarrow> bool" where
  "classify_resonance_step G s effect_catalog resonance_catalog request
    next_snapshot_id s' resonance_catalog' \<longlefttrightarrow>
    classify_resonance_input_valid G s effect_catalog resonance_catalog request
    next_snapshot_id \<and>
    s' = add_classified_resonance s request next_snapshot_id \<and>
    resonance_catalog' = extend_resonance_catalog resonance_catalog request"

subsection \<open>Score, classification, constructor, and exact delta\<close>

lemma clamp11_bounded:
  "-1 \<le> clamp11 value \<and> clamp11 value \<le> 1"
  by (simp add: clamp11_def)

theorem directional_resonance_score_bounded:
  "-1 \<le> directional_resonance_score profile source_vector target_vector \<and>
    directional_resonance_score profile source_vector target_vector \<le> 1"
  by (simp add: directional_resonance_score_def clamp11_bounded)

theorem uncertainty_adjusted_resonance_strength_bounded:
  "0 \<le> uncertainty_adjusted_resonance_strength score uncertainty \<and>
    uncertainty_adjusted_resonance_strength score uncertainty \<le> 1"
  unfolding uncertainty_adjusted_resonance_strength_def clamp01_def
  by auto

theorem classify_directional_score_total:
  "classify_directional_score profile score
    \<in> {Amplification, Complement, ProductiveTension, Inhibition, Neutral}"
  by (auto simp: classify_directional_score_def)

lemma classified_lifecycle_cases:
  "classified_resonance_lifecycle s request \<in> {Supported, Validated}"
  by (auto simp: classified_resonance_lifecycle_def)

lemma add_resonance_objects [simp]:
  "objects_of (add_classified_resonance s request next_snapshot_id) =
    insert (classified_resonance_object s request) (objects_of s)"
  by (simp add: add_classified_resonance_def)

lemma add_resonance_snapshot_id [simp]:
  "snapshot_id_of (add_classified_resonance s request next_snapshot_id) =
    next_snapshot_id"
  by (simp add: add_classified_resonance_def)

lemma add_resonance_background [simp]:
  "historical_versions_of
    (add_classified_resonance s request next_snapshot_id) =
    historical_versions_of s"
  "perspectives_of (add_classified_resonance s request next_snapshot_id) =
    perspectives_of s"
  "contexts_of (add_classified_resonance s request next_snapshot_id) =
    contexts_of s"
  "models_of (add_classified_resonance s request next_snapshot_id) = models_of s"
  "authorities_of (add_classified_resonance s request next_snapshot_id) =
    authorities_of s"
  "contains_edges_of (add_classified_resonance s request next_snapshot_id) =
    contains_edges_of s"
  "projections_of (add_classified_resonance s request next_snapshot_id) =
    projections_of s"
  "couplings_of (add_classified_resonance s request next_snapshot_id) =
    couplings_of s"
  "transmissions_of (add_classified_resonance s request next_snapshot_id) =

```

```

        transmissions_of s"
    "boundaries_of (add_classified_resonance s request next_snapshot_id) =
        boundaries_of s"
    by (simp_all add: add_classified_resonance_def)

lemma add_resonance_version_ids [simp]:
    "version_ids (add_classified_resonance s request next_snapshot_id) =
        insert (requested_resonance_version_id_of request) (version_ids s)"
    by (auto simp: version_ids_def classified_resonance_object_def)

lemma classified_resonance_object_simps [simp]:
    "object_id_of (classified_resonance_object s request) =
        requested_resonance_object_id_of request"
    "version_id_of (classified_resonance_object s request) =
        requested_resonance_version_id_of request"
    "kind_of (classified_resonance_object s request) = K_RES"
    "lifecycle_of (classified_resonance_object s request) =
        classified_resonance_lifecycle s request"
    "previous_version_of (classified_resonance_object s request) = None"
    "references_of (classified_resonance_object s request) =
        requested_resonance_references request"
    "evidence_of (classified_resonance_object s request) =
        requested_resonance_rationale_refs_of request"
    "perspective_of (classified_resonance_object s request) =
        Some (requested_resonance_perspective_of request)"
    "context_of (classified_resonance_object s request) =
        Some (requested_resonance_context_of request)"
    "model_of (classified_resonance_object s request) =
        Some (resonance_profile_model_of (requested_resonance_profile_of request))"
    "valid_from_of (classified_resonance_object s request) =
        requested_resonance_valid_from_of request"
    "valid_to_of (classified_resonance_object s request) = None"
    "source_ref_of (classified_resonance_object s request) =
        Some (requested_source_effect_of request)"
    "target_ref_of (classified_resonance_object s request) =
        Some (requested_target_effect_of request)"
    "effect_origin_of (classified_resonance_object s request) = None"
    "basis_refs_of (classified_resonance_object s request) =
        {requested_source_effect_of request, requested_target_effect_of request}"
    "authority_of (classified_resonance_object s request) = None"
    "execution_ref_of (classified_resonance_object s request) = None"
    "observation_refs_of (classified_resonance_object s request) = {}"
    "object_status_of (classified_resonance_object s request) = Some Applicable"
    by (simp_all add: classified_resonance_object_def)

lemma classified_resonance_payload_simps [simp]:
    "classified_source_effect_of (classified_resonance_payload request) =
        requested_source_effect_of request"
    "classified_target_effect_of (classified_resonance_payload request) =
        requested_target_effect_of request"
    "classified_source_vector_of (classified_resonance_payload request) =
        requested_source_vector_of request"
    "classified_target_vector_of (classified_resonance_payload request) =
        requested_target_vector_of request"
    "classified_resonance_profile_of (classified_resonance_payload request) =
        requested_resonance_profile_of request"
    "classified_resonance_perspective_of (classified_resonance_payload request) =
        requested_resonance_perspective_of request"
    "classified_resonance_context_of (classified_resonance_payload request) =
        requested_resonance_context_of request"
    "classified_resonance_form_of (classified_resonance_payload request) =
        requested_resonance_form request"
    "classified_resonance_score_of (classified_resonance_payload request) =
        requested_directional_score request"
    "classified_resonance_strength_of (classified_resonance_payload request) =
        requested_resonance_strength request"
    "classified_resonance_horizon_of (classified_resonance_payload request) =
        requested_resonance_horizon_of request"
    "classified_resonance_uncertainty_of (classified_resonance_payload request) =
        requested_resonance_uncertainty_of request"
    "classified_resonance_rationale_refs_of
        (classified_resonance_payload request) =
        requested_resonance_rationale_refs_of request"
    "classified_resonance_rationale_of (classified_resonance_payload request) =
        requested_resonance_rationale_of request"
    by (simp_all add: classified_resonance_payload_def)

lemma active_classifiable_effect_version:
    assumes "active_classifiable_effect s effect_catalog version vector context"
    shows "version <in> version_ids s"

```

```

using assms
by (auto simp: active_classifiable_effect_def version_ids_def)

lemma active_rationale_versions:
  assumes "active_resonance_rationale s source target rationale_refs"
  shows "rationale_refs \<subteq> version_ids s"
  using assms
  by (auto simp: active_resonance_rationale_def version_ids_def)

lemma classify_resonance_fresh_object:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
     next_snapshot_id"
  shows "classified_resonance_object s request \<notin> objects_of s"
proof
  assume member:
    "classified_resonance_object s request \<in> objects_of s"
  from valid have fresh_id:
    "requested_resonance_object_id_of request \<notin>
     object_id_of ` objects_of s"
  unfolding classify_resonance_input_valid_def by blast
  from member have
    "object_id_of (classified_resonance_object s request) \<in>
     object_id_of ` objects_of s"
  by blast
  then show False using fresh_id by simp
qed

theorem classify_resonance_exact_delta:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
     next_snapshot_id s' resonance_catalog'"
  shows
    "objects_of s' =
     insert (classified_resonance_object s request) (objects_of s) \<and>
     classified_resonance_object s request \<notin> objects_of s \<and>
     snapshot_id_of s' = next_snapshot_id \<and>
     resonance_catalog' (requested_resonance_version_id_of request) =
       Some (classified_resonance_payload request)"
  using concrete classify_resonance_fresh_object
  by (auto simp: classify_resonance_step_def extend_resonance_catalog_def)

theorem classify_resonance_direction_explicit:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
     next_snapshot_id s' resonance_catalog'"
  shows
    "source_ref_of (classified_resonance_object s request) =
     Some (requested_source_effect_of request) \<and>
     target_ref_of (classified_resonance_object s request) =
     Some (requested_target_effect_of request) \<and>
     requested_source_effect_of request \<noteq> requested_target_effect_of request \<and>
     classified_resonance_form_of (classified_resonance_payload request) =
       classify_directional_score
         (requested_resonance_profile_of request)
         (directional_resonance_score
           (requested_resonance_profile_of request)
           (requested_source_vector_of request)
           (requested_target_vector_of request))"
  using concrete
  by (auto simp: classify_resonance_step_def classify_resonance_input_valid_def
    requested_resonance_form_def requested_directional_score_def)

subsection \<open>Source integrity and payload derivation\<close>

lemma classified_resonance_source_integrity:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
     next_snapshot_id"
  shows
    "references_complete s (classified_resonance_object s request) \<and>
     evidence_well_formed s (classified_resonance_object s request) \<and>
     time_well_formed (classified_resonance_object s request) \<and>
     perspective_well_formed s (classified_resonance_object s request) \<and>
     context_well_formed s (classified_resonance_object s request)"
proof -
  from valid have source_active:
    "active_classifiable_effect s effect_catalog
     (requested_source_effect_of request)
     (requested_source_vector_of request)"

```

```

        (requested_resonance_context_of request)"
    and target_active:
        "active_classifiable_effect s effect_catalog
        (requested_target_effect_of request)
        (requested_target_vector_of request)
        (requested_resonance_context_of request)"
    and rationale:
        "active_resonance_rationale s
        (requested_source_effect_of request)
        (requested_target_effect_of request)
        (requested_resonance_rationale_refs_of request)"
    and perspective:
        "requested_resonance_perspective_of request \<in> perspectives_of s"
    and context_ok:
        "requested_resonance_context_of request \<in> contexts_of s"
    unfolding classify_resonance_input_valid_def by blast+
have source_version:
    "requested_source_effect_of request \<in> version_ids s"
    using active_classifiable_effect_version[OF source_active] .
have target_version:
    "requested_target_effect_of request \<in> version_ids s"
    using active_classifiable_effect_version[OF target_active] .
have rationale_versions:
    "requested_resonance_rationale_refs_of request \<subseq> version_ids s"
    using active_rationale_versions[OF rationale] .
have refs:
    "references_complete s (classified_resonance_object s request)"
    using source_version target_version rationale_versions
    by (auto simp: references_complete_def requested_resonance_references_def)
have evidence:
    "evidence_well_formed s (classified_resonance_object s request)"
    using rationale rationale_versions classified_lifecycle_cases[of s request]
    by (auto simp: evidence_well_formed_def evidence_references_complete_def
        active_resonance_rationale_def)
have time:
    "time_well_formed (classified_resonance_object s request)"
    by (simp add: time_well_formed_def)
have perspective_wf:
    "perspective_well_formed s (classified_resonance_object s request)"
    using perspective
    by (simp add: perspective_well_formed_def perspective_bound_kind_def)
have context_wf:
    "context_well_formed s (classified_resonance_object s request)"
    using context_ok
    by (simp add: context_well_formed_def context_bound_kind_def)
show ?thesis using refs evidence time perspective_wf context_wf by blast
qed

lemma classified_resonance_payload_is_valid:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
    next_snapshot_id"
  shows
    "resonance_payload_valid
    (classified_resonance_object s request)
    (classified_resonance_payload request)"
proof -
  from valid have distinct:
    "requested_source_effect_of request \<noteq> requested_target_effect_of request"
  and source_vector:
    "effect_vector_valid (requested_source_vector_of request)"
  and target_vector:
    "effect_vector_valid (requested_target_vector_of request)"
  and profile:
    "resonance_profile_well_formed (requested_resonance_profile_of request)"
  and horizon: "requested_resonance_horizon_of request > 0"
  and uncertainty:
    "0 \<le> requested_resonance_uncertainty_of request \<and>
    requested_resonance_uncertainty_of request \<le> 1"
  and rationale:
    "active_resonance_rationale s
    (requested_source_effect_of request)
    (requested_target_effect_of request)
    (requested_resonance_rationale_refs_of request)"
  and rationale_text: "requested_resonance_rationale_of request \<noteq> []"
  unfolding classify_resonance_input_valid_def by blast+
have strength:
    "0 \<le> requested_resonance_strength request \<and>
    requested_resonance_strength request \<le> 1"
  unfolding requested_resonance_strength_def

```

```

    using uncertainty_adjusted_resonance_strength_bounded by blast
show ?thesis
    using distinct source_vector target_vector profile horizon uncertainty
    rationale rationale_text strength classified_lifecycle_cases[of s request]
    by (auto simp: resonance_payload_valid_def requested_resonance_references_def
        requested_resonance_form_def requested_directional_score_def
        requested_resonance_strength_def active_resonance_rationale_def)
qed

subsection \<open>Monotonicity of inherited state conditions\<close>

lemma add_resonance_present_ref_mono:
  assumes "present_ref s ref"
  shows "present_ref (add_classified_resonance s request next_snapshot_id) ref"
  using assms
  by (cases ref) (auto simp: present_ref_def)

lemma add_resonance_references_mono:
  assumes "references_complete s obj"
  shows "references_complete
    (add_classified_resonance s request next_snapshot_id) obj"
  using assms
  by (auto simp: references_complete_def)

lemma add_resonance_evidence_mono:
  assumes "evidence_well_formed s obj"
  shows "evidence_well_formed
    (add_classified_resonance s request next_snapshot_id) obj"
  using assms
  by (auto simp: evidence_well_formed_def evidence_references_complete_def)

lemma add_resonance_relation_mono:
  assumes "relation_well_formed s obj"
  shows "relation_well_formed
    (add_classified_resonance s request next_snapshot_id) obj"
  using assms add_resonance_present_ref_mono
  unfolding relation_well_formed_def by blast

lemma add_resonance_effect_mono:
  assumes "effect_well_formed s obj"
  shows "effect_well_formed
    (add_classified_resonance s request next_snapshot_id) obj"
  using assms add_resonance_present_ref_mono
  unfolding effect_well_formed_def by blast

lemma add_resonance_perspective_iff [simp]:
  "perspective_well_formed
    (add_classified_resonance s request next_snapshot_id) obj
    \<longleftarrowrightarrow> perspective_well_formed s obj"
  unfolding perspective_well_formed_def
  by (simp only: add_resonance_background(2))

lemma add_resonance_context_iff [simp]:
  "context_well_formed
    (add_classified_resonance s request next_snapshot_id) obj
    \<longleftarrowrightarrow> context_well_formed s obj"
  unfolding context_well_formed_def
  by (simp only: add_resonance_background(3))

lemma add_resonance_relevance_iff [simp]:
  "relevance_well_formed
    (add_classified_resonance s request next_snapshot_id) obj
    \<longleftarrowrightarrow> relevance_well_formed s obj"
  unfolding relevance_well_formed_def
  by (simp only: add_resonance_perspective_iff add_resonance_context_iff
    add_resonance_background(4))

lemma add_resonance_orientation_iff [simp]:
  "orientation_well_formed
    (add_classified_resonance s request next_snapshot_id) obj
    \<longleftarrowrightarrow> orientation_well_formed s obj"
  by (simp add: orientation_well_formed_def)

lemma add_resonance_action_iff [simp]:
  "action_well_formed
    (add_classified_resonance s request next_snapshot_id) obj
    \<longleftarrowrightarrow> action_well_formed s obj"
  unfolding action_well_formed_def
  by (simp only: add_resonance_background(5))
    
```

```

lemma add_resonance_feedback_mono:
  assumes "feedback_well_formed s obj"
  shows "feedback_well_formed
    (add_classified_resonance s request next_snapshot_id) obj"
  using assms add_resonance_present_ref_mono
  unfolding feedback_well_formed_def by blast

lemma add_resonance_preserves_active_history_disjoint:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
      next_snapshot_id"
  shows "active_history_disjoint
    (add_classified_resonance s request next_snapshot_id)"
  using valid
  by (auto simp: classify_resonance_input_valid_def active_history_disjoint_def)

subsection \<open>Fresh identity and target-state preservation\<close>

lemma add_resonance_preserves_unique_versions:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
      next_snapshot_id"
  shows "unique_versions
    (add_classified_resonance s request next_snapshot_id)"
proof -
  let ?new = "classified_resonance_object s request"
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
    and fresh: "version_id_of ?new \<notin> version_ids s"
    unfolding classify_resonance_input_valid_def state_well_formed_def
    unique_versions_def
    by auto
  show ?thesis
    unfolding unique_versions_def
  proof (rule inj_onI)
    fix left right
    assume left_mem:
      "left \<in> objects_of (add_classified_resonance s request next_snapshot_id)"
      and right_mem:
      "right \<in> objects_of (add_classified_resonance s request next_snapshot_id)"
      and same_version: "version_id_of left = version_id_of right"
    show "left = right"
    proof (cases "left = ?new")
      case True
      show ?thesis
      proof (cases "right = ?new")
        case True
        then show ?thesis using \<open>left = ?new\<close> by simp
      next
        case False
        then have right_old: "right \<in> objects_of s" using right_mem by simp
        have "version_id_of ?new \<in> version_ids s"
          unfolding version_ids_def
          using same_version \<open>left = ?new\<close> right_old by (metis imageI)
        then show ?thesis using fresh by blast
      qed
    next
      case False
      then have left_old: "left \<in> objects_of s" using left_mem by simp
      show ?thesis
      proof (cases "right = ?new")
        case True
        have "version_id_of ?new \<in> version_ids s"
          unfolding version_ids_def
          using same_version left_old True by (metis imageI)
        then show ?thesis using fresh by blast
      next
        case False
        then have right_old: "right \<in> objects_of s" using right_mem by simp
        show ?thesis
          using inj_onD[OF source_unique same_version left_old right_old] .
      qed
    qed
  qed
qed
qed
qed

lemma classify_resonance_new_object_conditions:
  assumes valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
      next_snapshot_id"
  shows

```

```

"kind_of (classified_resonance_object s request) \<in> supported_kinds_of G \<and>
references_complete
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
evidence_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
time_well_formed (classified_resonance_object s request) \<and>
relation_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
effect_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
perspective_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
context_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
relevance_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
orientation_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
action_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request) \<and>
feedback_well_formed
  (add_classified_resonance s request next_snapshot_id)
  (classified_resonance_object s request)"
proof -
  let ?new = "classified_resonance_object s request"
  from valid have supported: "K_RES \<in> supported_kinds_of G"
  unfolding classify_resonance_input_valid_def by blast
  from classified_resonance_source_integrity[OF valid] have refs:
    "references_complete s ?new"
    and evidence: "evidence_well_formed s ?new"
    and time: "time_well_formed ?new"
    and perspective: "perspective_well_formed s ?new"
    and context_ok: "context_well_formed s ?new"
  by blast+
  have refs_target:
    "references_complete (add_classified_resonance s request next_snapshot_id) ?new"
  using add_resonance_references_mono[OF refs] .
  have evidence_target:
    "evidence_well_formed (add_classified_resonance s request next_snapshot_id) ?new"
  using add_resonance_evidence_mono[OF evidence] .
  have typed_conditions:
    "relation_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new \<and>
    effect_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new \<and>
    relevance_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new \<and>
    orientation_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new \<and>
    action_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new \<and>
    feedback_well_formed
      (add_classified_resonance s request next_snapshot_id) ?new"
  by (simp add: relation_well_formed_def effect_well_formed_def
    relevance_well_formed_def orientation_well_formed_def
    action_well_formed_def feedback_well_formed_def)
  show ?thesis
  using supported_refs_target evidence_target time perspective context_ok
    typed_conditions
  by simp
qed

subsection \<open>Operation-specific preservation theorems\<close>

theorem classify_resonance_preserves_state_well_formed:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
      next_snapshot_id s' resonance_catalog"
  shows "state_well_formed G s'"
proof -
  from concrete have valid:

```



```

"classify_resonance_input_valid G s effect_catalog resonance_catalog request
  next_snapshot_id"
and target: "s' = add_classified_resonance s request next_snapshot_id"
by (auto simp: classify_resonance_step_def)
from valid have source_wf: "state_well_formed G s"
by (simp add: classify_resonance_input_valid_def)
have unique:
  "unique_versions (add_classified_resonance s request next_snapshot_id)"
using add_resonance_preserves_unique_versions[OF valid] .
have all_objects:
  "\<forall>obj\<in>objects_of (add_classified_resonance s request next_snapshot_id).
  kind_of obj \<in> supported_kinds_of G \<and>
  references_complete
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  evidence_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  time_well_formed obj \<and>
  relation_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  effect_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  perspective_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  context_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  relevance_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  orientation_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  action_well_formed
  (add_classified_resonance s request next_snapshot_id) obj \<and>
  feedback_well_formed
  (add_classified_resonance s request next_snapshot_id) obj"
proof (intro ballI)
  fix obj
  assume obj_mem:
    "obj \<in> objects_of (add_classified_resonance s request next_snapshot_id)"
  consider (new) "obj = classified_resonance_object s request"
  | (old) "obj \<in> objects_of s"
  using obj_mem by auto
  then show
    "kind_of obj \<in> supported_kinds_of G \<and>
    references_complete
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    evidence_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    time_well_formed obj \<and>
    relation_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    effect_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    perspective_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    context_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    relevance_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    orientation_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    action_well_formed
    (add_classified_resonance s request next_snapshot_id) obj \<and>
    feedback_well_formed
    (add_classified_resonance s request next_snapshot_id) obj"
  proof cases
    case new
    then show ?thesis
      using classify_resonance_new_object_conditions[OF valid] by simp
  next
    case old
    from source_wf old have source_conditions:
      "kind_of obj \<in> supported_kinds_of G \<and>
      references_complete s obj \<and> evidence_well_formed s obj \<and>
      time_well_formed obj \<and> relation_well_formed s obj \<and>
      effect_well_formed s obj \<and> perspective_well_formed s obj \<and>
      context_well_formed s obj \<and> relevance_well_formed s obj \<and>
      orientation_well_formed s obj \<and> action_well_formed s obj \<and>
      feedback_well_formed s obj"
    unfolding state_well_formed_def by blast
    from source_conditions have supported:
      "kind_of obj \<in> supported_kinds_of G"

```

```

    and refs: "references_complete s obj"
    and evidence: "evidence_well_formed s obj"
    and time: "time_well_formed obj"
    and relation: "relation_well_formed s obj"
    and effect: "effect_well_formed s obj"
    and perspective: "perspective_well_formed s obj"
    and context_ok: "context_well_formed s obj"
    and relevance: "relevance_well_formed s obj"
    and orientation: "orientation_well_formed s obj"
    and action: "action_well_formed s obj"
    and feedback: "feedback_well_formed s obj"
    by blast+
have refs_target:
  "references_complete
  (add_classified_resonance s request next_snapshot_id) obj"
  using add_resonance_references_mono[OF refs] .
have evidence_target:
  "evidence_well_formed
  (add_classified_resonance s request next_snapshot_id) obj"
  using add_resonance_evidence_mono[OF evidence] .
have relation_target:
  "relation_well_formed
  (add_classified_resonance s request next_snapshot_id) obj"
  using add_resonance_relation_mono[OF relation] .
have effect_target:
  "effect_well_formed
  (add_classified_resonance s request next_snapshot_id) obj"
  using add_resonance_effect_mono[OF effect] .
have feedback_target:
  "feedback_well_formed
  (add_classified_resonance s request next_snapshot_id) obj"
  using add_resonance_feedback_mono[OF feedback] .
show ?thesis
  using supported refs_target evidence_target time relation_target
    effect_target perspective context_ok relevance orientation action
    feedback_target
  by simp
qed
qed
from source_wf have source_secondary:
  "(\\<forall>p\\<in>projections_of s. projection_well_formed s p) \\<and>
  (\\<forall>c\\<in>couplings_of s. coupling_well_formed s c) \\<and>
  (\\<forall>t\\<in>transmissions_of s. transmission_well_formed s t) \\<and>
  (\\<forall>b\\<in>boundaries_of s. boundary_well_formed b)"
  unfolding state_well_formed_def by blast
have target_secondary:
  "(\\<forall>p\\<in>projections_of
  (add_classified_resonance s request next_snapshot_id).
  projection_well_formed
  (add_classified_resonance s request next_snapshot_id) p) \\<and>
  (\\<forall>c\\<in>couplings_of
  (add_classified_resonance s request next_snapshot_id).
  coupling_well_formed
  (add_classified_resonance s request next_snapshot_id) c) \\<and>
  (\\<forall>t\\<in>transmissions_of
  (add_classified_resonance s request next_snapshot_id).
  transmission_well_formed
  (add_classified_resonance s request next_snapshot_id) t) \\<and>
  (\\<forall>b\\<in>boundaries_of
  (add_classified_resonance s request next_snapshot_id).
  boundary_well_formed b)"
  using source_secondary
  by (simp add: projection_well_formed_def coupling_well_formed_def
    transmission_well_formed_def)
show ?thesis
  unfolding target state_well_formed_def
  using unique_all_objects target_secondary by blast
qed

theorem classify_resonance_preserves_identity:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
    next_snapshot_id s' resonance_catalog"
  shows "unique_versions s' \\<and> immutable_preserved s s'"
proof -
  let ?new = "classified_resonance_object s request"
  from concrete have valid:
    "classify_resonance_input_valid G s effect_catalog resonance_catalog request
    next_snapshot_id"
  and target: "s' = add_classified_resonance s request next_snapshot_id"

```

```

    by (auto simp: classify_resonance_step_def)
  from valid have source_unique: "inj_on version_id_of (objects_of s)"
    and fresh: "version_id_of ?new \<notin> version_ids s"
    unfolding classify_resonance_input_valid_def state_well_formed_def
    unique_versions_def
    by auto
  have immutable: "immutable_preserved s s'"
    unfolding immutable_preserved_def target
  proof (intro ballI allI impI)
    fix old_obj target_obj
    assume old_mem: "old_obj \<in> objects_of s"
    and target_mem:
      "target_obj \<in>
        objects_of (add_classified_resonance s request next_snapshot_id)"
    and same_version:
      "version_id_of old_obj = version_id_of target_obj"
    show "old_obj = target_obj"
    proof (cases "target_obj = ?new")
      case True
      have "version_id_of ?new \<in> version_ids s"
        unfolding version_ids_def
        using old_mem same_version True by (metis imageI)
      then show ?thesis using fresh by blast
    next
      case False
      then have target_old: "target_obj \<in> objects_of s"
        using target_mem by simp
      show ?thesis
        using inj_onD[OF source_unique same_version old_mem target_old] .
    qed
  qed
  have target_unique: "unique_versions s'"
    unfolding target
    using add_resonance_preserves_unique_versions[OF valid] .
  show ?thesis using target_unique immutable by blast
qed

theorem classify_resonance_preserves_effect_catalog:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
      next_snapshot_id s' resonance_catalog'"
  shows "effect_catalog_well_formed s' effect_catalog"
  proof -
    from concrete have valid:
      "classify_resonance_input_valid G s effect_catalog resonance_catalog request
        next_snapshot_id"
    and target: "s' = add_classified_resonance s request next_snapshot_id"
    by (auto simp: classify_resonance_step_def)
  from valid have source_catalog:
    "effect_catalog_well_formed s effect_catalog"
    by (simp add: classify_resonance_input_valid_def)
  show ?thesis
    unfolding effect_catalog_well_formed_def target
    using source_catalog
    unfolding effect_catalog_well_formed_def
    by auto
qed

theorem classify_resonance_preserves_catalog:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
      next_snapshot_id s' resonance_catalog'"
  shows
    "resonance_catalog_well_formed s' resonance_catalog' \<and>
      resonance_catalog' (requested_resonance_version_id_of request) =
        Some (classified_resonance_payload request)"
  proof -
    from concrete have valid:
      "classify_resonance_input_valid G s effect_catalog resonance_catalog request
        next_snapshot_id"
    and target: "s' = add_classified_resonance s request next_snapshot_id"
    and catalog_target:
      "resonance_catalog' = extend_resonance_catalog resonance_catalog request"
    by (auto simp: classify_resonance_step_def)
  from valid have source_catalog:
    "resonance_catalog_well_formed s resonance_catalog"
    and fresh_key:
      "resonance_catalog (requested_resonance_version_id_of request) = None"
    by (auto simp: classify_resonance_input_valid_def)
  have payload_valid:

```

```

"resonance_payload_valid
(classified_resonance_object s request)
(classified_resonance_payload request)"
using classified_resonance_payload_is_valid[OF valid] .
from classified_resonance_source_integrity[OF valid] have source_refs:
"references_complete s (classified_resonance_object s request)"
and source_evidence:
"evidence_well_formed s (classified_resonance_object s request)"
by blast+
have new_refs:
"references_complete s' (classified_resonance_object s request)"
unfolding target
using add_resonance_references_mono[OF source_refs] .
have new_evidence:
"evidence_well_formed s' (classified_resonance_object s request)"
unfolding target
using add_resonance_evidence_mono[OF source_evidence] .
have target_catalog: "resonance_catalog_well_formed s' resonance_catalog'"
unfolding resonance_catalog_well_formed_def catalog_target
extend_resonance_catalog_def
proof (intro allI impI)
fix version payload
assume updated:
"(resonance_catalog
(requested_resonance_version_id_of request \<mapsto>
classified_resonance_payload request)) version = Some payload"
show "\<exists>resonance\<in>objects_of s'.
version_id_of resonance = version \<and>
resonance_payload_valid resonance payload \<and>
references_complete s' resonance \<and>
evidence_well_formed s' resonance"
proof (cases "version = requested_resonance_version_id_of request")
case True
with updated have payload_eq:
"payload = classified_resonance_payload request"
by simp
have new_mem:
"classified_resonance_object s request \<in> objects_of s'"
unfolding target by simp
show ?thesis
proof (rule bexI[where x = "classified_resonance_object s request"])
show
"version_id_of (classified_resonance_object s request) = version \<and>
resonance_payload_valid (classified_resonance_object s request)
payload \<and>
references_complete s' (classified_resonance_object s request) \<and>
evidence_well_formed s' (classified_resonance_object s request)"
using True payload_eq payload_valid new_refs new_evidence by simp
show "classified_resonance_object s request \<in> objects_of s'"
using new_mem .
qed
next
case False
with updated have old_entry: "resonance_catalog version = Some payload"
by simp
from source_catalog old_entry obtain resonance where
old_mem: "resonance \<in> objects_of s"
and version: "version_id_of resonance = version"
and payload: "resonance_payload_valid resonance payload"
and refs: "references_complete s resonance"
and evidence: "evidence_well_formed s resonance"
unfolding resonance_catalog_well_formed_def by blast
have refs_target: "references_complete s' resonance"
unfolding target using add_resonance_references_mono[OF refs] .
have evidence_target: "evidence_well_formed s' resonance"
unfolding target using add_resonance_evidence_mono[OF evidence] .
have old_mem_target: "resonance \<in> objects_of s'"
unfolding target using old_mem by simp
show ?thesis
proof (rule bexI[where x = resonance])
show
"version_id_of resonance = version \<and>
resonance_payload_valid resonance payload \<and>
references_complete s' resonance \<and>
evidence_well_formed s' resonance"
using version payload refs_target evidence_target by blast
show "resonance \<in> objects_of s'" using old_mem_target .
qed
qed
qed
qed

```

```

show ?thesis
  using target_catalog catalog_target
  by (simp add: extend_resonance_catalog_def)
qed

theorem classify_resonance_preserves_references_time_context:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
     next_snapshot_id s' resonance_catalog"
  shows
    "\<forall>obj\<in>objects_of s'.
     references_complete s' obj \<and>
     evidence_well_formed s' obj \<and>
     time_well_formed obj \<and>
     perspective_well_formed s' obj \<and>
     context_well_formed s' obj"
  using classify_resonance_preserves_state_well_formed[OF concrete]
  unfolding state_well_formed_def by blast

theorem classify_resonance_refines_generic_step:
  assumes concrete:
    "classify_resonance_step G s effect_catalog resonance_catalog request
     next_snapshot_id s' resonance_catalog"
  shows "step G s ClassifyResonance s'"
proof -
  from concrete have source_wf: "state_well_formed G s"
  and allowed: "ClassifyResonance \<in> allowed_operations_of G"
  by (auto simp: classify_resonance_step_def
    classify_resonance_input_valid_def)
  have target_wf: "state_well_formed G s'"
  using classify_resonance_preserves_state_well_formed[OF concrete] .
  have immutable: "immutable_preserved s s'"
  using classify_resonance_preserves_identity[OF concrete] by blast
  show ?thesis
  unfolding step_def
  using source_wf target_wf allowed immutable by blast
qed

end

```

Beweisstatus dieser D.10-Fassung: kernelgeprüft. Der Isabelle2025-2-Fresh-Profile-Build verarbeitete die vollständige elfteilige Session und endete mit Exit-Code 0. T6 enthält 1.289 Quellzeilen und keine Vorkommen von sorry, oops, admit, axiomatization oder quick_and_dirty. Der Quellhash lautet

3f3583b42f066eaf41208a22bcb53a2d49baed374aa1b41e00a434256741b1cd. Belegt sind die formulierten konstruktiven Rechen-, Delta-, Katalog-, Zustands-, Identitäts-, Referenz-, Evidenz-, Zeit-, Perspektiv-, Kontext- und Refinementsätze. Nicht belegt sind empirische Richtigkeit eines Resonanzprofils, Vollständigkeit der Formen oder Schwellen, allgemeine Konservativität oder Produktivengine-Äquivalenz.

D.11 Nicht-vakuoser Richtungszeuge für T6

Der endliche Zeuge verhindert, dass T6 nur als konditionales Schema mit unerfüllten Vorbedingungen bestehen bleibt. Der Ausgangszustand enthält zwei Erscheinungen, zwei gerichtete Relationsursachen und zwei verschiedene validierte Effekte samt auflösbarer Evidenz, Perspektive, Kontext, Modell und konkreten V10-Payloads. Der Kernel besitzt genau einen Nichtnull-Eintrag $K(\text{Connection}, \text{Stability})=1$. Der Quellvektor trägt $\text{Connection}=3$, der Zielvektor $\text{Stability}=3$.

Mechanisch bewiesen werden $\text{kernel_mass}=1$, $\text{forward_weighted_sum}=9$, $\text{reverse_weighted_sum}=0$, $\text{forward_score}=1$, $\text{reverse_score}=0$, Amplifikation in Vorwärtsrichtung, Neutral in Rückwärtsrichtung, Stärke 9/10, Lifecycle Validated, die tatsächliche Erfüllung aller T6-Eingabebedingungen, der konkrete ClassifyResonance-Schritt, Zielzustands- und Katalogwohlgeformtheit, generisches step-Refinement sowie die exakte Einfügung von Version 7 in Snapshot 2.

```
theory UZA_0_9_1_Classify_Resonance_Witness
  imports UZA_0_9_1_Classify_Resonance UZA_0_9_1_Base_Model
begin

section \<open>A non-vacuous directional witness for T6\<close>

text \<open>
  This finite witness activates the substantive T6 antecedents with two
  distinct validated effects and a non-symmetric, versioned interaction
  kernel. The kernel maps Connection in the source to Stability in the
  target. Consequently A-to-B has score 1 and form Amplification, whereas
  B-to-A has score 0 and form Neutral. The witness therefore establishes
  semantic directionality rather than merely storing ordered references.
\<close>

definition t6_source_vector :: effect_vector where
  "t6_source_vector dimension = (if dimension = Connection then 3 else 0)"

definition t6_target_vector :: effect_vector where
  "t6_target_vector dimension = (if dimension = Stability then 3 else 0)"

definition t6_directional_kernel :: resonance_kernel where
  "t6_directional_kernel source_dimension target_dimension =
    (if source_dimension = Connection \& target_dimension = Stability
     then 1 else 0)"

definition t6_resonance_profile :: resonance_model_profile where
  "t6_resonance_profile =
    \<lparr>
      resonance_profile_model_of = 30,
      resonance_kernel_of = t6_directional_kernel,
      amplification_cutoff_of = 1 / 2,
      complement_cutoff_of = 3 / 20,
      productive_tension_cutoff_of = -3 / 20,
      inhibition_cutoff_of = -1 / 2,
      resonance_validation_uncertainty_max_of = 1 / 4
    \<rparr>"

definition t6_entity_a :: uza_object where
  "t6_entity_a =
    base_object\<lparr>
      object_id_of := 1,
      version_id_of := 1,
      kind_of := K_ENT,
      lifecycle_of := Draft,
      perspective_of := Some 10,
      context_of := Some 20,
      object_status_of := Some Applicable
    \<rparr>"

definition t6_entity_b :: uza_object where
  "t6_entity_b =
```

```

base_object\<lparr>
  object_id_of := 2,
  version_id_of := 2,
  kind_of := K_ENT,
  lifecycle_of := Draft,
  perspective_of := Some 10,
  context_of := Some 20,
  object_status_of := Some Applicable
\<rparr>"

definition t6_relation_a :: uza_object where
  "t6_relation_a =
    base_object\<lparr>
      object_id_of := 3,
      version_id_of := 3,
      kind_of := K_REL,
      lifecycle_of := Hypothesis,
      references_of := {1, 2},
      context_of := Some 20,
      source_ref_of := Some 1,
      target_ref_of := Some 2,
      object_status_of := Some Applicable
    \<rparr>"

definition t6_effect_a :: uza_object where
  "t6_effect_a =
    base_object\<lparr>
      object_id_of := 4,
      version_id_of := 4,
      kind_of := K_EFF,
      lifecycle_of := Validated,
      references_of := {1, 3},
      evidence_of := {1},
      perspective_of := Some 10,
      context_of := Some 20,
      model_of := Some 30,
      source_ref_of := Some 3,
      effect_origin_of := Some 3,
      basis_refs_of := {3},
      object_status_of := Some Applicable
    \<rparr>"

definition t6_relation_b :: uza_object where
  "t6_relation_b =
    base_object\<lparr>
      object_id_of := 5,
      version_id_of := 5,
      kind_of := K_REL,
      lifecycle_of := Hypothesis,
      references_of := {1, 2},
      context_of := Some 20,
      source_ref_of := Some 2,
      target_ref_of := Some 1,
      object_status_of := Some Applicable
    \<rparr>"

definition t6_effect_b :: uza_object where
  "t6_effect_b =
    base_object\<lparr>
      object_id_of := 6,
      version_id_of := 6,
      kind_of := K_EFF,
      lifecycle_of := Validated,
      references_of := {2, 5},
      evidence_of := {2},
      perspective_of := Some 10,
      context_of := Some 20,
      model_of := Some 30,
      source_ref_of := Some 5,
      effect_origin_of := Some 5,
      basis_refs_of := {5},
      object_status_of := Some Applicable
    \<rparr>"

definition t6_witness_objects :: "uza_object set" where
  "t6_witness_objects =
    {t6_entity_a, t6_entity_b, t6_relation_a,
     t6_effect_a, t6_relation_b, t6_effect_b}"

definition t6_witness_source :: snapshot where

```

```

"t6_witness_source =
  \<lparr>
    snapshot_id_of = 1,
    objects_of = t6_witness_objects,
    historical_versions_of = {},
    perspectives_of = {10},
    contexts_of = {20},
    models_of = {30},
    authorities_of = {},
    contains_edges_of = {},
    projections_of = {},
    couplings_of = {},
    transmissions_of = {},
    boundaries_of = {}
  \<rparr>"

definition t6_witness_grammar :: grammar where
"t6_witness_grammar =
  \<lparr>
    grammar_id_of = 6,
    supported_kinds_of = {K_ENT, K_REL, K_EFF, K_RES},
    allowed_operations_of = {ClassifyResonance},
    patch_budget_of = 0
  \<rparr>"

definition t6_effect_payload_a :: effect_hypothesis_payload where
"t6_effect_payload_a =
  \<lparr>
    hypothesis_cause_of = 3,
    hypothesis_target_of = None,
    hypothesis_vector_of = Some t6_source_vector,
    hypothesis_uncertainty_of = 1 / 10,
    hypothesis_profile_model_of = 30
  \<rparr>"

definition t6_effect_payload_b :: effect_hypothesis_payload where
"t6_effect_payload_b =
  \<lparr>
    hypothesis_cause_of = 5,
    hypothesis_target_of = None,
    hypothesis_vector_of = Some t6_target_vector,
    hypothesis_uncertainty_of = 1 / 10,
    hypothesis_profile_model_of = 30
  \<rparr>"

definition t6_effect_catalog :: effect_catalog where
"t6_effect_catalog =
  Map.empty(4 \<mapsto> t6_effect_payload_a, 6 \<mapsto> t6_effect_payload_b)"

definition t6_source_resonance_catalog :: resonance_catalog where
"t6_source_resonance_catalog = Map.empty"

definition t6_request :: resonance_request where
"t6_request =
  \<lparr>
    requested_resonance_object_id_of = 7,
    requested_resonance_version_id_of = 7,
    requested_source_effect_of = 4,
    requested_target_effect_of = 6,
    requested_source_vector_of = t6_source_vector,
    requested_target_vector_of = t6_target_vector,
    requested_resonance_profile_of = t6_resonance_profile,
    requested_resonance_perspective_of = 10,
    requested_resonance_context_of = 20,
    requested_resonance_horizon_of = 4,
    requested_resonance_uncertainty_of = 1 / 10,
    requested_resonance_rationale_refs_of = {1, 2},
    requested_resonance_rationale_of = 'directed Connection-to-Stability interaction'',
    requested_resonance_valid_from_of = 10
  \<rparr>"

definition t6_witness_target :: snapshot where
"t6_witness_target =
  add_classified_resonance t6_witness_source t6_request 2"

definition t6_target_resonance_catalog :: resonance_catalog where
"t6_target_resonance_catalog =
  extend_resonance_catalog t6_source_resonance_catalog t6_request"

lemmas t6_object_defs =

```



```

t6_entity_a_def t6_entity_b_def t6_relation_a_def t6_effect_a_def
t6_relation_b_def t6_effect_b_def

lemma t6_witness_source_simps [simp]:
  "snapshot_id_of t6_witness_source = 1"
  "objects_of t6_witness_source = t6_witness_objects"
  "historical_versions_of t6_witness_source = {}"
  "perspectives_of t6_witness_source = {10}"
  "contexts_of t6_witness_source = {20}"
  "models_of t6_witness_source = {30}"
  "authorities_of t6_witness_source = {}"
  "contains_edges_of t6_witness_source = {}"
  "projections_of t6_witness_source = {}"
  "couplings_of t6_witness_source = {}"
  "transmissions_of t6_witness_source = {}"
  "boundaries_of t6_witness_source = {}"
  by (simp_all add: t6_witness_source_def)

lemma t6_witness_version_ids [simp]:
  "version_ids t6_witness_source = {1, 2, 3, 4, 5, 6}"
  by (auto simp: version_ids_def t6_witness_objects_def t6_object_defs
    base_object_def)

lemma t6_source_vector_valid:
  "effect_vector_valid t6_source_vector"
  by (auto simp: effect_vector_valid_def t6_source_vector_def)

lemma t6_target_vector_valid:
  "effect_vector_valid t6_target_vector"
  by (auto simp: effect_vector_valid_def t6_target_vector_def)

lemma t6_effect_dimension_delta:
  fixes selected :: effect_dimension
  and coefficient :: real
  shows
    "(\<Sum>dimension\<in>UNIV.
      if dimension = selected then coefficient else 0) = coefficient"
proof -
  have finite_dimensions: "finite (UNIV :: effect_dimension set)"
  by simp
  note delta =
    sum.delta[OF finite_dimensions, of selected "\<lambda>_. coefficient"]
  show ?thesis using delta by simp
qed

lemma t6_directional_kernel_mass:
  "resonance_kernel_mass t6_directional_kernel = 1"
proof -
  have inner:
    "\<And>source_dimension.
      (\<Sum>target_dimension\<in>UNIV.
        abs (t6_directional_kernel source_dimension target_dimension)) =
      (if source_dimension = Connection then 1 else 0)"
  by (case_tac source_dimension)
  (simp_all add: t6_directional_kernel_def t6_effect_dimension_delta)
  show ?thesis
  unfolding resonance_kernel_mass_def
  by (simp add: inner t6_effect_dimension_delta)
qed

lemma t6_profile_well_formed:
  "resonance_profile_well_formed t6_resonance_profile"
  using t6_directional_kernel_mass
  by (auto simp: resonance_profile_well_formed_def t6_resonance_profile_def
    t6_directional_kernel_def)

lemma t6_witness_source_well_formed:
  "state_well_formed t6_witness_grammar t6_witness_source"
  unfolding state_well_formed_def
  by (auto simp: unique_versions_def inj_on_def t6_witness_grammar_def
    t6_witness_objects_def t6_object_defs base_object_def present_ref_def
    references_complete_def evidence_well_formed_def
    evidence_references_complete_def time_well_formed_def
    relation_well_formed_def effect_well_formed_def
    perspective_well_formed_def perspective_bound_kind_def
    context_well_formed_def context_bound_kind_def relevance_well_formed_def
    orientation_well_formed_def action_well_formed_def
    feedback_well_formed_def)

lemma t6_effect_catalog_well_formed:

```

```

"effect_catalog_well_formed t6_witness_source t6_effect_catalog"
unfolding effect_catalog_well_formed_def
using t6_source_vector_valid t6_target_vector_valid
by (auto simp: t6_effect_catalog_def t6_effect_payload_a_def
    t6_effect_payload_b_def t6_witness_objects_def t6_object_defs
    base_object_def effect_payload_valid_def)

lemma t6_forward_term:
  "t6_directional_kernel source_dimension target_dimension *
    of_int (t6_source_vector source_dimension *
      t6_target_vector target_dimension) =
    (if source_dimension = Connection <and> target_dimension = Stability
    then 9 else 0)"
  by (case_tac source_dimension; case_tac target_dimension)
    (simp_all add: t6_directional_kernel_def t6_source_vector_def
      t6_target_vector_def)

lemma t6_reverse_term:
  "t6_directional_kernel source_dimension target_dimension *
    of_int (t6_target_vector source_dimension *
      t6_source_vector target_dimension) = 0"
  by (case_tac source_dimension; case_tac target_dimension)
    (simp_all add: t6_directional_kernel_def t6_target_vector_def
      t6_source_vector_def)

lemma t6_forward_weighted_sum:
  "(<Sum>source_dimension<in>UNIV.
    <Sum>target_dimension<in>UNIV.
      t6_directional_kernel source_dimension target_dimension *
        of_int (t6_source_vector source_dimension *
          t6_target_vector target_dimension)) = 9"
  proof -
    have inner:
      "\<And>source_dimension.
        (<Sum>target_dimension<in>UNIV.
          t6_directional_kernel source_dimension target_dimension *
            of_int (t6_source_vector source_dimension *
              t6_target_vector target_dimension)) =
          (if source_dimension = Connection then 9 else 0)"
      by (simp only: t6_forward_term)
      (case_tac source_dimension;
        simp_all add: t6_effect_dimension_delta)
    show ?thesis by (simp only: inner t6_effect_dimension_delta)
  qed

lemma t6_reverse_weighted_sum:
  "(<Sum>source_dimension<in>UNIV.
    <Sum>target_dimension<in>UNIV.
      t6_directional_kernel source_dimension target_dimension *
        of_int (t6_target_vector source_dimension *
          t6_source_vector target_dimension)) = 0"
  by (simp only: t6_reverse_term) simp

lemma t6_forward_score:
  "directional_resonance_score t6_resonance_profile
    t6_source_vector t6_target_vector = 1"
  using t6_directional_kernel_mass t6_forward_weighted_sum
  by (simp add: directional_resonance_score_def
    directional_resonance_raw_score_def clamp11_def
    t6_resonance_profile_def
    t6_directional_kernel_def t6_source_vector_def t6_target_vector_def)

lemma t6_reverse_score:
  "directional_resonance_score t6_resonance_profile
    t6_target_vector t6_source_vector = 0"
  using t6_directional_kernel_mass t6_reverse_weighted_sum
  by (simp add: directional_resonance_score_def
    directional_resonance_raw_score_def clamp11_def
    t6_resonance_profile_def
    t6_directional_kernel_def t6_source_vector_def t6_target_vector_def)

lemma t6_witness_input_valid:
  "classify_resonance_input_valid t6_witness_grammar t6_witness_source
    t6_effect_catalog t6_source_resonance_catalog t6_request 2"
  unfolding classify_resonance_input_valid_def
  using t6_witness_source_well_formed t6_effect_catalog_well_formed
    t6_source_vector_valid t6_target_vector_valid t6_profile_well_formed
  by (auto simp: active_history_disjoint_def resonance_catalog_well_formed_def
    t6_witness_grammar_def t6_source_resonance_catalog_def t6_request_def
    t6_resonance_profile_def)

```

```

t6_witness_objects_def t6_object_defs base_object_def
distinct_active_effect_objects_def active_classifiable_effect_def
t6_effect_catalog_def t6_effect_payload_a_def t6_effect_payload_b_def
effect_payload_valid_def active_resonance_rationale_def)

theorem t6_witness_concrete_step:
  "classify_resonance_step t6_witness_grammar t6_witness_source
   t6_effect_catalog t6_source_resonance_catalog t6_request 2
   t6_witness_target t6_target_resonance_catalog"
  unfolding classify_resonance_step_def t6_witness_target_def
   t6_target_resonance_catalog_def
  using t6_witness_input_valid by simp

theorem t6_witness_directionality_is_nonvacuous:
  "requested_source_effect_of t6_request = 4 \<and>
   requested_target_effect_of t6_request = 6 \<and>
   directional_resonance_score t6_resonance_profile
    t6_source_vector t6_target_vector = 1 \<and>
   directional_resonance_score t6_resonance_profile
    t6_target_vector t6_source_vector = 0 \<and>
   requested_resonance_form t6_request = Amplification \<and>
   classify_directional_score t6_resonance_profile
    (directional_resonance_score t6_resonance_profile
     t6_target_vector t6_source_vector) = Neutral \<and>
   requested_resonance_strength t6_request = 9 / 10 \<and>
   lifecycle_of
    (classified_resonance_object t6_witness_source t6_request) = Validated"
  using t6_forward_score t6_reverse_score
  by (simp add: t6_request_def requested_resonance_form_def
   requested_directional_score_def classify_directional_score_def
   requested_resonance_strength_def
   uncertainty_adjusted_resonance_strength_def clamp01_def
   t6_resonance_profile_def classified_resonance_lifecycle_def
   effect_validated_at_def t6_witness_objects_def t6_object_defs
   base_object_def)

theorem t6_witness_target_certificates:
  "state_well_formed t6_witness_grammar t6_witness_target \<and>
   effect_catalog_well_formed t6_witness_target t6_effect_catalog \<and>
   resonance_catalog_well_formed t6_witness_target
    t6_target_resonance_catalog \<and>
   step t6_witness_grammar t6_witness_source ClassifyResonance
    t6_witness_target"
  using classify_resonance_preserves_state_well_formed
   [OF t6_witness_concrete_step]
   classify_resonance_preserves_effect_catalog
   [OF t6_witness_concrete_step]
   classify_resonance_preserves_catalog
   [OF t6_witness_concrete_step]
   classify_resonance_refines_generic_step
   [OF t6_witness_concrete_step]
  by blast

theorem t6_witness_exact_delta:
  "objects_of t6_witness_target =
   insert (classified_resonance_object t6_witness_source t6_request)
    (objects_of t6_witness_source) \<and>
   classified_resonance_object t6_witness_source t6_request
    \<notin> objects_of t6_witness_source \<and>
   snapshot_id_of t6_witness_target = 2 \<and>
   t6_target_resonance_catalog 7 =
    Some (classified_resonance_payload t6_request)"
  using classify_resonance_exact_delta [OF t6_witness_concrete_step]
  by (simp add: t6_request_def)

end

```

Beweisstatus dieser D.11-Fassung: kernelgeprüft und nicht-vakuos im ausgewiesenen Umfang.
 Der Zeuge enthält 438 Quellzeilen; sein SHA-256 lautet
 494da00eeb17205001ae85090d940a12cd07215ec283d76833aa480126a716ed. Er ist ein
 Existenz-, Rechen- und Ausführbarkeitsbeleg für eine endliche gerichtete Konfiguration, kein
 empirischer Universalitätsbeweis und keine Aussage, dass jeder reale Resonanzzusammenhang
 mit diesem Kernel modelliert werden sollte.

Anhang E – Offene mathematische Beweisaufgaben

Der Isabelle2025-2-Fresh-Profile-Build des aktuellen integrierten Theoriestands, die Korrektur aller dabei gefundenen Typ- und Beweisfehler, das Pinning der elf Theoriequellen, die referenzielle Schließung von A9, der minimale und der reichhaltige Modellzeuge, die konstruktiven Erhaltungsbeweise für T1 RegisterAppearance bis T6 ClassifyResonance sowie die nicht-vakuoosen T5- und T6-Ausführungszeugen sind abgeschlossen. Aufgabe 4 ist damit für T1–T6 geschlossen und für T7–T13 offen. Verbleibende Aufgaben, unter Beibehaltung ihrer Referenznummern, sind:

2. Eindeutigkeit von version_id und Eigenschaften der Revisionskette jenseits der Locale-Annahmen.
3. Zeitliche Wohlgeformtheit aller Gültigkeitsintervalle als Erhaltungssatz.
4. Erhaltung von state_well_formed durch jede Kernoperation – T1, T2, T3, T4, T5 und T6 kernelgeprüft abgeschlossen; T7–T13 offen.
5. Azyklizität der direkten Einbettung.
6. Irreflexivität von strict_contains aus konstruktiven Graphvorbedingungen.
7. Antisymmetrie von contains_eq aus konstruktiven Graphvorbedingungen.
8. Lifecycle-Erhaltung von Transmissionen.
9. Vollständigkeit der Transmission Lineage.
10. Fusionsdispositionsvollständigkeit.
11. Fissions-Lineage-Vollständigkeit.
12. Wohldefiniertheit von Aggregation, Embedding und Detachment.
13. Weiterführende Eigenschaften der describe-Aggregation.
14. Paarweise Exklusivität der Emergenzklassen.
15. Hinreichende Bedingungen für Vollständigkeit von E0–E3.
16. Modell-Expansion vom Kern zur Interaktionserweiterung.
17. Relative Konsistenz.
18. Optionaler Konservativitätsnachweis für definierte Kernformeln.
19. Abgrenzung und Korrektheit einer FOL-Kernübersetzung.
20. Verknüpfung von Engine-Conformance-Tests mit formalisierten Pre-/Postconditions.

Anhang F – Glossar

Begriff	Konsolidierte Bedeutung
Zusammenhang	versionierte Konfiguration aus Struktur, Prozess, Potential und Geschichte
Erscheinung	registrierte unterscheidbare Gestalt oder Beobachtungseinheit
Beziehung	typisierte, zeit- und kontextgebundene Verbindung
Wirkung	gerichtete Veränderung mit Herkunft, Ziel und Wirkungsprofil
Resonanz	perspektivisch-kontextuelle Wechselwirkung zwischen Wirkungen
Relevanz	begründete, modellgebundene Wichtigkeits- und Reichweitenbewertung
Orientierung	hergeleiteter Möglichkeits- und Handlungsraum
Gestaltung	ausgewählte und autorisierte Option mit Ausführungsbezug
Rückwirkung	beobachtete Folge einer Gestaltung und ihre Abweichung zur Simulation
Emergenz	strukturelle Neuheit oder Grenze der aktiven Grammatik gemäß E0–E3
Geltungsstatus	explizites Ergebnis APPLICABLE, OUT_OF_SCOPE, INSUFFICIENT_BASIS oder MODEL_FALSIFIED
Perspektive	versionierter Beobachtungs-, Wissens-, Wertungs- und Handlungsstandpunkt
Kontext	Bedingungsrahmen semantischer Gültigkeit
Potential	unter Regeln und Bedingungen erreichbarer Zustandsraum
GrammarPatch	kontrollierter Vorschlag zur Erweiterung oder Revision einer Grammatik
Patch-Abbruch	nachvollziehbare Beendigung als PATCH_REJECTED, CORE_CONFLICT oder UNRESOLVED_E3
PowerProfile	von V10 getrenntes relationales Profil für Ressourcen, Zugang, Entscheidung, Risiko und Korrekturfähigkeit
ScaleProfile	räumliche, zeitliche, populationsbezogene und analytische Skalierung
SUBZ	formale Teilzusammenhangsrelation
Projektion	ausgewählte Containeransicht eines Teilzusammenhangs
LossProfile	dokumentierter Struktur-, Semantik-, Perspektiv- und Zeitverlust
Kopplung	versionierte Interaktionsbeziehung zwischen Systemzusammenhängen
Transmission	kanalgebundene, transformierte Wirkungsübertragung
BoundaryEvent	Fusion, Fission, Aggregation, Embedding oder Detachment mit Lineage

Begriff	Konsolidierte Bedeutung
InteractionComplex	Gesamtstruktur gekoppelter und eingebetteter Zusammenhänge
Zusammenhangskompeten z	getrennte Fähigkeit von Architektur, Engine und Akteur, Zusammenhang angemessen zu erfassen und zu gestalten

Anhang G – Konsolidierungsprotokoll 0.9.1

Diese Fassung nimmt folgende verbindliche Bereinigungen vor:

- Sie führt die detailliertere Entwicklungsmatrix v0.1–v0.9.1 als maßgeblich.
- Sie unterscheidet demonstrativ vollständig durchgespieltes Implementierungsdesign von produktiv ausgeführtem Code.
- Sie behandelt die Dynamik als engine-integrierte Spezifikation mit weiterhin offener formaler Axiomatik.
- Sie definiert ein allgemeines EffectProfile-Interface und V10 als Referenzprofil, begründet dessen Achsengruppen und behauptet keine unbelegte mathematische Orthogonalität.
- Sie erhält V10-POLAR als Legacy-Profil mit eindeutigen Statuswerten Mapped, Derived, Unmapped und Conflicted; fehlende Dimensionen werden niemals still mit null belegt.
- Sie behandelt Macht nicht als elfte V10-Dimension, sondern als separates relationales PowerProfile mit dokumentiertem PowerShift.
- Sie erklärt die Beziehung zwischen frühem Acht-Ebenen-Zyklus, semantischem Kernzyklus und operativer S0–S9-Maschine.
- Sie trennt die architektonischen RA1–RA12 von den als Locale-Annahmen formalisierten A1–A22 und weist deren gemeinsame Erfüllbarkeit relativ zu HOL durch einen expliziten Modellzeugen nach.
- Sie führt vollständige Engine-, Datenmodell-, API-, Validierungs-, Dynamics-, Scale-&-Interaction- und Demonstratorabschnitte ein.
- Sie vereinheitlicht die operative Reihenfolge auf T11/FDB → T12/EMG und korrigiert Zyklusgrafik, Zustandssemantik, Pseudocode und Golden Run entsprechend.
- Sie ergänzt Minimality in Formel und Erläuterung der Architekturkompetenz und benennt den inneren Kern korrekt als viergliedrig.
- Sie führt Geltungs-, Falsifikations-, Patch-Annahme- und Patch-Abbruchregeln als normative Gates ein.
- Sie integriert die HOL Theory 0.9.1 mit zentraler Semantik und A1–A22 und dokumentiert den erfolgreich bestandenen Isabelle2025-2-Session-Build samt kernelgeprüftem Quellhash.
- Sie schließt A9 durch `evidence_references_complete` und `evidence_well_formed`, weist unbekannte Evidence-IDs zurück und erzwingt für Supported und Validated eine nichtleere Evidenzmenge.
- Sie ergänzt eine zweite Isabelle-Theorie mit nichtleerem Zustand, versioniertem Objekt, zulässigem Übergang, vollständiger Locale-Interpretation und dem Existenztheorem `explicit_uz_model_exists`; der Zeuge wird ausdrücklich als minimal und weitgehend vakuos ausgewiesen.
- Sie ergänzt eine dritte Isabelle-Theorie, die T1 RegisterAppearance konstruktiv definiert und Zustandswohlgeformtheit, Identität, Evidenzintegrität, Referenzen, Zeit-, Perspektiv- und Kontextbindung sowie das Refinement zu step mechanisch beweist.

- Sie ergänzt eine vierte Isabelle-Theorie mit zwölf versionierten Objekten, Projektion, Kopplung, validierter Transmission und erfolgreichem Fusion-Event; neun Nichtvakuositätszertifikate aktivieren A5–A13 und A17–A21, bevor Rich: uza_model und rich_uza_model_exists die vollständige Interpretation sichern.
- Sie ergänzt eine fünfte Isabelle-Theorie, die T2 DifferentiatePotential als endliche nichtleere Kandidatenmengentransition definiert, Beobachtung und Möglichkeitskandidat formal trennt und exaktes Delta, Identität, Evidenzintegrität, Bindungen, vollständige Zielwohlgeformtheit sowie das Refinement zu step beweist.
- Sie stellt dem Isabelle-Quellanhang eine explizite Leistungsübersicht voran, die **abgeschlossen**, **teilweise abgeschlossen** und **offen** samt Beleg und verbleibendem Ziel getrennt ausweist.
- Sie ergänzt drei tatsächlich ausgeführte Golden Runs einschließlich Negativfällen und gepinnter Replayergebnisse.
- Sie ersetzt pauschale Vollständigkeitsbehauptungen durch klar prüfbare Reifestände.

Sie ergänzt eine sechste konservative Isabelle-Theorie für T3 CreateRelation. Diese führt relation_type und Richtung in einem versionierten Relationenkatalog, erzwingt zwei verschiedene aktive ENT-Endpunkte und beweist exaktes Delta, Katalog-, Zustands-, Identitäts-, Referenz-, Zeit-, Kontext- und Evidenzerhaltung sowie Refinement zu step.

Sie dokumentiert den realen T3-Prüfpfad: zunächst Ablehnung der flachen Mehrsession-Struktur, dann zwei vom Kernel entdeckte lokale Beweislücken und schließlich den erfolgreichen integrierten Isabelle2025-2-Clean-Build mit sieben Theorien und Exit-Code 0.

Sie ergänzt eine siebte konservative Isabelle-Theorie für T4 HypothesizeEffect. Diese erzwingt eine aktive typisierte REL-Ursache, explizite option-Offenheit von Ziel und Wirkungsvektor, Perspektiv-, Kontext- und Profilmodellbindung, begrenzte Unsicherheit und frische K_EFF-Identität; sie beweist exaktes Delta, Katalog-, Zustands-, Identitäts-, Referenz-, Zeit-, Perspektiv-, Kontext- und Evidenzerhaltung sowie Refinement zu step. Der integrierte Isabelle2025-2-Clean-Build mit sieben Theorien endete mit Exit-Code 0.

Sie ergänzt eine achte Isabelle-Theorie für T5 IntegrateEvidence. Diese realisiert Copy-on-Write-Effektversionierung, vollständige Objekt- und Payload-Archive, einen Evidenz-Ledger, kontrollierte Unsicherheitsbewegung, abgeleitete Lifecycle-Übergänge, Ausschluss von Selbststützung sowie Erhaltung und Refinement ohne vorausgesetzte Zielwohlgeformtheit.

Sie ergänzt eine neunte Isabelle-Theorie als nicht-vakuozen T5-Ausführungszeugen. Der konkrete Hypothesis → Supported-Lauf aktiviert die Evidenz-, Archiv-, Katalog-, Lifecycle- und step-Bedingungen und wurde gemeinsam mit der gesamten Session kernelgeprüft.

Sie ergänzt eine zehnte Isabelle-Theorie für T6 ClassifyResonance. Diese realisiert erstmals die gerichtete Resonanzforderung als versionierte Interaktionsmatrix, schließt die Endlichkeit der V10-Domäne formal, konstruiert RES-Objekt und Payload und beweist Score-, Form-, Stärke-, Delta-, Katalog-, Zustands- und Refinamenteigenschaften ohne vorausgesetzte Zielwohlgeformtheit.

Sie ergänzt eine elfte Isabelle-Theorie als nicht-vakuozen T6-Richtungszeugen. Der konkrete Lauf beweist $A \rightarrow B = 1/\text{Amplification}$ und $B \rightarrow A = 0/\text{Neutral}$ sowie Stärke 0,9, Lifecycle Validated,

exaktes Delta und alle Zielzertifikate. Die frühere symmetrische V10-Dot-Product-Funktion bleibt ausdrücklich nur Baseline; normative T6-Operationen verwenden das gerichtete Profil.

Damit ist die bisherige konzeptionelle und spezifikatorische Arbeit in einer konsistenten 0.9.1-Referenzdatei zusammengeführt, in einer elfteiligen Isabelle2025-2-Fresh-Profile-Session mechanisch geprüft, durch einen minimalen und einen reichhaltigen Modellzeugen ergänzt, in A9 referenziell geschlossen und für T1 bis T6 mit nichtzirkulären Operations-Erhaltungsbeweisen verbunden. T5 und T6 besitzen darüber hinaus nicht-vakuose Ausführungszeugen; T6 beweist eine konkrete gerichtete Asymmetrie. Der nächste objektive Operationsabschluss ist T7 EvaluateRelevance; parallel bleiben allgemeine Modell-Expansion und Konservativität eigenständige Beweisziele.

Anhang H – Validierungsprotokoll 0.9.1

H.1 Formale Quell-, Modell- und Operationsprüfung

Prüfung	Ergebnis
Struktur-Preflight der Theorie	bestanden
A1–A22 vollständig und eindeutig gefunden	bestanden
zentrale Definitionen (step, Evidence Integrity, Describe-Tripel, emergence_of, Relevanz, Resonanz, Macht, Patch)	bestanden
sechs Haupttheorie-Sentinels gefunden	bestanden
A9-Integritätslemmata	unresolved_evidence_reference_rejected und A9_upgraded_evidence_resolves kernelgeprüft
Isabelle-Version	Isabelle2025-2
integrierter Fresh-Profile-Build (leeres Benutzerprofil, -j 1)	bestanden; elf Theorien; Exit-Code 0
sechs Haupttheorie-Sentinelbeweise	vom Isabelle-Kernel akzeptiert
kernelgeprüfter Theorie-Hash	21dcfb175d6e754adf20042b941cf4a23329831d0165ebaf2023b0048213ed24
Struktur-Preflight des Basismodellzeugen	bestanden
Basismodelltheorie in integrierter Session	100 % verarbeitet; Exit-Code 0
base_states und Objektmenge	beide nichtleer bewiesen
Wohlgeformtheit und zulässiger Übergang	base_state_well_formed und base_self_step kernelgeprüft
vollständige Locale-Interpretation A1–A22	Base: uza_model kernelgeprüft
Existenztheorem	explicit_uza_model_exists kernelgeprüft
Modellgrenze	minimaler Erfüllbarkeitszeuge; die meisten objekttypspezifischen Bedingungen bleiben vakuos
kernelgeprüfter Basismodell-Hash	322d2a8fc31dbca1ea6c7cc2463b24c891701a79eebb761a94f5256a62632a4b
Struktur-Preflight des Rich-Model-Zeugen	bestanden

Prüfung	Ergebnis
Rich-Model-Theorie in integrierter Session	100 % verarbeitet; Exit-Code 0
reichhaltiger Zustand	zwölf versionierte Objekte, zwei Kontexte, Projektion, Kopplung, validierte Transmission und Fusion
A5–A13-Nichtvakuosität	neun konkrete Zertifikate decken Relation, Wirkung, Bindungen, Evidenz, Relevanz, Orientierung, Handlung und Rückwirkung ab
A17–A21-Nichtvakuosität	Projektion, Kopplung, validierte Transmission mit Lineage und Boundary-Disposition kernelgeprüft
vollständige Rich-Locale-Interpretation	Rich: uza_model kernelgeprüft
Rich-Existenztheorem	rich_uza_model_exists kernelgeprüft
kernelgeprüfter Rich-Model-Hash	3a3671b918d9639607fb6152dedd039cdf6145b0f040db8edf6ba296b8dc9a8c
Struktur-Preflight der T1-Operationstheorie	bestanden
konkrete T1-Konstruktion ohne Ziel-Wohlgeformtheit als Vorbedingung	bestanden
T1-Theorie in integrierter Session	100 % verarbeitet; Exit-Code 0
exakte Zustandsdifferenz und frische Objekt-/Versionsidentität	kernelgeprüft
Erhaltung von state_well_formed und bestehender Objektversionen	kernelgeprüft
Evidence Integrity sowie Referenz-, Zeit-, Perspektiv- und Kontexterhaltung	kernelgeprüft
unbekannte Evidence-ID	durch unresolved_evidence_reference_rejected ausgeschlossen
Refinement zu step G s RegisterAppearance s'	kernelgeprüft
Beobachtungs-/Potentialtrennung in T1	Beobachtungen besitzen keine Potentialquelle, -basis oder Analysemarkierung
kernelgeprüfter T1-Theorie-Hash	9b8cc6167b99070a47f63b4e63bd5a783a000f6ec84f7620945bee224f116cd0
Struktur-Preflight der T2-Operationstheorie	bestanden
konkrete T2-Konstruktion ohne Ziel-Wohlgeformtheit als Vorbedingung	bestanden
T2-Theorie in integrierter Session	100 % verarbeitet; Exit-Code 0

Prüfung	Ergebnis
Kandidatenmenge	endlich, nichtleer, intern eindeutig und gegenüber dem Ausgangszustand frisch
Beobachtung/Kandidat	potential_candidate_not_observed kernelgeprüft
exakte Mengendifferenz und Snapshot-Fortschreibung	differentiate_potential_exact_delta kernelgeprüft
Erhaltung von state_well_formed und bestehender Objektversionen	kernelgeprüft
Evidence Integrity sowie Referenz-, Zeit-, Perspektiv- und Kontexterhaltung	kernelgeprüft
Refinement zu step G s DifferentiatePotential s'	kernelgeprüft
kernelgeprüfter T2-Theorie-Hash	ef483b279d0581752bfff655db6672fa57823dac348ace29268e68e486a40f9b
Struktur-Preflight der T3-Operationstheorie	bestanden
integrierte Importkette	lokale Theorieimporte; eine zulässige Session mit elf Theorien
T3-Quellumfang	580 Zeilen; relation_type, relation_catalog, aktive Endpunkte, exaktes Delta und Erhaltungssätze
unvollständige Beweisplatzhalter	keine Vorkommen von sorry, oops oder admit
kernelgeprüfter T3-Theorie-Hash	a3c801eaf1640624ee7bf41ba91a20d2ba1e000e489ee40f4d3c209b15911ca3
T3-Theorie in integriertem Clean-Build	100 % verarbeitet; Exit-Code 0
T3-Parser-, Typ- und Kernelstatus	Isabelle2025-2 akzeptiert; Buildlog und Exit-Code als Einzeldateien
Struktur-Preflight der T4-Operationstheorie	bestanden
T4-Payload und Offenheitssemantik	effect_hypothesis_payload; Ziel und Wirkungsvektor als option; keine Sentinel-ID und kein stilles Zero-Fill
T4-Quellumfang	718 Zeilen; aktive K_REL-Ursache, Perspektiv-, Kontext- und Profilmodellbindung, begrenzte Unsicherheit, exaktes Delta und Erhaltungssätze

Prüfung	Ergebnis
unvollständige T4-Beweisplatzhalter	keine Vorkommen von sorry, oops oder admit
kernelgeprüfter T4-Theorie-Hash	2eb8e1d262cb3790f6c446839fb6809de3c2837d20705925507e272c860e2fc4
T4-Theorie in integriertem Clean-Build	100 % verarbeitet; Exit-Code 0
T4-Erhaltung und Refinement	state_well_formed, Identität, Katalog, Referenzen, Zeit, Perspektive, Kontext und Evidence Integrity sowie Refinement zu step kernelgeprüft
T4-Buildlog-Hash	7a49a99e15790d7324def03c5078600e4cb75bd2715b9439380fe711e85daa06
Struktur-Preflight der T5-Operationstheorie	bestanden
T5-Quellumfang	1.564 Zeilen; Copy-on-Write, Archive, Evidenz-Ledger, Lifecycle-, Unsicherheits-, Delta- und Erhaltungssätze
unvollständige T5-Beweisplatzhalter	keine Vorkommen von sorry, oops, axiomatization oder quick_and_dirty
T5-Lifecycle-Grenze	Hypothesis → Supported; Validated nur aus Supported bei Schwellen; Contradiction → Rejected; Validated-Revision an T13 delegiert
T5-Selbststützung	aktuelle und unmittelbare Vorgängerversion als Evidence ausgeschlossen; Evidenzquelle aktives anderes logisches Objekt
T5-Erhaltung und Refinement	state_well_formed, Identität, Aktiv/Historie-Disjunktheit, Katalog, Archive, Evidenz-Ledger, Referenzen, Zeit, Perspektive, Kontext und Evidence Integrity sowie Refinement zu step kernelgeprüft
kernelgeprüfter T5-Theorie-Hash	4422076e2bd6cb2ff9fe5e2d5e2e135e3bf3c05919a1ca03a42b1faccd5fddc0
T5-Zeugenquellumfang	281 Zeilen; endlicher nicht-vakuoser Hypothesis → Supported-Lauf
T5-Zeugenzertifikate	Eingabevalidität, konkreter Schritt, Lifecycle, exakte Audit-Differenz, Zielwohlgeformtheit und generischer step kernelgeprüft
kernelgeprüfter T5-Zeugen-Hash	f5e037e578335431903754153deaedeada461fed8d359

Prüfung	Ergebnis
	a0511e36b59a825a7d
T5-Clean-Build und Buildlog	Isabelle2025-2; Exit-Code 0; Buildlog-SHA-256 f15ccd5b811e988c85abd3232adbc5c1cf8b2b2d2a7bfca 7b2adb3d99fc35166
Struktur-Preflight der T6- Operationstheorie	bestanden
T6-Quellumfang	1.289 Zeilen; gerichtetes Profil, Score, Form, Stärke, Konstruktoren, exaktes Delta und Erhaltungssätze
unvollständige T6- Beweisplatzhalter	keine Vorkommen von sorry, oops, admit, axiomatization oder quick_and_dirty
V10-Summationsdomäne	UNIV exakt als zehn effect_dimension-Konstruktoren charakterisiert; Endlichkeit kernelgeprüft
T6-Richtung und Legacy-Grenze	normative Operation mit K(source,target); symmetrischer V10-Dot-Product nur Baseline, keine stille Gleichsetzung
T6-Erhaltung und Refinement	state_well_formed, Identität, Effect-/Resonance-Katalog, Referenzen, Evidenz, Zeit, Perspektive und Kontext sowie Refinement zu step kernelgeprüft
kernelgeprüfter T6-Theorie-Hash	3f3583b42f066eaf41208a22bcb53a2d49baed374aa1b4 1e00a434256741b1cd
T6-Zeugenquellumfang	438 Zeilen; zwei validierte Effekte und nicht- symmetrisches Kernelprofil
T6-Asymmetriezertifikat	$A \rightarrow B=1$ /Amplification; $B \rightarrow A=0$ /Neutral; Stärke 9/10; Lifecycle Validated
T6-Zeugenzertifikate	Eingabevalidität, konkreter Schritt, Rechenwerte, exaktes Delta, Zielwohlgeformtheit, Katalog und generischer step kernelgeprüft
kernelgeprüfter T6-Zeugen-Hash	494da00eeb17205001ae85090d940a12cd07215ec283d 76833aa480126a716ed
T6-Fresh-Profile-Build und Buildlog	Isabelle2025-2; elf Theorien; Exit-Code 0; Buildlog- SHA-256 e5945cfd1c2a62588862592a6bdf4c89b3dab15f34c0b34 2060144b033ef2f21

H.1 führt nun den aktuellen Gesamtstatus: Die früheren Sessionstände bis einschließlich T5 bleiben historisch nachvollziehbar; maßgeblich ist der Isabelle2025-2-Fresh-Profile-Build des T6-Gesamtstands vom 9. August 2026. Er verarbeitete Haupttheorie, Basismodell, T1, Rich Model,

T2, T3, T4, T5, T5-Zeuge, T6 und T6-Zeuge und endete mit Exit-Code 0. Der T6-Hash lautet 3f3583b42f066eaf41208a22bcb53a2d49baed374aa1b41e00a434256741b1cd; der T6-Zeugenhash lautet 494da00eeb17205001ae85090d940a12cd07215ec283d76833aa480126a716ed; der Buildlog-Hash lautet e5945cfd1c2a62588862592a6bdf4c89b3dab15f34c0b342060144b033ef2f21; der ROOT-Hash lautet 3c5e6445691abc6aaffcc7261d2bb63dcda3d7687a632d3f7521f0498c5055f5. Rohlog, Exit-Code, ROOT, Theoriequellen, Hashregister, Source Audit und Verifikationsbericht werden als getrennte Einzeldateien geführt. Die Beweisgrenzen bleiben ausdrücklich ausgewiesen.

H.2 Ausführbare Conformance

Prüfung	Ergebnis
Golden-Run-Replay	3/3 bestanden
automatisierte Testgruppen	3/3 bestanden
Replay-Abweichungen	0
FDB-vor-EMG-Invariante	bestanden
Transmission-Lineage und Nicht-Fusion	bestanden
Geltungs-, Falsifikations- und Patch-Abbruchgates	bestanden
verlustbewusste Legacy-Migration ohne stilles Zero-Fill	bestanden